



MaxPatrol SIEM

версия 27.6

Руководство разработчика

© Positive Technologies, 2026.

Настоящий документ является собственностью Positive Technologies и защищен законодательством Российской Федерации и международными соглашениями об авторских правах и интеллектуальной собственности.

Копирование документа либо его фрагментов в любой форме, распространение, в том числе в переводе, а также их передача третьим лицам возможны только с письменного разрешения Positive Technologies.

Документ может быть изменен без предварительного уведомления.

Товарные знаки, использованные в тексте, приведены исключительно в информационных целях, исключительные права на них принадлежат соответствующим правообладателям.

Дата редакции документа: 19.03.2026

Содержание

1.	Об этом документе.....	14
1.1.	Условные обозначения.....	15
1.2.	Другие источники информации о MaxPatrol SIEM.....	15
2.	Этапы обработки событий.....	16
3.	Программные компоненты для обработки событий.....	18
4.	Язык eXtraction and Processing.....	20
4.1.	Имена.....	20
4.2.	Математические и логические операторы.....	21
4.2.1.	Присваивание «=».....	22
4.2.2.	Сложение «+».....	23
4.2.3.	Вычитание «-».....	23
4.2.4.	Умножение «*».....	24
4.2.5.	Деление «div».....	25
4.2.6.	Деление по модулю «mod».....	25
4.2.7.	Равенство «==».....	26
4.2.8.	Неравенство «!=».....	26
4.2.9.	Сравнение: «<», «>».....	27
4.2.10.	Сравнение: «>=», «<=».....	28
4.2.11.	Конъюнкция: «and», «&&».....	28
4.2.12.	Дизъюнкция: «or», « ».....	29
4.2.13.	Отрицание «not».....	30
4.3.	Конструкции языка.....	30
4.3.1.	Символ комментария «#».....	31
4.3.2.	Объявление строки.....	31
4.3.3.	Объединение в список «[... , ...]».....	32
4.3.4.	Условие «if».....	33
4.3.5.	Множественный выбор «switch».....	33
4.4.	Функции работы с временем.....	35
4.4.1.	datetime.....	35
4.4.2.	datetime_to_epoch, datetime_to_epoch_ms.....	36
4.4.3.	datetime_to_win_ticks.....	37
4.4.4.	duration.....	37
4.4.5.	epoch_to_datetime, epoch_ms_to_datetime.....	38
4.4.6.	hour, minute, second, timezone.....	39
4.4.7.	win_ticks_to_datetime.....	40
4.4.8.	year, month, day.....	40
4.5.	Функции преобразования типов данных.....	41
4.5.1.	bool.....	41
4.5.2.	buffer_from_base64.....	42
4.5.3.	decode.....	43
4.5.4.	hex_to_string.....	43
4.5.5.	ipv4.....	44
4.5.6.	ipv6.....	44

4.5.7.	join	45
4.5.8.	macaddr	46
4.5.9.	number	47
4.5.10.	number8	47
4.5.11.	number16	48
4.5.12.	string	48
4.6.	Функции для работы со строками	49
4.6.1.	csv	49
4.6.2.	find_substr	50
4.6.3.	keyvalue	51
4.6.4.	length	52
4.6.5.	lower	52
4.6.6.	match	53
4.6.7.	regex	54
4.6.8.	replace	57
4.6.9.	rot13	59
4.6.10.	strip	60
4.6.11.	strip_all	60
4.6.12.	substr	61
4.6.13.	upper	62
4.7.	Функции для работы со списками	62
4.7.1.	append	63
4.7.2.	extend	63
4.7.3.	in_list	64
4.7.4.	intersect	65
4.7.5.	is_list	65
4.7.6.	remove	66
4.7.7.	slice	67
4.7.8.	sort	67
4.7.9.	split	68
4.7.10.	unique	69
4.8.	Функции для анализа данных	69
4.8.1.	coalesce	70
4.8.2.	flip_endianness16, flip_endianness32, flip_endianness64	70
4.8.3.	in_subnet	71
4.8.4.	set_bit	72
4.8.5.	clear_bit	72
4.8.6.	get_bit	73
4.8.7.	check_bit_mask	74
4.9.	Функции для работы с табличными списками	74
4.9.1.	Префикс table::, инлайн-запрос	75
4.9.2.	Функция exes_query, запрос к табличному списку	76
4.9.3.	Функция exes_query, арифметические операции над результатами запроса	78
4.9.4.	Функция select_query_first, запрос значения из табличного списка	81
4.9.5.	Функция regex_match, проверка строки по регулярному выражению из табличного списка	83

5.	Создание правила нормализации	85
5.1.	Алгоритм работы службы нормализации событий.....	85
5.2.	Структура правила нормализации.....	87
5.3.	Заполнение полей нормализованного события	87
5.4.	Ключевое слово COND. Условие использования правила нормализации	88
5.5.	Ключевое слово TEXT	89
5.5.1.	Операторы токена.....	91
5.5.2.	BASE64.....	93
5.5.3.	CSV	94
5.5.4.	DATETIME	95
5.5.5.	DURATION	97
5.5.6.	HOSTNAME	98
5.5.7.	IPV4.....	98
5.5.8.	IPV6.....	99
5.5.9.	KEYVALUE.....	100
5.5.10.	LITERAL.....	101
5.5.11.	MACADDR	101
5.5.12.	NTUSER.....	102
5.5.13.	NUMBER.....	102
5.5.14.	REST	103
5.5.15.	STRING.....	103
5.5.16.	UNTIL.....	104
5.5.17.	WORD	104
5.5.18.	WORDDASH	105
5.6.	Ключевое слово TABULAR	105
5.7.	Ключевое слово JSON	107
5.8.	Ключевое слово EVENTLOG	108
5.9.	Ключевое слово XML	112
5.10.	Команда drop.....	115
5.11.	Вложенное правило нормализации.....	115
5.12.	Пример правила нормализации	117
6.	Создание макроса.....	120
7.	Создание шаблона исключений для заполнения табличного списка	122
8.	Создание правила агрегации	126
8.1.	Алгоритм работы службы агрегации.....	126
8.2.	Структура правила агрегации	127
8.3.	Директива event. Объявление события, подлежащего агрегации.....	128
8.4.	Директива aggregate. Условие правила агрегации	129
9.	Создание правила обогащения.....	133
9.1.	Структура правила обогащения	133
9.2.	Директива event. Объявление события для обогащения.....	134
9.3.	Директива enrichment. Название правила обогащения	135
9.4.	Директива enrich. Обработчик события	135
9.4.1.	Инструкция enrich_fields. Обогащение события	136
9.4.2.	Инструкция insert_into. Запись в табличный список	138

9.4.3.	Инструкция insert_into. Условная запись в табличный список.....	143
9.4.4.	Инструкция insert_into. Арифметические операции при записи в табличный список	149
9.4.5.	Инструкция remove_from. Удаление строк из табличного списка.....	155
9.4.6.	Команда drop	160
9.5.	Директива query. Объявление запроса к табличному списку.....	160
9.5.1.	Инструкции qhandler, limit, skip.....	165
9.5.2.	Агрегатные функции для результатов запроса к табличному списку.....	170
10.	Создание правила корреляции	171
10.1.	Алгоритм работы службы корреляции	171
10.2.	Структура правила корреляции.....	172
10.3.	Заполнение полей корреляционного события	173
10.4.	Директива event. Объявление события, подлежащего корреляции.....	174
10.4.1.	Инструкция key.....	177
10.4.2.	Инструкция filter	178
10.5.	Директива rule. Условие правила корреляции	179
10.5.1.	Логические операторы для последовательности событий	183
10.5.2.	Квантификаторы событий последовательности	185
10.5.3.	Операторы отсчета времени для последовательности	188
10.5.4.	Оператор as. Псевдонимы события.....	190
10.5.5.	Инструкция init. Объявление переменных.....	190
10.5.6.	Инструкция on. Обработчик события.....	191
10.5.7.	Команда close.....	192
10.6.	Директива rule. Изменение табличного списка	192
10.6.1.	Инструкция insert_into. Запись в табличный список	193
10.6.2.	Инструкция insert_into. Условная запись в табличный список.....	198
10.6.3.	Инструкция insert_into. Арифметические операции при записи в табличный список	204
10.6.4.	Инструкция insert_into. Добавление строк в табличный список.....	211
10.6.5.	Инструкция remove_from. Удаление строк из табличного списка.....	217
10.6.6.	Инструкция clear_table. Очистка табличного списка.....	222
10.7.	Директива emit. Корреляционное событие	226
10.8.	Команда copy_fields.....	227
10.9.	Директива query. Объявление запроса к табличному списку.....	228
10.9.1.	Инструкции qhandler, limit, skip.....	234
10.9.2.	Агрегатные функции для результатов запроса к табличному списку.....	240
11.	Создание файла расширения аудита	241
11.1.	Секция \$converters	242
11.1.1.	Audit Extension SDK.....	243
11.1.2.	Функции API	244
11.1.2.1.	mpx.ldap_exec	247
11.1.2.2.	mpx.odbc.mssql_exec	248
11.1.2.3.	mpx.odbc.mysql_exec	249
11.1.2.4.	mpx.odbc.oracle_exec.....	249
11.1.2.5.	mpx.odbc.postgre_exec	250
11.1.2.6.	mpx.scanner.check_port	251
11.1.2.7.	mpx.scanner.dns_lookup.....	251

11.1.2.8.	mpx.scanner.dns_reverse_lookup	252
11.1.2.9.	mpx.scanner.get_profile_data	252
11.1.2.10.	mpx.scanner.get_scanning_ip	252
11.1.2.11.	mpx.unix.get_file_by_path	253
11.1.2.12.	mpx.unix.ssh_exec	253
11.1.2.13.	mpx.windows.delete_file	254
11.1.2.14.	mpx.windows.dir_exist	254
11.1.2.15.	mpx.windows.eight_dot_three_to_fullpath	254
11.1.2.16.	mpx.windows.file_exist	255
11.1.2.17.	mpx.windows.find_files	255
11.1.2.18.	mpx.windows.get_account_name_by_sid	256
11.1.2.19.	mpx.windows.get_audit_policy	256
11.1.2.20.	mpx.windows.get_binary_file_by_path	258
11.1.2.21.	mpx.windows.get_dirs	258
11.1.2.22.	mpx.windows.get_file_by_path	259
11.1.2.23.	mpx.windows.get_file_info_by_path	259
11.1.2.24.	mpx.windows.get_file_info_recursive	260
11.1.2.25.	mpx.windows.get_filetime	260
11.1.2.26.	mpx.windows.get_lockout_policy	260
11.1.2.27.	mpx.windows.get_registry_values	261
11.1.2.28.	mpx.windows.get_security_info	263
11.1.2.29.	mpx.windows.get_sid_string_by_account_name	264
11.1.2.30.	mpx.windows.get_system_temp_directory	265
11.1.2.31.	mpx.windows.get_user_sids_by_privilege	265
11.1.2.32.	mpx.windows.get_users_by_privilege	267
11.1.2.33.	mpx.windows.normalize_path	268
11.1.2.34.	mpx.windows.powershell_exec	268
11.1.2.35.	mpx.windows.shell_exec	269
11.1.2.36.	mpx.windows.wmi_exec	270
11.1.2.37.	mpx.windows.wmi_query	270
11.1.2.38.	mpx.windows.write_file	271
11.2.	Секция \$extractors	271
11.2.1.	Экстракторы классов File и FileInfo	272
11.2.2.	ODBC-экстракторы	274
11.2.3.	Registry Extractor	276
11.2.4.	Shell adapters	277
11.2.5.	Экстракторы WMI	278
11.3.	Секция \$transformers	280
11.3.1.	DictValue (JSON) Parser	280
11.3.2.	KeyValue Parsers	281
11.3.3.	Search Parser	282
11.3.4.	XML-парсеры классов XMLFindone и XMLFinditer	283
11.3.5.	Resolvers	285
11.4.	Секция \$loaders	288
11.5.	Примеры заполнения файлов расширения аудита с секциями \$converters и \$loaders	293

11.5.1.	Сбор данных о процессах в Windows.....	294
11.5.2.	Сбор данных о процессах в ОС семейства Unix	295
11.5.3.	Сбор данных о Microsoft Sysmon	297
11.5.4.	Сбор данных о ПО из Uninstall-веток реестра	299
11.5.5.	Сбор данных о БД под управлением Microsoft SQL Server.....	302
11.5.6.	Дополнительный сбор данных о Microsoft Exchange.....	304
11.6.	Примеры заполнения файлов расширения аудита с секциями \$extractors, \$transformers и \$loaders	307
12.	Настройка регистрации инцидентов.....	311
12.1.	Алгоритм регистрации инцидента	312
12.2.	Поля для заполнения информации об инцидентах.....	312
12.3.	Поля для настройки агрегации инцидентов	317
13.	Утилиты SDK.....	320
13.1.	Экспорт событий из БД. Утилита export_data	320
13.2.	Отладка правил нормализации.....	323
13.2.1.	гсс	324
13.2.2.	normalizer-cli	326
13.3.	Отладка правил агрегации	328
13.3.1.	гсс	329
13.3.2.	aggregator-cli.....	330
13.4.	Отладка правил обогащения	331
13.4.1.	fpta_filler.....	332
13.4.2.	гсс	335
13.4.3.	enricher-cli.....	337
13.5.	Отладка правил корреляции.....	338
13.5.1.	гсс	339
13.5.2.	router-cli	340
13.5.3.	correlator-cli.....	342
13.6.	Структура файла schema.json	344
14.	Утилита PTSIEMSDK GUI	349
14.1.	Интерфейс	349
14.1.1.	Вкладка Редактор объектов	350
14.1.2.	Вкладка Сценарии.....	351
14.1.3.	Вкладка Настройка	354
14.1.4.	Управление пакетами.....	357
14.1.5.	Справка	357
14.2.	Предварительная настройка	358
14.3.	Работа с пакетами.....	359
14.3.1.	Распаковка пакета.....	359
14.3.2.	Формирование пакета с объектами из папок экспертной структуры	360
14.3.3.	Формирование пакета с объектами из произвольных папок	360
14.3.4.	Формирование пакета с пользовательским пакетом экспертизы	361
14.4.	Отладка отдельного правила	362
14.4.1.	Создание схемы табличных списков и файла со значениями по умолчанию	363
14.4.2.	Создание набора данных для отладки и отладка правила	364
14.4.3.	Составление тестового сценария	365

14.4.4.	Отладка правила с помощью тестов	367
14.4.5.	Метаданные для локализации	368
14.4.6.	Проверка регулярного выражения	371
14.5.	Отладка совместной работы правил	372
14.5.1.	Сценарий для отладки правил нормализации и локализации	372
14.5.2.	Сценарий для отладки правил для всех этапов обработки событий.....	373
15.	Утилита kbtools	376
15.1.	exportPackage, экспорт объектов из БД.....	376
15.2.	importPackage, импорт объектов в БД	378
15.3.	unpack, распаковка пакета	379
15.4.	pack, формирование пакета для импорта	380
15.5.	createRootDB, создание БД	380
15.6.	dropRootDB, удаление БД.....	381
16.	Виды запросов к API	383
16.1.	Создание токена доступа	384
16.2.	Работа с группами активов	384
16.2.1.	Получение идентификатора инфраструктуры	385
16.2.2.	Получение идентификатора группы активов.....	386
16.2.3.	Создание группы активов.....	389
16.2.4.	Проверка статуса создания группы.....	393
16.2.5.	Удаление группы активов.....	394
16.3.	Получение токена PDQL-запросов.....	395
16.4.	Импорт активов из CSV	397
16.4.1.	Получение пользовательских полей.....	397
16.4.2.	Запрос формата CSV-файла	399
16.4.3.	Загрузка CSV-файла	400
16.4.4.	Выгрузка файла ошибок	403
16.4.5.	Запуск импорта активов	404
16.4.6.	Проверка статуса импорта.....	404
16.5.	Работа с табличными списками в MaxPatrol SIEM	406
16.5.1.	Поиск табличного списка	406
16.5.2.	Получение информации о табличном списке	408
16.5.3.	Экспорт табличного списка	411
16.5.4.	Импорт табличного списка.....	413
16.5.5.	Очистка табличных списков	415
16.5.6.	Обновление табличного списка	415
16.6.	Работа с табличными списками в Knowledge Base.....	417
16.6.1.	Создание табличного списка	417
16.6.2.	Поиск объекта БД	421
16.6.3.	Получение информации о табличном списке	426
16.6.4.	Экспорт табличного списка	432
16.6.5.	Открытие сессии импорта табличного списка.....	433
16.6.6.	Загрузка CSV-файла	434
16.6.7.	Запуск импорта табличного списка	436
16.6.8.	Установка объектов в MaxPatrol SIEM	437

16.7.	Работа с инцидентами	439
16.7.1.	Регистрация инцидентов	440
16.7.2.	Обновление информации об инциденте	445
16.7.3.	Удаление инцидентов	448
16.7.4.	Получение списка инцидентов	449
16.7.5.	Получение данных инцидента	453
16.7.6.	Получение событий по инциденту	458
16.7.7.	Экспорт инцидентов	459
16.7.8.	Открытие сессии импорта инцидентов	466
16.7.9.	Объявление загрузки файла с инцидентами	467
16.7.10.	Загрузка файла с инцидентами	468
16.7.11.	Импорт инцидентов	470
16.7.12.	Удаление инцидентов по фильтру	472
16.7.13.	Удаление связи события с инцидентом	473
16.7.14.	Удаление связи актива с инцидентом	474
16.7.15.	Изменение мер инцидента	475
16.7.16.	Обновление групп инцидентов	476
16.7.17.	Привязка событий к инциденту	476
16.7.18.	Изменение статуса инцидента	477
16.7.19.	Создание задачи	478
16.7.20.	Обновление задачи	480
16.7.21.	Получение данных о событии	480
16.8.	Работа с событиями	481
16.8.1.	Получение списка событий	481
16.8.2.	Получение списка агрегированных событий	486
16.8.3.	Привязка событий к инциденту	490
16.9.	Управление табличными списками для получения информации об изменении активов	492
16.9.1.	Получение списка подписчиков	492
16.9.2.	Получение информации о подписчике	493
16.9.3.	Удаление подписчика и его табличных списков	494
16.9.4.	Регистрация подписчика	494
16.9.5.	Получение информации о табличном списке подписчика	495
16.9.6.	Создание или обновление табличного списка	496
16.9.7.	Удаление табличных списков подписчика	497
16.9.8.	Получение информации о табличных списках подписчика	497
16.9.9.	Удаление табличного списка подписчика	498
16.9.10.	Получение изменений табличного списка	499
16.10.	Работа с учетными записями сканирования	500
16.10.1.	Добавление учетной записи типа «логин — пароль»	501
16.10.2.	Получение учетной записи типа «логин — пароль»	502
16.10.3.	Обновление учетной записи типа «логин — пароль»	503
16.10.4.	Добавление учетной записи типа «пароль»	504
16.10.5.	Получение учетной записи типа «пароль»	505
16.10.6.	Обновление учетной записи типа «пароль»	506
16.10.7.	Добавление учетной записи типа «сертификат»	507

16.10.8.	Получение учетной записи типа «сертификат»	508
16.10.9.	Обновление учетной записи типа «сертификат»	509
16.10.10.	Получение всех учетных записей	510
16.10.11.	Удаление учетной записи	511
16.10.12.	Получение списка меток учетных записей	511
16.11.	Работа со сканами	511
16.11.1.	Добавление нового скана	512
16.11.2.	Получение метаданных скана по идентификатору	513
16.11.3.	Получение содержания обработанного скана	514
16.11.4.	Получение коллекции данных обработанных сканов	514
16.11.5.	Получение пакета информации об обработанных сканах	515
16.11.6.	Получение метаданных сырого скана по идентификатору	517
16.11.7.	Получение содержания сырого скана в формате XML	518
16.11.8.	Получение коллекции данных сырых сканов	518
16.11.9.	Получение пакета информации о сырых сканах	519
16.12.	Работа с профилями сканирования	520
16.12.1.	Получение списка профилей сканирования	521
16.12.2.	Создание профиля сканирования	522
16.12.3.	Получение расширенной информации о профиле сканирования	523
16.12.4.	Обновление профиля сканирования	525
16.12.5.	Удаление профиля сканирования	526
16.12.6.	Генерация профиля PenTest	527
16.12.7.	Проверка наличия уязвимости при сканировании в режиме пентеста	528
16.12.8.	Получение списка схем профилей сканирования	529
16.12.9.	Получение локализации параметров схемы модуля	530
16.12.10.	Удаление из базы ошибок, возникших при миграции профиля	531
16.13.	Управление задачами сканирования	531
16.13.1.	Запрос отчета обо всех задачах	532
16.13.2.	Запрос отчета обо всех задачах (API версии 4)	535
16.13.3.	Создание задачи	542
16.13.4.	Создание задачи (API версии 4)	549
16.13.5.	Запрос количества задач	553
16.13.6.	Запрос отчета обо всех точках сохранения всех задач	554
16.13.7.	Запрос количества задач по инфраструктуре	555
16.13.8.	Запрос отчета по задаче по идентификатору	555
16.13.9.	Запрос отчета по задаче по идентификатору (API версии 4)	558
16.13.10.	Обновление задачи	561
16.13.11.	Обновление задачи (API версии 4)	565
16.13.12.	Удаление задачи	568
16.13.13.	Валидация задачи	569
16.13.14.	Сброс закладок для задачи сканирования	570
16.13.15.	Запуск задачи сканирования	570
16.13.16.	Остановка задачи	571
16.13.17.	Приостановка задачи	572
16.13.18.	Удаление из базы ошибок, возникших при миграции задачи	572

16.13.19.	Получение времени следующего запуска задачи (API версии 4).....	572
16.14.	Мониторинг задач сканирования.....	574
16.14.1.	Получение истории запусков задачи	574
16.14.2.	Получение списка подзадач в составе задачи.....	576
16.14.3.	Получение информации о подзадаче.....	578
16.14.4.	Получение ошибок, произошедших при выполнении подзадачи.....	579
16.14.5.	Получение информации об используемых в задаче коллекторах	580
16.15.	Журналирование действий пользователя.....	580
16.15.1.	Получение дерева категорий действий.....	581
16.15.2.	Получение возможного значения дополнительного поля.....	582
16.15.3.	Получение описания всех дополнительных полей для всех приложений	582
16.15.4.	Получение отчетов о действиях пользователей	583
16.15.5.	Получение отчетов о действиях пользователей по фильтру	585
16.15.6.	Регистрация событий пользователя.....	588
16.16.	Работа с активами	588
16.16.1.	Запуск задачи на удаление активов	589
16.16.2.	Получение состояния задачи по удалению активов	590
16.16.3.	Запуск задачи на изменение расположения активов в группах.....	591
16.16.4.	Получение состояния задачи по обновлению расположения активов в группах	592
16.16.5.	Запуск задачи на обновление паспортов активов	593
16.16.6.	Получение состояния задачи по обновлению паспорта активов	594
16.16.7.	Отмена задачи обновления паспорта активов	596
16.16.8.	Сохранение статуса завершенной задачи.....	596
16.17.	Получение метаданных актива.....	596
16.18.	Экспорт таблицы активов в CSV	598
16.19.	Массовые операции над задачами	599
16.19.1.	Запуск задач сканирования.....	599
16.19.2.	Остановка задач сканирования.....	600
16.19.3.	Приостановка задач сканирования.....	600
16.19.4.	Удаление задач сканирования.....	601
16.19.5.	Установка расписания задач сканирования	601
16.19.6.	Сравнение расписаний запуска задач	602
16.19.7.	Сброс закладок для задач сканирования.....	603
16.19.8.	Получение информации о массовой операции.....	604
16.19.9.	Удаление массовой операции	604
16.20.	Работа с выбором задач	605
16.20.1.	Выбор задач для использования в массовых операциях	605
16.20.2.	Изменение параметров выбора задач	607
16.20.3.	Получение информации о выбранных задачах по их идентификатору.....	610
16.20.4.	Отмена выбора задач.....	610
17.	Отправка уведомлений через POST-запрос	612
17.1.	Поля POST-запроса для уведомления.....	613
17.2.	Тестирование приема POST-запросов	614

18. О технической поддержке	617
Приложение А. Типы данных	621
Приложение Б. Схема полей событий	623
Приложение В. Значения полей с типом данных Enum	647
Приложение Г. Правила заполнения поля event_src.category	653
Приложение Д. Правила заполнения полей category.generic, category.high, category.low для нормализованных событий	655
Приложение Е. Правила заполнения полей инцидента category и type	664
Приложение Ж. Экранирование специальных символов в регулярных выражениях	669
Предметный указатель	671

1. Об этом документе

Руководство разработчика содержит информацию, необходимую для тонкой настройки анализа данных в Positive Technologies MaxPatrol SIEM (далее также — MaxPatrol SIEM).

Руководство содержит рекомендации по созданию правил нормализации, корреляции, агрегации и обогащения событий и описание утилит MaxPatrol SIEM SDK для их отладки.

В руководстве приводятся сведения о функциях языка eXtraction and Processing, используемых при создании правил нормализации, корреляции и обогащения. В приложении дан перечень полей событий и указаны правила их заполнения.

Руководство также содержит информацию о доступных в MaxPatrol SIEM функциях сервиса REST API.

Руководство адресовано разработчикам, выполняющим настройку MaxPatrol SIEM для сбора, обработки и анализа событий источников, а также выполняющих интеграцию MaxPatrol SIEM со сторонними приложениями.

Комплект документации MaxPatrol SIEM включает в себя следующие документы:

- Этот документ.
- Руководство по внедрению — содержит информацию для внедрения продукта в инфраструктуре организации: от типовых схем развертывания до инструкций по установке, первоначальной настройке и обновлению продукта.
- Руководство администратора — содержит справочную информацию и инструкции по настройке и администрированию продукта.
- Руководство оператора — содержит сценарии использования продукта для управления информационными активами организации и событиями информационной безопасности.
- Руководство по настройке источников — содержит рекомендации по интеграции элементов IT-инфраструктуры организации с MaxPatrol SIEM для сбора событий с источников и аудита активов.
- Синтаксис языка запроса PDQL — содержит справочную информацию и примеры синтаксиса, основных функций и операторов языка PDQL, используемых при работе с MaxPatrol SIEM.
- PDQL-запросы для анализа активов — содержит информацию о стандартных запросах на языке PDQL, предназначенных для проверки конфигураций активов при работе в MaxPatrol SIEM.

В этом разделе

[Условные обозначения \(см. раздел 1.1\)](#)

[Другие источники информации о MaxPatrol SIEM \(см. раздел 1.2\)](#)

1.1. Условные обозначения

В документе приняты условные обозначения.

Таблица 1. Условные обозначения

Пример	Описание
Внимание! При выключении модуля снижается уровень защищенности сети	Предупреждения. Содержат информацию о действиях или событиях, которые могут иметь нежелательные последствия
Примечание. Вы можете создать дополнительные отчеты	Примечания. Содержат советы, описания важных частных случаев, дополнительную или справочную информацию, которая может быть полезна при работе с продуктом
► Чтобы открыть файл:	Начало инструкции выделено специальным значком
Нажмите OK	Названия элементов интерфейса (например, кнопок, полей, пунктов меню) выделены полужирным шрифтом
Выполните команду <i>Stop-Service</i>	Текст командной строки, примеры кода, прочие данные, которые нужно ввести с клавиатуры, выделены специальным шрифтом. Также выделены специальным шрифтом имена файлов и пути к файлам и папкам
Ctrl+Alt+Delete	Комбинация клавиш. Чтобы использовать комбинацию, клавиши нужно нажимать одновременно
<Название программы>	Переменные заключены в угловые скобки

1.2. Другие источники информации о MaxPatrol SIEM

Вы можете найти дополнительную информацию о MaxPatrol SIEM [на портале технической поддержки](#).

Портал содержит статьи базы знаний, новости обновлений продуктов Positive Technologies, ответы на часто задаваемые вопросы пользователей. Для доступа к базе знаний и всем новостям нужно создать на портале учетную запись.

Если вы не нашли нужную информацию или решение проблемы самостоятельно, обратитесь [в службу технической поддержки \(см. раздел 18\)](#).

2. Этапы обработки событий

MaxPatrol SIEM выполняет непрерывный мониторинг информационной безопасности IT-инфраструктуры организации. Мониторинг осуществляется путем сбора событий с источников и отслеживания состояния активов.

Источником событий может являться любой элемент IT-инфраструктуры, для которого настроены журналирование и сбор событий. Для описания IT-инфраструктуры используется модель. Активами называются объекты модели, соответствующие элементам IT-инфраструктуры. Изменения состояний активов фиксируются на основе данных аудита IT-инфраструктуры организации.

Для обнаружения событий информационной безопасности используется метод rule-based reasoning. Специалисты по информационной безопасности заранее создают правила, описывающие признаки события. Событие информационной безопасности фиксируется, если в потоке событий от источников появляется событие (или последовательность событий), указанное в одном из правил, или фиксируется изменение состояния актива, описанное в одном из правил.

Сбор событий

Сбор событий с источников выполняется компонентом MP 10 Collector. Компонент имеет модульную структуру и в зависимости от используемых модулей может выполнять активный или пассивный сбор необработанных событий с источников или аудит активов. В зависимости от особенностей хранения событий на источнике (например, сетевая папка, журнал ОС, СУБД) используются специализированные алгоритмы сбора событий. Для описания особенностей того или иного источника для модулей создаются шаблоны настройки — профили.

Нормализация событий

Для событий, собранных с разных источников, могут отличаться формат (TXT, XML, JSON) и стандарт записи. Для анализа потока событий требуется преобразовать все события к единому виду.

Нормализация события — процедура приведения необработанного события к нормализованному виду в соответствии с заранее заданным для источника и типа события правилом нормализации.

Нормализованное событие — событие представляет собой совокупность полей, заполненных данными из необработанного события согласно правилу нормализации.

Примечание. Для записи событий (нормализованных, агрегированных, корреляционных) в MaxPatrol SIEM используется внутренний стандарт, разработанный на основе стандарта Common Event Expression (его описание см. на сайте mitre.org).

Агрегация событий

Поток событий может содержать однотипные события, отличающиеся значением одного или нескольких полей (например, временем регистрации). Для сокращения количества таких событий выполняется агрегация событий.

Агрегация событий — это процесс отбора (в потоке нормализованных и корреляционных событий) событий, которые удовлетворяют условию заранее настроенного правила агрегации, и объединения их в одно агрегированное событие.

Обогащение событий

Обогащение событий — заполнение полей нормализованных, агрегированных и корреляционных событий согласно правилам обогащения. Поля заполняются данными, указанными в правиле обогащения или полученными из табличных списков.

Табличный список — двумерный массив данных, хранящийся в памяти MaxPatrol SIEM и доступный для использования в правилах корреляции и правилах обогащения.

Корреляция событий

Корреляция событий — это процесс обнаружения событий информационной безопасности путем анализа потока нормализованных событий. При обнаружении в потоке событий такой их последовательности, которая указана в условии одного из заранее настроенных правил корреляции, регистрируется корреляционное событие.

Корреляционное событие — событие информационной безопасности, представляющее собой совокупность полей, заполненных данными о возможном нарушении согласно правилам, указанным в правиле корреляции.

Инцидент — корреляционное событие, связанное с нарушением информационной безопасности. Сведения об инцидентах оперативно передаются оператору MaxPatrol SIEM.

3. Программные компоненты для обработки событий

MP 10 Collector — компонент MaxPatrol SIEM для сбора событий с источников и аудита IT-инфраструктуры предприятия. Формирует пакеты необработанных событий и отправляет их через MP 10 Core на MP SIEM Server.

MP 10 Core — сервер управления MaxPatrol SIEM, обеспечивающий доступ пользователей через веб-интерфейс и хранение информации об активах.

SIEM Server receiver — служба для приема событий. Выполняет предобработку событий и передает необработанные события службе нормализации.

SIEM Server normalizer — служба для нормализации необработанных событий источников в соответствии с указанными заранее для каждого источника и типа событий правилами нормализации. Нормализованные события передаются службе агрегации, ненормализованные — службе SIEM Server resolver.

SIEM Server correlator — служба для регистрации событий информационной безопасности в IT-инфраструктуре организации путем проверки потока нормализованных (и корреляционных) событий на соответствие указанным заранее правилам корреляции. Зарегистрированные корреляционные события передаются службе агрегации.

SIEM Server aggregator — служба для анализа потока нормализованных и корреляционных событий на соответствие указанным заранее правилам агрегации. События, удовлетворяющие правилу агрегации, объединяются в одно агрегированное событие, исходные события удаляются. Агрегированные события и события, не удовлетворяющие правилам агрегации (нормализованные, и корреляционные), передаются службе SIEM Server resolver.

SIEM Server resolver — служба для поиска связи событий с зарегистрированными в MaxPatrol SIEM активами. Для каждого нормализованного события выполняется запрос к MP 10 Core на наличие актива, параметры которого соответствуют параметрам источника события. Если актив существует, то в событии указывается UUID актива. Нормализованные и корреляционные события передаются службе обогащения, ненормализованные — службе SIEM Server storage.

SIEM Server enricher — служба для обогащения нормализованных и корреляционных событий данными в соответствии с указанными заранее правилами обогащения. Нормализованные события передаются службе SIEM Server storage, корреляционные — SIEM Server router.

SIEM Server router — служба для маршрутизации и предварительной обработки событий. Корреляционные события передаются службе SIEM Server storage. Из потока корреляционных событий отбираются удовлетворяющие условиям правил корреляции и передаются службе корреляции. Зарегистрированные корреляционные события передаются службе Notifier.

SIEM Server storage — служба для работы с хранилищем событий. Все поступающие нормализованные, ненормализованные и корреляционные события сохраняются в БД.

Notifier — сервис в составе службы корреляции для передачи информации об инцидентах в MP 10 Core.

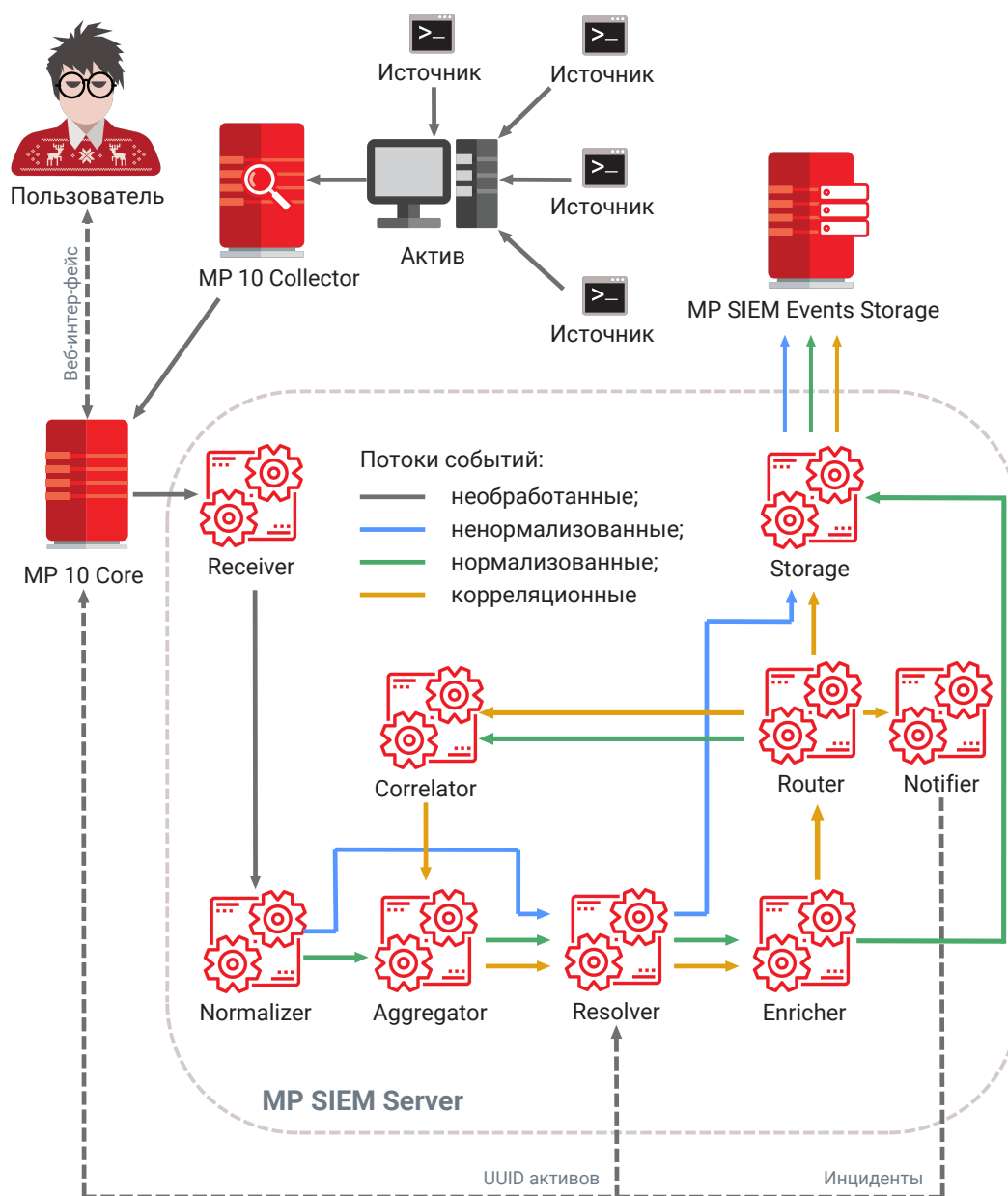


Рисунок 1. Обработка событий в MaxPatrol SIEM

4. Язык eXtraction and Processing

Язык eXtraction and Processing (далее также — XP) разработан в Positive Technologies для создания правил преобразования данных в процессе обработки событий источников в MaxPatrol SIEM. Язык XP используется при написании правил нормализации, корреляции и обогащения событий.

В разделе описаны синтаксис, основные функции и операторы языка XP, приводятся примеры использования. Некоторые функции языка XP используются только при написании правил нормализации, другие только при написании правил корреляции или обогащения событий, это будет указано дополнительно в описании функций. Полный перечень функций, используемых при написании правил нормализации, корреляции и обогащения, приведен в предметном указателе.

В этом разделе

[Имена \(см. раздел 4.1\)](#)

[Математические и логические операторы \(см. раздел 4.2\)](#)

[Конструкции языка \(см. раздел 4.3\)](#)

[Функции работы с временем \(см. раздел 4.4\)](#)

[Функции преобразования типов данных \(см. раздел 4.5\)](#)

[Функции для работы со строками \(см. раздел 4.6\)](#)

[Функции для работы со списками \(см. раздел 4.7\)](#)

[Функции для анализа данных \(см. раздел 4.8\)](#)

[Функции для работы с табличными списками \(см. раздел 4.9\)](#)

4.1. Имена

В языке XP используются следующие имена:

- Переменные, например `$f1`. Имена всех создаваемых переменных нужно начинать со знака доллара (`$`). В именах можно использовать латинские буквы, цифры, точку и символ подчеркивания. Переменным можно присваивать значения любых типов данных.
- Функции, например: `number($f1)`, `regex($f1, $f2, $f3)`, `coalesce($f1, $f2, $f3, ...)`. В круглых скобках указываются аргументы функции. Имя, количество аргументов (кроме `coalesce`) и приведение типов данных предопределены для каждой функции.
- Поля событий, например: `action`, `event_src.ip`, `correlation_name`, `datafield1`. Перечни полей событий предопределены для каждого типа события и отличаются друг от друга. Каждое поле может принимать значения только одного типа данных.

- Ключевые слова форматной строки правил нормализации для указания типа анализируемого события, например: `EVENTLOG`, `JSON`, `TEXT`, `TABULAR`, `XML`. Ключевое слово строки с условием выполнения правила нормализации: `COND`.

Примечание. Имена ключевых слов в правиле нормализации могут начинаться с восклицательного знака (!). Это устаревший вариант синтаксиса языка XP.

- Токены форматной строки правила нормализации для анализа текстовой строки события, например: `{CSV}`, `{DATETIME}`, `{DURATION}`, `{HOSTNAME}`, `{IPV4}`, `{IPV6}`, `{KEYVALUE}`, `{LITERAL}`, `{NTUSER}`, `{NUMBER}`, `{REST}`, `{STRING}`, `{WORD}`, `{WORDDASH}`.
- Директивы и инструкции правила корреляции и обогащения, например: `query`, `event`, `rule`, `emit`, `enrichment`, `enrich`.
- Названия правил корреляции и обогащения (`Critical_login_success`, `Map_protocol_number_to_name`), названия событий и запросов, объявляемые в правилах корреляции и обогащения, названия табличных списков (`Log_names`, `Known_Malwares`). Все названия должны начинаться с прописной буквы, в них можно использовать латинские буквы, цифры и символ подчеркивания.
- Префиксы, например: `column::` — для обращения к колонке табличного списка, `filter::` — для вызова макроса или `table::` — для выполнения инлайн-запроса к табличному списку.

4.2. Математические и логические операторы

В разделе описаны математические и логические операторы языка XP. При вычислении выражений операторы применяются согласно приоритету.

Таблица 2. Приоритет математических и логических операторов

Приоритет	Оператор	Описание
1	Вызов функции	—
2	<code>-</code> , <code>not</code>	Унарный минус и логическое отрицание
3	<code>*</code> , <code>div</code> , <code>mod</code>	Умножение, деление и деление по модулю
4	<code>+</code> , <code>-</code>	Сложение и вычитание
5	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Логическое сравнение
6	<code>==</code> , <code>!=</code>	Логические равенство и неравенство
7	<code>and</code>	Конъюнкция (логическое И)
8	<code>or</code>	Дизъюнкция (логическое ИЛИ)
9	<code>=</code>	Присваивание

В этом разделе

- Присваивание «=» (см. раздел 4.2.1)
- Сложение «+» (см. раздел 4.2.2)
- Вычитание «-» (см. раздел 4.2.3)
- Умножение «*» (см. раздел 4.2.4)
- Деление «div» (см. раздел 4.2.5)
- Деление по модулю «mod» (см. раздел 4.2.6)
- Равенство «==» (см. раздел 4.2.7)
- Неравенство «!=» (см. раздел 4.2.8)
- Сравнение: «<», «>» (см. раздел 4.2.9)
- Сравнение: «>=», «<=» (см. раздел 4.2.10)
- Конъюнкция: «and», «&&» (см. раздел 4.2.11)
- Дизъюнкция: «or», «||» (см. раздел 4.2.12)
- Отрицание «not» (см. раздел 4.2.13)

4.2.1. Присваивание «=»

Результатом применения оператора является присваивание переменной значения литерала, другой переменной, функции или выражения.

Формат вызова:

```
$f1 = $f2
```

Пример

```
# Необработанное событие:
# Example: 12
TEXT = 'Example: {$f2=NUMBER}'
$f1 = $f2
datafield1 = $f1
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "12",
#   "time": "2017-10-18T12:00:00Z"}
```

4.2.2. Сложение «+»

Результатом применения оператора к числовому типу данных Number является сумма операндов. Если оба операнда имеют тип данных String, результатом является конкатенация (сцепление) операндов в одну строку. Если первый операнд имеет тип данных List, результатом является список, дополненный вторым операндом.

Формат вызова:

```
$f1 + $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 + $f2
datafield2 = string($f1) + string($f2)
$f3 = [144, 72] + $f1 * $f2
datafield3 = $f3[2]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "14",
#   "datafield2": "122",
#   "datafield2": "24",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 3. Приведение типов данных оператора сложения

Первый операнд	Второй операнд	Результат
Number	Number, Null	Number
String	String, Null	String
DateTime	Number, Null	DateTime
List	Number, String, DateTime	List

4.2.3. Вычитание «-»

Результатом применения оператора к числовому типу данных Number является разность операндов. При использовании в качестве унарного оператора результатом является арифметическое отрицательное значение операнда.

Формат вызова:

```
$f1 - $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 - $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "10,
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 4. Приведение типов данных оператора вычитания

Первый операнд	Второй операнд	Результат
Number	Number, Null	Number
String	Null	String

4.2.4. Умножение «*»

Результатом применения оператора к числовому типу данных Number является произведение операндов.

Формат вызова:

```
$f1 * $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 * $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "24",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 5. Приведение типов данных оператора умножения

Первый операнд	Второй операнд	Результат
Number, Null	Number	Number
Number	Number, Null	Number

4.2.5. Деление «div»

Результатом применения оператора `div` является отношение первого операнда ко второму. Если оба операнда относятся к типу данных `Number`, результатом является целая часть от деления операндов. Деление на ноль не допускается.

Формат вызова:

```
div($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = div($f1, $f2)
datafield2 = div($f1, 7)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "6",
#   "datafield2": "1",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 6. Приведение типов данных оператора деления

Первый операнд	Второй операнд	Результат
Number	Number	Number

4.2.6. Деление по модулю «mod»

Результатом применения оператора `mod` к целым числам является остаток от целочисленного деления операндов. Если первый операнд меньше второго, то результатом является его значение. Деление на ноль не допускается.

Формат вызова:

```
mod($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 12 7 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER} {$f3=NUMBER}'
datafield1 = mod($f1, $f2)
datafield2 = mod($f1, $f3)
datafield3 = mod($f3, $f2)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
```

```
# "datafield1": "5",
# "datafield2": "0",
# "datafield3": "2",
# "time": "2017-10-18T12:00:00Z"}
```

Таблица 7. Приведение типов данных оператора деления по модулю

Первый операнд	Второй операнд	Результат
Number	Number	Number

4.2.7. Равенство «==»

Если первый операнд равен второму, результатом применения оператора равенства является значение True; в противном случае — значение False.

Формат вызова:

```
$f1 == $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 == $f2
datafield2 = $f1 == 6 * $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "false",
#   "datafield2": "true",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 8. Приведение типов данных логического оператора равенства

Оба операнда	Результат
Number, DateTime, String, IPAddress	Bool

4.2.8. Неравенство «!=»

Если операнды не равны, результатом применения оператора является значение True; в противном случае — значение False.

Формат вызова:

```
$f1 != $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 != $f2
datafield2 = $f1 != 6 * $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 9. Приведение типов данных логического оператора неравенства

Оба операнда	Результат
Number, DateTime, String, IPAddress	Bool

4.2.9. Сравнение: «<», «>»

Результатом применения оператора «меньше» (<) является значение True, если первый операнд меньше второго, в противном случае — значение False.

Результатом применения оператора «больше» (>) является значение True, если первый операнд больше второго, в противном случае — значение False.

Формат вызова:

```
$f1 < $f2
$f1 > $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 > $f2
datafield2 = $f1 < $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 10. Приведение типов данных операторов сравнения

Оба операнда	Результат
Number, DateTime	Bool

4.2.10. Сравнение: «>=», «<=»

Результатом применения оператора «меньше или равно» (<=) является значение True, если первый операнд меньше или равен второму, в противном случае — значение False.

Результатом применения оператора «больше или равно» (>=) является значение True, если первый операнд больше или равен второму, в противном случае — значение False.

Формат вызова:

```
$f1 >= $f2
$f1 <= $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 >= $f2
datafield2 = $f1 >= 6 * $f2
datafield3 = $f1 >= 7 * $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "true",
#   "datafield3": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 11. Приведение типов данных операторов сравнения

Оба операнда	Результат
Number, DateTime	Bool

4.2.11. Конъюнкция: «and», «&&»

Результатом применения логического оператора И является значение True, если оба операнда имеют значение True, в противном случае — значение False. Если один из операндов не задан (null), результатом применения оператора является значение False.

Формат вызова:

```
$f1 and $f2
```

или

```
$f1 && $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
datafield1 = $f1 > 0 and $f1 < $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 12. Приведение типов данных логического оператора И

Первый операнд	Второй операнд	Результат
Bool	Bool	Bool
Bool	Null	Bool

4.2.12. Дизъюнкция: «or», «||»

Результатом применения логического оператора ИЛИ является значение False, если оба операнда имеют значение False, в противном случае — значение True. Если один из операндов не задан, результатом применения оператора является значение второго операнда.

Формат вызова:

```
$f1 or $f2
```

или

```
$f1 || $f2
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2= NUMBER}'
datafield1 = $f1 > 0 or $f1 < $f2
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 13. Приведение типов данных логического оператора ИЛИ

Первый операнд	Второй операнд	Результат
Bool	Bool	Bool
Bool	Null	Bool

4.2.13. Отрицание «not»

Результатом применения оператора `not` является смена логического значения операнда на противоположное.

Формат вызова:

```
not $f1
```

Пример

```
# Необработанное событие:
# Example: 12 2
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
$f3 = $f1 > $f2
datafield1 = $f3
datafield2 = not $f3
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 14. Приведение типов данных логического оператора отрицания

Операнд	Результат
Bool	Bool

4.3. Конструкции языка

В разделе описаны конструкции для написания выражений в языке XP.

В этом разделе

[Символ комментария «#» \(см. раздел 4.3.1\)](#)

[Объявление строки \(см. раздел 4.3.2\)](#)

[Объединение в список «\[..., ...\]» \(см. раздел 4.3.3\)](#)

Условие «if» (см. раздел 4.3.4)

Множественный выбор «switch» (см. раздел 4.3.5)

4.3.1. Символ комментария «#»

Символ предназначен для объявления однострочного комментария. Все последующие символы в строке игнорируются при компиляции кода.

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
# Заполните поля
datafield1 = 'значениями' # из событий
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "значениями",
#   "time": "2017-10-18T12:00:00Z"}
```

4.3.2. Объявление строки

Результатом применения оператора " " (' ') является переменная с типом данных String. Аргументом оператора может быть только набор символов.

Формат вызова:

```
$string = "..."
```

или

```
$string = '...'
```

Пример

```
# Необработанное событие:
# Example: Click
TEXT = 'Example: {$f1=STRING}'
$f2 = " 10 "
$f3 = 'times'
datafield1 = $f1 + $f2 + $f3
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Click 10 times",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 15. Приведение типов данных при объявлении строки

Аргумент	Результат
—	String

4.3.3. Объединение в список «[..., ...]»

Результатом применения оператора [] является объединение аргументов в список. Отсчет элементов списка ведется от 0.

Формат вызова для формирования списка:

```
$list1 = [$f1, $f2, ...]
$list2 = [[$f11, $f12, ...], $f2, ...]
```

Формат вызова для обращения к элементу списка:

```
$f1 = $list1[0]
$f12 = $list2[0][1]
```

Пример

```
# Необработанное событие:
# Example: Click 10 times
TEXT = 'Example: {$f1=STRING} {$f2=NUMBER} {$f3=STRING}'
$list1 = [$f1, $f2, $f3]
datafield1 = $list1[0]
datafield2 = $list1[1]
datafield3 = $list1[2]
$list2 = [[$f1, $f2], $f3]
datafield4 = $list2[0][0]
datafield5 = $list2[0][1]
datafield6 = $list2[1]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Click",
#   "datafield2": "10",
#   "datafield3": "times",
#   "datafield4": "Click",
#   "datafield5": "10",
#   "datafield6": "times",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 16. Приведение типов данных при объединении в список

Аргумент	Результат
Любой	List

4.3.4. Условие «if»

Результатом применения условного оператора `if` является выполнение той или иной последовательности операторов, в зависимости от того, какое из условий возвращает значение `True`. Если первое условие возвращает значение `True`, выполняется первый блок операторов, если второе условие возвращает значение `True` — второй блок операторов, и так далее. Если указан альтернативный блок операторов, то он выполняется, если все условия возвращают значения `False`.

Формат вызова:

```
if <Условие 1> then
    <Блок операторов 1>
elif <Условие 2> then
    <Блок операторов 2>
...
else
    <Альтернативный блок операторов>
endif
```

Пример

```
# Необработанное событие:
# Example: Click 10 times
TEXT = 'Example: {STRING} {$f1=NUMBER} {STRING}'
if $f1 == 11 then
    $f2 = 10
elif $f1 == 10 then
    $f2 = 9
elif $f1 == 9 then
    $f2 = 8
else
    $f2 = 0
endif
datafield1 = string($f2) + " times left"
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "9 times left",
#   "time": "2017-10-18T12:00:00Z"}
```

4.3.5. Множественный выбор «switch»

Результатом применения оператора множественного выбора `switch` является выполнение той или иной последовательности операторов, в зависимости от значения операнда. Если операнд принимает первое значение, выполняется первый блок операторов, если операнд принимает второе значение — второй блок операторов, и так далее. Количество указанных значений операнда не ограничено.

Формат вызова:

```
switch $f1
  case <Значение 1>
    <Блок операторов 1>
  case <Значение 2>
    <Блок операторов 2>
  case <Значение 3>
    <Блок операторов 3>
  ...
endswitch
```

Пример

```
# Необработанное событие:
# Example: Click 8 times
TEXT = 'Example: {STRING} {$f1=NUMBER} {STRING}'
switch $f1
  case 10
    $f2 = 9
  case 9
    $f2 = 8
  case 8
    $f2 = 7
endswitch
datafield1 = string($f2) + ' times left'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "7 times left",
#   "time": "2017-10-18T12:00:00Z"}
```

Упрощенный формат вызова оператора

Результатом применения оператора множественного выбора является присваивание переменной того или иного значения, в зависимости от значения операнда. Если операнд принимает первое значение, переменной присваивается одно значение, если операнд принимает второе значение — другое, и так далее. Количество указанных значений операнда не ограничено.

Формат вызова:

```
$f2 = switch $f1
  case <Значение $f1 1> <Значение $f2 1>
  case <Значение $f1 2> <Значение $f2 2>
  case <Значение $f1 3> <Значение $f2 3>
  ...
endswitch
```

Пример

```
# Необработанное событие:
# Example: Click 8 times
TEXT = 'Example: {STRING} {$f1=NUMBER} {STRING}'
$f2 = switch $f1
    case 10 9
    case 9 8
    case 8 7
endswitch
datafield1 = string($f2) + ' times left'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "7 times left",
#   "time": "2017-10-18T12:00:00Z"}
```

4.4. Функции работы с временем

В разделе описаны функции языка XР для преобразования и анализа данных, содержащих время и дату.

В этом разделе

[datetime](#) (см. раздел 4.4.1)

[datetime_to_epoch](#), [datetime_to_epoch_ms](#) (см. раздел 4.4.2)

[datetime_to_win_ticks](#) (см. раздел 4.4.3)

[duration](#) (см. раздел 4.4.4)

[epoch_to_datetime](#), [epoch_ms_to_datetime](#) (см. раздел 4.4.5)

[hour](#), [minute](#), [second](#), [timezone](#) (см. раздел 4.4.6)

[win_ticks_to_datetime](#) (см. раздел 4.4.7)

[year](#), [month](#), [day](#) (см. раздел 4.4.8)

4.4.1. datetime

Функция `datetime` используется для преобразования значения аргумента с типом данных `String` в тип данных `DateTime`.

Формат вызова:

```
datetime($f1)
```

Пример

```
# Необработанное событие:
# Example: 1990-10-31T23:13:04+03:00
TEXT = 'Example: {$f1=STRING}'
$f2 = datetime($f1)
datafield1 = $f2
datafield2 = year($f2)
datafield3 = month($f2)
datafield4 = day($f2)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "1990-10-31T23:13:04+03:00",
#   "datafield2": "1990",
#   "datafield3": "10",
#   "datafield4": "31",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 17. Приведение типов данных функции datetime

Аргумент	Результат
String	DateTime
DateTime	DateTime

4.4.2. datetime_to_epoch, datetime_to_epoch_ms

Функции `datetime_to_epoch` и `datetime_to_epoch_ms` используются для преобразования формата времени. Аргументом функций является время с типом данных `DateTime`. Функции возвращают в Unix-время, функция `datetime_to_epoch` — в секундах, функция `datetime_to_epoch_ms` — в миллисекундах. Отсчет Unix-времени начинается с 00:00:00 UTC 1 января 1970 года.

Формат вызова:

```
datetime_to_epoch($f1)
datetime_to_epoch_ms($f1)
```

Пример

```
# Необработанное событие:
# Example: 2015-01-25T01:06:40
TEXT = 'Example: {$f1=DATETIME}'
datafield1 = $f1
datafield2 = datetime_to_epoch($f1)
datafield3 = datetime_to_epoch_ms($f1)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
```

```
# "datafield1": "2015-01-25T01:06:40Z",
# "datafield2": "1422148000",
# "datafield3": "1422148000000",
# "time": "2017-10-18T12:00:00Z"}
```

Таблица 18. Приведение типов данных функций datetime_to_epoch, datetime_to_epoch_ms

Аргумент	Результат
DateTime	Number

4.4.3. datetime_to_win_ticks

Функция `datetime_to_win_ticks` используется для преобразования формата времени. Аргументом функции является время с типом данных `DateTime`. Функция возвращает время в формате `filetime` с типом данных `Number`. Время `filetime` указывается в сотнях наносекунд (10^{-7} секунды) начиная с 00:00:00 UTC 1 января 1601 года.

Формат вызова:

```
datetime_to_win_ticks($f1)
```

Пример

```
# Необработанное событие:
# Example: 2015-01-25T01:06:40
TEXT = 'Example: {$f1=DATETIME}'
datafield1 = $f1
datafield2 = datetime_to_win_ticks($f1)
datafield3 = div(datetime_to_win_ticks($f1), 10000000)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
# "datafield1": "2015-01-25T01:06:40Z",
# "datafield2": "1306662160000000000",
# "datafield3": "13066621600",
# "time": "2017-10-18T12:00:00Z"}
```

Таблица 19. Приведение типов данных функций datetime_to_win_ticks

Аргумент	Результат
DateTime	Number

4.4.4. duration

Функция `duration` используется для преобразования значения аргумента с типом данных `String`, содержащего интервал времени в формате `XXXd XXh:XXm:XXs`, в число секунд. Если преобразование не удалось, функция возвращает `null`.

Формат вызова:

```
duration($f1)
```

Пример

```
# Необработанное событие:
# Example: 0d 1h:2m:15s
TEXT = 'Example: {datafield1=REST}'
$f = duration(datafield1)
datafield2 = $f
datafield3 = div($f,60*60)
datafield4 = div(mod($f,3600),60)
datafield5 = mod(mod($f,3600),60)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "0d 1h:2m:15s",
#   "datafield2": "3735",
#   "datafield3": "1",
#   "datafield4": "2",
#   "datafield5": "15",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 20. Приведение типов данных функции duration

Аргумент	Результат
String	Number
Number	Number

4.4.5. epoch_to_datetime, epoch_ms_to_datetime

Функции `epoch_to_datetime` и `epoch_ms_to_datetime` используются для преобразования формата времени. Аргументом функций является Unix-время. Нужно указать промежуток времени начиная с 00:00:00 UTC 1 января 1970 года, для функции `epoch_to_datetime` — в секундах, для функции `epoch_ms_to_datetime` — в миллисекундах. Функции возвращают значение времени с типом данных `DateTime`. При некорректном аргументе функции возвращают `null`.

Формат вызова:

```
epoch_to_datetime($f1)
epoch_ms_to_datetime($f1)
```

Пример

```
# Необработанное событие:
# Example: 1422148000
```

```

TEXT = 'Example: {$f1=NUMBER}'
datafield1 = epoch_to_datetime($f1)
datafield2 = epoch_ms_to_datetime($f1*1000)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "2015-01-25T01:06:40Z",
#   "datafield2": "2015-01-25T01:06:40Z",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 21. Приведение типов данных функций epoch_to_datetime, epoch_ms_to_datetime

Аргумент	Результат
Number	DateTime

4.4.6. hour, minute, second, timezone

Функции `hour`, `minute`, `second` и `timezone` используются для получения из даты значения часов, минут, секунд и отклонения от UTC. В аргументе функций нужно указать дату с типом данных `DateTime`. Функции возвращают значения в секундах.

Формат вызова:

```

hour($f1)
minute($f1)
second($f1)
timezone($f1)

```

Пример

```

# Необработанное событие:
# Example: 1990-10-31T23:13:04+03:00
TEXT = 'Example: {$f1=DATETIME}'
datafield1 = hour($f1)
datafield2 = minute($f1)
datafield3 = second($f1)
datafield4 = timezone($f1)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "20",
#   "datafield2": "13",
#   "datafield3": "4",
#   "datafield4": "10800",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 22. Приведение типов данных функций hour, minute, second, timezone

Аргумент	Результат
DateTime	Number

4.4.7. win_ticks_to_datetime

Функция `win_ticks_to_datetime` используется для преобразования формата времени. Аргументом функций является время в формате `filetime`. Время `filetime` указывается в сотнях наносекунд (10^{-7} секунды) начиная с 00:00:00 UTC 1 января 1601 года. Время ранее 1 января 1970 года считается невалидным. Функции возвращают время с типом данных `DateTime`.

Формат вызова:

```
win_ticks_to_datetime($f1)
```

Пример

```
# Необработанное событие:
# Example: 130666216000000000
TEXT = 'Example: {$f1=NUMBER}'
datafield1 = win_ticks_to_datetime($f1)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "2015-01-25T01:06:40Z"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 23. Приведение типов данных функций win_ticks_to_datetime

Аргумент	Результат
Number	DateTime

4.4.8. year, month, day

Функции `year`, `month` и `day` используются для получения из даты значения года, месяца и числа (в часовом поясе UTC). В аргументе функций нужно указать дату с типом данных `DateTime`.

Формат вызова:

```
year($f1)
month($f1)
day($f1)
```

Пример

```
# Необработанное событие:
# Example: 1990-10-31T02:13:04+03:00
```



```

TEXT = 'Example: {$f=DATETIME}'
datafield1 = year($f)
datafield2 = month($f)
datafield3 = day($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "1990",
#   "datafield2": "10",
#   "datafield3": "30",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 24. Приведение типов данных функций year, month, day

Аргумент	Результат
DateTime	Number

4.5. Функции преобразования типов данных

В разделе описаны функции языка XР для преобразования типов данных.

В этом разделе

[bool](#) (см. раздел 4.5.1)

[buffer_from_base64](#) (см. раздел 4.5.2)

[decode](#) (см. раздел 4.5.3)

[hex_to_string](#) (см. раздел 4.5.4)

[ipv4](#) (см. раздел 4.5.5)

[ipv6](#) (см. раздел 4.5.6)

[join](#) (см. раздел 4.5.7)

[macaddr](#) (см. раздел 4.5.8)

[number](#) (см. раздел 4.5.9)

[number8](#) (см. раздел 4.5.10)

[number16](#) (см. раздел 4.5.11)

[string](#) (см. раздел 4.5.12)

4.5.1. bool

Функция `bool` используется для преобразования значения аргумента в логическое значение `True` или `False`.

Формат вызова:

```
bool($f1)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
datafield1 = bool(tRuE)
datafield2 = bool(12)
datafield3 = bool('Hello')
datafield4 = bool(0)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "true",
#   "datafield3": "true",
#   "datafield4": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 25. Приведение типов данных функции bool

Тип данных аргумента	Значение аргумента	Результат
Bool	True	True
Bool	False	False
String	Любое, кроме: 0, false, False, пустой строки ("")	True
String	0, false, False, пустая строка	False
Number	1, любое число	True
Number	0	False
Null	null	False

4.5.2. buffer_from_base64

Функция `buffer_from_base64` используется для преобразования строки (данные типа `String`) в массив байтов типа `Buffer`. Данные типа `Buffer` не отображаются в полях нормализованного сообщения, но могут быть преобразованы в тип `String` с помощью функции [decode](#) (см. [раздел 4.5.3](#)).

Формат вызова:

```
buffer_from_base64($f)
```

Таблица 26. Приведение типов данных функции `buffer_from_base64`

Аргумент	Результат
String	Buffer

4.5.3. decode

Функция `decode` используется для преобразования данных типа `Buffer` в строку (данные типа `String`). В первом аргументе необходимо указать преобразуемые данные типа `Buffer`. Во втором аргументе необходимо указать кодировку получаемой строки — UTF-8 или UCS2-LE.

Формат вызова:

```
decode($f, "UTF-8")
```

Пример

```
# Необработанное событие:
# Example: aGVsbG8gd29ybGQ=
TEXT = 'Example: {$f=BASE64}'
time = "2018-04-27T10:52:47Z"
datafield1 = decode(buffer_from_base64($f), "UTF-8")
# Нормализованное событие: {
#   "time": "2018-04-27T10:52:47Z",
#   "datafield1": "hello world"}
```

Таблица 27. Приведение типов данных функции `decode`

Аргумент	Результат
Buffer	String

4.5.4. hex_to_string

Функция `hex_to_string` используется для преобразования строки (данные типа `String`), содержащей число в шестнадцатеричном виде в кодировке UTF-8, в обычную строку.

Формат вызова:

```
hex_to_string ($f1)
```

Пример

```
# Необработанное событие:
# Example: hex=6465636f6465642074657874
TEXT = 'Example: hex={$f1=WORD}'
datafield1 = hex_to_string($f1)
time = "2017-10-18T12:00:00Z"
```

```
# Нормализованное событие: {
#   "datafield1": "decoded text",
#   "time": "2017-10-18T12:00:00.000Z"
#}
```

Таблица 28. Приведение типов данных функции hex_to_string

Аргумент	Результат
String	String
Bool, Number, Datetime, List, Map	Null

4.5.5. ipv4

Функция `ipv4` используется для преобразования значения аргумента, содержащего IP-адрес стандарта IPv4 с типом данных String, в IP-адрес с типом данных IPAddress. Если IP-адрес распознать не удалось, функция возвращает null.

Формат вызова:

```
ipv4($f)
```

Пример

```
# Необработанное событие:
# Example: 192.168.0.1
TEXT = 'Example: {$f=STRING}'
if ipv4($f) != null then
    src.host = ipv4($f)
elif ipv6($f) != null then
    src.host = ipv6($f)
endif
time = "2021-10-18T12:00:00Z"
# Нормализованное событие: {
#   "src.host": "192.168.0.1"
#   "time": "2021-10-18T12:00:00Z"}
```

Таблица 29. Приведение типов данных функции ipv4

Аргумент	Результат
String, IPAddress	IPAddress

4.5.6. ipv6

Функция `ipv6` используется для преобразования значения аргумента, содержащего IP-адрес стандарта IPv6 с типом данных String, в IP-адрес с типом данных IPAddress. Если IP-адрес распознать не удалось, функция возвращает null.

В аргументе функции поддерживаются IP-адреса в форматах X:X:X:X:X:X, X:X:X:X:X:D.D.D.D и X:X:X:X:X:X:X%S, где X — шестнадцатеричные значения для 16-битовых частей адреса, D — десятичные значения для 8-битовых частей адреса (D.D.D.D — адрес стандарта IPv4), S — четырехзначное десятичное число. Для замены одного или нескольких нулей в расположенных рядом 16-битовых частях адреса допускается использование двух двоеточий (::).

Формат вызова:

```
ipv6($f)
```

Пример

```
# Необработанное событие:
# Example: 211:DB8:0:0:8:800:200C:47A 211:DB8:0:0:8:800:192.168.0.1
211:DB8:0:0:8:800:200C:47A%5141
TEXT = 'Example: {$f1=STRING} {$f2=STRING} {$f3=STRING}'
event_src.host = ipv6($f1)
src.host = ipv6($f2)
dst.host = ipv6($f3)
time = "2021-10-18T12:00:00Z"
# Нормализованное событие: {
#   "event_src.host": "211:DB8:0:0:8:800:200C:47A",
#   "src.host": "211:DB8:0:0:8:800:192.168.0.1",
#   "dst.host": "211:DB8:0:0:8:800:200C:47A",
#   "time": "2021-10-18T12:00:00Z"}
```

Таблица 30. Приведение типов данных функции ipv6

Аргумент	Результат
String, IPAddress	IPAddress

4.5.7. join

Функция `join` используется для объединения в строку элементов списка, указанного в первом аргументе. Во втором аргументе можно указать строку или число, которое будут добавлено в строку в качестве разделителя между элементами списка.

Формат вызова:

```
join($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["AC", "DC"]
datafield1 = join($f1)
```

```

datafield2 = join($f1, "|")
datafield3 = $f1[0] + "|" + $f1[1]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "ACDC",
#   "datafield2": "AC|DC",
#   "datafield3": "AC|DC",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 31. Приведение типов данных функции join

Первый аргумент	Второй аргумент	Результат
List	String, Number, Null	String
Null	String, Number	String

4.5.8. macaddr

Внимание! Функция может использоваться только при создании правил нормализации.

Функция `macaddr` используется для преобразования значения аргумента, содержащего MAC-адрес с типом данных String, в тип данных MACAddress. MAC-адрес может вводиться в следующих форматах: `aa:bb:cc:dd:ee:ff`, `aa-bb-cc-dd-ee-ff`, `aabb.ccdd.eeff`.

Формат вызова:

```
macaddr($f1)
```

Пример

```

# Необработанное событие:
# Example: a1-f6-33-4d-21-56
TEXT = 'Example: {$f=STRING?}'
datafield1 = macaddr($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "A1:F6:33:4D:21:56",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 32. Приведение типов данных функции macaddr

Аргумент	Результат
String, MACAddress	MACAddress

4.5.9. number

Функция `number` используется для преобразования значение аргумента, содержащего число в десятичной системе счисления с типом данных `String`, в число с типом данных `Number`. Преобразуются все цифры, расположенные в строке слева от первого нечислового символа. Если число распознать не удалось, функция возвращает `null`.

Формат вызова:

```
number($f1)
```

Пример

```
# Необработанное событие:
# Example: -245
TEXT = 'Example: {$f=REST}'
datafield1 = number($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "-245"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 33. Приведение типов данных функции `number`

Аргумент	Результат
String, Number	Number

4.5.10. number8

Функция `number8` используется для преобразования значения аргумента, содержащего число в восьмеричной системе счисления с типом данных `String`, в число с типом данных `Number`. Преобразуются все цифры от 0 до 7, расположенные в строке слева от первого символа, не относящегося к указанным. Если число распознать не удалось, функция возвращает `null`.

Формат вызова:

```
number8($f1)
```

Пример

```
# Необработанное событие:
# Example: -245
TEXT = 'Example: {$f=REST}'
datafield1 = number8($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "-165",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 34. Приведение типов данных функции number8

Аргумент	Результат
String, Number	Number

4.5.11. number16

Функция `number16` используется для преобразования значения аргумента, содержащего число в шестнадцатеричной системе счисления с типом данных `String`, в число с типом данных `Number`. В число преобразуется набор цифр от 0 до 9 и символов от a до f, расположенных в строке слева от первого символа, не относящегося к указанным. Если число распознать не удалось, функция возвращает `null`.

Формат вызова:

```
number16($f1)
```

Пример

```
# Необработанное событие:
# Example: -245e
TEXT = 'Example: {$f=REST}'
datafield1 = number16($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "-9310",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 35. Приведение типов данных функции number16

Аргумент	Результат
String, Number	Number

4.5.12. string

Функция `string` используется для преобразования значение аргумента в тип данных `String`.

Формат вызова:

```
string($f1)
```

Пример

```
# Необработанное событие:
# Example: 192.168.0.1
TEXT = 'Example: {$f1=IPV4}'
datafield1 = string($f1) + " – IP-адрес"
```



```
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "192.168.0.1 — IP-адрес"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 36. Приведение типов данных функции string

Аргумент	Результат
Bool, Number, String, DateTime	String

4.6. Функции для работы со строками

В разделе описаны функции языка XP для преобразования данных с типом String.

В этом разделе

[csv](#) (см. раздел 4.6.1)

[find_substr](#) (см. раздел 4.6.2)

[keyvalue](#) (см. раздел 4.6.3)

[length](#) (см. раздел 4.6.4)

[lower](#) (см. раздел 4.6.5)

[match](#) (см. раздел 4.6.6)

[regex](#) (см. раздел 4.6.7)

[replace](#) (см. раздел 4.6.8)

[rot13](#) (см. раздел 4.6.9)

[strip](#) (см. раздел 4.6.10)

[strip_all](#) (см. раздел 4.6.11)

[substr](#) (см. раздел 4.6.12)

[upper](#) (см. раздел 4.6.13)

4.6.1. csv

Внимание! Функция может использоваться только при создании правил нормализации.

Функция `csv` используется для разбора строки с CSV-разметкой на элементы. В первом аргументе нужно указать строку для разбора. Во втором аргументе — символ, разделяющий элементы строки, в третьем аргументе — символ экранирования, стоящий перед каждым элементом строки и после него. Если элементы строки не экранированы, в третьем аргументе нужно указать любой символ, не встречающийся в строке.

Функция возвращает список элементов строки. Если первым аргументом функции является список, то значения остальных аргументов при разборе игнорируются, функция возвращает этот же список.

Формат вызова:

```
csv($f1, $f2, $f3)
```

Пример

```
# Необработанное событие:
# Example: /login=admin/, /password=supercleveradmin/
TEXT = 'Example: {$f1=REST}'
$f2 = ','
$f3 = '/'
datafield1 = csv($f1, $f2, $f3)[0]
datafield2 = csv($f1, $f2, $f3)[1]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "login=admin",
#   "datafield2": "password=supercleveradmin",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 37. Приведение типов данных функции csv

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	String	String	List
List	String	String	List

4.6.2. find_substr

Функция `find_substr` используется для поиска сочетания символов в строке. В первом аргументе указывается строка для поиска, во втором — искомое сочетание символов. Функция возвращает номер символа в строке, с которого начинается искомое сочетание. Отсчет символов начинается от нуля. Если строка не содержит указанного сочетания символов, функция возвращает `null`.

Формат вызова:

```
find_substr($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 7896453223, abcdefghijk
TEXT = 'Example: {$f1=WORD}, {$f2=STRING}'
datafield1 = find_substr($f1, "64")
datafield2 = find_substr($f2, "de")
```

```
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "3",
#   "datafield2": "3",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 38. Приведение типов данных функции find_substr

Первый аргумент	Второй аргумент	Результат
String	String	Number, Null

4.6.3. keyvalue

Внимание! Функция может использоваться только при создании правил нормализации.

Функция `keyvalue` используется для анализа строки и создания ассоциативного массива пар «ключ — значение».

Функция имеет четыре аргумента: в первом нужно указать строку для анализа, во втором — внешний разделитель между парами, в третьем — внутренний разделитель в паре, в четвертом — символ экранирования значения, стоящий перед каждым значением в паре и после него. Функция возвращает ассоциативный массив, каждый элемент которого содержит пару «ключ — значение».

Примечание. Для ввода одинарных кавычек используйте обратную косую черту (`\`), для ввода двойных кавычек — две обратные косые черты (`\\`).

Формат вызова:

```
$result_list = keyvalue($f1, "<Разделитель между парами>", "<Разделитель в паре>",
"<Экранирование значения>")
```

Формат обращения к элементам списка:

```
<Значение> = $result_list["<Ключ>"]
```

Пример

```
# Необработанное событие:
# Example: login='admin'; password='supercleveradmin'
TEXT = 'Example: {$f=REST}'
$result_list = keyvalue($f, ";", "=", "\\")
datafield1 = $result_list["login"]
datafield2 = $result_list["password"]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "admin",
#   "datafield2": "supercleveradmin",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 39. Приведение типов данных функции keyvalue

Первый аргумент	Второй аргумент	Третий аргумент	Четвертый аргумент	Результат
String	String	String	String	KeyValue

4.6.4. length

Функция `length` для аргумента, имеющего тип данных String, возвращает размер строки в символах, для аргумента, имеющего тип данных List — количество элементов в списке.

Формат вызова:

```
length($f1)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1_string = "12 Angry Men"
$f1_list = [12, " Angry", " Men"]
$f2_string = "2128506"
$f2_list = [2, 12, "85", 06]
datafield1 = length($f1_string)
datafield2 = length($f1_list)
datafield3 = length($f2_string)
datafield4 = length($f2_list)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "12",
#   "datafield2": "3",
#   "datafield3": "7",
#   "datafield4": "4",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 40. Приведение типов данных функции length

Аргумент	Результат
List, String	Number
Null	Null

4.6.5. lower

Функция `lower` используется для преобразования в строке, указанной в аргументе, всех прописных буквы в строчные.

Формат вызова:

```
lower($f1)
```

Пример

```
# Необработанное событие:
# Example: Online_SERVICE
TEXT = 'Example: {$f=STRING}'
datafield1 = lower($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "online_service",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 41. Приведение типов данных функции lower

Аргумент	Результат
String	String

4.6.6. match

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функция `match` используется для сравнения строки с заданным шаблоном. В первом аргументе указывается строка для поиска. Во втором аргументе указывается шаблон для поиска. В шаблоне можно использовать замещающие символы: `?` — один символ, `*` — несколько символов. Функция возвращает `True`, если строка соответствует шаблону, в противном случае — `False`. Если какой-либо из аргументов не задан, функция возвращает `null`.

Формат вызова:

```
match($f1, $f2)
```

Пример

```
# Нормализованное событие: {
#   "time": "2017-10-18T12:00:00Z",
#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
```

```

event Event:
    key:
        event_src.title
    filter {
        event_src.title == "test"
    }
rule Example: Event
    init {
        $f1 = "rh24!keyk1l;k5v7ke6!"
        $f2 = "*k?y*"
    }
    on Event{
        $datafield1 = match($f1, $f2)
    }
emit {
    $correlation_type = "event"
    $object = "client"
    $action = "check"
    $status = "success"
    $importance = "info"
}
# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"true",
#   ...}

```

Таблица 42. Приведение типов данных функции match

Первый аргумент	Второй аргумент	Результат
String	String	Bool
String	Null	Null
Null	String	Null

4.6.7. regex

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функция `regex` используется для поиска в строке по заданному шаблону. В первом аргументе функции указывается строка для поиска. Во втором аргументе — шаблон для поиска. В шаблоне нужно использовать синтаксис регулярных выражений библиотеки RE2. Описание

библиотеки см. на сайте github.com в разделе компании Google. В третьем аргументе указывается номер группы символов шаблона: 0 — возвращается результат для всего шаблона; 1 — возвращается результат для первой группы символов; 2 — возвращается результат для второй группы символов, и так далее. Для объединения символов шаблона в группу используются круглые скобки.

Функция возвращает строку, содержащую первую найденную последовательность символов, удовлетворяющую шаблону. Если указан отличный от нуля номер группы символов шаблона, функция возвращает часть строки, соответствующую этой группе в шаблоне. Если какой-либо из аргументов не задан или удовлетворяющая шаблону последовательность символов не найдена, функция возвращает null.

Формат вызова:

```
regex($f1, $f2, $f3)
```

Пример

```
# Нормализованное событие: {
#   "time": "2017-10-18T12:00:00Z",
#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  init {
    $f1 = "Outlook 14.0; Windows 8.1; Office 14.0; Windows Vista"
    $f2 = "Windows ([0-9\\.]+)"
  }
  on Event{
    $datafield1 = regex($f1, $f2, 0)
    $datafield2 = regex($f1, $f2, 1)
  }
  emit {
    $correlation_type = "event"
    $object = "client"
    $action = "check"
```

```

    $status = "success"
    $importance = "info"
}
# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"Windows 8.1",
#   "datafield2":"8.1",
#   ...}

```

Таблица 43. Приведение типов данных функции regex

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	String	Number	String

regex для получения словаря с именованными группами

Вы можете использовать функцию `regex`, чтобы получить в результате поиска словарь с именованными группами. В этом случае в первом аргументе нужно указать строку для поиска, во втором — шаблон для поиска с именованием групп. В шаблоне нужно использовать синтаксис регулярных выражений библиотеки RE2. Описание библиотеки см. на сайте github.com в разделе компании Google.

Если совпадение найдено, функция возвращает словарь со значениями именованных групп, указанных в шаблоне регулярного выражения. Если совпадение не найдено, функция возвращает `null`.

Формат вызова:

```
regex($f1, $f2)
```

Пример

```

# Нормализованное событие
# {"time": "2017-10-18T12:00:00Z", "id": "developer_guide", "importance": "info",
#  "action": "start", "object": "task", "status": "success", "event_src.vendor": "Best
#  Company", "event_src.title": "test", "event_src.category": "Other", "uuid":
#  "00000005-9f0a-0e90-f000-0000abbac077"}
# expect 1 {"correlation_name": "Example"}
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }

```



```

rule Example: Event
  init {
    $f1 = "Outlook 14.0; Windows 8.1; Office 14.0; Windows Vista"
    $f2 = "Windows (?P<windowsVersion>[0-9\\.]+); Office (?P<officeVersion>[0-9\\.]+)"
  }
  on Event{
    $result = regex($f1, $f2)
    $datafield1 = "Windows " + $result["windowsVersion"]
    $datafield2 = "Office " + $result["officeVersion"]
  }
  emit {
    $correlation_type = "event"
    $object = "client"
    $action = "check"
    $status = "success"
    $importance = "info"
  }
  # Корреляционное событие:
  # {"correlation_type": "event",
  # "count": 1,
  # "datafield1": "Windows 8.1",
  # "importance": "info",
  # "subevents.time": ["2017-10-18T12:00:00.000Z"],
  # "subevents": ["00000005-9f0a-0e90-f000-0000abbac077"],
  # "_rule": "Example",
  # "time": "2017-10-18T12:00:00.000Z",
  # "datafield2": "Office 14.0",
  # ... }

```

Таблица 44. Приведение типов данных функции regex для получения словаря именованных групп

Первый аргумент	Второй аргумент	Результат
String	String	KeyValue, Null

4.6.8. replace

Функция `replace` используется для замены в строке сочетания символов, удовлетворяющих шаблону, другим сочетанием символов.

Функция имеет четыре аргумента: в первом нужно указать строку для анализа; во втором — сочетание символов, которое будет подставлено в строку; в третьем — шаблон для поиска сочетания символов, которое будет удалено из строки (в шаблоне можно использовать регулярные выражения, указанные в таблице ниже); в четвертом можно указать максимальное количество вхождений, которое может быть заменено (отсчет выполняется слева направо; если значение аргумента не указано или указан ноль, количество вхождений не ограничено).

Примечание. В шаблоне для экранирования символов \, *, ?, [, ! нужно использовать \.

Функция возвращает копию строки, указанной в первом аргументе, в которой сочетания символов, удовлетворяющие шаблону, указанному в третьем аргументе, заменены на сочетание символов, указанное во втором аргументе.

Таблица 45. Регулярные выражения для составления шаблона

Выражение	Назначение
?	Замена одного любого символа. Например, для регулярного выражения <Символ 1>?<Символ 2> в строке будут заменены три символа, два указанных и один любой символ между ними
*	Замена любого количества любых символов подряд (или ни одного символа). Например, для регулярного выражения <Символ>* в строке будут заменены указанный символы и все последующие; для выражения *<Символ> — указанный символ и все перед ним; для выражения <Символ 1>*<Символ 2> — указанные символы и любое количество любых символов между ними
[abcd]	Замена любого из перечисленных символов
[a-d]	Замена любого символа из диапазона
[0-4]	
[!abcd]	Замена любого символа, кроме перечисленных
[!a-d]	Замена любого символа, кроме символов из диапазона
[!0-4]	

Формат вызова:

```
replace($f1, $f2, $f3, $f5)
```

Пример

```
# Необработанное событие:
# Example: a()bb(c)ddd(ee)ffff(gggg)
TEXT = 'Example: {$f1=STRING}'
datafield1 = replace($f1, ".", "?")
datafield2 = replace($f1, ".", "(?)")
datafield3 = replace($f1, ".", "(?)")
datafield4 = replace($f1, ".", "(*)")
datafield5 = replace($f1, ".", "(*?)")
datafield6 = replace($f1, ".", "(*)")
datafield7 = replace($f1, ".", "(*)")
datafield8 = replace($f1, ".", "(*?)")
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
```

```
# "datafield1": ".....",
# "datafield2": "a()bb.ddd(ee)ffff(gggg)",
# "datafield3": "a.bb.)ddd.e)ffff.ggg)",
# "datafield4": "a.bb.ddd.ffff.",
# "datafield5": "a.ddd.ffff.",
# "datafield6": "....",
# "datafield7": "a.",
# "datafield8": "a.bb.)ddd.e)ffff.ggg)",
# "time": "2017-10-18T12:00:00Z"
# "time": "2017-10-18T12:00:00Z"}
```

Таблица 46. Приведение типов данных функции replace

Первый аргумент	Второй аргумент	Третий аргумент	Четвертый аргумент	Результат
String	String	String	Number	String

4.6.9. rot13

Функция `rot13` используется для смещения символов латинского алфавита в строке (данные типа `String`) на 13 позиций. Другие символы (пробелы, специальные символы, кириллица) не попадают под действие функции и не меняются.

Формат вызова:

```
rot13($f)
```

Пример

```
# Необработанное событие:
# Example: ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
TEXT = 'Example: {$f=STRING}'
datafield1 = rot13($f)
time = "2018-04-27T10:52:47Z"
# Нормализованное событие: {
# "datafield1": "NOPQRSTUVWXYZABCDEFGHIJKLMnopqrstuvwxyzabcdefghijklm",
# "time": "2018-04-27T10:52:47Z"}
```

Таблица 47. Приведение типов данных функции rot13

Аргумент	Результат
String	String

4.6.10. strip

Функция `strip` используется для удаления из значения первого аргумента, с типом данных `String`, префикса, указанного во втором аргументе, и (или) постфикса, указанного в третьем аргументе.

Формат вызова:

```
strip($f1, $f2, $f3)
```

Пример

```
# Необработанное событие:
# Example: <<<<Текст примера>>>>
TEXT = 'Example: {$f1=REST?}'
$f2 = "<<<<"
$f3 = ">>>>"
datafield1 = strip($f1, $f2, $f3)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Текст примера",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 48. Приведение типов данных функции `strip`

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	String	String	String
String	String	Null	String
String	Null	String	String
Null	String, Null	String, Null	Null

4.6.11. strip_all

Функция `strip_all` используется для удаления из строки всех повторений префикса и постфикса. Исходная строка указывается в первом аргументе, префикс — во втором, постфикс — в третьем.

Вы можете указать в аргументах функции только префикс или только постфикс.

Формат вызова:

```
strip_all($f1, $f2, $f3)
```

Пример

```
# Необработанное событие:
# Example
```

```

TEXT = 'Example'
$f1 = "<<<<Текст примера..."
$f4 = strip_all($f1, "<", ".")
datafield1 = $f4
time = "2023-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Текст примера",
  "time": "2023-10-18T12:00:00.000Z"
}

```

Таблица 49. Приведение типов данных функции strip_all

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	String, Null	String, Null	String

4.6.12. substr

Функция `substr` возвращает часть строки, указанной в первом аргументе, начиная с символа, указанного во втором аргументе, и длиной в число символов, указанное в третьем аргументе. Если второй аргумент отрицательный, отсчет символов выполняется с конца строки.

Формат вызова:

```
substr($f1, $f2, $f3)
```

Пример

```

# Необработанное событие:
# Example: 7896453223, abcdefghijk
TEXT = 'Example: {$f1=WORD}, {$f2=STRING}'
$f3 = 4
$f4 = 6
datafield1 = substr($f1, $f3, $f4)
datafield2 = substr($f1, -$f3, $f4)
datafield3 = substr($f2, $f3, $f4)
datafield4 = substr($f2, -$f3, $f4)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "453223",
#   "datafield2": "3223",
#   "datafield3": "efghij",
#   "datafield4": "hijk",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 50. Приведение типов данных функции substr

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	Number	Number	String

4.6.13. upper

Функция `upper` используется для преобразования в строке, указанной в аргументе, всех строчных букв в прописные.

Формат вызова:

```
upper($f1)
```

Пример

```
# Необработанное событие:
# Example: Online_service
TEXT = 'Example: {$f1=STRING}'
datafield1 = upper($f1)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "ONLINE_SERVICE",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 51. Приведение типов данных функции upper

Аргумент	Результат
String	String

4.7. Функции для работы со списками

В разделе описаны функции языка XP для работы со списками и преобразования данных с типом List.

В этом разделе

[append](#) (см. раздел 4.7.1)

[extend](#) (см. раздел 4.7.2)

[in_list](#) (см. раздел 4.7.3)

[intersect](#) (см. раздел 4.7.4)

[is_list](#) (см. раздел 4.7.5)

[remove](#) (см. раздел 4.7.6)

[slice \(см. раздел 4.7.7\)](#)

[sort \(см. раздел 4.7.8\)](#)

[split \(см. раздел 4.7.9\)](#)

[unique \(см. раздел 4.7.10\)](#)

4.7.1. append

Функция `append` используется для добавления в конец списка, указанного в первом аргументе, элемента, указанного во втором аргументе. Если первый аргумент функции не определен, функция возвращает список из одного элемента.

Формат вызова:

```
append($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: rows
TEXT = 'Example: {$f2=REST?}'
$f1 = ["Delete the last ", 10]
$f3 = append($f1, $f2)
datafield1 = $f3[0] + string($f3[1]) + " " + $f3[2]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Delete the last 10 rows",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 52. Приведение типов данных функции `append`

Первый аргумент	Второй аргумент	Результат
List, Null	Любой	List

4.7.2. extend

Функция `extend` используется для объединения списков. Она возвращает новый список, в котором в конец первого списка добавлены элементы второго с сохранением дубликатов.

Формат вызова:

```
extend($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example
```

```

TEXT = 'Example'
$f1 = ["Red", "Green", "Orange"]
$f2 = ["Blue", "Red", "Black"]
$f3 = extend($f1, $f2)
# $f3 == ["Red", "Green", "Orange", "Blue", "Red", "Black"]
datafield1 = join($f3, ", ")
time = "2023-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Red, Green, Orange, Blue, Red, Black",
  "time": "2023-10-18T12:00:00.000Z"
}

```

Таблица 53. Приведение типов данных функции extend

Первый аргумент	Второй аргумент	Результат
List	List	List

4.7.3. in_list

Функция `in_list` используется для поиска в списке. В первом аргументе функции указывается список с типом данных List, во втором аргументе — искомое значение. Если значение найдено в списке, функция возвращает True, в ином случае — False. При поиске учитывается тип данных аргументов.

Формат вызова:

```
in_list($f1, $f2)
```

Пример

```

# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange"]
datafield1 = in_list($f1, "Green")
datafield2 = in_list($f1, "Blue")
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield1": "false",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 54. Приведение типов данных функции in_list

Первый аргумент	Второй аргумент	Результат
List	Любой	Bool

4.7.4. intersect

Функция `intersect` используется для поиска элементов, которые есть в обоих списках, указанных в качестве аргументов. Функция возвращает список, состоящий из элементов, общих для обоих списков. При этом строки будут сортированы по алфавиту, числа — по возрастанию.

Вы можете использовать `intersect` только для списков, состоящих из элементов одного типа: строк или чисел. Если список-аргумент состоит из элементов разного типа или содержит элементы неподдерживаемых типов, функция возвращает `null`.

Формат вызова:

```
intersect($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange", "Black"]
$f2 = ["Blue", "Orange", "Yellow", "Red"]
$f3 = intersect($f1, $f2)
datafield1 = join($f3, ", ")
time = "2024-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Orange, Red",
  "time": "2024-10-18T12:00:00.000Z"
}
```

Таблица 55. Приведение типов данных функции `intersect`

Первый аргумент	Второй аргумент	Результат
List	List	List, Null

4.7.5. is_list

Внимание! Функция может использоваться только при создании правил нормализации.

Функция `is_list` используется для проверки соответствия аргумента типу данных `List`. Если аргумент относится к типу данных `List`, функция возвращает значение `True`, в ином случае — `False`.

Формат вызова:

```
is_list($f1)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f = ["Red", "Green", "Orange"]
datafield1 = is_list($f)
datafield2 = is_list("Blue")
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "datafield2": "false",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 56. Приведение типов данных функции is_list

Аргумент	Результат
Любой	Bool

4.7.6. remove

Функция `remove` используется для удаления из списка, указанного в первом аргументе, всех элементов, значение которых совпадает со вторым аргументом.

Формат вызова:

```
remove($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange"]
$f3 = remove($f1, "Green")
datafield1 = in_list($f3, "Green")
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "false"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 57. Приведение типов данных функции remove

Первый аргумент	Второй аргумент	Результат
List	Любой, кроме Null, List	List

4.7.7. slice

Функция `slice` используется для возвращения части списка, ограниченной начальным и конечным номерами элементов списка (индексами). Исходный список нужно указывать в первом аргументе функции, начальный индекс — во втором, конечный индекс — в третьем. Элемент списка с начальным индексом будет включен в новый список, элемент с конечным индексом — не будет.

Если конечный индекс не указан, его значение считается равным длине списка.

Примечание. Нумерация элементов списка начинается с нуля.

Формат вызова:

```
slice($f1, $f2, $f3)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange", "Blue", "Black"]
$f4 = slice($f1, 1, 3)
# $f4 == ["Green", "Orange"]
datafield1 = join($f4, ", ")
time = "2023-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Green, Orange",
  "time": "2023-10-18T12:00:00.000Z"
}
```

Таблица 58. Приведение типов данных функции `slice`

Первый аргумент	Второй аргумент	Третий аргумент	Результат
List	Number	Number, Null	List

4.7.8. sort

Функция `sort` используется для сортировки элементов списка.

Вы можете использовать сортировку только для списков, состоящих из элементов одного типа: строк или чисел. Строки будут отсортированы по алфавиту, числа — по возрастанию.

Если список состоит из элементов разного типа или содержит элементы неподдерживаемых типов, функция возвращает `null`.

Формат вызова:

```
sort($f1)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange", "Black"]
$f2 = [2, 45, 8, 120]
$f4 = sort($f1)
$f5 = sort($f2)
datafield1 = join($f4, ", ")
datafield2 = join($f5, ", ")
time = "2024-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Black, Green, Orange, Red",
  "datafield2": "2, 8, 45, 120",
  "time": "2024-10-18T12:00:00.000Z"
}
```

Таблица 59. Приведение типов данных функции sort

Аргумент	Результат
List	List, Null

4.7.9. split

Функция `split` используется для разделения строки на элементы списка по разделителю. Строку нужно указывать в первом аргументе функции, разделитель — во втором.

Формат вызова:

```
split($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = "Red|Green|Orange"
$f2 = split($f1, "|")
# $f2 == ["Red", "Green", "Orange"]
datafield1 = join($f2, ", ")
time = "2023-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Red, Green, Orange",
  "time": "2023-10-18T12:00:00.000Z"
}
```

Таблица 60. Приведение типов данных функции split

Первый аргумент	Второй аргумент	Результат
String	String	List

4.7.10. unique

Функция `unique` используется для удаления из списка повторяющихся элементов с сохранением последовательности.

Формат вызова:

```
unique($f1)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f1 = ["Red", "Green", "Orange", "Red", "Black", "Green"]
$f2 = unique($f1)
# f2 == ["Red", "Green", "Orange", "Black"]
datafield1 = join($f2, ", ")
time = "2023-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "Red, Green, Orange, Black",
  "time": "2023-10-18T12:00:00.000Z"
}
```

Таблица 61. Приведение типов данных функции unique

Аргумент	Результат
List	List

4.8. Функции для анализа данных

В разделе описаны функции языка XP для анализа данных.

В этом разделе

[coalesce](#) (см. раздел 4.8.1)

[flip_endianness16](#), [flip_endianness32](#), [flip_endianness64](#) (см. раздел 4.8.2)

[in_subnet](#) (см. раздел 4.8.3)

[set_bit](#) (см. раздел 4.8.4)

`clear_bit` (см. раздел 4.8.5)

`get_bit` (см. раздел 4.8.6)

`check_bit_mask` (см. раздел 4.8.7)

4.8.1. coalesce

Функция `coalesce` проверяет значения аргументов слева направо и возвращает значение первого отличного от `null`. Количество аргументов может быть любым. Если аргументы не заданы, функция возвращает `null`.

Формат вызова:

```
coalesce($f1, $f2, $f3,...)
```

Пример

```
# Необработанное событие:
# Example
TEXT = 'Example'
$f2 = "second"
$f3 = [3, "third"]
$f4 = 4
datafield1 = coalesce($f1, $f2, $f3, $f4)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "second",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 62. Приведение типов данных функции `coalesce`

Аргумент	Результат
Null	Null
Любой	Как у аргумента

4.8.2. flip_endianness16, flip_endianness32, flip_endianness64

Внимание! Функции могут использоваться только при создании правил нормализации.

Функции `flip_endianness16`, `flip_endianness32`, `flip_endianness64` используются для изменения порядка байтов 16-, 32- и 64-разрядных чисел на обратный.

Формат вызова:

```
flip_endianness16($f1)
flip_endianness32($f1)
flip_endianness64($f1)
```

Пример

```
# Необработанное событие:
# Example: 256
TEXT = 'Example: {$f1=NUMBER?}'
datafield1 = flip_endianness16($f1)
datafield2 = flip_endianness32($f1)
datafield3 = flip_endianness64($f1)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "1",
#   "datafield2": "65536",
#   "datafield3": "281474976710656",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 63. Приведение типов данных функций flip_endianness16, flip_endianness32, flip_endianness64

Аргумент	Результат
Number	Number

4.8.3. in_subnet

Функция `in_subnet` используется для проверки принадлежности IP-адреса к подсети. В первом аргументе указывается IP-адрес с типом данных `IPAddress`. Во втором аргументе указывается адрес подсети в формате CIDR-нотации в виде: `a.b.c.d/<Маска>` с типом данных `String`. Если один из аргументов не задан (`null`), функция возвращает значение `False`.

Формат вызова:

```
in_subnet($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 10.0.2.3 10.0.0.0/22
TEXT = 'Example: {$f1=IPV4} {$f2=STRING}'
datafield1 = in_subnet($f1, $f2)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "true",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 64. Приведение типов данных функции in_subnet

Первый аргумент	Второй аргумент	Результат
IPAddress, Null	String	Bool

4.8.4. set_bit

Функция `set_bit` используется, чтобы установить для конкретного бита числа значение 1. Число указывается в первом аргументе функции, а номер бита, для которого устанавливается значение, — во втором. Нумерация битов начинается с наименьшего значащего бита, имеющего номер 0 (LSb 0).

Функция возвращает число, которому соответствует измененная последовательность битов.

Формат вызова:

```
set_bit($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 1
TEXT = 'Example: {$f1=NUMBER}'
# $f1 == 0000000000000001
numfield1 = set_bit($f1, 2)
# numfield1 == 0000000000000101
time = "2017-10-18T12:00:00Z"
# Нормализованное событие {
  "numfield1": 5,
  "time": "2017-10-18T12:00:00.000Z"
}
```

Таблица 65. Приведение типов данных функции `set_bit`

Первый аргумент	Второй аргумент	Результат
Number	Number	Number

4.8.5. clear_bit

Функция `clear_bit` используется, чтобы установить для конкретного бита числа значение 0. Число указывается в первом аргументе функции, а номер бита, для которого устанавливается значение, — во втором. Нумерация битов начинается с наименьшего значащего бита, имеющего номер 0 (LSb 0).

Функция возвращает число, которому соответствует измененная последовательность битов.

Формат вызова:

```
clear_bit($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 5
```



```

TEXT = 'Example: {$f1=NUMBER}'
# $f1 == 00000000000000101
numfield1 = clear_bit($f1, 2)
# numfield1 == 000000000000001
time = "2017-10-18T12:00:00Z"
# Нормализованное событие {
  "numfield1": 1,
  "time": "2017-10-18T12:00:00.000Z"
}

```

Таблица 66. Приведение типов данных функции clear_bit

Первый аргумент	Второй аргумент	Результат
Number	Number	Number

4.8.6. get_bit

Функция `get_bit` используется для проверки значения конкретного бита числа. Число указывается в первом аргументе функции, а номер бита, значение которого проверяется, — во втором. Нумерация битов начинается с наименьшего значащего бита, имеющего номер 0 (LSb 0).

Если значение указанного бита равно 1, функция возвращает `true`, если 0 — `false`.

Формат вызова:

```
get_bit($f1, $f2)
```

Пример

```

# Необработанное событие:
# Example: 1
TEXT = 'Example: {$f1=NUMBER}'
# $f1 == 0000000000000001

datafield1 = string(get_bit($f1, 0))
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
  "datafield1": "true",
  "time": "2017-10-18T12:00:00.000Z"
}

```

Таблица 67. Приведение типов данных функции get_bit

Первый аргумент	Второй аргумент	Результат
Number	Number	Bool

4.8.7. check_bit_mask

Функция `check_bit_mask` используется для проверки соответствия числа битовой маске. Число, которое нужно проверить, указывается в первом аргументе функции, а число, которое служит битовой маской для проверки, — во втором.

Если число соответствует маске, функция возвращает `true`, если не соответствует — `false`.

Формат вызова:

```
check_bit_mask($f1, $f2)
```

Пример

```
# Необработанное событие:
# Example: 27 9
TEXT = 'Example: {$f1=NUMBER} {$f2=NUMBER}'
# $f1 == 0000000000011011
# $f2 == 000000000001001
datafield1 = string(check_bit_mask($f1, $f2))
time = "2017-10-18T12:00:00Z"
# Нормализованное событие {
  "datafield1": "true",
  "time": "2017-10-18T12:00:00.000Z"
}
```

Таблица 68. Приведение типов данных функции `check_bit_mask`

Первый аргумент	Второй аргумент	Результат
Number	Number	Bool

4.9. Функции для работы с табличными списками

В разделе описаны инструменты языка XPR, которые вы можете использовать в правилах корреляции и обогащения для получения информации из табличных списков, созданных в MaxPatrol SIEM.

Для получения информации можно, либо выполнить инлайн-запрос, добавив к названию табличного списка префикс `table::`, либо с помощью функций `exes_query`, `select_query_first` обратиться к запросу, объявленному в правиле корреляции или обогащения в директиве `query`.

В этом разделе

[Префикс `table::`, инлайн-запрос \(см. раздел 4.9.1\)](#)

[Функция `exes\_query`, запрос к табличному списку \(см. раздел 4.9.2\)](#)

[Функция `exes\_query`, арифметические операции над результатами запроса \(см. раздел 4.9.3\)](#)

Функция `select_query_first`, запрос значения из табличного списка (см. раздел 4.9.4)

Функция `regex_match`, проверка строки по регулярному выражению из табличного списка (см. раздел 4.9.5)

4.9.1. Префикс `table::`, инлайн-запрос

Инлайн-запрос — запрос к табличному списку, условие которого указывается непосредственно в строке вызова запроса. Условие запроса проверяется для записей табличного списка, и если хотя бы для одной из записей условие выполнено, запрос возвращает `True`, в ином случае — `False`.

Для создания инлайн-запроса к табличному списку используется название этого табличного списка с префиксом `table::`. После названия в фигурных скобках нужно указать условие запроса.

Условие запроса может состоять из нескольких логических выражений, объединенных операторами `or`, `and`. В условии можно использовать круглые скобки и оператор `not`. В каждом выражении слева от логического оператора должны располагаться имена колонок табличного списка (с префиксом `column::`), справа от оператора можно использовать:

- поля события;
- литералы (с типом данных `Bool`, `Null`, `Number`, `String`);
- переменные (должны начинаться со знака доллара `$`);
- выражения языка `XP` с математическими операторами;
- функции языка `XP`.

Примечание. При использовании в условии запроса функции `in_subnet` в первом аргументе функции нужно указать имя переменной или поле события, во втором — имя колонки табличного списка (с префиксом `column::`).

Формат запроса:

```
table::<Название табличного списка> {
    <Условие запроса>
}
```

Пример

```
# Табличный список:
# "Tabular_List_Name": [
#   {"IP": "192.0.2.10", "Port": 21, "Status": "Close"},
#   {"IP": "192.0.2.11", "Port": 22, "Status": "Open"},
#   {"IP": "192.0.2.12", "Port": 23, "Status": "Close"}]
# Нормализованное событие: {
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "time": "2017-10-18T12:00:00Z",
```

```

#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event{
    # Возвращает True, если порт в состоянии Open
    $datafield1 = table::Tabular_List_Name {
      column::IP == dst.ip and
      column::Port == dst.port and
      column::Status == "Open"
    }
  }
emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}
# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"true",
#   ...}

```

4.9.2. Функция `exes_query`, запрос к табличному списку

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функция `exes_query` используется для выполнения запроса к табличному списку. В первом аргументе нужно указать название запроса, описанного в директиве `query`. Во втором аргументе — список значений полей, по которым выполняется запрос. Количество передаваемых значений должно совпадать с количеством аргументов запроса.

Если для переданных значений полей условие в указанном запросе выполнено, функция возвращает `True`, в ином случае — `False`.

Формат вызова функции:

```
exes_query(<Название запроса>, [$f1, $f2, ...])
```

Формат запроса:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
    <Условие запроса>
}
```

Пример

```
# Табличный список:
# "Tabular_List_Name": [
#     {"IP": "192.0.2.10", "Port": 21, "Status": "Close"},
#     {"IP": "192.0.2.11", "Port": 22, "Status": "Open"},
#     {"IP": "192.0.2.12", "Port": 23, "Status": "Close"}]
# Нормализованное событие: {
#     "dst.ip": "192.0.2.11",
#     "dst.port": 22,
#     "time": "2017-10-18T12:00:00Z",
#     "id": "developer_guide",
#     "importance": "info",
#     "action": "start",
#     "object": "task",
#     "status": "success",
#     "event_src.vendor": "Best Company",
#     "event_src.title": "test",
#     "event_src.category": "Other",
#     "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
# Запрос TestQuery к табличному списку Tabular_List_Name
query TestQuery ($check_ip, $check_port) from Tabular_List_Name {
    column::IP == $check_ip and
    column::Port == $check_port and
    column::Status == "Open"
}
event Event:
    key:
        event_src.title
    filter {
        event_src.title == "test"
```

```

    }
rule Example: Event
  on Event{
    # Возвращает True, если переданный в запрос порт в состоянии Open
    $datafield1 = exec_query("TestQuery", [dst.ip, dst.port])
  }
emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}
# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"true",
#   ...}

```

Таблица 69. Приведение типов данных функции `exec_query`

Первый аргумент	Второй аргумент	Результат
String	List = [String, String, ...]	Bool

4.9.3. Функция `exec_query`, арифметические операции над результатами запроса

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функцию `exec_query` можно использовать для выполнения арифметических операций над элементами табличного списка, удовлетворяющих запросу.

В первом аргументе функции нужно указать название запроса, описанного в директиве `query`. В запросе нужно добавить инструкцию `qhandler`, в которой указать [агрегатные функции](#) (см. [раздел 9.5.2](#)) (например, `min`, `max`, `avg`) для выполнения арифметических операций над результатами запроса. Значение каждой функции нужно присвоить переменной, имя которой является ключом для получения значения. В аргументе каждой функции нужно указать имя колонки табличного списка (с префиксом `column:`), над элементами которого выполняется арифметическая операция. Поддерживаются операции над элементами колонок с типом данных `Number`.

Во втором аргументе функции нужно указать список значений полей, по которым выполняется запрос. Количество передаваемых значений должно совпадать с количеством аргументов запроса.

Если для переданных значений полей условие в указанном запросе выполнено, функция возвращает ассоциативный массив с типом данных KeyValue. Массив содержит результаты с типом данных Number для агрегатных функций, указанных в инструкции `qhandler`, над элементами указанной колонки из строк табличного списка, удовлетворяющих запросу. Для обращения к элементам массива в качестве ключей нужно использовать имена переменных инструкции `qhandler`.

Формат вызова функции:

```
$result_list = exec_query(<Название запроса к табличному списку>, [<Аргументы запроса>])
```

Формат обращения к элементам списка:

```
<Значение> = $result_list["<Ключ>"]
```

Формат запроса:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
    <Условие запроса>
}

qhandler {
    <Ключ 1> = <Агрегатная функция 1>(column::<Имя колонки 1>)
    <Ключ 2> = <Агрегатная функция 2>(column::<Имя колонки 2>)
    ...
}
```

Пример

```
# Табличный список "Tabular_List_Duration": [
#   {"IP":"192.0.2.10", "Port":22, "Status":"Open", "Duration":41},
#   {"IP":"192.0.2.11", "Port":22, "Status":"Open", "Duration":33},
#   {"IP":"192.0.2.11", "Port":22, "Status":"Open", "Duration":67}]
# Нормализованное событие: {
#   "dst.ip":"192.0.2.11",
#   "time": "2017-10-18T12:00:00Z",
#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
# Запрос TestQuery к табличному списку Tabular_List_Duration
query TestQuery ($check_ip) from Tabular_List_Duration {
```

```

column::IP == $check_ip and
column::Port == 22 and
column::Status == "Open"
}

qhandler {
    $get_max_duration = max(column::Duration)
    $get_min_duration = min(column::Duration)
    $get_avg_duration = avg(column::Duration)
}

event Event:
    key:
        event_src.title
    filter {
        event_src.title == "test"
    }

rule Example: Event
    on Event{
        # Присваивание переменным максимального, минимального и среднего времени
        # открытия порта 22 для переданного в запрос IP-адреса узла
        $get_duration = exec_query("TestQuery", [dst.ip])
        $datafield1 = "IP_" + string(dst.ip)
        $datafield2 = "Max_duration_" + string($get_duration["$get_max_duration"])
        $datafield3 = "Min_duration_" + string($get_duration["$get_min_duration"])
        $datafield4 = "Avg_duration_" + string($get_duration["$get_avg_duration"])
    }

emit {
    $correlation_type = "event"
    $object = "client"
    $action = "check"
    $status = "success"
    $importance = "info"
}

# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"IP_192.0.2.11",
#   "datafield2":"Max_duration_67",
#   "datafield3":"Min_duration_33",
#   "datafield4":"Avg_duration_50",
#   ...}

```


Таблица 70. Приведение типов данных функции `exec_query` с инструкцией `qhandler`

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	List = [String, String, ...]	String	KeyValue

4.9.4. Функция `select_query_first`, запрос значения из табличного списка

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функция `select_query_first` используется для получения значения ячейки табличного списка, удовлетворяющего запросу. В первом аргументе нужно указать название запроса, описанного в директиве `query`. Во втором аргументе — список значений полей, по которым выполняется запрос. Количество и тип данных передаваемых значений должно совпадать с количеством и типом данных аргументов запроса. В третьем аргументе нужно указать заголовок колонки табличного списка (в кавычках), значение которой требуется получить.

Если для переданных значений полей условие в указанном запросе выполнено, функция возвращает значение ячейки указанной колонки из первой строки табличного списка, удовлетворяющей запросу. Тип данных возвращаемого значения совпадает с типом данных колонки. Если условие запроса не выполнено, функция возвращает `null`.

Формат вызова функции:

```
select_query_first(<Название запроса>, [$f1, $f2, ...], "<Имя колонки>")
```

Формат запроса:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
  <Условие запроса>
}
```

Пример

```
# Табличный список:
# "Tabular_List_Name": [
#   {"IP": "192.0.2.10", "Port": 21, "Status": "Close"},
#   {"IP": "192.0.2.11", "Port": 22, "Status": "Open"},
#   {"IP": "192.0.2.12", "Port": 23, "Status": "Close"}]
# Нормализованное событие: {
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "time": "2017-10-18T12:00:00Z",
#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
```

```

#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
# Запрос TestQuery к табличному списку Tabular_List_Name
query TestQuery ($check_ip, $check_port) from Tabular_List_Name {
  column::IP == $check_ip and
  column::Port == $check_port}
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event{
    # Присваивание переменной IP-адреса узла, если переданный в запрос порт на
    этом узле в состоянии Open
    $datafield1 = "Port_Status: " + select_query_first("TestQuery", [dst.ip,
dst.port], "Status")
  }
emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}
# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"Port_Status: Open",
#   ...}

```

Таблица 71. Приведение типов данных функции exes_query_first

Первый аргумент	Второй аргумент	Второй аргумент	Результат
String	List = [String, String, ...]	String	Number, String, DateTime, Null (зависит от типа данных колонки табличного списка)

4.9.5. Функция `regex_match`, проверка строки по регулярному выражению из табличного списка

Внимание! Функция может использоваться только при создании правил корреляции и обогащения.

Функция `regex_match` используется в запросе к табличному списку для сравнения строки с указанным в табличном списке шаблоном. Функцию можно использовать в инлайн-запросе или при описании запроса в директиве `query`. В первом аргументе функции указывается строка для сравнения с шаблоном, во втором — название колонки с шаблонами. Функция возвращает `True`, если строка соответствует одному из шаблонов в указанной колонке, в противном случае — `False`. Если первый аргумент не задан, функция возвращает `False`.

Примечание. Вы можете выполнять запрос с функцией `regex_match` только к табличному списку типа «справочник». Для колонки с регулярными выражениями в табличном списке должен быть выбран тип `Regex`. Поддерживаются Perl-совместимые регулярные выражения (PCRE).

Формат запроса:

```
query <Название запроса>($f) from <Название табличного списка> {
    regex_match($f, column::<Название колонки>)
}
```

или

```
table::<Название табличного списка> {
    regex_match($f, column::<Название колонки>)
}
```

Пример

```
# Табличный список:
# "Tabular_List_Service_Names": [
#   {"Object": ".*first_service.exe"},
#   {"Object": ".*second_service.exe"},
#   {"Object": ".*third_service.exe"}]
# Нормализованное событие: {
#   "object.value": "path\\folder\\third_service.exe"
#   "time": "2017-10-18T12:00:00Z",
#   "id": "developer_guide",
#   "importance": "info",
#   "action": "start",
#   "object": "task",
#   "status": "success",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "test",
#   "event_src.category": "Other",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077"}
```

```

query TestQuery ($check_object) from Tabular_List_Service_Names {
  regex_match($check_object, column::Object)
}
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test" and
    exec_query("TestQuery", [object.value])
  }
rule Example: Event
on Event{
  $datafield1 = "Обнаружен объект " + object.value
}
emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}

# Корреляционное событие: {
#   "correlation_type":"event",
#   "object":"client",
#   "action":"check",
#   "status":"success",
#   "importance":"info",
#   "datafield1":"Обнаружен объект path\\folder\\third_service.exe",
#   ...}

```

Таблица 72. Приведение типов данных функции regex_match

Первый аргумент	Второй аргумент	Результат
String	—	Bool
Null	—	Bool (False)

5. Создание правила нормализации

В этом разделе описаны ключевые слова, токены и функции, используемые при создании правил нормализации, даны примеры и рекомендации.

Для отладки нового правила вы можете использовать утилиты SDK или утилиту PTSIEMSDK GUI. Добавить новое правило в MaxPatrol SIEM вы можете через веб-интерфейс Knowledge Base (см. Руководство оператора).

В этом разделе

[Алгоритм работы службы нормализации событий \(см. раздел 5.1\)](#)

[Структура правила нормализации \(см. раздел 5.2\)](#)

[Заполнение полей нормализованного события \(см. раздел 5.3\)](#)

[Ключевое слово COND. Условие использования правила нормализации \(см. раздел 5.4\)](#)

[Ключевое слово TEXT \(см. раздел 5.5\)](#)

[Ключевое слово TABULAR \(см. раздел 5.6\)](#)

[Ключевое слово JSON \(см. раздел 5.7\)](#)

[Ключевое слово EVENTLOG \(см. раздел 5.8\)](#)

[Ключевое слово XML \(см. раздел 5.9\)](#)

[Команда drop \(см. раздел 5.10\)](#)

[Вложенное правило нормализации \(см. раздел 5.11\)](#)

[Пример правила нормализации \(см. раздел 5.12\)](#)

5.1. Алгоритм работы службы нормализации событий

При сборе событий MP 10 Collector, в зависимости от используемого модуля и метода сбора, для каждого необработанного события указывает его формат (в поле `mime`).

Алгоритм работы службы нормализации событий зависит от формата необработанного события. Для разных форматов используются специализированные алгоритмы обработки текста события (парсеры). Правила нормализации создаются с учетом особенностей работы алгоритмов обработки, форматы необработанных событий указываются с помощью ключевых слов форматной строки:

- TEXT — для неструктурированного текста, например стандарта syslog;
- TABULAR — для ассоциативного массива «ключ — значение» формата JSON;
- JSON — для формата JSON;

- EVENTLOG — для событий журнала Windows;
- XML — для событий формата XML.

Алгоритм обработки неструктурированного текста

В ходе обработки событий:

- Для каждого события по графу нормализации выполняется поиск подходящего правила нормализации. Текст события должен соответствовать шаблону, указанному в форматной строке (с ключевым словом **TEXT**). Если такое правило не найдено, событие не нормализуется.
- Выполняется последовательное распознавание текста необработанного события по шаблону, указанному в форматной строке.
- Если в правиле нормализации объявлено условие применения (в строке с ключевым словом **COND**), проверяется это условие. Если условие не выполнено, событие не нормализуется.
- Создается нормализованное событие, поля события заполняются значениями, указанными в блоке параметров правила нормализации.

Алгоритм обработки структурированного текста

В ходе обработки событий:

- Для каждого события по графу нормализации выполняется поиск правила нормализации с подходящей форматной строкой. Если такое правило не найдено, событие не нормализуется.
- Необработанные события журнала Windows и формата XML преобразуются в формат JSON.
- Параметры события заносятся в словарь в памяти службы нормализации событий.
- Если в правиле нормализации объявлено условие применения правила (в строке с ключевым словом **COND**), проверяется это условие. Если условие не выполнено, событие не нормализуется.
- Создается нормализованное событие, поля события заполняются значениями, указанными в блоке параметров правила нормализации.

См. также

[Этапы обработки событий \(см. раздел 2\)](#)

[Программные компоненты для обработки событий \(см. раздел 3\)](#)

5.2. Структура правила нормализации

Правило нормализации должно начинаться с ключевого слова, указывающего на формат необработанного события. После ключевого слова в одинарных или двойных кавычках указывается форматная строка, вид которой зависит от формата события.

Правило нормализации может содержать строку, начинающуюся с ключевого слова `COND` (для объявления условия применения этого правила). Если форматная строка не содержит никакого значения, то строка со словом `COND` — обязательна.

Примечание. Ключевое слово форматной строки и слово `COND` могут начинаться с восклицательного знака (!). Такое написание считается устаревшим.

Правило нормализации может содержать функцию `submessage` — для объявления вложенного правила нормализации для распознавания фрагмента необработанного события, а также ключевые слова `subformula`, `endsubformula` — для ввода блока операторов вложенного правила нормализации.

В блоке операторов правила нормализации вы можете использовать функции, операторы и конструкции языка XPR, переменные (должны начинаться с `$`) и поля нормализованного события.

```
<Ключевое слово> = '<Форматная строка>'
COND = (<Условие>)
submessage("<Формат фрагмента>", "<Название вложенного правила>", <Фрагмент события>)
subformula <Название вложенного правила>
    <Ключевое слово вложенного правила> = '<Форматная строка вложенного правила>'
    COND = (<Условие вложенного правила>)
    <Блок операторов вложенного правила>
endsubformula
<Блок операторов>
```

5.3. Заполнение полей нормализованного события

В правиле нормализации необходимо указать значения обязательных для заполнения [полей нормализованного события](#) (см. [приложение Б](#)). Кроме того, вы можете указать значения необязательных полей или изменить значения отдельных сервисных полей, заполняемых автоматически.

Заполнение полей нормализованного события выполняется в блоке операторов правила нормализации (и вложенного правила).

Примечание. Для текстовых событий вы можете заполнять поля нормализованного события в форматной строке, присваивая им значения токенов внутри шаблонов.

В правиле нормализации нужно указать значения для следующих обязательных полей нормализованного события:

- `time` — для указания времени регистрации события на источнике;
- `event_src.title` — для указания названия продукта источника события; указывается в нижнем регистре без пробелов (можно использовать символ подчеркивания), например `windows, asa, oracle_database`;
- `event_src.vendor` — для указания производителя источника; указывается в нижнем регистре без пробелов, например `microsoft, cisco`;
- `action, object, status` — для описания характера, объекта и результата воздействия; могут принимать только [определенные значения \(см. приложение В\)](#);

Примечание. Вы можете заполнить необязательное поле `subject`, чтобы указать источник воздействия.

- `event_src.category` — для назначения категории источника события; может принимать только [определенные значения \(см. приложение Г\)](#);
- `importance` — для указания важности события для информационной безопасности; может принимать значения `info, low, medium, high, critical`;
- `id` — для ввода идентификатора правила нормализации в виде:
`<Пространство_имен>_<Производитель>_<Название_источника>_<Транспорт>_<Тип_события>`, например `PT_Cisco_IOS_syslog_SEC_LOGIN_QUIET_MODE_OFF`.

5.4. Ключевое слово COND. Условие использования правила нормализации

Ключевое слово `COND` используется для объявления условия применения правила нормализации (и вложенного правила). Если условие возвращает `True`, создается нормализованное событие, поля события заполняются данными согласно правилу нормализации.

Ключевое слово `COND` применяется также для предотвращения конфликта правил нормализации, если их форматные строки идентичны.

Пример

```
{ "time": "2017-10-18T12:00:00Z", "source": "my_source", "data": "foobar" }
{ "time": "2017-10-18T12:00:00Z", "source": "another_source", "data": "barbaz" }
```


Форматные строки правил нормализации источников `my_source` и `another_source` идентичны, поэтому неизбежен конфликт правил нормализации. Чтобы этого избежать, нужно добавить в правило нормализации источника `my_source` ключевое слово `COND`:

```
JSON = <>
COND = $source == "<Источник my_source>"
time = $time
datafield1 = $data
```

Условие может содержать переменные, математические и логические операторы языка XPR или значение `null`.

Формат вызова:

```
COND = <Условие>
```

или

```
COND = (
    <Условие>
)
```

Пример

```
# Необработанное событие:
# Example: Air temperature 28 degrees
TEXT = 'Example: {WORD+}{temperature=NUMBER}{WORD}'
COND = (number($temperature) >= 23)
datafield1 = "Too_hot_" + string($temperature) + "_degrees"
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Too_hot_28_degrees",
#   "time": "2017-10-18T12:00:00Z"}
```

5.5. Ключевое слово TEXT

Ключевое слово `TEXT` используется для объявления форматной строки события в формате простого текста. При нормализации текстового события выполняется последовательное распознавание слов и символов события для заполнения полей нормализованного события. Форматная строка в этом случае является шаблоном для распознавания текста события, она должна повторять его структуру и содержать шаблоны для распознавания отдельных слов или символов события (в фигурных скобках) и сами слова (символы), если их расположение в событии фиксировано.

Примечание. Если форматная строка заключена в двойные кавычки, внутри шаблона слова нужно использовать одинарные кавычки.

Шаблон слова события может содержать токены языка XPR, операторы токенов, любые символы (в двойных кавычках), переменные и поля нормализованного события. Элементы в шаблоне можно разделять пробелом.

Токен языка XP — лексема, определяющая специализированный алгоритм, который используется для распознавания последовательности символов в шаблоне слова.

Формат вызова:

```
TEXT = '{<Токен 1>}{<Токен 2>}...'
```

Пример

```
# Необработанное событие:
# Example: Temperature - 28 degrees; Pressure - 1 bar.
TEXT = 'Example: {WORD "-" $tmp = NUMBER}{WORD};
{WORD|WORDDASH}{"-"}{datafield2 = NUMBER}{WORD}{LITERAL}'
datafield1 = $tmp
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1":"28",
#   "datafield2":"1",
#   "time": "2017-10-18T12:00:00Z"}
```

В этом разделе

[Операторы токена \(см. раздел 5.5.1\)](#)

[BASE64 \(см. раздел 5.5.2\)](#)

[CSV \(см. раздел 5.5.3\)](#)

[DATETIME \(см. раздел 5.5.4\)](#)

[DURATION \(см. раздел 5.5.5\)](#)

[HOSTNAME \(см. раздел 5.5.6\)](#)

[IPV4 \(см. раздел 5.5.7\)](#)

[IPV6 \(см. раздел 5.5.8\)](#)

[KEYVALUE \(см. раздел 5.5.9\)](#)

[LITERAL \(см. раздел 5.5.10\)](#)

[MACADDR \(см. раздел 5.5.11\)](#)

[NTUSER \(см. раздел 5.5.12\)](#)

[NUMBER \(см. раздел 5.5.13\)](#)

[REST \(см. раздел 5.5.14\)](#)

[STRING \(см. раздел 5.5.15\)](#)

[UNTIL \(см. раздел 5.5.16\)](#)

[WORD \(см. раздел 5.5.17\)](#)

[WORDDASH \(см. раздел 5.5.18\)](#)

5.5.1. Операторы токена

В шаблоне слова форматной строки текстового события можно использовать операторы токенов.

Оператор «=»

Результатом применения оператора «=» является присваивание переменной или полю нормализованного события значения токена.

Формат вызова:

```
TEXT = '{$f=<Токен>}'
```

или

```
TEXT = '{<Поле события>=<Токен>}'
```

Пример:

```
# Необработанное событие:
# Example: Hello
TEXT = 'Example: {datafield1 = WORD}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Hello",
#   "time": "2017-10-18T12:00:00Z"}
```

Оператор «+»

Результатом применения оператора «+» является использование токена для распознавания одной и более последовательностей символов, разделенных пробелами.

Формат вызова:

```
TEXT = '{<Токен>+}'
```

Пример:

```
# Необработанное событие:
# Example: Hello wonderful world
TEXT = 'Example: {datafield1=WORD+}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Hello wonderful world",
#   "time": "2017-10-18T12:00:00Z"}
```

Оператор «*»

Результатом применения оператора «*» является использование токена для распознавания нескольких последовательностей символов, разделенных пробелами. В отличие от оператора «+» последовательность символов может отсутствовать в событии, это не приведет к ошибке распознавания.

Формат вызова:

```
TEXT='{<Токен>*}'
```

Пример:

```
# Необработанное событие:
# Example
TEXT = 'Example {datafield1=WORD*}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "time": "2017-10-18T12:00:00Z"}
```

Оператор «?»

Применение оператора «?» к токenu делает необязательным его использование при распознавании события. Последовательность символов может отсутствовать в событии, это не приведет к ошибке распознавания.

Формат вызова:

```
TEXT='{<Токен>?}'
```

Пример:

```
# Необработанное событие:
# Example: Hello world !
TEXT = 'Example: {datafield1=WORD+}{datafield2=LITERAL?}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Hello world",
#   "datafield2": "!",
#   "time": "2017-10-18T12:00:00Z"}
```

Операторы конъюнкции

Оператором конъюнкции (логическим И) в шаблоне слова форматной строки является пробел. Применение оператора позволяет использовать в одном шаблоне несколько токенов.

Формат вызова:

```
TEXT='{<Токен 1> <Токен2>}'
```

Пример:

```
# Необработанное событие:
# Example: Hello world !!!
```

```
TEXT = 'Example: {datafield1=WORD datafield2=WORD "!!!"}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Hello",
#   "datafield2": "world",
#   "time": "2017-10-18T12:00:00Z"}
```

Оператор дизъюнкции «|»

Применение оператора дизъюнкции (логического ИЛИ) позволяет использовать один или другой токен, в зависимости от формата данных последовательности символов.

Формат вызова:

```
TEXT = '{<Токен 1>|<Токен2>}'
```

При распознавании может использоваться токен 1 или токен 2.

```
TEXT = '{<Токен 1>|<Токен2>|}'
```

При распознавании может использоваться токен 1, токен 2 или последовательность символов может отсутствовать в событии.

Пример:

```
# Необработанное событие:
# Example: 10.12.13.14
TEXT = 'Example: {datafield1=IPV4|datafield1=IPV6}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "10.12.13.14",
#   "time": "2017-10-18T12:00:00Z"}
```

5.5.2. BASE64

Токен BASE64 используется для распознавания строки, закодированной по стандарту Base64.

Токен имеет один аргумент, в котором можно указать тип определяемой последовательности Base64:

- STRICT — последовательности Base64, сформированные по стандарту RFC 4648.
- URL — последовательности типа Base64 URL.
- ALL — последовательности Base64 обоих типов (STRICT и URL); используется по умолчанию.

Данные, полученные с помощью токена BASE64, имеют тип Buffer и при необходимости могут быть преобразованы в строку (тип String) с помощью функции decode.

Формат вызова:

```
TEXT = '{$f=BASE64("STRICT")}'
```

Пример

```
# Необработанное событие:
# Example: Q29tbW9uIEJBU0U2NA== U3RyaWN0IEJBU0U2NA== dXJsLXNhZmUgbWVzc2FnZQ
TEXT = 'Example: {$f1=BASE64} {$f2=BASE64("STRICT")} {$f3=BASE64("URL")} '
time = "2018-04-27T10:52:47Z"
datafield1 = decode(buffer_from_base64($f1), "UTF-8")
datafield2 = decode(buffer_from_base64($f2), "UTF-8")
datafield3 = decode(buffer_from_base64($f3), "UTF-8")
# Нормализованное событие: {
#   "time": "2018-04-27T10:52:47Z",
#   "datafield1": "Common BASE64",
#   "datafield2": "Strict BASE64",
#   "datafield3": "url-safe message"}
```

Таблица 73. Приведение типов данных токена BASE64

Аргумент	Результат
String	Buffer

5.5.3. CSV

Токен **CSV** используется для распознавания строки с CSV-разметкой и возвращает список элементов строки. Распознаются все символы до конца строки.

Токен имеет два аргумента: в первом нужно указать символ, разделяющий элементы строки, во втором — символ экранирования, стоящий перед каждым элементом строки и после него. Если элементы строки не экранированы, во втором аргументе нужно указать любой символ, не встречающийся в строке. Если токен используется без аргументов, считается, что элементы строки разделены запятой и не экранированы.

Формат вызова:

```
TEXT = '{$f=CSV}'
```

или

```
TEXT = '{$f=CSV("<Разделитель элементов>","<Экранирование элементов>")}'
```

Пример

```
# Необработанное событие:
# Example: /login=admin/; /password=supercleveradmin/
TEXT = 'Example: {$f=CSV(";", "/" )}'
datafield1 = $f[0]
datafield2 = $f[1]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "login=admin",
```

```
# "datafield2": "password=supercleveradmin",
# "time": "2017-10-18T12:00:00Z"}
```

Таблица 74. Приведение типов данных токена CSV

Аргумент	Результат
String	List

5.5.4. DATETIME

Токен `DATETIME` используется для распознавания даты и времени и возвращает тип данных `DateTime` (формат ISO 8601). Формат даты и времени в событии определяется автоматически. Для сокращения времени распознавания вы можете указать имя формата из таблицы ниже.

Формат вызова:

```
TEXT = '{$f=DATETIME}'
```

или

```
TEXT = '{$f=DATETIME_<Имя формата>}'
```

Пример

```
# Необработанное событие:
# Example: 1990-10-31T23:13:04+03:00
TEXT = 'Example: {$f=DATETIME_ISO8601}'
datafield1 = $f
datafield2 = year($f)
datafield3 = month($f)
datafield4 = day($f)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "1990-10-31T23:13:04+03:00",
#   "datafield2": "1990",
#   "datafield3": "10",
#   "datafield4": "31",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 75. Приведение типов данных токена DATETIME

Аргумент	Результат
String	DateTime

Таблица 76. Форматы даты и времени токена DATETIME

Имя формата	Описание	Пример
CHECKPOINT_OPSEC	Встречается в системах Check Point (часовой пояс не указан)	25Jan2015 01:01:01 25Jan2015 17:10:10
ISO8601	Соответствует ISO 8601	2015-01-25T12:12:12 2015-01-25T12:12:12Z 2015-01-25T12:12:12+03:00 2015-01-25T16:53:47.2146003+03:00
MMDDYYYY_HHMMSS	Разделителем между датой и временем может быть один любой символ. Разделителем внутри даты и времени может быть любой знак препинания. Разделителем между секундами и долями секунды может быть только точка (доли секунды — от 2 до 6 знаков)	01/25/2015 16:12:12 01-25-2015-16!12!12 01/25/2015 16:12:12.121 01/25/2015 16:12:12 MSK 01/25/2015 16:12:12 +03:00
SAP	Формат YYYYMMDDHHMMSS	20150125074639
SAP_PO	Встречается в системах SAP Process Orchestration	2015 10 25 04:09:22:502#+0300
SEDO	Встречается в системе электронного документооборота «Электронная Москва»	3/Jul/2015:11:02:47 +0300 03/Jul/2015:11:02:47 +0300 25/Jul/2015:11:02:47 +0300 25/Jul/2015:08:02:47
SYSLOG	Встречается в сетевых устройствах и ОС семейства Unix. Поддерживаются сокращения для часовых поясов, например: MSK — UTC+03:00; GMT — UTC+00:00	Jan 25 01:00:00 Jan 25 16:12:12 Jan 25 16:12:12 MSK Jan 25 2015 16:12:12 Jan 25 2015 16:12:12 MSK Jan 25 16:12:12.133 Jan 25 16:12:12.133 MSK Jan 25 16:12:12.133331

Имя формата	Описание	Пример
		Jan 25 16:12:12.133331 MSK Jan 25 2015 16:12:12.133331 Jan 25 2015 16:12:12.133331 MSK
SYSLOG_HUAWEI	Встречается в сетевых устройствах Huawei и Juniper	Jun 25 02:29:19 2000 Jan 25 16:50:56:487 2011 Jan 25 16:50:56.487 2011 Jan 25 16:50:56,487 2011
SYSLOG_NEXUS	Встречается в сетевых устройствах Cisco Nexus	2015 Aug 2 13:10:44 MSK 2015 Aug 25 16:51:55 UTC 2015 Aug 25 17:45:23 2015 Aug 25 16:51:55.123 UTC 2015 Aug 25 17:45:23.123
WINDOWS_CIM	Формат времени параметра CIM_DATETIME из WMI вида YYYYMMDDHHMMSS.mmmmmmsUU	20150125173254.006182+180
YYYYMMDD_HHMMSS	Встречается в ПО компании «Лаборатория Касперского». Разделителем между датой и временем может быть любой символ, кроме буквы „Т”	2015/06/25 16:12:12 2015-06-25-16!12!12 2015/06/25@16:12:12.121 2015-06-25 12:10:14.4707

5.5.5. DURATION

Токен **DURATION** используется для преобразования строки, содержащей интервал времени в формате XXXd XXh:XXm:XXs или [HHH]HH:MM:SS, в число секунд.

Формат вызова:

```
TEXT = '{$f=DURATION}'
```

Пример

```
# Необработанное событие:
# Example: 0d 1h:2m:15s
```

```

TEXT = 'Example: {$f=DURATION}'
datafield1 = div($f,60*60)
datafield2 = div(mod($f,3600),60)
datafield3 = mod(mod($f,3600),60)
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1":"1",
#   "datafield2":"2",
#   "datafield3":"15",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 77. Приведение типов данных токена DURATION

Аргумент	Результат
String	Number

5.5.6. HOSTNAME

Токен **HOSTNAME** используется для распознавания имени узла в виде последовательности букв и цифр, которые могут быть разделены точками, знаками подчеркивания (_) и дефисами (-).

Формат вызова:

```
TEXT = '{$f=HOSTNAME}'
```

Пример

```

# Необработанное событие:
# Example: Logon Some-Host-Name_com.co
TEXT = 'Example: {WORD+}{datafield1=HOSTNAME}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1":"Some-Host-Name_com.co",
#   "time": "2017-10-18T12:00:00Z"}

```

Таблица 78. Приведение типов данных токена HOSTNAME

Аргумент	Результат
String	String

5.5.7. IPV4

Токен **IPV4** используется для распознавания IP-адреса стандарта IPv4 и преобразования его в тип данных IPAddress.

Формат вызова:

```
TEXT = '{$f=IPV4}'
```

Пример

```
# Необработанное событие:
# Example: 192.168.0.1
TEXT = 'Example: {datafield1=IPV4}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "192.168.0.1"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 79. Приведение типов данных токена IPV4

Аргумент	Результат
String	IPAddress

5.5.8. IPV6

Токен **IPV6** используется для распознавания IP-адреса стандарта IPv6 и преобразования его в тип данных IPAddress.

Поддерживаются IP-адреса в форматах X:X:X:X:X:X:X, X:X:X:X:X:D.D.D.D и X:X:X:X:X:X.X%S, где X — шестнадцатеричные значения для 16-битовых частей адреса, D — десятичные значения для 8-битовых частей адреса (стандарта IPv4), S — четырехзначное десятичное число. Для замены одного или нескольких нулей в расположенных рядом 16-битовых частях адреса допускается использование двух двоеточий (::).

Формат вызова:

```
TEXT = '{$f=IPV6}'
```

Пример

```
# Необработанное событие:
# Example: 211:DB8:0:0:8:800:200C:47A 211:DB8:0:0:8:800:192.168.0.1
211:DB8:0:0:8:800:200C:47A%5141
TEXT = 'Example: {event_src.host=IPV6} {src.host=IPV6} {dst.host=IPV6}'
time = "2021-10-18T12:00:00Z"
# Нормализованное событие: {
#   "event_src.host": "211:DB8:0:0:8:800:200C:47A",
#   "src.host": "211:DB8:0:0:8:800:192.168.0.1",
#   "dst.host": "211:DB8:0:0:8:800:200C:47A",
#   "time": "2021-10-18T12:00:00Z"}
```

Таблица 80. Приведение типов данных токена IPV6

Аргумент	Результат
String	IPAddress

5.5.9. KEYVALUE

Токен **KEYVALUE** используется для распознавания ассоциативного массива, содержащего пары «ключ — значение». Распознаются все символы до конца строки.

Токен имеет три аргумента: в первом нужно указать символ, разделяющий элементы строки, во втором — символ, разделяющий пару «ключ — значение», в третьем — символ экранирования значения, стоящий перед каждым значением в паре и после него.

Примечание. Для ввода одинарных кавычек используйте обратную косую черту (\), для ввода двойных кавычек — две обратные косые черты (\\).

Если токен используется без аргументов, считается, что элементы строки разделены запятой, пары «ключ — значение» разделены знаком равенства (=) и не экранированы.

Формат вызова:

```
TEXT = '{$f1=KEYVALUE}'
```

или

```
TEXT = '{$f1=KEYVALUE("<Разделитель между парами>", "<Разделитель в паре>", "<Экранирование значения>")}'
```

Пример

```
# Необработанное событие:
# Example: login:'admin'/password:'supercleveradmin'
TEXT = 'Example: {$f=KEYVALUE("/", ":", "\"")}'
datafield1 = $f["login"]
datafield2 = $f["password"]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "admin",
#   "datafield2": "supercleveradmin",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 81. Приведение типов данных токена KEYVALUE

Аргумент	Результат
String	KeyValue

5.5.10. LITERAL

Токен `LITERAL` используется для распознавания одного символа. Токен можно использовать для распознавания символов не разделенных пробелами.

Формат вызова:

```
TEXT = '{$f=LITERAL}'
```

Пример

```
# Необработанное событие:
# Example: Hello world !!!
TEXT = 'Example: {WORD WORD}{datafield1=LITERAL+}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "!!!",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 82. Приведение типов данных токена LITERAL

Аргумент	Результат
String	String

5.5.11. MACADDR

Токен `MACADDR` используется для распознавания MAC-адреса и преобразования его в тип данных `MACAddress`.

Формат вызова:

```
TEXT = '{$f=MACADDR}'
```

Пример

```
# Необработанное событие:
# Example: 00:0C:29:27:6E:D0
TEXT = 'Example: {datafield1=MACADDR}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "00:0C:29:27:6E:D0",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 83. Приведение типов данных токена MACADDR

Аргумент	Результат
String	MACAddress

5.5.12. NTUSER

Токен NTUSER используется для распознавания имени пользователя в формате <Имя домена>\<Имя пользователя>. Имя может содержать цифры, буквы и обратную косую черту (\).

Формат вызова:

```
TEXT = '{$f=NTUSER}'
```

Пример

```
# Необработанное событие:
# Example: This is AC\DC
TEXT = 'Example: {WORD+}{datafield1=NTUSER}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "AC\\DC"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 84. Приведение типов данных токена NTUSER

Аргумент	Результат
String	String

5.5.13. NUMBER

Токен NUMBER используется для распознавания целого числа.

Формат вызова:

```
TEXT = '{$f=NUMBER}'
```

Пример

```
# Необработанное событие:
# Example: Route 66
TEXT = 'Example: {WORD+}{datafield1=NUMBER}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "66",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 85. Приведение типов данных токена NUMBER

Аргумент	Результат
String	Number

5.5.14. REST

Токен `REST` используется для распознавания последовательности символов и пробелов до конца строки. Если символов не найдено, возвращает пустое значение.

Формат вызова:

```
TEXT = '{$f=REST}'
```

Пример

```
# Необработанное событие:
# Example: System check is finished!
TEXT = 'Example: {WORD WORD}{datafield1=REST}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "is finished!",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 86. Приведение типов данных токена REST

Аргумент	Результат
String	String

5.5.15. STRING

Токен `STRING` используется для распознавания последовательности символов до пробела.

Формат вызова:

```
TEXT = '{$f=STRING}'
```

Пример

```
# Необработанное событие:
# Password: As4r2@6E
TEXT = '{WORD}:{datafield1=STRING}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "As4r2@6E"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 87. Приведение типов данных токена STRING

Аргумент	Результат
String	String

5.5.16. UNTIL

Токен `UNTIL` используется для распознавания последовательности символов (и пробелов) до ограничителя, указанного в аргументе токена. Ограничителем может быть любая последовательность символов, ограничитель не попадает в возвращаемое токеном значение.

Примечание. Вы можете применить к токenu только оператор «?».

Формат вызова:

```
TEXT = '{$f=UNTIL("<Ограничитель>")}'
```

Пример

```
# Необработанное событие:
# Example: System check is finished
TEXT = 'Example: {datafield1=UNTIL("check is")}{datafield2=REST}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "System ",
#   "datafield2": "finished"
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 88. Приведение типов данных токена UNTIL

Аргумент	Результат
String	String

5.5.17. WORD

Токен `WORD` используется для распознавания отдельных слов, разделенных пробелами. Слова могут состоять из последовательности символов, состоящей из букв, цифр и знака подчеркивания.

Формат вызова:

```
TEXT = '{$f=WORD}'
```

Пример

```
# Необработанное событие:
# Example: System_check is finished
TEXT = 'Example: {datafield1=WORD}{datafield2=WORD*}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "System_check"
#   "datafield2": "is finished",
#   "time": "2017-10-18T12:00:00Z"}
```


Таблица 89. Приведение типов данных токена WORD

Аргумент	Результат
String	String

5.5.18. WORDDASH

Токен **WORDDASH** используется для распознавания отдельных слов, разделенных пробелами. Слова могут состоять из последовательности символов, включающей буквы, цифры, дефисы и знаки подчеркивания.

Формат вызова:

```
TEXT = '{$f=WORDDASH}'
```

Пример

```
# Необработанное событие:
# Example: S-y-s-t-e-m check_is finished
TEXT = 'Example: {datafield1=WORDDASH}{datafield2=WORDDASH}{datafield3=WORDDASH*}'
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "S-y-s-t-e-m",
#   "datafield2": "check_is",
#   "datafield3": "finished",
#   "time": "2017-10-18T12:00:00Z"}
```

Таблица 90. Приведение типов данных токена WORDDASH

Аргумент	Результат
String	String

5.6. Ключевое слово TABULAR

Ключевое слово **TABULAR** используется для объявления форматной строки правила нормализации событий, представленных в виде ассоциативного массив в формате JSON, например:

```
{
  "<Ключ 1>": "<Значение 1>",
  "<Ключ 2>": "<Значение 2>",
  "<Ключ 3>": [<Значение 31>, <Значение 32>, <Значение 33>]
}
```

При поступлении события в службу нормализации параметры события заносятся в словарь в памяти службы в виде ассоциативного массива, повторяющего структуру необработанного события. Значения параметров могут иметь типы данных Bool, String, Number или List.

Синтаксис форматной строки

В форматной строке вы можете указать параметры, которые должны находиться в событии (не равны null) для использования правила нормализации, или оставить ее пустой. Вы можете присвоить значения этих параметров переменным или полям нормализованного события.

Примечание. С помощью ключевого слова `COND` вы можете объявить дополнительное условие использования правила нормализации.

Формат вызова:

```
TABULAR = ' '
```

или

```
TABULAR = '<Ключ 1>,<Ключ 2>,...,{<Название поля>=<Ключ N-1>},{<$f=<Ключ N>}'
```

Обращение к словарю параметров

Из блока операторов правила нормализации или из условия `COND` вы можете обратиться к словарю для получения значения параметра необработанного события по его ключу. При указании ключа его нужно начинать со знака доллара (`$`).

Формат обращения к словарю:

```
$f=$<Ключ>
```

или

```
$f=$<Ключ>[Номер элемента списка]
```

если ключ содержит спецсимволы:

```
$f=${ ' <Ключ> ' }
```

Примечание. Отсчет элементов списка начинается от нуля, то есть для обращения к первому элементу нужно указать 0, ко второму — 1 и так далее.

Пример

```
# Необработанное событие:
# {"firstName":"Vasily",
#  "lastName":"Ivanov",
#  "birthDate":"1976-08-23T00:00:00",
#  "phoneNumbers":["812 123-1234","991 123-4567"],
#  "user_id/user_code":"74328042/VI43"}
TABULAR = '{datafield1=firstName}, phoneNumbers'
COND = ($lastName=="Ivanov" and month(datetime($birthDate))>=5)
datafield2 = $phoneNumbers[0]
datafield3 = $phoneNumbers[1]
datafield4 = $['user_id/user_code']
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "Vasily",
```

```
# "datafield2": "812 123-1234",
# "datafield3": "991 123-4567",
# "datafield4": "74328042/VI43",
# "time": "2017-10-18T12:00:00Z"}
```

5.7. Ключевое слово JSON

Ключевое слово JSON используется для объявления форматной строки правила нормализации событий в формате JSON. Событие представляет собой вложенную структуру параметров (многоуровневый ассоциативный массив), например:

```
{
  "<Ключ 11>": "<Значение 11>",
  "<Ключ 12>": {
    "<Ключ 21>": "<Значение 21>",
    "<Ключ 22>": {
      "<Ключ 31>": "<Значение 31>",
      "<Ключ 32>": [<Значение 321>, <Значение 322>, <Значение 323>]
    },
    "<Ключ 23>": "<Значение 23>"
  },
  "<Ключ 13>": "<Значение 13>"
}
```

При поступлении события в службу нормализации параметры события заносятся в словарь в памяти службы в виде многоуровневого ассоциативного массива, повторяющего структуру необработанного события. Значения параметров могут иметь типы данных Bool, String, Number или List.

Синтаксис форматной строки

В форматной строке вы можете указать одно условие использования правила нормализации или оставить ее пустой. С помощью условия в форматной строке вы можете проверить наличие параметра в событии или проверить его значение с типом данных Bool, String или Number (обращение к элементам списков не поддерживается). При указании составного ключа параметра используется точка.

Примечание. С помощью ключевого слова COND вы можете объявить дополнительное условие использования правила нормализации.

Формат вызова:

```
JSON = ''
```

или

```
JSON = '<Ключ 1x>.<Ключ 2x>....<Ключ Nx>'
```

или

```
JSON = '<Ключ 1x>.<Ключ 2x>....<Ключ Nx>=<Значение Nx>''
```

или

```
JSON = '<Ключ 1х>.<Ключ 3х>....<Ключ Nx>=<Значение Nx>'
```

Обращение к словарю параметров

Из блока операторов правила нормализации или из условия `COND` вы можете обратиться к словарю для получения значения параметра необработанного события по его ключу. При указании ключа его нужно начинать со знака доллара (\$). При указании составного ключа используются квадратные скобки (как при обращении к ассоциативному массиву).

Формат обращения к словарю:

```
$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"]
```

или

```
$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"] [<Номер элемента списка>]
```

если ключ содержит спецсимволы:

```
$f=$['<Ключ>']
```

Примечание. Отсчет элементов списка начинается от нуля, то есть для обращения к первому элементу нужно указать 0, ко второму — 1 и так далее.

Пример

```
# Необработанное событие:
# {"firstName": "Vasily",
# "lastName": "Ivanov",
# "birthDate": {"year": 1976, "month": 8, "day": null},
# "phoneNumbers": ["812 123-1234", "991 123-4567"],
# "user_id/user_code": "74328042/VI43"}
JSON = 'birthDate.year=1976'
COND = ($lastName=="Ivanov" and $birthDate["month"]>=5)
datafield1 = $phoneNumbers[0]
datafield2 = $phoneNumbers[1]
datafield3 = $['user_id/user_code']
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "812 123-1234",
#   "datafield2": "991 123-4567",
#   "datafield3": "74328042/VI43",
#   "time": "2017-10-18T12:00:00Z"}
```

5.8. Ключевое слово EVENTLOG

Ключевое слово `EVENTLOG` используется для объявления форматной строки правила нормализации событий из журнала Windows.

При получении MP 10 Collector события автоматически преобразуются в формат JSON, например:

```
{
  "Event": {
    "xmlns": "<Адрес пространства имен XML>",
    "System": {
      "Provider": {
        "Name": "<Имя провайдера>"
      },
      "EventID": {
        "text": "<Идентификатор события>",
        "Qualifiers": "0"
      },
      "Level": "4",
      "Task": "0",
      "Keywords": "0x8000000000000000",
      "TimeCreated": {
        "SystemTime": "<Дата и время регистрации>"
      },
      "EventRecordID": "72365",
      ...
      "Channel": "<Название журнала>",
      "Computer": "<Имя узла>",
      "Security": {
        "UserID": "S-1-5-18"
      }
    },
    "EventData": {
      "Data": [
        { "Name": "<Имя 1>", "text": "<Значение 1>" },
        { "Name": "Имя 2", "text": "Значение 2" },
        { "Name": "Имя 3", "text": "Значение 3" },
      ],
      ...
    }
  }
}
```

При поступлении события в службу нормализации событий выполняется автоматическая обработка события для упрощения его структуры. Из события удаляется общий для всех параметров ключ Event, удаляются ключи EventData и System, а относящиеся к ним параметры заносятся в словарь в виде многоуровневого ассоциативного массива. Если параметр Data

содержит вложенные параметры Name — text, то они преобразуются в ассоциативный массив «ключ — значение». Значения параметров в словаре могут иметь типы данных Bool, String, Number или List. Структура параметров в словаре для указанного выше события примет вид:

```
{
  "Provider": {
    "Name": "<Имя провайдера>"
  },
  "EventID": {
    "text": "<Идентификатор события>",
    "Qualifiers": "0"
  },
  "Level": "4",
  "Task": "0",
  "Keywords": "0x8000000000000000",
  "TimeCreated": {
    "SystemTime": "<Дата и время регистрации>"
  },
  "EventRecordID": "72365",
  ...
  "Channel": "<Название журнала>",
  "Computer": "<Имя узла>",
  "Security": {
    "UserID": "S-1-5-18"
  },
  "Data": [
    "<Имя 1>": "<Значение 1>",
    "<Имя 2>": "<Значение 2>",
    "<Имя 3>": "<Значение 3>"
  ]
}
```

Синтаксис форматной строки

Внимание! При указании ключей в правиле нормализации (в том числе в форматной строке) нужно использовать упрощенную структуру параметров из словаря, а не из исходного JSON-события.

В форматной строке вы можете указать одно условие использования правила нормализации или оставить ее пустой. С помощью условия в форматной строке вы можете проверить наличие параметра в событии или проверить его значение с типом данных Bool, String или Number (обращение к элементам списков не поддерживается). При указании составного ключа параметра используется точка.

В качестве условия нормализации события рекомендуется использовать идентификатор события, указанный в параметре EventID.text (EventID).

Примечание. С помощью ключевого слова `COND` вы можете объявить дополнительное условие использования правила нормализации.

Формат вызова:

```
EVENTLOG = 'EventID.text="Идентификатор события"'
```

или

```
EVENTLOG = ''
```

или

```
EVENTLOG = '<Ключ 1х>.<Ключ 2х>....<Ключ Nx>'
```

или

```
EVENTLOG = '<Ключ 1х>.<Ключ 2х>....<Ключ Nx>=<Значение Nx>''
```

или

```
EVENTLOG = '<Ключ 1х>.<Ключ 3х>....<Ключ Nx>=<Значение Nx>'
```

Обращение к словарю параметров

Из блока операторов правила нормализации или из условия `COND` вы можете обратиться к словарю для получения значения параметра необработанного события по его ключу. При указании ключа его нужно начинать со знака доллара (\$). При указании составного ключа используются квадратные скобки (как при обращении к ассоциативному массиву).

Формат обращения к словарю:

```
$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"]
```

или

```
$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"][Номер элемента списка]
```

Примечание. Отсчет элементов списка начинается от нуля, то есть для обращения к первому элементу нужно указать 0, ко второму — 1 и так далее.

Пример

```
# Необработанное событие:
# {"Event": {
#   "System": {
#     "Provider": {
#       "Name": "Developer Guide"
#     },
#     "EventID": {
#       "Qualifiers": "0",
#       "text": "777"
#     },
#     "TimeCreated": {
#       "SystemTime": "2016-04-14T16:35:40.000000000Z"
#     }
#   }
# }
```

```

# },
# "EventData": {
#   "Data": [{
#     "Name": "firstName",
#     "text": "Vasily"
#   }, {
#     "Name": "lastName",
#     "text": "Ivanov"
#   }, {
#     "Name": "birthDate",
#     "text": "1976-08-23T00:00:00"
#   }, {
#     "Name": "phoneNumbers",
#     "text": ["812 123-1234", "991 123-4567"]
#   }
# ]
# }
# }
# }
EVENTLOG = 'EventID.text="777"'
COND = ($Data["lastName"]=="Ivanov" and
        month(datetime($Data["birthDate"]))>=5)
datafield1 = $Data["phoneNumbers"][0]
datafield2 = $Data["phoneNumbers"][1]
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "812 123-1234",
#   "datafield2": "991 123-4567",
#   "time": "2017-10-18T12:00:00Z"}

```

5.9. Ключевое слово XML

Ключевое слово XML используется для объявления форматной строки правила нормализации событий в формате XML. Событие представляет собой вложенную структуру параметров (многоуровневый ассоциативный массив), например:

```

<<Ключ 1>>
  <<Ключ 21>><Значение 21><</Ключ 21>
  <<Ключ 22> <Имя атрибута 221>=<Значение атрибута 221> <Имя атрибута
222>=<Значение атрибута 222>><Значение 22><</Ключ 22>
  <<Ключ 23>>
    <<Ключ 31>><Значение 31><</Ключ 31>>
    <<Ключ 32>><Значение 321><</Ключ 32>>
    <<Ключ 32>><Значение 322><</Ключ 32>>
  <</Ключ 23>>
<</Ключ 1>>

```


При поступлении события в службу нормализации выполняется автоматическая обработка события. Для каждого параметра создается ассоциативный массив, в который заносятся значение параметра с ключом **!** и атрибуты параметра, ключами которых являются их названия с добавленным в начале названия символом **@**. Параметры события заносятся в словарь в виде многоуровневого ассоциативного массива в формате JSON, повторяющего структуру необработанного события. Значения параметров могут иметь тип данных String.

```
{
  "<Ключ 1>":{
    "<Ключ 21>":{
      "!=":"<Значение 21>"
    },
    "<Ключ 22>":{
      "!=":"<Значение 22>",
      "@<Имя атрибута 221>":"<Значение атрибута 221>",
      "@<Имя атрибута 222>":"<Значение атрибута 222>"
    },
    "<Ключ 23>":{
      "<Ключ 31>":{
        "!=":"<Значение 31>"
      },
      "<Ключ 32>":{
        "!=":"<Значение 32>"
      },
      "<Ключ 32>":{
        "!=":"<Значение 32>"
      },
    },
  },
}
```

Синтаксис форматной строки

В форматной строке вы можете указать параметр, который должен находиться в событии для использования правила нормализации, или оставить ее пустой. При указании составного ключа параметра используется точка.

Примечание. С помощью ключевого слова **COND** вы можете объявить дополнительное условие использования правила нормализации.

Формат вызова:

XML = ''

или

XML = '<Ключ 1х>.<Ключ 2х>....<Ключ Nx>'

Обращение к словарю параметров

Из блока операторов правила нормализации или из условия `COND` вы можете обратиться к словарю для получения значения параметра необработанного события по его ключу. При указании ключа его нужно начинать со знака доллара (\$). При указании составного ключа используются квадратные скобки (как при обращении к ассоциативному массиву).

Формат обращения к словарю:

`$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"]["!"]` — получение значения параметра,

`$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"]["@<Имя атрибута Nx>"]` — получение значения атрибута параметра,

`$f=$<Ключ 1х>["<Ключ 2х>"]...["<Ключ Nx>"] [<Номер параметра в списке>]` — получение значения параметра в списке параметров с одинаковыми названиями. При обращении к первому элементу в списке номер можно не указывать.

Примечание. Отсчет элементов списка начинается от нуля, то есть для обращения к первому элементу нужно указать 0, ко второму — 1 и так далее.

Альтернативный формат обращения к словарю:

`$f=$<Ключ 1х>/<Ключ 2х>/.../<Ключ Nx>` — получение значения параметра,

`$f=$<Ключ 1х>/<Ключ 2х>/.../<Ключ Nx>/@<Имя атрибута Nx>` — получение значения атрибута параметра,

`$f=$<Ключ 1х>/<Ключ 2х>/.../<Ключ Nx>/<Номер параметра в списке>` — получение значения параметра в списке параметров с одинаковыми названиями. При обращении к первому элементу в списке номер можно не указывать.

Пример

```
# Необработанное событие:
# <user>
#   <firstName>Vasily</firstName>
#   <lastName>Ivanov</lastName>
#   <birthDate>
#     <year>1976</year>
#     <month>8</month>
#     <day>1</day>
#   </birthDate>
#   <phoneNumbers>
#     <number type="landline">812 123-1234</number>
#     <number type="cell">991 123-4567</number>
#   </phoneNumbers>
# </user>
XML = "user.birthDate.year"
COND = $user["lastName"]["!"] == "Ivanov"
      and number($user["birthDate"]["month"]["!"]) >= 5
```

```

datafield1 = $user["phoneNumbers"]["number"][0]["@type"]
datafield2 = $user["phoneNumbers"]["number"][0]
datafield3 = $user/phoneNumbers/number/1/@type
datafield4 = $user/phoneNumbers/number/1
time = "2017-10-18T12:00:00.000Z"
# Нормализованное событие: {
#   "datafield1": "landline",
#   "datafield2": "812 123-1234",
#   "datafield3": "cell",
#   "datafield4": "991 123-4567",
#   "time": "2017-10-18T12:00:00.000Z"}

```

5.10. Команда drop

Команда `drop` используется для прекращения нормализации события. Нормализованное событие не создается.

Формат вызова:

```
drop
```

Пример

```

# Необработанное событие:
# Example: Click 2 times
TEXT = 'Example: {STRING} {$f=NUMBER} {STRING}'
if $f == 1 then
    datafield1 = "Click again"
elif $f == 2 then
    datafield1 = "OK"
else
    drop
endif
time = "2017-10-18T12:00:00Z"
# Нормализованное событие: {
#   "datafield1": "OK",
#   "time": "2017-10-18T12:00:00Z"}

```

5.11. Вложенное правило нормализации

В правило нормализации событий вы можете добавить несколько вложенных правил для распознавания фрагментов необработанного события, имеющих другой формат. Для создания вложенного правила нормализации нужно объявить его с помощью функции `submessage` и ввести блок операторов правила внутри конструкции из ключевых слов `subformula`, `endsubformula`.

submessage

Функция `submessage` используется для объявления вложенного правила нормализации для распознавания фрагмента необработанного события, имеющего другой формат. В первом аргументе нужно указать ключевое слово для формата фрагмента события, во втором — название вложенного правила нормализации, в третьем — имя переменной с фрагментом исходного события для распознавания с помощью вложенного правила нормализации.

Формат вызова:

```
submessage("<Формат фрагмента>", "<Название вложенного правила>", $fragment)
```

Таблица 91. Приведение типов данных функции `submessage`

Первый аргумент	Второй аргумент	Третий аргумент	Результат
String	String	String	—

subformula ... endsubformula

Конструкция из ключевых слов `subformula`, `endsubformula` используется для ограничения вложенного правила нормализации. В блоке операторов вложенного правила вы можете использовать функции, операторы и конструкции языка XPR, переменные (должны начинаться с `$`) и поля нормализованного события.

Формат вызова:

```
subformula '<Название вложенного правила>'
    <Ключевое слово вложенного правила> = '<Форматная строка вложенного правила>'
    COND = (<Условие вложенного правила>)
    <Блок операторов вложенного правила>
endsubformula
```

Таблица 92. Приведение типов данных функции `subformula`

Аргумент	Результат
String	—

Пример

```
# Необработанное событие:
# {"timegenerated":"2016-05-22 08:44:26.640",
# "userID":334,
# "userHost":"10.19.15.24",
# "UserData":"<user><firstName>Vasily</firstName><lastName>Ivanov</
lastName><birthDate><year>1976</year><month>8</month><day>1</day></
birthDate><phoneNumbers><number type=\"landline\">812 123-1234</number><number
type=\"cell\">991 123-4567</number></phoneNumbers></user>"}
TABULAR = 'UserData, {time=timegenerated},{event_src.hostname=userHost}'
submessage("XML", "Example_subformula", $UserData)
```

```

subformula 'Example_subformula'
    XML = 'user.birthDate.year'
    COND = $user["lastName"]["!"] == "Ivanov"
    datafield1 = $user["phoneNumbers"]["number"][0]
    datafield2 = $user/phoneNumbers/number/1
endsubformula
# Нормализованное событие: {
#   "event_src.hostname": "10.19.15.24",
#   "datafield1": "812 123-1234",
#   "datafield2": "991 123-4567",
#   "time": "2016-05-22T08:44:26.640Z"}

```

5.12. Пример правила нормализации

Используя примеры из раздела, с помощью утилиты PTSIEMSDK GUI вы можете проверить работу правила нормализации при получении исходных событий.

Исходные события

Для каждого исходного события в утилите PTSIEMSDK GUI необходимо создать отдельный тест. Код каждого из исходных событий, приведенных ниже, необходимо вставить в поле **Исходное событие** на вкладке **Редактор объектов**.

```

<45> Oct 18 2017 12:00:00 10.0.12.45 Session closed for user super_admin @ 192.0.2.11
Port:21 duration 00:00:55 protocol 17 ID32571
<45> Oct 18 2017 12:00:00 10.0.12.45 Session opened for user super_user @ 192.0.2.11
Port:22 protocol 17 ID32571

```

Правило нормализации

Код правила нормализации необходимо вставить в поле **Формула**:

```

TEXT = '{"<NUMBER">"?}{time=DATETIME} {event_src.ip=IPV4} Session {$s=WORD} for user
{object.account.domain=STRING} @ {dst.ip=IPV4} Port:{dst.port=NUMBER} {"duration"
$dur=DURATION?} protocol {protocol = NUMBER} ID{object.process.id=NUMBER}'

```

```

# Так как события одного источника отражают разные действия, используется конструкция
if ... then для обеспечения нормализации разных событий. На основании действия
заполняется поле action
if $s=="opened" then
    action = "open"
elif $s=="closed" then
    action = "close"
endif

```

```

subject = "account"
object = "connection"
status = "success"

```

```
duration = $dur # Переменной $dur присваивается значение токена DURATION, при этом
duration может быть null
```

```
importance = "info"
```

```
event_src.vendor = "Best Company"
event_src.title = "NG Session Controller"
event_src.category = "Other"
```

```
id = "PT_normalized_event_example"
```

Для получения нормализованных событий из исходных необходимо нажать **Запуск**.

Нормализованные события

После применения правила к исходным событиям в поле **Результат** отобразятся нормализованные события:

```
# Нормализованное событие 1:
{
  "subject": "account",
  "action": "close",
  "object": "connection",
  "status": "success",
  "dst.ip": "192.0.2.11",
  "dst.port": 21,
  "duration": 55,
  "event_src.category": "Other",
  "event_src.ip": "10.0.12.45",
  "event_src.title": "NG Session Controller",
  "event_src.vendor": "Best Company",
  "id": "PT_normalized_event_example",
  "importance": "info",
  "object.account.domain": "super_admin",
  "object.process.id": "32571",
  "protocol": "17",
  "time": "2017-10-18T12:00:00.000Z"
}
```

```
# Нормализованное событие 2:
{
  "subject": "account",
  "action": "open",
  "object": "connection",
  "status": "success",
  "dst.ip": "192.0.2.11",
  "dst.port": 22,
  "event_src.category": "Other",
```

```
"event_src.ip": "10.0.12.45",  
"event_src.title": "NG Session Controller",  
"event_src.vendor": "Best Company",  
"id": "PT_normalized_event_example",  
"importance": "info",  
"object.account.domain": "super_user",  
"object.process.id": "32571",  
"protocol": "17",  
"time": "2017-10-18T12:00:00.000Z"  
}
```

6. Создание макроса

В этом разделе описана структура макроса. Макрос представляет собой фрагмент кода, сохраненный в отдельном файле `<Название макроса>.flt`.

Создать новый макрос вы можете в веб-интерфейсе Knowledge Base (см. Руководство оператора) или с помощью утилиты PTSIEMSDK GUI. Добавление макросов, созданных с помощью утилиты, в MaxPatrol SIEM выполняется через веб-интерфейс Knowledge Base.

Вы можете вызывать макросы из условия отбора событий в инструкции `filter` директивы `event` в правилах обогащения и корреляции. Для вызова макроса нужно использовать его название с префиксом `filter::`.

Структура макроса

Макрос определяется директивой `filter`. Определение макроса состоит из ключевого слова `filter`, за которым следуют список параметров и условие макроса в фигурных скобках.

```
filter <Название макроса>(<Параметры макроса>){
    <Условие макроса>
}
```

Формат вызова макроса:

```
filter::<Название макроса>(<Параметры макроса>)
```

Директива filter

Директива `filter` предназначена для определения макроса. В директиве нужно указать название, параметры и условие макроса.

Название макроса может состоять из букв латинского алфавита, цифр и знака подчеркивания, должно начинаться с прописной буквы и быть уникальным в рамках всего набора названий макросов.

Параметры макроса используются при составлении условия макроса и при вызове макроса заменяются указанными аргументами (например, полями события, математическими или логическими выражениями). Имена параметров должны начинаться со знака доллара (\$). Параметры перечисляются через запятую, перед каждым параметром должен быть указан тип данных (`bool`, `number` или `string`). Макрос может не содержать параметров.

Условие макроса может состоять из нескольких логических выражений, объединенных операторами `or`, `and`. Если при проверке условие возвращает значение `True`, вызов макроса также возвращает `True`, в ином случае — `False`. В логических выражениях можно использовать:

- математические и логические операторы;
- поля события;
- литералы (с типом данных `Bool`, `Null`, `Number`, `String`);
- параметры макроса (должны начинаться со знака доллара \$);

- инлайн-запросы к табличным спискам (с префиксом `table::`);

Примечание. Макросы поддерживают работу только с табличными списками типа «справочник».

- вызовы других макросов, рекурсивные вызовы не допускаются (с префиксом `filter::`);
- функции языка XР.

Пример

```
filter Example(string $action){
    action == $action
    and table::Example_TableName {
        (column::username == subject.name and column::domain == null)
        or (column::username == subject.name and column::domain == subject.domain)
    }
}
```

См. также

[Префикс `table::`, инлайн-запрос \(см. раздел 4.9.1\)](#)

7. Создание шаблона исключений для заполнения табличного списка

После создания шаблона исключений для табличного списка и добавления его в Knowledge Base вы сможете по ссылкам из сводок о корреляционных событиях, зарегистрированных в MaxPatrol SIEM, автоматически добавлять исключения в белый список или удалять исключения из черного списка.

Для создания шаблона исключений используется разметка YAML. Шаблон состоит из секции `criteria`, которая должна содержать хотя бы одну секцию `filter` для условия применения шаблона и одну секцию `exceptions` для описания исключений.

Примечание. В шаблоне исключений вы можете использовать символ `#` для объявления однострочного комментария. Все последующие символы в строке игнорируются при использовании шаблона.

Структура шаблона исключений

```
criteria:
  - filter: <Условие шаблона 1>
    exceptions:
      <Ключ 11>
      ...
  - filter: <Условие шаблона 2>
    exceptions:
      <Ключ 21>
      ...
  ...
```

Секция `filter`

Секция предназначена для ввода условия применения шаблона исключений. В условии рекомендуется указывать системные названия правил корреляции, которые используют табличный список для хранения исключений.

Условие может состоять из одного или нескольких предикатов вида: `<Поле события><Оператор><Значение>`. В предикате можно использовать логические операторы равенства (`=`) и неравенства (`!=`) или оператор `in` для проверки на наличие значения поля события в списке. Для формирования списка с оператором `in` нужно использовать квадратные скобки. Между предикатами можно использовать операторы `and` или `or`. С оператором `or` также можно использовать круглые скобки.

Пример:

```
- filter: >
  correlation_name in [
    "Bruteforce_attempt_atomic",
    "Bruteforce_attempt_by_user",
```

```

    "Bruteforce_attempt_from_src",
    "Bruteforce_attempt_to_dst",
    "Bruteforce_success",
    "Cisco_FW_Execute_show_run_after_success_bruteforce",
    "User_enumeration"
]

```

Примечание. При использовании шаблона каждый разрыв строки после символа `>` и до пустой строки заменяется пробелом.

Секция `exceptions`

Секция предназначена для ввода описаний исключений. Для каждого исключения создается отдельная секция с уникальным в рамках шаблона названием. Это название является идентификатором (ключом) для сопоставления описания исключения и текста исключения, отображаемого для корреляционных событий в веб-интерфейсе MaxPatrol SIEM.

Каждая секция с описанием исключения должна содержать секцию `insert` для ввода правил заполнения колонок белого списка или секцию `remove` для ввода условия удаления записи из черного списка. Также секция `insert` может содержать секцию `filter` для ввода дополнительного условия для исключения.

Формат ввода:

```

<Ключ 1>
  filter: <Условие исключения 1>
  insert:
    <Название колонки 1>: <Значение 11>
    <Название колонки 2>: <Значение 12>
    ...
<Ключ 2>
  filter: <Условие исключения 2>
  remove:
    <Название колонки 1>: <Значение 21>
    <Название колонки 2>: <Значение 22>
    ...

```

Секция `filter` предназначена для ввода дополнительного условия для исключения. При проверке соответствия данных события условию исключения это условие с помощью оператора `and` объединяется с условием шаблона. В условии рекомендуется указывать данные события, по которым создается исключение, например имя узла, на котором зарегистрировано событие, или идентификатор учетной записи, с которой связана подозрительная активность.

Условие может состоять из одного или нескольких предикатов вида: `<Поле события><Оператор><Значение>`. В предикате можно использовать логические операторы равенства (`=`) и неравенства (`!=`) или оператор `in` для проверки на наличие значения поля

события в списке. Для формирования списка с оператором `in` нужно использовать квадратные скобки. Между предикатами можно использовать операторы `and` или `or`. С оператором `or` также можно использовать круглые скобки.

Секция `insert` предназначена для ввода правил заполнения колонок белого списка данными из события при добавлении исключения. Для каждой колонки нужно создать отдельную секцию с названием колонки и указать в ней правило ее заполнения. В секции вы можете указать:

- Конкретное значение, соответствующее типу данных колонки (в кавычках). Формат ввода:
`<Название колонки>:`
`"<Значение>"`

Примечание. Если данные в колонке могут принимать любые значения, необходимо ввести звездочку (*) для колонки с типом данных `String` и ноль для колонки с типом данных `Number`.

- Значение из поля события. Формат ввода:
`<Название колонки>:`
`field:<Название поля>`
- Значения из нескольких полей события, объединенные в одну строку с разделителем. Формат ввода:

```
<Название колонки>:
join:
  fields: [<Название поля 1>, <Название поля 2>]
  separator: "<Разделитель>"
```

- Значение из поля события в нижнем регистре. Формат ввода:
`<Название колонки>:`
`lower:`
`field: <Название поля>`
- Значение из поля события в верхнем регистре. Формат ввода:
`<Название колонки>:`
`upper:`
`field: <Название поля>`

Секция `remove` предназначена для ввода условия (по данным из события) удаления записей из черного списка. Для каждой колонки табличного списка нужно создать отдельную секцию с названием колонки и указать в ней условие на соответствия данных события значению в колонке. При удалении исключения указанные для каждой колонки условия объединяются с помощью оператора `and`. Первая запись, удовлетворяющая такому составному условию, удаляется из табличного списка. При составлении условия для колонки вы можете указывать конкретные значения или значения из полей события, используя указанный выше для секции `insert` формат ввода.

Пример:

```
exceptions:
  # Фильтрация по пользователю
  By_object_user:
    filter: object.name = "account"
```

```

insert:
  host: "*"
  rule: "*"
  specific_value: "*"
  user_domain:
    lower:
      field: object.domain
  user_id:
    lower:
      field: object.id
  user_name:
    lower:
      field: object.name
# Фильтрация по пути к объекту
By_object_path_name_in_specific_value_for_rule:
  filter: >
    correlation_name in [
      "Detect_Abusing_CredSSP",
      "Detect_Account_Discovery"
    ]
insert:
  host: "*"
  rule:
    field: correlation_name
  specific_value:
    lower:
      join:
        fields: [object.path, object.name]
        separator: ""
  user_domain: "*"
  user_id: "*"
  user_name: "*"

```

8. Создание правила агрегации

В этом разделе описаны директивы и инструкции, используемые при создании правил агрегации, даны примеры и рекомендации.

Для отладки нового правила вы можете использовать утилиты SDK или утилиту PTSIEMSDK GUI. Добавить новое правило в MaxPatrol SIEM вы можете через веб-интерфейс Knowledge Base (см. Руководство оператора).

В этом разделе

[Алгоритм работы службы агрегации \(см. раздел 8.1\)](#)

[Структура правила агрегации \(см. раздел 8.2\)](#)

[Директива event. Объявление события, подлежащего агрегации \(см. раздел 8.3\)](#)

[Директива aggregate. Условие правила агрегации \(см. раздел 8.4\)](#)

8.1. Алгоритм работы службы агрегации

Из потока нормализованных и корреляционных событий служба агрегации отбирает те, которые подлежат агрегации. Эти события должны удовлетворять условию отбора, указанному при объявлении события хотя бы в одном из правил агрегации (в директиве `event`). Для каждого правила агрегации отобранные события разделяются на потоки событий с одинаковыми значениями всех полей, указанных при объявлении события (в инструкции `key` директив `event`).

Условие правила агрегации (в директиве `aggregate`) содержит описание последовательности событий, при появлении которой регистрируется агрегированное событие. При составлении одной последовательности используются события из одного потока, одновременно могут составляться несколько последовательностей для разных потоков. Каждое отобранное событие используется при составлении последовательности, указанной в условии правила агрегации.

При регистрации первого события в последовательности начинается отсчет времени регистрации всей последовательности и количества событий в последовательности.

Если время регистрации всей последовательности не превышено, выполняется подсчет количества событий в последовательности:

- Если номер события меньше числа пропускаемых событий, событие остается в потоке событий. Количество событий в последовательности увеличивается на единицу.
- Если номер события равен или больше числа пропускаемых событий, но меньше числа событий для регистрации агрегированного события, событие удаляется из потока событий. Количество событий в последовательности увеличивается на единицу.
- Если номер события равен числу событий для регистрации агрегированного события, событие удаляется из потока событий, регистрируется агрегированное событие.

Если время регистрации всей последовательности превышено, то в зависимости от количества событий в последовательности принимается решение о регистрации агрегированного события:

- Если количество событий меньше числа пропускаемых событий, агрегированное событие не регистрируется.
- Если количество событий больше числа пропускаемых событий, регистрируется агрегированное событие.

См. также

[Этапы обработки событий \(см. раздел 2\)](#)

[Программные компоненты для обработки событий \(см. раздел 3\)](#)

8.2. Структура правила агрегации

Правило агрегации должно состоять из директив `event` и `aggregate`.

Директива `event` используется для объявления события, подлежащего агрегации. Директива должна содержать название события, условие его отбора и может содержать инструкцию `key` для указания полей, по совокупности значений которых разделяется поток событий.

Директива `aggregate` используется для описания правила агрегации. Директива должна содержать уникальное название правила агрегации и условие его выполнения. В условии правила агрегации нужно указать, какое число раз и в течение какого времени объявленное событие должно появиться в потоке событий для регистрации агрегированного события.

Примечание. Все поля агрегированного события заполняются автоматически. В событии указываются все поля, перечисленные в инструкции `key` и указанные в условии отбора события директивы `event`. В поле `aggregation_name` указывается название правила агрегации, по которому зарегистрировано событие, в поле `count` — количество событий в последовательности, в поле `start_time` — время регистрации первого события в последовательности.

Структура правила агрегации:

```
event <Название события>:
    key:
        <Поле события 1>, <Поле события 2>, ...
        <Условие отбора события>
aggregate <Название правила>: <Условие правила агрегации>
```

8.3. Директива event. Объявление события, подлежащего агрегации

Директива `event` предназначена для объявления события, подлежащего агрегации. Директива должна содержать название события (должно начинаться с прописной буквы латинского алфавита), условие его отбора из потока нормализованных (и корреляционных) событий и может содержать инструкцию `key` для указания полей события, по совокупности значений которых для каждого правила агрегации поток отобранных событий разделяется на несколько потоков.

Формат вызова:

```
event <Название события>:
    key:
        <Поле события 1>, <Поле события 2>, ...
        <Условие отбора события>
```

Условие отбора события

Условия отбора события может состоять из нескольких пар «поле события — значение», записанных каждая в отдельной строке. Для ввода значения поля события в паре используется присваивание (вместо логического равенства «==»). Все пары в условии объединяются операцией конъюнкции (логическое И). Если условие отбора возвращает `True`, событие используется при составлении последовательности в условии правила агрегации.

Инструкция key

В инструкции `key` вы можете в одной строке, через запятую указать поля события, по совокупности значений которых в каждом правиле агрегации поток отобранных событий разделяется на несколько потоков.

Примечание. Если при разделении на несколько потоков значения двух полей события в инструкции `key` могут взаимозаменять друг друга, вы можете перед такими полями добавить восклицательный знак. Например, при записи `!src.host, !dst.host` в один поток попадут события, соответствующие потокам `src.host, dst.host` и `dst.host, src.host`.

Пример

```
event Host_info:
    key:
        event_src.host, event_src.hostname, event_src.ip, object, object.value,
        object.group, object.domain, action, status, importance, event_src.category,
        event_src.vendor, event_src.title
        msgid = "OTHER_EV_UPDATE_INFO_ABOUT_HOST"
        object.type = "host"
```


8.4. Директива aggregate. Условие правила агрегации

Директива `aggregate` предназначена для описания правила агрегации. Директива должна содержать название правила агрегации и условие его выполнения. Название правила агрегации должно быть уникальным в рамках всего набора правил агрегации (может состоять из букв латинского алфавита, цифр, знаков подчеркивания и точки, должно начинаться с прописной буквы). В условии правила агрегации нужно указать:

- название события (событие должно быть объявлено в директиве `event`);
- максимальное количество появлений этого события в последовательности, при котором событие будет пропускаться;
- общее количество появлений этого события в последовательности;
- максимальное время регистрации всей последовательности. Для ввода времени используйте оператор отсчета времени `within` и сокращения: `s` — секунды, `m` — минуты, `h` — часы, `d` — дни. Время регистрации всей последовательности рассчитывается как разница между значениями полей `time` последнего и первого событий последовательности.

Условие правила агрегации возвращает `True`, если регистрируются все события последовательности за время, не превышающее указанного в операторе `within`, иначе — `False`. При выполнении условия регистрируется агрегированное событие.

Примечание. Вы можете использовать правило агрегации для удаления из потока событий всех событий, удовлетворяющих условию отбора, указанному в директиве `event`. Формат вызова: `aggregate <Название правила>: <Название события>[-1, <Любое число>]`.

Формат вызова:

```
aggregate <Название правила>: <Название события>[<Количество пропускаемых событий>,
<Общее количество событий>] within <Время последовательности>
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом агрегации — в поле **Формула**, после чего нажать **Запустить**. Агрегированное событие отобразится в поле **Результат**.

Исходные события:

```
{ "time": "2017-10-18T12:00:30.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 21, "protocol": "17", "object.process.id": "32571", "action": "close", "subject": "account", "object": "connection", "status": "success", "duration": 55, "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "PT_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11", "taxonomy_version": "26.0.220-release-26.0", "generator.type": "logcollector", "generator.version": "N26.2.3090", "origin_app_id": "00000005-d160-0de3-f000-0000abbac07a", "primary_siem_app_id": "00000005-d160-0de3-f000-0000abbac07b", "uuid": "00000005-9f0a-0e90-f000-0000abbac075",
```

```
"scope_id":"00000005-d160-0de3-f000-0000abbac07d","siem_id":"00000005-d160-0de3-f000-0000abbac07e","site_id":"00000005-d160-0de3-f000-0000abbac07f","tenant_id":"00000005-d160-0de3-f000-0000abbac07g"}
{"time":"2017-10-18T12:00:40.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":23,"protocol":"17","object.process.id":"32571","action":"close","subject":"account","object":"connection","status":"success","duration":55,"importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"PT_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11","taxonomy_version":"26.0.220-release-26.0","generator.type":"logcollector","generator.version":"N26.2.3090","origin_app_id":"00000005-d160-0de3-f000-0000abbac07a","primary_siem_app_id":"00000005-d160-0de3-f000-0000abbac07b","uuid":"00000005-9f0a-0e90-f000-0000abbac076","scope_id":"00000005-d160-0de3-f000-0000abbac07d","siem_id":"00000005-d160-0de3-f000-0000abbac07e","site_id":"00000005-d160-0de3-f000-0000abbac07f","tenant_id":"00000005-d160-0de3-f000-0000abbac07g"}
{"time":"2017-10-18T12:00:50.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":23,"protocol":"17","object.process.id":"32571","action":"close","subject":"account","object":"connection","status":"success","duration":55,"importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"PT_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11","taxonomy_version":"26.0.220-release-26.0","generator.type":"logcollector","generator.version":"N26.2.3090","origin_app_id":"00000005-d160-0de3-f000-0000abbac07a","primary_siem_app_id":"00000005-d160-0de3-f000-0000abbac07b","uuid":"00000005-9f0a-0e90-f000-0000abbac077","scope_id":"00000005-d160-0de3-f000-0000abbac07d","siem_id":"00000005-d160-0de3-f000-0000abbac07e","site_id":"00000005-d160-0de3-f000-0000abbac07f","tenant_id":"00000005-d160-0de3-f000-0000abbac07g"}
```

Правило агрегации:

```
event Event:
  key:
    event_src.vendor
  dst.host = "192.0.2.11"
  dst.port = 23
```

aggregate Aggregation_rule: Event[0,2] within 1m # Первое событие будет пропущено, агрегированы будут только два следующих события, так как они удовлетворяют условию

Результат работы правила:

```
# Исходное событие 1:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "dst.host": "192.0.2.11",
#   "dst.port": 21,
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "event_src.vendor": "Best Company",
```

```

#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example",
#   "origin_app_id": "00000005-d160-0de3-f000-0000abbac07a",
#   "primary_siem_app_id": "00000005-d160-0de3-f000-0000abbac07b",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac075",
#   "scope_id": "00000005-d160-0de3-f000-0000abbac07d",
#   "siem_id": "00000005-d160-0de3-f000-0000abbac07e",
#   "site_id": "00000005-d160-0de3-f000-0000abbac07f",
#   "tenant_id": "00000005-d160-0de3-f000-0000abbac07g"
#}
# Исходное событие 2:
#{
#   "time": "2017-10-18T12:00:30.000Z",
#   "dst.host": "192.0.2.11",
#   "dst.port": 23,
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac076",
#   ...
#}
# Исходное событие 3:
#{
#   "time": "2017-10-18T12:00:50.000Z",
#   "dst.host": "192.0.2.11",
#   "dst.port": 23,
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "uuid": "00000005-9f0a-0e90-f000-0000abbac077",
#   ...
#}
#
# Агрегированное событие:
#{
#   "dst.host": "192.0.2.11",
#   "dst.port": 23,
#   "aggregation_name": "Aggregation_rule",
#   "count": 2,
#   "start_time": "2017-10-18T12:00:40.000Z",
#   "time": "2017-10-18T12:00:50.000Z",

```

```
# ...  
# }
```

9. Создание правила обогащения

В этом разделе описаны директивы и инструкции, используемые при создании правил обогащения, даны примеры и рекомендации.

Справочная информация, используемая в правилах, может храниться в табличных списках. Подробную информацию о табличных списках и работе с ними вы можете найти в разделах «Работа с табличными списками» и «Настройка справочников для обработки событий» Руководства оператора.

Для отладки нового правила вы можете использовать утилиты SDK или утилиту PTSIEMSDK GUI. Добавить новое правило в MaxPatrol SIEM вы можете через веб-интерфейс Knowledge Base (см. Руководство оператора).

В этом разделе

[Структура правила обогащения \(см. раздел 9.1\)](#)

[Директива event. Объявление события для обогащения \(см. раздел 9.2\)](#)

[Директива enrichment. Название правила обогащения \(см. раздел 9.3\)](#)

[Директива enrich. Обработчик события \(см. раздел 9.4\)](#)

[Директива query. Объявление запроса к табличному списку \(см. раздел 9.5\)](#)

9.1. Структура правила обогащения

Правило обогащения состоит из последовательности директив, содержащих инструкции. Для работы правила обогащения необходимо указать обязательные директивы и инструкции.

Директива `event` предназначена для объявления события для обогащения. В одном правиле можно использовать несколько директив `event` для объявления нескольких событий с разными названиями. Директива должна содержать инструкцию `filter` для настройки условия отбора события.

Директива `enrichment` предназначена для объявления правила обогащения. В директиве нужно указать название правила, уникальное в рамках всех правил обогащения.

Директива `enrich` (обработчик события) предназначена для выполнения инструкций при регистрации события, указанного в директиве. Директиву `enrich` нужно указать для каждого объявленного события. Директива может содержать инструкции:

- `enrich_fields` — для заполнения полей события значениями, указанными в блоке операторов инструкции или полученными из табличного списка; также в инструкции можно настроить регистрацию инцидента по событию;
- `insert_into` — для записи в табличный список значений полей события;
- `remove_from` — для удаления строк из табличного списка.

Правило обогащения может содержать директиву `query` для объявления запроса к табличному списку, данными из которого обогащаются события. В одном правиле можно объявить несколько запросов с разными названиями.

Структура правила обогащения:

```

query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
  <Условие запроса>
}
event <Название события>:
  filter {
    <Условие отбора события>
  }
enrichment <Название правила обогащения>
  enrich <Название события>:
    enrich_fields {
      <Блок операторов для обогащения события>
      <Блок для настройки инцидента>
    }
    insert_into <Название табличного списка> {
      <Блок операторов insert_into>
    }
    remove_from <Название табличного списка> {
      <Условие для удаления>
    }
  }

```

9.2. Директива `event`. Объявление события для обогащения

Директива `event` предназначена для объявления события для обогащения. В каждом правиле обогащения нужно объявить хотя бы одно событие.

В директиве нужно указать название события и условие отбора события. Название должно начинаться с прописной буквы латинского алфавита и быть уникальным в рамках одного правила обогащения. Для указания условия отбора используется инструкция `filter`.

Инструкция `filter`

Инструкция `filter` директивы `event` предназначена для настройки условия отбора события. Если условие возвращает значение `True`, выполняется обогащение события. При написании условия вы можете использовать:

- математические и логические операторы;
- поля события;
- литералы (с типом данных `Bool`, `Null`, `Number`, `String`);
- инлайн-запросы к табличным спискам (с префиксом `table::`);

Примечание. Макросы поддерживают работу только с табличными списками типа «справочник».

- вызовы макросов (с префиксом `filter::`);
- функции языка XPL, в том числе для выполнения запроса к [табличному списку](#) (см. [раздел 4.9.2](#)).

Формат вызова:

```
event <Название события>:
  filter {
    <Условие отбора события>
  }
```

См. также

[Префикс table::, инлайн-запрос \(см. раздел 4.9.1\)](#)

[Создание макроса \(см. раздел 6\)](#)

9.3. Директива `enrichment`. Название правила обогащения

Директива `enrichment` предназначена для объявления названия правила обогащения. Название должно быть уникальным в рамках всего набора названий правил обогащения (может состоять из букв латинского алфавита, цифр, знаков подчеркивания и точки, должно начинаться с прописной буквы).

Формат вызова:

```
enrichment <Название правила обогащения>
```

9.4. Директива `enrich`. Обработчик события

Директива `enrich` (обработчик события) предназначена для выполнения блока операторов при регистрации указанного события. Событие должно быть объявлено в директиве `event`. В блоке операторов директивы `enrich` вы можете использовать инструкцию `enrich_fields`, `insert_into` и `remove_from`.

Формат вызова:

```
enrich <Название события>:
  enrich_fields {
    <Блок операторов для обогащения события>
    <Блок для настройки инцидента>
  }
  insert_into <Название табличного списка> {
    <Блок операторов insert_into>
  }
  remove_from <Название табличного списка> {
```

```

    <Условие для удаления>
  }

```

В этом разделе

Инструкция `enrich_fields`. Обогащение события (см. раздел 9.4.1)

Инструкция `insert_into`. Запись в табличный список (см. раздел 9.4.2)

Инструкция `insert_into`. Условная запись в табличный список (см. раздел 9.4.3)

Инструкция `insert_into`. Арифметические операции при записи в табличный список (см. раздел 9.4.4)

Инструкция `remove_from`. Удаление строк из табличного списка (см. раздел 9.4.5)

Команда `drop` (см. раздел 9.4.6)

9.4.1. Инструкция `enrich_fields`. Обогащение события

Инструкция `enrich_fields` директивы `enrichment` предназначена для заполнения полей указанного события значениями из табличного списка. В блоке операторов инструкции `enrich_fields` вы можете использовать переменные (названия нужно начинать со знака доллара), поля указанного в инструкции события, функции и операторы языка XPL или литералы с типом данных, соответствующим заполняемым полям.

Формат вызова:

```

enrich <Название события>
  enrich_fields {
    <Блок операторов для обогащения события>
    <Блок для настройки инцидента>
  }

```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**.

Исходное событие:

```

{"time":"2017-10-18T12:00:00.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":21,"protocol":"17","object.process.id":"32571","action":"close","subject":"account","object":"connection","status":"success","duration":55,"importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"ID_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11"}

```


Правило обогащения:

```

event AnythingWithProtocol:
  filter {
    protocol != null and protocol != ""
  }
  enrichment Protocol_number_to_name
  enrich AnythingWithProtocol:
    enrich_fields {
      switch protocol # Применяется конструкция switch ... case для смены
значения 17 поля protocol на имя протокола
        case "1" protocol = "ICMP" # Internet Control Message
        case "2" protocol = "IGMP" # Internet Group Management
        case "6" protocol = "TCP" # Transmission Control
        case "17" protocol = "UDP" # User Datagram
      endswitch
    }

```

Результат работы правила:

```

# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "protocol": "17",
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example"
#}
#
# Обогащенное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "protocol": "UDP",
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example",

```

```
# ...
#}
```

9.4.2. Инструкция `insert_into`. Запись в табличный список

Внимание! Вы можете изменять только табличные списки, предназначенные для правил обогащения. Назначение табличного списка вы можете выбрать при его создании в MaxPatrol SIEM.

Инструкция `insert_into` используется для записи в табличный список. Для использования инструкции нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- имена колонок (с префиксом `column: :`), в которые нужно добавить значения;
- значения ячеек для добавления в табличный список (тип данных должен совпадать с типом данных колонки табличного списка). При вычислении значения вы можете использовать поля события, функции и операторы языка XPL.

Примечание. Если указано значение для ключевой колонки табличного списка и оно есть в табличном списке — значения строки перезаписывается, если нет — добавляется новая строка в конец табличного списка.

При добавлении строки в табличный список нужно указать значения ячеек для всех колонок. Если значения для колонок не указаны, результат записи зависит от свойств колонки, настроенных при создании табличного списка: если колонка может содержать `null` — в добавляемые ячейки будет записано значение `null`, если нет — появится сообщение об ошибке.

Формат вызова:

```
enrichment <Название правила обогащения>
  enrich <Название события>:
    insert_into <Название табличного списка> {
      column::<Имя колонки 1> = <Значение для записи 1>
      column::<Имя колонки 2> = <Значение для записи 2>
      ...
    }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_EnrichmentRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходные события:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 24,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}

{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 25,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 4684,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило обогащения:

```
event Event: # Определение событий
  filter {
    dst.ip == "192.0.2.11" # Фильтрация событий по значению dst.ip
  }

enrichment Example
  enrich Event:
    insert_into Tabular_List_EnrichmentRule { # Присваивание значений из событий
      столбцам табличного списка
      column::ip = string(dst.ip)
      column::port = number(dst.port)
      column::action = string(action)
    }
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#     {
#       "ip": "192.0.2.11",
#       "port": 21,
#       "action": "close"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 22,
#       "action": "open"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 23,
#       "action": "close"
#     }
#   ]
# }
```

```

#     }
#   ]
#}
#
# Нормализованное событие 1:
# {
#   "event_src.vendor":"Best Company",
#   "time":"2017-10-18T12:00:00Z",
#   "uuid":"00000005-9f0a-0e90-f000-0000abbac077",
#   "dst.ip":"192.0.2.11",
#   "dst.port":24,
#   "action":"open",
#   ...}
# Нормализованное событие 2:
# {
#   "event_src.vendor":"Best Company",
#   "time":"2017-10-18T12:00:00Z",
#   "uuid":"00000005-9f0a-0e90-f000-0000abbac078",
#   "dst.ip":"192.0.2.11",
#   "dst.port":25,
#   "action":"close",
#   ...}
#
# Конечное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#     {
#       "ip": "192.0.2.11",
#       "port": 21,
#       "action": "open"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 22,
#       "action": "open"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 23,
#       "action": "close"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 24,
#       "action": "open"
#     },
#   ],
# }

```

```
#      {
#          "ip": "192.0.2.11",
#          "port": 25,
#          "action": "close"
#      }
#  ]
#}
```

Табличный список Tabular_List_EnrichmentRule в формате YAML:

```
name: Tabular_List_EnrichmentRule
fillType: EnrichmentRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- action:
    index: true
    nullable: false
    primaryKey: false
    type: String
```

Табличный список Tabular_List_EnrichmentRule в формате JSON:

```
"Tabular_List_EnrichmentRule": {
  "description": "Table list for enrichment rule - main",
  "fields": {
    "_id": {
      "index": true,
```

```

        "nullable": false,
        "primaryKey": false,
        "type": "Number",
        "unique": true
    },
    "_last_changed": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Datetime"
    },
    "action": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    }
},
"fillType": "EnrichmentRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",

```

```

        "ip",
        "port",
        "action",
        "_last_changed",
        "_id"
    ],
    "ttl": 259200,
    "type": 2,
    "typicalSize": 80000
}

```

9.4.3. Инструкция `insert_into`. Условная запись в табличный список

Внимание! Вы можете изменять только табличные списки, предназначенные для правил обогащения. Назначение табличного списка вы можете выбрать при его создании в MaxPatrol SIEM.

Инструкция `insert_into` может использоваться для записи в табличный список при выполнении условия. Для использования инструкции нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- условие записи в табличный список. Если условие возвращает `true`, выполняется запись в табличный список, если `false` — не выполняется. Условие может состоять из поля события с типом данных `Bool` или переменной, объявленной в инструкции `enrich_fields` и содержащей данные с типом `Bool`;
- имена колонок (с префиксом `column::`), в которые нужно добавить значения;
- значения ячеек для добавления в табличный список (тип данных должен совпадать с типом данных колонки табличного списка). При вычислении значения вы можете использовать поля события, функции и операторы языка XPL.

Примечание. Если указано значение для ключевой колонки табличного списка и оно есть в табличном списке — значения строки перезаписывается, если нет — добавляется новая строка в конец табличного списка.

При добавлении строки в табличный список нужно указать значения ячеек для всех колонок. Если значения для колонок не указаны, результат записи зависит от свойств колонки, настроенных при создании табличного списка: если колонка может содержать `null` — в добавляемые ячейки будет записано значение `null`, если нет — появится сообщение об ошибке.

Формат вызова:

```

enrichment <Название правила обогащения>
    enrich <Название события>:
        insert_into <Название табличного списка> if <Условие записи> {
            column::<Имя колонки 1> = <Значение для записи 1>
            column::<Имя колонки 2> = <Значение для записи 2>
        }
    }

```

```
...
}
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_EnrichmentRule (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходные события:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 24,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 25,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 4684,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило обогащения:

```
event Event:
  filter {
    dst.ip == "192.0.2.11" # Фильтрация событий по значению 192.0.2.11 поля
    dst.ip
  }

enrichment Example
  enrich Event:
    enrich_fields {
      # Формирование условия для записи в табличный список, присваивание
      # переменной булево значения по полю dst.ip
      $f_cond = bool(dst.ip == "192.0.2.11")
    }
    # Добавление данных в табличный список, если переменная удовлетворяет
    # описанному ранее условию
    insert_into Tabular_List_EnrichmentRule if $f_cond {
      column::ip = string(dst.ip)
      column::port = number(dst.port)
      column::action = string(action)
    }
```


Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "action": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "action": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "action": "close"
#       }
#   ]
#}

# Исходные события:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 24,
#   "protocol": "17",
#   "object.process.id": "32571",
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example",
#   "event_src.host": "10.0.12.45",
#   "dst.host": "192.0.2.11"
#}

#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
```

```

#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 25,
#   "protocol": "17",
#   ...
#}
# Конечное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "action": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "action": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "action": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 24,
#           "action": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 25,
#           "action": "close"
#       }
#   ]
#}

```

Табличный список Tabular_List_EnrichmentRule в формате YAML:

```

name: Tabular_List_EnrichmentRule
fillType: EnrichmentRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:

```

```

    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
      - ip
      - port
  - ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
  - port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
  - action:
    index: true
    nullable: false
    primaryKey: false
    type: String

```

Табличный список Tabular_List_EnrichmentRule в формате JSON:

```

"Tabular_List_EnrichmentRule": {
  "description": "Table list for enrichment rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "action": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    }
  }
}

```

```

    },
    "complex_key": {
      "compositeFields": [
        "ip",
        "port"
      ],
      "index": false,
      "nullable": false,
      "primaryKey": true,
      "type": "composite",
      "unique": true
    },
    "ip": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "port": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    }
  },
  "fillType": "EnrichmentRule",
  "maxSize": 100000,
  "objectId": "LOC-TL-1",
  "order": [
    "complex_key",
    "ip",
    "port",
    "action",
    "_last_changed",
    "_id"
  ],
  "ttl": 259200,
  "type": 2,
  "typicalSize": 80000
}

```

9.4.4. Инструкция insert_into. Арифметические операции при записи в табличный список

Внимание! Вы можете изменять только табличные списки, предназначенные для правил обогащения. Назначение табличного списка вы можете выбрать при его создании в MaxPatrol SIEM.

Для выполнения арифметических операций при записи в табличный список в инструкции `insert_into` используются функции, указанные в таблице ниже. Для использования функций нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- имя ключевой колонки (с префиксом `column::`) в указанном табличном списке;

Примечание. Какая из колонок является ключевой, определяется при создании табличного списка. В отличие от обычной колонки, все значения ключевой колонки уникальны в рамках одного табличного списка.

- значение ключевой колонки для выбора строки, в которую будут записаны значения;

Примечание. Если ключевая колонка не содержит указанного в инструкции значения, в табличный список добавляется строка с этим ключом.

- имя колонки (с префиксом `column::`), в ячейку которой требуется записать результат применения функции;
- значение для записи (с типом данных `Number`), к которому применяется функция.

Примечание. Если к ячейке табличного списка, содержащей максимальное значение, поддерживаемое для [типа данных Number](#) (см. [приложение A](#)), будет применена функция `insert_inc`, значение ячейки не изменится. Если к ячейке табличного списка, содержащей минимальное значение, будет применена функция `insert_dec`, значение ячейки также не изменится.

Формат вызова:

```
enrichment <Название правила обогащения>
  enrich <Название события>:
    insert_into <Название табличного списка> {
      <Имя ключевой колонки> = <Значение ключа>
      insert_min(column::<Имя колонки 1>, <Значение для записи 1>)
      insert_max(column::<Имя колонки 2>, <Значение для записи 2>)
      insert_inc(column::<Имя колонки 3>)
      insert_dec(column::<Имя колонки 4>)
    }
```

Таблица 93. Функции для выполнения арифметических операций при записи в табличный список в директиве insert_into

Функция	Арифметическая операция
insert_min(column::<Имя колонки>, \$f)	Сравниваются два значения: указанное во втором аргументе функции и содержащееся в ячейке колонки, указанной в первом аргументе. В ячейку заносится меньшее из значений. Если ячейка содержит null или в табличный список добавляется новая строка, в ячейку заносится значение, указанное во втором аргументе функции
insert_max(column::<Имя колонки>, \$f)	Сравниваются два значения: указанное во втором аргументе функции и содержащееся в ячейке колонки, указанной в первом аргументе. В ячейку заносится большее из значений. Если ячейка содержит null или в табличный список добавляется новая строка, в ячейку заносится значение, указанное во втором аргументе функции
insert_inc(column::<Имя колонки>)	Значение ячейки в указанной колонке увеличивается на единицу. Если в табличный список добавляется новая строка, в ячейку заносится 1
insert_dec(column::<Имя колонки>)	Значение ячейки в указанной колонке уменьшается на единицу. Если в табличный список добавляется новая строка, в ячейку заносится -1

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_EnrichmentRule_Arithm (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{ "time": "2017-10-18T12:00:00.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 21, "protocol": "17", "object.process.id": "32571", "action": "close", "subject": "account", "object": "connection", "status": "success", "duration": 55, "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11" }
```

Правило обогащения:

```
event Event:
  filter {
    dst.ip == "192.0.2.11"
  }
```

enrichment Example

enrich Event:

```
insert_into Tabular_List_EnrichmentRule_Arithm { # Добавление строк в
табличный список Tabular_List_EnrichmentRule_Arithm с вычислением минимального,
максимального значений и увеличением счетчика
    column::ip = string(dst.ip)
    column::port = number(dst.port)
    insert_min(column::minduration, duration)
    insert_max(column::maxduration, duration)
    insert_inc(column::count)
}
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule_Arithm": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "minduration": 12,
#           "maxduration": 51,
#           "count": 11
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "minduration": 63,
#           "maxduration": 71,
#           "count": 23
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "minduration": 9,
#           "maxduration": 35,
#           "count": 6
#       }
#   ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
```

```

#   "dst.port": 22,
#   "protocol": "17",
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}
# Конечное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule_Arithm": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "minduration": 12,
#           "maxduration": 55,
#           "count": 12
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "minduration": 63,
#           "maxduration": 71,
#           "count": 23
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "minduration": 9,
#           "maxduration": 35,
#           "count": 6
#       }
#   ]
#}

```

Табличный список Tabular_List_EnrichmentRule_Arithm в формате YAML:

```

name: Tabular_List_EnrichmentRule_Arithm
fillType: EnrichmentRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false

```



```

    primaryKey: true
    type: composite
    unique: true
    compositeFields:
      - ip
      - port
  - ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
  - port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
  - minduration:
    index: true
    nullable: false
    primaryKey: false
    type: Number
  - maxduration:
    index: true
    nullable: false
    primaryKey: false
    type: Number
  - count:
    index: true
    nullable: false
    primaryKey: false
    type: Number

```

Табличный список Tabular_List_EnrichmentRule_Arithm в формате JSON:

```

"Tabular_List_EnrichmentRule_Arithm": {
  "description": "Table list enrichment rule - arithmetic operations",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,

```

```

        "primaryKey": false,
        "type": "Datetime"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "count": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "maxduration": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "minduration": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    }
},
"fillType": "EnrichmentRule",

```

```

    "maxSize": 100000,
    "objectId": "LOC-TL-1",
    "order": [
      "complex_key",
      "ip",
      "port",
      "minduration",
      "maxduration",
      "count",
      "_last_changed",
      "_id"
    ],
    "ttl": 259200,
    "type": 1,
    "typicalSize": 80000
  }

```

9.4.5. Инструкция `remove_from`. Удаление строк из табличного списка

Внимание! Вы можете изменять только табличные списки, предназначенные для правил обогащения. Назначение табличного списка вы можете выбрать при его создании в MaxPatrol SIEM.

Инструкция `remove_from` предназначена для удаления строк из табличного списка. При выполнении правила корреляции удаляются строки, удовлетворяющие указанному в инструкции условию. В инструкции нужно указать название табличного списка и условие удаления строк. В условии можно использовать переменные, имена колонок табличного списка (с префиксом `column::`) и логические операторы.

Формат вызова:

```

enrichment <Название правила обогащения>
  enrich <Название события>:
    remove_from <Название табличного списка> {
      <Условие для удаления>
    }

```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_EnrichmentRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 22,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило обогащения:

```
event Event:
  filter {
    dst.ip == "192.0.2.11"
  }

enrichment Example
  enrich Event:
    remove_from Tabular_List_EnrichmentRule {
      column::Port > 22 # Из TC Tabular_List_EnrichmentRule Удаление записей со
      значением Port больше 22
    }
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#     {
#       "ip": "192.0.2.11",
#       "port": 21,
#       "action": "close"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 22,
#       "action": "open"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 23,
#       "action": "close"
#     }
#   ]
# }
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
```

```

#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}
# Конечное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "action": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "action": "open"
#       }
#   ]
#}
# Обогащенное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}

```

Табличный список Tabular_List_EnrichmentRule в формате YAML:

```

name: Tabular_List_EnrichmentRule
fillType: EnrichmentRule
type: 2
maxSize: 100000
ttl: 259200

```

```

typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
      - ip
      - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- action:
    index: true
    nullable: false
    primaryKey: false
    type: String

```

Табличный список Tabular_List_EnrichmentRule в формате JSON:

```

"Tabular_List_EnrichmentRule": {
  "description": "Table list for enrichment rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "action": {
      "index": true,

```

```

        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    }
},
"fillType": "EnrichmentRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",
    "ip",
    "port",
    "action",
    "_last_changed",
    "_id"
],
"ttl": 259200,
"type": 2,
"typicalSize": 80000
}

```

9.4.6. Команда drop

Команда `drop` используется в инструкции `enrich_fields` для удаления обогащаемого события. Событие может быть использовано при корреляции событий, но не будет сохранено в хранилище событий.

Формат вызова:

```
drop
```

Пример

```
event Event:
  filter {
    protocol == null or protocol == ""
  }
  enrichment Example
  enrich Event:
    enrich_fields {
      drop
    }
```

9.5. Директива query. Объявление запроса к табличному списку

Директива `query` может использоваться для объявления запроса к табличному списку. В директиве нужно указать:

- название запроса, уникальное в рамках правила обогащения. В одном правиле обогащения можно объявить несколько запросов с разными названиями к одному или нескольким табличным спискам. Обращение к запросу выполняется по его названию;
- название табличного списка, к которому выполняется запрос (должно начинаться с прописной буквы латинского алфавита);
- аргументы запроса — переменные, используемые при составлении условия запроса. Значения переменных передаются в запрос при выполнении запроса. Запрос можно выполнить с помощью функций языка XP `exec_query` или `select_query_first`;
- условие, проверяемое при выполнении запроса. В условии запроса можно использовать значения переменных, переданные в аргументах запроса, имена колонок табличного списка (с префиксом `column:`), логические операторы и функции языка XP `in_subnet`, `regex_match`.

Примечание. При использовании функции `in_subnet` в условии запроса `query` к табличному списку в первом аргументе функции нужно указать имя переменной, переданной в запрос, во втором — имя колонки табличного списка (с префиксом `column:`).

Формат вызова:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
    <Условие запроса>
}
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_EnrichmentRule_PID (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{ "time": "2017-10-18T12:00:00.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 22, "protocol": "17", "object.process.id": "32571", "action": "open", "subject": "account", "object": "connection", "status": "success", "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11" }
```

Правило обогащения:

```
query GetProcessPath($host, $pid) from Tabular_List_EnrichmentRule_PID{
    column::host == $host and
    column::pid == $pid
} # Получение значений из табличного списка Tabular_List_EnrichmentRule_PID и
присваивание их переменным

event ModuleLoadedByProcess:
    filter {
        action == "open"
        and object == "connection" # Фильтрация событий
    }

enrichment Map_process_pid_to_path
    enrich ModuleLoadedByProcess:
        enrich_fields {
            datafield1 = select_query_first("GetProcessPath", [event_src.host,
number(object.process.id)], "Path") # Выполнение запроса GetProcessPath, который
получает путь к приложению и приложение по узлу и номеру процесса из табличного
списка Tabular_List_EnrichmentRule_PID
        }
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#     "Tabular_List_EnrichmentRule_PID": [
```

```

#      {
#          "key": 1,
#          "host": "10.0.12.45",
#          "path": " C:\\WINDOWS\\system32\\svchost.exe",
#          "pid": 32571
#      }
#  ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",
#   "object.process.id": "32571",
#   ...
#}
#
# Обогащенное событие:
#{
#   "action": "open",
#   "dst.host": "192.0.2.11",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "event_src.category": "Other",
#   "event_src.host": "10.0.12.45",
#   "event_src.ip": "10.0.12.45",
#   "object.process.id": "32571",
#   "time": "2017-10-18T12:00:00.000Z",
#   "datafield1": " C:\\WINDOWS\\system32\\svchost.exe"
#   ...
#}

```

Табличный список Tabular_List_EnrichmentRule_PID в формате YAML:

```

name: Tabular_List_EnrichmentRule_PID
fillType: EnrichmentRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- key:
    index: false

```

```

    nullable: false
    primaryKey: true
    unique: true
    type: Number
- Host:
    index: false
    nullable: false
    primaryKey: false
    type: String
- Path:
    index: false
    nullable: false
    primaryKey: false
    type: String
- PID:
    index: false
    nullable: false
    primaryKey: false
    type: Number

```

Табличный список Tabular_List_EnrichmentRule_PID в формате JSON:

```

"Tabular_List_EnrichmentRule_PID": {
  "description": "Tabular list enrichment rule - with processes",
  "fields": {
    "Host": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "PID": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    },
    "Path": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,

```

```

        "type": "Number",
        "unique": true
    },
    "_last_changed": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Datetime"
    },
    "key": {
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "Number",
        "unique": true
    }
},
"fillType": "EnrichmentRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "key",
    "Host",
    "Path",
    "PID",
    "_last_changed",
    "_id"
],
"ttl": 259200,
"type": 2,
"typicalSize": 80000
}

```

В этом разделе

[Инструкции qhandler, limit, skip](#) (см. раздел 9.5.1)

[Агрегатные функции для результатов запроса к табличному списку](#) (см. раздел 9.5.2)

См. также

[in_subnet](#) (см. раздел 4.8.3)

[Функция regex_match](#), проверка строки по регулярному выражению из табличного списка (см. раздел 4.9.5)

[Функция exes_query](#), запрос к табличному списку (см. раздел 4.9.2)

[Функция select_query_first](#), запрос значения из табличного списка (см. раздел 4.9.4)

9.5.1. Инструкции qhandler, limit, skip

Инструкция `qhandler` используется для выполнения арифметических операций над результатом запроса к табличному списку. При выполнении запроса с инструкцией `qhandler` функция `exec_query` возвращает ассоциативный массив (с типом данных `KeyValue`), содержащий результаты вычислений (с типом данных `Number`) для указанных в инструкции агрегатных функций. Для использования инструкции нужно указать:

- агрегатную функцию для выполнения арифметической операции;
- имя колонки табличного списка (с префиксом `column::`), элементам которого применяется агрегатная функция (поддерживаются операции над элементами колонок с типом данных `Number`);
- ключ для получения результата применения агрегатной функции из ассоциативного массива, возвращаемого функцией `exec_query`.

Инструкция `limit` используется для указания количества первых строк в результате запроса, к которым применяются агрегатные функции, указанные в инструкции `qhandler`.

Инструкция `skip` используется для исключения указанного количества первых строк из результата запроса, к оставшимся строками применяются агрегатные функции, указанные в инструкции `qhandler`.

Примечание. При использовании инструкций `limit` и `skip` нужно учитывать, что сортировка строк в результате запроса не определена.

Формат вызова:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
  <Условие запроса>
}
qhandler {
  <Ключ 1> = <Агрегатная функция 1>(column:<Имя колонки 1>)
  <Ключ 2> = <Агрегатная функция 2>(column:<Имя колонки 2>)
  ...
}
limit <Максимальное число возвращаемых строк>
skip <Количество строк, исключаемых из результата>
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом обогащения — в поле **Формула**, после чего нажать **Запустить**. Обогащенное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_EnrichmentRule_Duration` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 21,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило обогащения:

```
query TestQuery ($check_ip) from Tabular_List_EnrichmentRule_Duration {
  column::ip == $check_ip and
  column::port == 22 and
  column::status == "Open"
} # Запрос данных из табличного списка Tabular_List_EnrichmentRule_Duration
qhandler { # Получение массива значений из табличного списка и применение
арифметических функций
  $get_max_duration = max(column::duration)
  $get_min_duration = min(column::duration)
  $get_avg_duration = avg(column::duration)
  $get_count = count()
}
event Event:
  filter {
    event_src.vendor == "Best Company"
  }

enrichment Example
  enrich Event:
    enrich_fields { # Присваивание переменным максимального, минимального и
среднего времени открытия порта 22 для переданного в запрос IP-адреса узла
      $get_duration = exec_query("TestQuery", [dst.ip])
      $max_duration = $get_duration["$get_max_duration"]
      $min_duration = $get_duration["$get_min_duration"]
      $avg_duration = $get_duration["$get_avg_duration"]
      $count = $get_duration["$get_count"]
      datafield1 = "IP_" + string(dst.ip)
      datafield2 = "Max_duration_" + string($max_duration)
      datafield3 = "Min_duration_" + string($min_duration)
      datafield4 = "Average_duration_" + string($avg_duration)
      datafield5 = "Result_count_" + string($count)
    }
}
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_EnrichmentRule_Duration": [
#       {
```

```

#         "key": 1,
#         "ip": "192.0.2.10",
#         "port": 22,
#         "status": "Open",
#         "duration": 41
#     },
#     {
#         "key": 2,
#         "ip": "192.0.2.11",
#         "port": 22,
#         "status": "Open",
#         "duration": 33
#     },
#     {
#         "key": 3,
#         "ip": "192.0.2.11",
#         "port": 22,
#         "status": "Open",
#         "duration": 67
#     }
# ]
#}
#
# Исходное событие:
#{
#     "time": "2017-10-18T12:00:00.000Z",
#     "event_src.ip": "10.0.12.45",
#     "object.account.domain": "super_admin",
#     "dst.ip": "192.0.2.11",
#     "dst.port": 21,
#     "protocol": "17",
#     "object.process.id": "32571",
#     "action": "close",
#     "subject": "account",
#     "object": "connection",
#     "status": "success",
#     "duration": 55,
#     ...
#}
#
# Обогащенное событие:
#{
#     "action": "close",
#     "dst.host": "192.0.2.11",
#     "dst.ip": "192.0.2.11",
#     "dst.port": 21,

```

```
#   "duration": 55,
#   "time": "2017-10-18T12:00:00.000Z",
#   "uuid": "74209e91-c848-42b0-9de7-527eec26fb92",
#   ...
#   "datafield1": "IP_192.0.2.11",
#   "datafield2": "Max_duration_67",
#   "datafield3": "Min_duration_33",
#   "datafield4": "Average_duration_50",
#   "datafield5": "Result_count_2"
#}
```

Табличный список Tabular_List_EnrichmentRule_Duration в формате YAML:

```
name: Tabular_List_EnrichmentRule_Duration
fillType: EnrichmentRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- key:
    index: false
    nullable: false
    primaryKey: true
    unique: true
    type: Number
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
    nullable: false
    primaryKey: false
    type: String
- duration:
    index: false
    nullable: false
    primaryKey: false
    type: Number
```


Табличный список Tabular_List_EnrichmentRule_Duration в формате JSON:

```
"Tabular_List_EnrichmentRule_Duration": {
  "description": "Table list for enrichment rule - duration",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "duration": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    },
    "ip": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "key": {
      "index": false,
      "nullable": false,
      "primaryKey": true,
      "type": "Number",
      "unique": true
    },
    "port": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    },
    "status": {
      "index": true,
      "nullable": false,
      "primaryKey": false,

```

```

        "type": "String"
    },
    "fillType": "EnrichmentRule",
    "maxSize": 100000,
    "objectId": "LOC-TL-1",
    "order": [
        "key",
        "ip",
        "port",
        "status",
        "duration",
        "_last_changed",
        "_id"
    ],
    "ttl": 259200,
    "type": 2,
    "typicalSize": 80000
}

```

9.5.2. Агрегатные функции для результатов запроса к табличному списку

Указанные в таблице агрегатные функции используются только в инструкции `qhandler` для выполнения арифметических операций над результатами запроса к табличному списку.

Таблица 94. Агрегатные функции для выполнения арифметических операций над результатами запроса к табличному списку

Агрегатная функция	Возвращаемое значение
<code>min(column::<Имя колонки>)</code>	Наименьший элемент в результатах запроса
<code>max(column::<Имя колонки>)</code>	Наибольший элемент в результатах запроса
<code>avg(column::<Имя колонки>)</code>	Среднее значение элементов, округленное до целого
<code>median(column::<Имя колонки>)</code>	Значение медианного элемента. При четном числе элементов выбирается нижнее медианное значение
<code>sum(column::<Имя колонки>)</code>	Сумма всех элементов результатов запроса
<code>count()</code>	Количество элементов в результатах запроса
<code>column::<Имя колонки></code>	Первый элемент из результатов запроса

10. Создание правила корреляции

В этом разделе описаны директивы и инструкции, используемые при создании правил корреляции, даны примеры и рекомендации.

Справочная информация, используемая в правилах, может храниться в табличных списках. Подробную информацию о табличных списках и работе с ними вы можете найти в разделах «Работа с табличными списками» и «Настройка справочников для обработки событий» Руководства оператора.

Для отладки нового правила вы можете использовать утилиты SDK или утилиту PTSIEMSDK GUI. Добавить новое правило в MaxPatrol SIEM вы можете через веб-интерфейс Knowledge Base (см. Руководство оператора).

В этом разделе

[Алгоритм работы службы корреляции \(см. раздел 10.1\)](#)

[Структура правила корреляции \(см. раздел 10.2\)](#)

[Заполнение полей корреляционного события \(см. раздел 10.3\)](#)

[Директива event. Объявление события, подлежащего корреляции \(см. раздел 10.4\)](#)

[Директива rule. Условие правила корреляции \(см. раздел 10.5\)](#)

[Директива rule. Изменение табличного списка \(см. раздел 10.6\)](#)

[Директива emit. Корреляционное событие \(см. раздел 10.7\)](#)

[Команда copy_fields \(см. раздел 10.8\)](#)

[Директива query. Объявление запроса к табличному списку \(см. раздел 10.9\)](#)

10.1. Алгоритм работы службы корреляции

Из потока нормализованных событий служба маршрутизации отбирает те, которые подлежат корреляции. Эти события должны удовлетворять условию, указанному хотя бы в одном из правил корреляции (в инструкции `filter` директивы `event`).

Отобранные события поступают в службу корреляции событий, где для каждого правила корреляции разделяются на потоки в соответствии со значениями полей, указанными при объявлении событий (в инструкциях `key` директив `event`). Для каждого события выполняется проверка условия правила корреляции (указанного в директиве `rule`):

- если событие является первым в последовательности событий, указанной в условии правила корреляции, в памяти MP SIEM Server создается цепочка из одного события (выполняются инструкция `init` директивы `rule` и инструкция `on` для события);

Примечание. В памяти MP SIEM Server одновременно сохраняются все цепочки, составленные из различных комбинаций событий, поступивших в службу корреляции, и соответствующие последовательностям событий, указанным в условиях всех правил корреляции.

- если событие является не первым и не последним в последовательности — в цепочку добавляется следующее событие (для события выполняется инструкция `on`);
- если событие является последним в последовательности, цепочка событий совпадает с указанной в условии правила последовательностью и собрана за указанное в условии время — регистрируется корреляционное событие (для события выполняется инструкция `on` и директива `emit`), цепочка событий удаляется из памяти MP SIEM Server.

Цепочка событий может быть удалена из памяти MP SIEM Server без регистрации корреляционного события:

- если истекло время, указанное в операторе отсчета времени в условии правила корреляции;
- если истекло время между моментами регистрации событий, удовлетворяющих условию отбора для правила корреляции (по умолчанию — один час);
- если истекло время жизни цепочки событий (по умолчанию — сутки);

Примечание. Вы можете изменить время между моментами регистрации событий и время жизни цепочки событий в конфигурационном файле службы корреляции событий, используя параметры `mtime_ttl` и `ctime_ttl` соответственно.

- если из правила корреляции из инструкции `on` подана команда `close`.

См. также

[Этапы обработки событий \(см. раздел 2\)](#)

[Программные компоненты для обработки событий \(см. раздел 3\)](#)

10.2. Структура правила корреляции

Правило корреляции состоит из последовательности директив, содержащих инструкции и команды. Для работы правила корреляции необходимо указать обязательные директивы и инструкции.

Директива `event` используется для объявления события, подлежащего корреляции. Для объявления нескольких событий с разными названиями в одном правиле можно использовать несколько директив `event`. Директива может содержать инструкцию `key` для указания полей события, по значению которых разделяется поток событий, и должна содержать инструкцию `filter` для настройки условия отбора события.

Директива `rule` используется для описания правила корреляции. В директиве нужно указать условие выполнения правила корреляции. Название, указанное в директиве, является уникальным названием правила корреляции. Все события, используемые при составлении условия правила корреляции, должны быть объявлены с помощью директив `event`. Директива

может содержать инструкцию `init` для объявления переменных и по одной инструкции `on` для каждого объявленного события. Инструкция `on` (обработчик события) используется для выполнения блока операторов при регистрации события, указанного в этой инструкции. Директива `rule` может содержать инструкции `insert_into`, `remove_from`, `clear_table` для редактирования табличного списка.

Директива `emit` используется для заполнения полей корреляционного события и [настройки регистрации инцидента \(см. раздел 12\)](#) при выполнении правила корреляции.

Правило корреляции может содержать директиву `query` для объявления запроса к табличному списку. В одном правиле можно объявить несколько запросов с разными названиями.

Структура правила корреляции:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
    <Условие запроса>
}
event <Название события>:
    key:
        <Поле события 1>, <Поле события 2>, ...
    filter {
        <Условие отбора события>
    }
rule <Название правила>: <Условие правила корреляции>
    init {
        <Блок операторов init>
    }
    on <Название события> {
        <Блок для заполнения полей события>
    }
    insert_into <Название табличного списка> {
        <Блок операторов insert_into>
    }
emit {
    <Блок для заполнения полей события>
    <Блок для настройки инцидента>
}
```

10.3. Заполнение полей корреляционного события

В правиле корреляции требуется указать значения обязательных для заполнения полей корреляционного события. Кроме того, вы можете указать значения необязательных полей или изменить значения отдельных сервисных полей, заполняемых автоматически. Заполнение полей корреляционного события выполняется в инструкциях `on` директивы `rule` и в директиве `emit` правила корреляции.

Внимание! При заполнении полей корреляционного события их названия нужно начинать со знака доллара (\$).

Нужно указать значения для следующих обязательных для заполнения полей корреляционного события:

- `correlation_type` — для настройки регистрации инцидента (см. раздел 12); если указано `event`, регистрируется только корреляционное событие, если `incident` — корреляционное событие и инцидент;
- `action`, `object`, `status` — для описания характера, объекта и результата воздействия; могут принимать только определенные значения (см. приложение B);

Примечание. Вы можете заполнить необязательное поле `subject`, чтобы указать источник воздействия.

- `category.generic`, `category.high`, `category.low` — для назначения категории нарушения информационной безопасности, в связи с которым регистрируется корреляционное событие. В поля категорий корреляционного события обычно помещают тактики и техники по классификации MITRE ATT&CK. Поле `category.generic` всегда содержит `Attack`, поле `category.high` содержит тактику, а поле `category.low` — технику, которые покрывает правило корреляции;
- `importance` — для указания важности события для информационной безопасности; может принимать значения `info`, `low`, `medium`, `high`, `critical`;
- `id` — для ввода идентификатора правила корреляции в виде: <Пространство имен>_SIEM_<Название правила корреляции>.

10.4. Директива `event`. Объявление события, подлежащего корреляции

Директива `event` предназначена для объявления события, подлежащего корреляции. В каждом правиле корреляции нужно объявить хотя бы одно событие.

В директиве нужно указать название события, поля события для разделения потока нормализованных событий и условие отбора события. Название должно начинаться с прописной буквы латинского алфавита и быть уникальным в рамках одного правила корреляции. Для указания полей события и условия отбора используются инструкции `key` и `filter`.

Формат вызова:

```
event <Название события>:
  key:
    <Поле события 1>, <Поле события 2>, ...
  filter {
    <Условие отбора события>
  }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**.

Исходные события:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 22,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}

{
  "time": "2017-10-18T12:00:10.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 23,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}

{
  "time": "2017-10-18T12:00:20.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 23,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило корреляции:

```
event Event_1:
  key:
    event_src.vendor
  filter {
    dst.port == 22
  }
```

```
event Event_2:
  key:
    event_src.vendor
  filter {
    dst.port == 23
  }
```

```
rule Example: Event_1 -> Event_2[2] # Формирование последовательности событий. При получении события Event_1, ожидается два события Event_2
emit {
  $correlation_type = "event"
  $object = "client"
}
```

```

$action = "check"
$status = "success"
$importance = "info"
}

```

Результат работы правила:

```

# Нормализованное событие 1:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",
#   "object.process.id": "32571",
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "duration": 55,
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example",
#   "event_src.host": "10.0.12.45",
#   "dst.host": "192.0.2.11"
#}
# Нормализованное событие 2:
#{
#   "time": "2017-10-18T12:00:10.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 23,
#   ...
#}
# Нормализованное событие 3:
#{
#   "time": "2017-10-18T12:00:20.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 23,
#   ...
#}

```



```
#
# Корреляционное событие:
#{
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z",
#       "2017-10-18T12:00:00.000Z",
#       "2017-10-18T12:00:20.000Z"
#   ],
#   "subevents": [
#       "d63d2fdd-5b3c-46a1-8569-e3363636e1a7",
#       "d63d2fdd-5b3c-46a1-8569-e3363636e1a8",
#       "d63d2fdd-5b3c-46a1-8569-e3363636e1a9"
#   ],
#   ...
#}
```

В этом разделе

[Инструкция key \(см. раздел 10.4.1\)](#)

[Инструкция filter \(см. раздел 10.4.2\)](#)

10.4.1. Инструкция key

Инструкция `key` директивы `event` предназначена для указания полей события, по значению которых поток нормализованных событий будет разделяться на несколько потоков. В инструкции вы можете указать одно поле события или несколько полей через запятую. Разделение потока событий выполняется по совокупности значений указанных полей. Наборы полей для событий, объявленных в одном правиле корреляции, могут отличаться названиями полей, но типы данных для полей в одинаковых позициях должны совпадать.

Примечание. Если условие правила корреляции выполняется при регистрации одного события (не указана последовательность событий), в качестве `key` необходимо указать `event_src.host`.

Пример:

```
event Account_login_failed:
  key:
    event_src.host, subject.name
  filter {
    ...
  }
event Account_login_failed_dst:
  key:
```

```

        dst.host, subject.name
    filter {
        ...
    }

```

10.4.2. Инструкция filter

Инструкция `filter` директивы `event` предназначена для настройки условия отбора события. Если условие возвращает значение `True`, событие используется в правиле корреляции. При написании условия вы можете использовать:

- математические и логические операторы;
- поля события;
- литералы (с типом данных `Bool`, `Null`, `Number`, `String`);
- инлайн-запросы к табличным спискам (с префиксом `table::`);

Примечание. Макросы поддерживают работу только с табличными списками типа «справочник».

- вызовы макросов (с префиксом `filter::`);
- функции языка XPL, в том числе для выполнения запроса к [табличному списку](#) (см. [раздел 4.9.2](#)).

Упрощенный вызов инструкции filter

Вы можете опустить название инструкции `filter` и операторы `and`, если в инструкции `filter` в условии используется только конъюнкция (логическое И) и указаны точные значения полей события. Для ввода значений полей в этом случае используется присваивание (вместо логического равенства `==`).

Пример:

```

event SessionStop:
    key: src.ip
        action = "stop"
        object = "session"
        status = "success"
        subject.domain = "net"

```

Многоуровневая корреляция

Многоуровневая корреляция — поиск событий информационной безопасности на основе анализа зарегистрированных ранее корреляционных событий.

Для отбора корреляционных событий в инструкции `filter` для поля `correlation_name` нужно указать название правила корреляции, по которому были зарегистрированы корреляционные события.

Пример:

```
event Event:
  key:
    correlation_name
  filter {
    correlation_name == "Cisco_WebVPN_start_success"
  }
```

Корреляция на активах

Корреляция на активах — поиск событий информационной безопасности, связанных с активами, зарегистрированными в MaxPatrol SIEM.

Для отбора связанных с активом событий нужно в инструкции `filter` для одного из полей `src.asset`, `dst.asset`, `event_src.asset` или `recv_asset` указать UUID актива.

Пример:

```
event Asset_event:
  key:
    event_src.asset
  filter {
    event_src.asset != null and
    action == "login" and
    status == "success"
  }
```

См. также

[Префикс `table::`, инлайн-запрос \(см. раздел 4.9.1\)](#)

[Создание макроса \(см. раздел 6\)](#)

10.5. Директива `rule`. Условие правила корреляции

Директива `rule` предназначена для описания правила корреляции. В директиве нужно указать название правила корреляции и условие его выполнения. Название должно быть уникальным в рамках всего набора правил корреляции (может состоять из букв латинского алфавита, цифр, знаков подчеркивания и точки, должно начинаться с прописной буквы).

Условие правила корреляции может содержать названия событий, объявленных в директивах `event`, и специальные операторы для описания последовательности событий.

Последовательность может содержать одно или несколько событий. При описании последовательности событий вы можете указать порядок регистрации событий, количество появлений каждого события, интервалы времени между моментами регистрации событий или ограничить время регистрации всей последовательности.

Если регистрируются все события последовательности в указанном порядке и за указанное время, условие правила корреляции возвращает True, иначе — False. При выполнении условия правила корреляции выполняется блок операторов директивы `emit` и регистрируется корреляционное событие.

В директиве `rule` можно использовать инструкции `init` и `on`. Если регистрируется первое (крайнее слева) событие последовательности, указанной в условии, выполняются блок операторов инструкции `init` и блок операторов инструкции `on` для зарегистрированного события. При регистрации следующих событий последовательности для них выполняются блоки операторов в инструкциях `on`.

Примечание. В директиве `rule` для редактирования табличного списка вы можете использовать инструкции `insert_into`, `remove_table` и `clear_table`. Инструкции выполняются при выполнении условия правила корреляции.

Формат вызова:

```
rule <Название правила>: <Условие правила корреляции>
  init {
    <Блок операторов init>
  }
  on <Название события> {
    <Блок операторов on>
  }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**.

Исходные события:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 22,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}

{
  "time": "2017-10-18T12:00:10.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 23,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}

{
  "time": "2017-10-18T12:00:20.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 23,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

```
Company", "event_src.title": "NG Session
Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.
c.host": "10.0.12.45", "dst.host": "192.0.2.11"}
```

Правило корреляции:

```
event Event_1:
  key:
    event_src.vendor
  filter {
    dst.port == 22
  }
```

```
event Event_2:
  key:
    event_src.vendor
  filter {
    dst.port == 23
  }
```

rule Example: Event_1 -> Event_2[2] #Формирование последовательности событий. При получении события Event_1, ожидается два события Event_2, увеличиваются значения счетчиков в обоих событиях

```
init {
  $f1 = 3
  $f2 = 0
  $f3 = 0
}

on Event_1 {
  $f1 = $f1+1
  $f2 = $f2+1
}

on Event_2 {
  $f1 = $f1+1
  $f3 = $f3+1
}

emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
  $datafield1 = $f1
  $datafield2 = $f2
  $datafield3 = $f3
}
```

Результат работы правила:

```

# Нормализованное событие 1:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",
#   "object.process.id": "32571",
#   "action": "close",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "duration": 55,
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   "event_src.title": "NG Session Controller",
#   "event_src.category": "Other",
#   "id": "PT_normalized_event_example",
#   "event_src.host": "10.0.12.45",
#   "dst.host": "192.0.2.11"
#}
# Нормализованное событие 2:
#{
#   "time": "2017-10-18T12:00:10.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 23,
#   ...
#}
# Нормализованное событие 3:
#{
#   "time": "2017-10-18T12:00:20.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 23,
#   ...
#}
#
# Корреляционное событие:
#{
#   "correlation_type": "event",
#   "datafield1": "6",

```

```
# "datafield2": "1",
# "datafield3": "2",
# "count": 1,
# "importance": "info",
# "subevents.time": [
#     "2017-10-18T12:00:00.000Z",
#     "2017-10-18T12:00:10.000Z",
#     "2017-10-18T12:00:20.000Z"
# ],
# "subevents": [
#     "c32a06cd-b672-414d-bc98-af2126beb378",
#     "c32a06cd-b672-414d-bc98-af2126beb379",
#     "c32a06cd-b672-414d-bc98-af2126beb37a"
# ],
# "_rule": "Example",
# "time": "2017-10-18T12:00:20.000Z",
# "correlation_name": "Example",
# ...
#}
```

В этом разделе

[Логические операторы для последовательности событий \(см. раздел 10.5.1\)](#)

[Квантификаторы событий последовательности \(см. раздел 10.5.2\)](#)

[Операторы отсчета времени для последовательности \(см. раздел 10.5.3\)](#)

[Оператор as. Псевдонимы события \(см. раздел 10.5.4\)](#)

[Инструкция init. Объявление переменных \(см. раздел 10.5.5\)](#)

[Инструкция on. Обработчик события \(см. раздел 10.5.6\)](#)

[Команда close \(см. раздел 10.5.7\)](#)

10.5.1. Логические операторы для последовательности событий

При составлении в условии правила корреляции последовательности из объявленных событий вы можете использовать логические операторы `->`, `or`, `and`, `not`, `with` `different`.

Оператор «->»

Результатом применения оператора `->` является `True`, если зарегистрировано событие, указанное слева от оператора, а затем событие, указанное справа от оператора. Вы можете использовать оператор для составления последовательности любой длины.

Формат вызова:

```
rule <Название правила>: <Название события 1> -> <Название события 2>
```

Пример:

```
rule Example: Event_1 -> Event_2
# Выполнено при регистрации: Event_1, Event_2.
```

Оператор «or»

Результатом применения оператора `or` является `True`, если зарегистрировано событие, указанное слева от оператора, или событие, указанное справа от оператора. Вы можете использовать оператор для составления последовательности любой длины.

Формат вызова:

```
rule <Название правила>: <Название события 1> or <Название события 2>
```

Пример:

```
rule Example: Event_1 or Event_2
# Выполнено при регистрации: Event_1 или Event_2.
```

Оператор «and»

Результатом применения оператора `and` является `True`, если зарегистрированы оба указанных события (в любом порядке). Вы можете использовать оператор для составления последовательности любой длины.

Формат вызова:

```
rule <Название правила>: <Название события 1> and <Название события 2>
```

Пример:

```
rule Example: Event_1 and Event_2
# Выполнено при регистрации: Event_1, Event_2 или Event_2, Event_1.
```

Оператор «not»

Результатом применения оператора `not` является `False` при регистрации события, указанного справа от оператора. Оператор нельзя использовать с первым или последним событием в последовательности.

Примечание. Для использования оператора `not` с последним событием в последовательности нужно применить оператор отсчета времени `timer`.

Формат вызова:

```
rule <Название правила>: <Последовательность событий 1> -> not <Название события> ->
<Последовательность событий 2>
```


Пример:

```
rule Example: Event_1 -> not Event_2 -> Event_3
# Выполнено при регистрации: Event_1, Event_3.
# Не выполнено при регистрации: Event_1, Event_2.
```

Оператор «with different»

Результатом применения оператора `with different` является `True`, если во всех событиях последовательности, указанной слева от оператора, отличаются значения поля, указанного справа от оператора. Не поддерживается вложение операторов `with different` друг в друга.

Внимание! Условие с оператором `with different` невыполнимо, если поле события в аргументе оператора совпадает с одним из полей, указанных в инструкции `key` директивы `event` для события, расположенного слева от оператора.

Формат вызова:

```
rule <Название правила>: (<Последовательность событий>) with different <Поле события>
```

Пример:

```
rule Example: (Event_1 -> Event_2) with different src.host
# Выполнено при регистрации: Event_1, Event_2 (с разными значениями поля src.host).
```

Пример:

```
rule Example: Event[3] with different(src.ip)
# Выполнено при регистрации: Event, Event, Event (с разными значениями поля src.ip).
```

Пример:

```
rule Example: (Event_1[2, 3] with different(src.ip)) -> Event_2
# Выполнено при регистрации: Event_1, Event_1 (с разными значениями поля src.ip),
Event_2 или Event_1, Event_1, Event_1 (с разными значениями поля src.ip), Event_2
```

Пример:

```
event Event:
  key:
    src.ip
  filter {
    event_src.vendor == "Best Company"
  }
rule Event[3] with different src.ip
# Всегда не выполнено
```

10.5.2. Квантификаторы событий последовательности

Квантификатор — оператор, применяемый к событию для указания количества его появлений подряд в последовательности событий. При составлении в условии правила корреляции последовательности из объявленных событий вы можете применять к событиям квантификаторы.

Примечание. Если количество событий в последовательности не определено (последовательность оканчивается одним из операторов ?, *, +, [N,]) и не указан оператор отсчета времени, собранная цепочка событий удаляется из памяти MP SIEM Server (без регистрации корреляционного события) по истечении времени жизни или по истечении времени между [моментами регистрации событий](#) (см. раздел 10.1).

Название события в условии

Результатом регистрации указанного события является True.

Формат вызова:

```
rule <Название правила>: <Название события>
```

Пример:

```
rule Example: Event
# Выполнено при регистрации: Event.
```

Оператор «+»

Результатом применения оператора + является True, если событие, указанное слева от оператора, зарегистрировано один раз или более. Оператор нельзя использовать для указания единственного или последнего события в последовательности с оператором отсчета времени within.

Формат вызова:

```
rule <Название правила>: <Название события>+
```

Пример:

```
rule Example: Event_1+ -> Event_2
# Выполнено при регистрации: Event_1, ..., Event_1, Event_2.
```

Оператор «?»

Результатом применения оператора ? всегда является True. Вы можете использовать оператор для указания одного необязательного события в последовательности. Оператор нельзя использовать для указания первого или единственного события в любой последовательности и для указания последнего события в последовательности с оператором отсчета времени within.

Формат вызова:

```
rule <Название правила>: <Название события 1> -> <Название события 2>?
```

Пример:

```
rule Example: Event_1 -> Event_2? -> Event_1
# Выполнено при регистрации: Event_1, Event_1 или Event_1, Event_2, Event_1.
```

Оператор «*»

Результатом применения оператора * всегда является True. Вы можете использовать оператор для указания последовательности любой длины из одного необязательного события. Оператор нельзя использовать для указания первого или единственного события в любой последовательности и для указания последнего события в последовательности с оператором отсчета времени *within*.

Формат вызова:

```
rule <Название правила>: <Название события 1> -> <Название события 2>*
```

Пример:

```
rule Example: Event_1 -> Event_2* -> Event_1
# Выполнено при регистрации: Event_1, Event_1 или Event_1, Event_2 ,..., Event_1.
```

Оператор «[N,M]»

Результатом применения оператора [N,M] является True, если указанное слева от оператора событие зарегистрировано указанное число раз (см. таблицу 95).

Формат вызова:

```
rule <Название правила>: <Название события 1> -> <Название события 2>[N,M] ->
<Название события 3>
```

Таблица 95. Варианты настройки оператора [N,M]

Синтаксис	Число событий	Пример последовательности	Примечание
<Название события>[N]	Ровно N	Event[3] — последовательность из 3 событий	—
<Название события>[N,] ...	N и больше	Event[3,] — последовательность из 3, 4, 5, 6 и так далее событий	Вариант настройки нельзя использовать для указания последнего события в последовательности с оператором отсчета времени <i>within</i>
<Название события>[, M]	M и меньше или отсутствует	Event[, 3] — последовательность из 1, 2 или 3 событий или отсутствие события	Вариант настройки нельзя использовать для указания первого или единственного события в любой последовательности и для указания последнего события в последовательности с оператором отсчета времени <i>within</i>

Синтаксис	Число событий	Пример последовательности	Примечание
<Название события>[N,M]	От N до M	Event[3,5] — последовательность из 3, 4 или 5 событий	Вариант настройки нельзя использовать для указания последнего события в последовательности с оператором отсчета времени <code>within</code>

Пример:

```
rule Example: Event[3]
# Выполнено при регистрации: Event, Event, Event.
```

Пример:

```
rule Example: Event_1[3,] -> Event_2
# Выполнено при регистрации: Event_1, Event_1, Event_1, Event_2 или Event_1, ..., Event_1, Event_2
```

Пример:

```
rule Example: Event_1 -> Event_2[,3] -> Event_1
# Выполнено при регистрации: Event_1, Event_1 или Event_1, Event_2, Event_1 или Event_1, Event_2, Event_2, Event_1 или Event_1, Event_2, Event_2, Event_2, Event_1.
```

Пример:

```
rule Example: Event [3, 5]
# Выполнено при регистрации: Event, Event, Event, или Event, Event, Event, Event или Event, Event, Event, Event, Event.
```

10.5.3. Операторы отсчета времени для последовательности

Внимание! В одном условии нельзя использовать несколько операторов отсчета времени.

При составлении в условии правила корреляции последовательности из объявленных событий вы можете использовать операторы отсчета времени.

Оператор `within`

Результатом применения оператора `within` является значение `True`, если разница между значениями полей `time` последнего и первого событий последовательности, указанной слева от оператора, меньше времени, указанного справа от оператора. Корреляционное событие регистрируется сразу при выполнении условия с оператором `within`.

Вы можете использовать оператор для событий последовательности для ограничения интервала времени между моментами их регистрации на источнике. Для ввода времени вы можете использовать сокращения: `s` — секунды, `m` — минуты, `h` — часы, `d` — дни. Последнее событие последовательности должно быть указано явно, для него нельзя использовать операторы `?`, `*`, `+`, `[N, ]`, `[ , M]`, `[N, M]`.

Формат вызова:

```
rule <Название правила>: <Последовательность событий> within <Время>
```

Пример:

```
rule Example: Event[3] within 5s
# Выполнено при регистрации не более чем за 5 секунд: Event, Event, Event.
```

Оператор timer

Результатом применения оператора `timer` является значение `True`, если все события последовательности, указанной слева от оператора, поступили в службу корреляции событий за указанное справа от оператора время.

Отсчет времени начинается с момента получения службой корреляции первого события последовательности и продолжается до истечения указанного времени. Корреляционное событие регистрируется после окончания указанного времени.

Вы можете использовать оператор для ограничения времени регистрации последовательности событий. Для ввода времени вы можете использовать сокращения: `s` — секунды, `m` — минуты, `1h` — 1 час.

Формат вызова:

```
rule <Название правила>: <Последовательность событий> timer <Время>
```

Пример:

```
rule Example: Event+ timer 5s
# Выполнено при регистрации за 5 секунд любого числа Event.
```

Пример:

```
rule Example: (Event_1 -> Event_2) timer 10m
# Выполнено при регистрации за 10 минут Event_1, Event_2.
```

Пример:

```
rule Example: ((Event_1 -> Event_2?) timer 1h) -> Event_3
# Выполнено при регистрации за 1 час Event_1 или Event_1, Event_2, после этого за любое время Event_3.
```

Оператор timeout_timer

Результатом применения оператора `timeout_timer` является значение `True`, если все события последовательности, указанной слева от оператора, поступают в службу корреляции событий с интервалом, не превышающим время, указанное справа от оператора.

Отсчет времени начинается с момента получения службой корреляции первого события последовательности и продолжается до истечения указанного времени. Если за это время в службу корреляции поступает следующее событие последовательности, отсчет времени перезапускается.

Вы можете использовать оператор для ограничения интервала времени между моментами регистрации событий последовательности. Для ввода времени вы можете использовать сокращения: *s* — секунды, *m* — минуты, *1h* — 1 час.

Формат вызова:

```
rule <Название правила>: <Последовательность событий> timeout_timer <Время>
```

Пример:

```
rule Example: Event[3] timeout_timer 10m
# Выполнено при регистрации за 30 минут: Event в течение первых 10 минут, затем Event
в течение следующих 10 минут, затем Event в течение последних 10 минут.
```

10.5.4. Оператор as. Псевдонимы события

Оператор *as* предназначен для объявления в условии правила корреляции директивы *rule* псевдонима объявленного события. Псевдонимы используются для выполнения блоков операторов в разных инструкциях *on* для одного события, указанного в разных частях последовательности.

Формат вызова:

```
rule <Название правила>: <Название события> as <Псевдоним>
  on <Псевдоним> {
    <Блок операторов on>
  }
```

Пример

```
rule Example: Event as Event_1 -> Event as Event_2
  on Event_1 {
    $f1=1
  }
  on Event_2 {
    $f1=2
  }
```

10.5.5. Инструкция init. Объявление переменных

Инструкция *init* директивы *rule* предназначена для объявления переменных, используемых в инструкциях *on* и *insert_into* или в директиве *emit*. Переменным вы можете присвоить определенные значения. Блок операторов инструкции *init* выполняется один раз при регистрации первого события последовательности в условии правила корреляции.

Формат вызова:

```
init {
  <Блок операторов init>
}
```

Пример

```
init {
    $count.subevents = 0
}
```

10.5.6. Инструкция on. Обработчик события

Инструкция `on` директивы `rule` (обработчик события) предназначена для выполнения блока операторов при регистрации указанного события (событие должно быть объявлено в директиве `event`).

В блоке операторов инструкции `on` вы можете объявлять переменные и присваивать значения полям корреляционного события. Для этого вы можете использовать поля указанного в инструкции события; переменные, объявленные ранее в инструкциях `init` и `on` (для других событий); функции и операторы языка XPL.

Кроме этого, в блоке операторов инструкции `on` вы можете использовать команду `close` для удаления уже собранной цепочки событий из памяти MP SIEM Server без регистрации корреляционного события.

Формат вызова:

```
rule <Название правила>: <Условие правила корреляции>
init {
    <Блок операторов init>
}
on <Название события> {
    <Блок операторов on>
}
```

Пример

```
rule Bruteforce_success_by_user: (Login_failure[5,] -> Login_success) within 5m
init{
    $count.subevents = 0
}
on Login_failure {
    $count.subevents = $count.subevents + 1
}
on Login_success {
    $subject.name = subject.name
    $subject.domain = subject.domain
    $count.subevents = $count.subevents + 1
}
```

10.5.7. Команда close

Команда `close` используется в инструкции `on` директивы `rule` для прекращения проверки последовательности событий и удаления уже собранной цепочки событий из памяти MP SIEM Server без регистрации корреляционного события.

Примечание. Вы можете использовать команду `close` в директиве `emit` правила корреляции.

Формат вызова:

```
close
```

Пример

```
rule Example: Event_1 -> Event_2
  on Event_1 {
    $hostname = src.hostname
  }
  on Event_2 {
    if $hostname != src.hostname then
      close
    endif
  }
```

10.6. Директива rule. Изменение табличного списка

Внимание! Из директивы `rule` вы можете изменять только табличные списки, предназначенные для правил корреляции. Назначение табличного списка вы можете выбрать при его создании в MaxPatrol SIEM.

В директиве `rule` для изменения табличного списка можно использовать инструкции `insert_into`, `remove_from` и `clear_table`. Инструкции выполняются при выполнении условия правила корреляции.

В этом разделе

[Инструкция `insert\_into`. Запись в табличный список \(см. раздел 10.6.1\)](#)

[Инструкция `insert\_into`. Условная запись в табличный список \(см. раздел 10.6.2\)](#)

[Инструкция `insert\_into`. Арифметические операции при записи в табличный список \(см. раздел 10.6.3\)](#)

[Инструкция `insert\_into`. Добавление строк в табличный список \(см. раздел 10.6.4\)](#)

[Инструкция `remove\_from`. Удаление строк из табличного списка \(см. раздел 10.6.5\)](#)

[Инструкция `clear\_table`. Очистка табличного списка \(см. раздел 10.6.6\)](#)

10.6.1. Инструкция insert_into. Запись в табличный список

Инструкция `insert_into` может использоваться для записи в существующую строку (ячейку) табличного списка. Для использования инструкции нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- переменные, в которых из директивы `rule` в инструкцию `insert_into` передаются значения ячеек табличного списка;
- имя ключевой колонки (с префиксом `column::`) в указанном табличном списке;

Примечание. Какая из колонок является ключевой, определяется при создании табличного списка. В отличие от обычной колонки, все значения ключевой колонки уникальны в рамках одного табличного списка.

- значение ключа для выбора строки, в которую будут записаны значения;

Примечание. Если ключевая колонка не содержит указанного в инструкции значения, в табличный список добавляется строка с этим ключом.

- имена колонок (с префиксом `column::`), в ячейки которых требуется записать значения;
- значения, которые будут записаны в ячейки табличного списка (тип данных должен совпадать с типом данных колонки табличного списка).

Формат вызова:

```
rule <Название правила>: <Условие правила>
  on <Название события> {
    $f_key = <Значение ключа>
    $f1 = <Значение 1>
    $f2 = <Значение 2>
    ...
  }
  insert_into <Название табличного списка> {
    column::<Имя ключевой колонки> = $f_key
    column::<Имя колонки 1> = $f1
    column::<Имя колонки 2> = $f2
    ...
  }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_CorrelationRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 21,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило корреляции:

```
event Event:
  key:
    event_src.vendor
  filter {
    dst.ip == "192.0.2.11"
  }

rule Example: Event
  on Event { # Обработка события, присваивание значений полей событий переменным
    $f_key = dst.port
    $f = action
    $i = string(dst.ip)
  }

insert_into Tabular_List_CorrelationRule { # Добавление значений в табличный список
  column::ip = $i
  column::port = $f_key
  column::status = $f
}

emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}
```

Результат работы правила:

```
# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#     {
#       "ip": "192.0.2.11",
#       "port": 21,
#       "status": "close"
#     },
#     {
```

```

#         "ip": "192.0.2.11",
#         "port": 22,
#         "status": "open"
#     },
#     {
#         "ip": "192.0.2.11",
#         "port": 23,
#         "status": "close"
#     }
# ]
#}
# Исходное событие:
#{
#     "time": "2017-10-18T12:00:00.000Z",
#     "event_src.ip": "10.0.12.45",
#     "object.account.domain": "super_admin",
#     "dst.ip": "192.0.2.11",
#     "dst.port": 21,
#     "action": "open",
#     "subject": "account",
#     "object": "connection",
#     "status": "success",
#     "duration": 55,
#     ...
#}
# Конечное состояние табличного списка:
#{
#     "Tabular_List_CorrelationRule": [
#         {
#             "ip": "192.0.2.11",
#             "port": 21,
#             "status": "open"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 22,
#             "status": "open"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 23,
#             "status": "close"
#         }
#     ]
#}
# Корреляционное событие:
#{

```

```
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "14133818-448e-4302-be7e-754cb4fb13ad"
#   ],
#   "_rule": "Example",
#   "correlation_name": "Example",
#   "object": "client",
#   "action": "check",
#   ...
#
#}
```

Табличный список Tabular_List_CorrelationRule в формате YAML:

```
name: Tabular_List_CorrelationRule
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
```

```

nullable: false
primaryKey: false
type: String

```

Табличный список Tabular_List_CorrelationRule в формате JSON:

```

"Tabular_List_CorrelationRule": {
  "description": "Tabular list for correlation rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "complex_key": {
      "compositeFields": [
        "ip",
        "port"
      ],
      "index": false,
      "nullable": false,
      "primaryKey": true,
      "type": "composite",
      "unique": true
    },
    "ip": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "port": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    },
    "status": {
      "index": true,

```

```

        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "fillType": "CorrelationRule",
    "maxSize": 100000,
    "objectId": "LOC-TL-1",
    "order": [
        "complex_key",
        "ip",
        "port",
        "status",
        "_last_changed",
        "_id"
    ],
    "ttl": 259200,
    "type": 1,
    "typicalSize": 80000
}

```

10.6.2. Инструкция `insert_into`. Условная запись в табличный список

Инструкция `insert_into` может использоваться для записи в табличный список при выполнении условия. Для использования инструкции нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- условие записи в табличный список. Если условие возвращает `true`, выполняется запись в табличный список, если `false` — не выполняется. Условие может состоять из переменной, объявленной в директиве `rule` и содержащей данные с типом `Bool`;
- переменные, в которых из директивы `rule` в инструкцию `insert_into` передаются значения ячеек табличного списка;
- имя ключевой колонки (с префиксом `column:`) в указанном табличном списке;

Примечание. Какая из колонок является ключевой, определяется при создании табличного списка. В отличие от обычной колонки, все значения ключевой колонки уникальны в рамках одного табличного списка.

- значение ключа для выбора строки, в которую будут записаны значения;

Примечание. Если ключевая колонка не содержит указанного в инструкции значения, в табличный список добавляется строка с этим ключом.

- имена колонок (с префиксом `column::`), в ячейки которых требуется записать значения;
- значения, которые будут записаны в ячейки табличного списка (тип данных должен совпадать с типом данных колонки табличного списка).

Формат вызова:

```
rule <Название правила>: <Условие правила>
  on <Название события> {
    $f_key = <Значение ключа>
    $f1 = <Значение 1>
    $f2 = <Значение 2>
    ...
  }
  insert_into <Название табличного списка> if <Условие записи> {
    column::<Имя ключевой колонки> = $f_key
    column::<Имя колонки 1> = $f1
    column::<Имя колонки 2> = $f2
    ...
  }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_CorrelationRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{ "time": "2017-10-18T12:00:00.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 21, "protocol": "17", "object.process.id": "32571", "action": "open", "subject": "account", "object": "connection", "status": "success", "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11" }
```

Правило корреляции:

```
event Event:
  key:
    event_src.vendor
  filter {
    dst.ip == "192.0.2.11"
  }
```

```
rule Example: Event
  on Event { # Обработка события, проверка значений полей и присваивание значений полей события переменным
```

```

        $f_cond = bool(dst.ip == "192.0.2.11")
        $f_key = dst.port
        $f = string(action)
        $i = string(dst.ip)
    }

    insert_into Tabular_List_CorrelationRule if $f_cond { # Значения добавляются в
табличный список только в том случае, если выполнено условие в переменной $f_cond
        column::ip = $i
        column::port = $f_key
        column::status = $f
    }

    emit {
        $correlation_type = "event"
        $object = "client"
        $action = "check"
        $status = "success"
        $importance = "info"
    }

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "status": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "status": "close"
#       }
#   ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",

```



```

#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 21,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "duration": 55,
#   ...
#}
#
# Конечное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "status": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "status": "close"
#       }
#   ]
#}
#
# Корреляционное событие:
#{
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "1cc36133-a82a-480a-8c26-c6cc361582a3"
#   ],
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   "correlation_name": "Example",

```

```
#   "object": "client",
#   "action": "check",
#   ...
#}
```

Табличный список Tabular_List_CorrelationRule в формате YAML:

```
name: Tabular_List_CorrelationRule
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
    nullable: false
    primaryKey: false
    type: String
```

Табличный список Tabular_List_CorrelationRule в формате JSON:

```
"Tabular_List_CorrelationRule": {
  "description": "Tabular list for correlation rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
```

```

        "unique": true
    },
    "_last_changed": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Datetime"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "status": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",
    "ip",
    "port",
    "status",

```

```

        "_last_changed",
        "_id"
    ],
    "ttl": 259200,
    "type": 1,
    "typicalSize": 80000
}

```

10.6.3. Инструкция insert_into. Арифметические операции при записи в табличный список

Для выполнения арифметических операций при записи в табличный список в инструкции `insert_into` используются функции, указанные в таблице ниже. Для использования функций нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- переменные, в которых из директивы `rule` в инструкцию `insert_into` передаются значения ячеек табличного списка;
- имя ключевой колонки (с префиксом `column: :`) в указанном табличном списке;

Примечание. Какая из колонок является ключевой, определяется при создании табличного списка. В отличие от обычной колонки, все значения ключевой колонки уникальны в рамках одного табличного списка.

- значение ключа для выбора строки, в которую будут записаны значения;

Примечание. Если ключевая колонка не содержит указанного в инструкции значения, в табличный список добавляется строка с этим ключом.

- имя колонки (с префиксом `column: :`), в ячейку которой требуется записать результат применения функции;
- значение для записи (с типом данных `Number`), к которому применяется функция.

Примечание. Если к ячейке табличного списка, содержащей максимальное значение, поддерживаемое для [типа данных Number](#) (см. [приложение A](#)), будет применена функция `insert_inc`, значение ячейки не изменится. Если к ячейке табличного списка, содержащей минимальное значение, будет применена функция `insert_dec`, значение ячейки также не изменится.

Формат вызова:

```

rule <Название правила>: <Условие правила>
  on <Название события> {
    $f_key = <Значение ключа>
    $f = <Значение для записи>
  }
  insert_into <Название табличного списка> {
    column:<Имя ключевой колонки> = $f_key

```

```

insert_min(column::<Имя колонки 1>, $f)
insert_max(column::<Имя колонки 2>, $f)
insert_inc(column::<Имя колонки 3>)
insert_dec(column::<Имя колонки 4>)
}

```

Таблица 96. Функции для выполнения арифметических операций при записи в табличный список в директиве insert_into

Функция	Арифметическая операция
insert_min(column::<Имя колонки>, \$f)	Сравниваются два значения: указанное во втором аргументе функции и содержащееся в ячейке колонки, указанной в первом аргументе. В ячейку заносится меньшее из значений. Если ячейка содержит null или в табличный список добавляется новая строка, в ячейку заносится значение, указанное во втором аргументе функции
insert_max(column::<Имя колонки>, \$f)	Сравниваются два значения: указанное во втором аргументе функции и содержащееся в ячейке колонки, указанной в первом аргументе. В ячейку заносится большее из значений. Если ячейка содержит null или в табличный список добавляется новая строка, в ячейку заносится значение, указанное во втором аргументе функции
insert_inc(column::<Имя колонки>)	Значение ячейки в указанной колонке увеличивается на единицу. Если в табличный список добавляется новая строка, в ячейку заносится 1
insert_dec(column::<Имя колонки>)	Значение ячейки в указанной колонке уменьшается на единицу. Если в табличный список добавляется новая строка, в ячейку заносится -1

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_Correlation_Arithm (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```

{"time":"2017-10-18T12:00:00.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":21,"protocol":"17","object.process.id":"32571","action":"close","subject":"account","object":"connection","status":"success","duration":55,"importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"ID_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11"}

```

Правило корреляции:

```

event Event:
  key:
    event_src.vendor
  filter {
    dst.ip == "192.0.2.11"
  }

rule Example: Event
  on Event {
    $f_key = dst.port
    $f = duration
  }
insert_into Tabular_List_Correlation_Arithm { # Изменение значений в табличном списке
Tabular_List_Correlation_Arithm в результате выполнения арифметических операций
  column::key = 1
  column::ip = dst.ip
  column::port = $f_key
  insert_min(column::minduration, $f)
  insert_max(column::maxduration, $f)
  insert_inc(column::count)
}
emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_Correlation_Arithm": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "minduration": 12,
#           "maxduration": 51,
#           "count": 11
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "minduration": 63,
#           "maxduration": 71,

```

```

#         "count": 23
#     },
#     {
#         "ip": "192.0.2.11",
#         "port": 23,
#         "minduration": 9,
#         "maxduration": 35,
#         "count": 6
#     }
# ]
#}
#
# Исходное событие:
#{
#     "time": "2017-10-18T12:00:00.000Z",
#     "event_src.ip": "10.0.12.45",
#     "object.account.domain": "super_admin",
#     "dst.ip": "192.0.2.11",
#     "dst.port": 21,
#     "action": "close",
#     "subject": "account",
#     "object": "connection",
#     "status": "success",
#     "duration": 55,
#     ...
#}
#
# Конечное состояние табличного списка:
#{
#     "Tabular_List_Correlation_Arithm": [
#         {
#             "ip": "192.0.2.11",
#             "port": 21,
#             "minduration": 12,
#             "maxduration": 55,
#             "count": 12
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 22,
#             "minduration": 63,
#             "maxduration": 71,
#             "count": 23
#         },
#         {
#             "ip": "192.0.2.11",

```

```

#         "port": 23,
#         "minduration": 9,
#         "maxduration": 35,
#         "count": 6
#     }
# ]
#}
#
# Корреляционное событие:
#{
#     "correlation_type": "event",
#     "count": 1,
#     "importance": "info",
#     "subevents.time": [
#         "2017-10-18T12:00:00.000Z"
#     ],
#     "subevents": [
#         "cbf44df9-928f-4bad-be99-7a1adc3bf39f"
#     ],
#     "_rule": "Example",
#     "time": "2017-10-18T12:00:00.000Z",
#     "correlation_name": "Example",
#     "object": "client",
#     "action": "check",
#     ...
#}

```

Табличный список Tabular_List_Correlation_Arithm в формате YAML:

```

name: Tabular_List_Correlation_Arithm
fillType: CorrelationRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- key:
    index: false
    nullable: false
    primaryKey: true
    unique: true
    type: Number
- ip:
    index: false
    nullable: true
    primaryKey: false
    type: String

```


- port:
 - index: false
 - nullable: true
 - primaryKey: false
 - type: Number
- minduration:
 - index: false
 - nullable: true
 - primaryKey: false
 - type: Number
- maxduration:
 - index: false
 - nullable: true
 - primaryKey: false
 - type: Number
- count:
 - index: false
 - nullable: true
 - primaryKey: false
 - type: Number

Табличный список Tabular_List_Correlation_Arithm в формате JSON:

```
"Tabular_List_Correlation_Arithm": {
  "description": "Tabular list correlation rule - arithmetic operations",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "count": {
      "index": false,
      "nullable": true,
      "primaryKey": false,
      "type": "Number"
    },
    "ip": {
      "index": false,
```

```

        "nullable": true,
        "primaryKey": false,
        "type": "String"
    },
    "key": {
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "Number",
        "unique": true
    },
    "maxduration": {
        "index": false,
        "nullable": true,
        "primaryKey": false,
        "type": "Number"
    },
    "minduration": {
        "index": false,
        "nullable": true,
        "primaryKey": false,
        "type": "Number"
    },
    "port": {
        "index": false,
        "nullable": true,
        "primaryKey": false,
        "type": "Number"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"order": [
    "key",
    "ip",
    "port",
    "minduration",
    "maxduration",
    "count",
    "_last_changed",
    "_id"
],
"ttl": 259200,
"type": 2,
"typicalSize": 80000
}

```

10.6.4. Инструкция `insert_into`. Добавление строк в табличный список

Инструкция `insert_into` может использоваться для добавления одной или нескольких строк в конец табличного списка. Для использования инструкции нужно указать:

- название табличного списка (должно начинаться с прописной буквы латинского алфавита);
- переменные, в которых из директивы `rule` в инструкцию `insert_into` передаются значения ячеек табличного списка;

Примечание. При добавлении нескольких значений переменные имеют тип данных `List`. Для формирования списка значений можно использовать функцию языка XPL [append](#) (см. раздел 4.7.1).

- имена колонок (с префиксом `column: :`), в которые нужно добавить значения;
- значения ячеек для добавления в табличный список (тип данных должен совпадать с типом данных колонки табличного списка).

Примечание. Колонка может содержать список значений для ключевой колонки табличного списка. Если ключ из списка есть в табличном списке — значения строк перезаписываются, если нет — для них добавляются строки в конец табличного списка.

При добавлении строк в табличный список нужно указать значения ячеек для всех колонок. Если значения для колонки не указаны, результат записи зависит от свойств колонки, настроенных при создании табличного списка: если колонка может содержать `null` — в добавляемые ячейки будет записано значение `null`, если нет — появится сообщение об ошибке. Если для колонки указано одно значение, оно будет записано в ячейки всех добавляемых строк.

Формат вызова:

```
rule <Название правила>: <Условие правила>
  on <Название события> {
    append($f1, <Значение 1>)
    append($f2, <Значение 2>)
    ...
  }
  insert_into <Название табличного списка> {
    column::<Имя колонки 1> = $f1
    column::<Имя колонки 2> = $f2
    ...
  }
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходными событиями в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**.

после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_CorrelationRule (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходные события:

```
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 24,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "open",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
{
  "time": "2017-10-18T12:00:00.000Z",
  "event_src.ip": "10.0.12.45",
  "object.account.domain": "super_admin",
  "dst.ip": "192.0.2.11",
  "dst.port": 25,
  "protocol": "17",
  "object.process.id": "32571",
  "action": "close",
  "subject": "account",
  "object": "connection",
  "status": "success",
  "duration": 55,
  "importance": "info",
  "event_src.vendor": "Best Company",
  "event_src.title": "NG Session Controller",
  "event_src.category": "Other",
  "id": "ID_normalized_event_example",
  "event_src.host": "10.0.12.45",
  "dst.host": "192.0.2.11"
}
```

Правило корреляции:

```
event Event:
  key:
    event_src.vendor
  filter {
    dst.ip == "192.0.2.11"
  }

rule Example: Event[2]
  on Event { # Обработка события, присваивание переменным значений полей события
    $f1 = dst.ip
    $f2 = append($f2, dst.port)
    $f3 = append($f3, action)
  }

insert_into Tabular_List_CorrelationRule { # Добавление новых строк в табличный
  список Tabular_List_CorrelationRule с использованием переменных и их значений
  column::ip = $f1
  column::port = $f2
  column::status = $f3
}

emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}
```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "status": "close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "status": "close"
#       }
#   ]
#}
#
# Исходное событие 1:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 24,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}
#
# Исходное событие 2:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 25,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",

```

```

#     "status": "success",
#     ...
#}
#
# Конечное состояние табличного списка:
#{
#     "Tabular_List_CorrelationRule": [
#         {
#             "ip": "192.0.2.11",
#             "port": 21,
#             "status": "close"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 22,
#             "status": "open"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 23,
#             "status": "close"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 24,
#             "status": "open"
#         },
#         {
#             "ip": "192.0.2.11",
#             "port": 25,
#             "status": "close"
#         }
#     ]
#}
#
# Корреляционное событие:
#{
#     "correlation_type": "event",
#     "count": 1,
#     "importance": "info",
#     "subevents.time": [
#         "2017-10-18T12:00:00.000Z"
#     ],
#     "subevents": [
#         "d2a4bc87-8971-469d-b819-bb0f8bd78881"
#     ],

```

```
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   "correlation_name": "Example",
#   "object": "client",
#   "action": "check",
#   ...
#}
```

Табличный список Tabular_List_CorrelationRule в формате YAML:

```
name: Tabular_List_CorrelationRule
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
    nullable: false
    primaryKey: false
    type: String
```

Табличный список Tabular_List_CorrelationRule в формате JSON:

```
"Tabular_List_CorrelationRule": {
  "description": "Tabular list for correlation rule - main",
  "fields": {
    "_id": {
      "index": true,
```

```

        "nullable": false,
        "primaryKey": false,
        "type": "Number",
        "unique": true
    },
    "_last_changed": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Datetime"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "status": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",

```



```

        "ip",
        "port",
        "status",
        "_last_changed",
        "_id"
    ],
    "ttl": 259200,
    "type": 1,
    "typicalSize": 80000
}

```

10.6.5. Инструкция `remove_from`. Удаление строк из табличного списка

Инструкция `remove_from` предназначена для удаления строк из табличного списка. При выполнении правила корреляции удаляются строки, удовлетворяющие указанному в инструкции условию. В инструкции нужно указать название табличного списка и условие удаления строк. В условии можно использовать переменные, имена колонок табличного списка (с префиксом `column::`) и логические операторы.

Формат вызова:

```

rule <Название правила>: <Условие правила>
...
remove_from <Название табличного списка> {
    <Условие для удаления>
}

```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_CorrelationRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```

{"time":"2017-10-18T12:00:00.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":22,"protocol":"17","object.process.id":"32571","action":"open","subject":"account","object":"connection","status":"success","importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"ID_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11"}

```

Правило корреляции:

```

event Event:
  key:
    event_src.vendor
  filter {
    event_src.vendor == "Best Company"
  }

rule Example: Event
  remove_from Tabular_List_CorrelationRule { # Удаление из табличного списка
    записей, которые соответствуют условию Port > 22
    column::Port > 22
  }

emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
  $importance = "info"
}

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#     {
#       "ip": "192.0.2.11",
#       "port": 21,
#       "status": "Close"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 22,
#       "status": "Open"
#     },
#     {
#       "ip": "192.0.2.11",
#       "port": 23,
#       "status": "Close"
#     }
#   ]
#}
#
# Исходное событие:
#{

```

```

#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}
#
# Конечное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "status": "Close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "Open"
#       }
#   ]
#}
#
# Корреляционное событие:
#{
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "f69176b8-6ce0-42dc-912d-3dbb68185e91"
#   ],
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   "correlation_name": "Example",
#   "object": "client",
#   "action": "check",
#   ...
#}

```

Табличный список Tabular_List_CorrelationRule в формате YAML:

```
name: Tabular_List_CorrelationRule
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
    nullable: false
    primaryKey: false
    type: String
```

Табличный список Tabular_List_CorrelationRule в формате JSON:

```
"Tabular_List_CorrelationRule": {
  "description": "Tabular list for correlation rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
```

```

        "nullable": false,
        "primaryKey": false,
        "type": "Datetime"
    },
    "complex_key": {
        "compositeFields": [
            "ip",
            "port"
        ],
        "index": false,
        "nullable": false,
        "primaryKey": true,
        "type": "composite",
        "unique": true
    },
    "ip": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "status": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",
    "ip",
    "port",
    "status",
    "_last_changed",
    "_id"
],
"ttl": 259200,

```

```

    "type": 1,
    "typicalSize": 80000
  }

```

10.6.6. Инструкция clear_table. Очистка табличного списка

Инструкция `clear_table` предназначена для удаления всех строк из табличного списка при выполнении правила корреляции. В инструкции нужно указать название табличного списка.

Формат вызова:

```

rule <Название правила>: <Условие правила>
...
clear_table <Название табличного списка>

```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_CorrelationRule` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```

{"time":"2017-10-18T12:00:00.000Z","event_src.ip":"10.0.12.45","object.account.domain":"super_admin","dst.ip":"192.0.2.11","dst.port":22,"protocol":"17","object.process.id":"32571","action":"open","subject":"account","object":"connection","status":"success","importance":"info","event_src.vendor":"Best Company","event_src.title":"NG Session Controller","event_src.category":"Other","id":"ID_normalized_event_example","event_src.host":"10.0.12.45","dst.host":"192.0.2.11"}

```

Правило корреляции:

```

event Event:
  key:
    event_src.vendor
  filter {
    event_src.vendor == "Best Company"
  }

rule Example: Event
  clear_table Tabular_List_CorrelationRule # Очистка таблицы

emit {
  $correlation_type = "event"
  $object = "client"
  $action = "check"
  $status = "success"
}

```

```

    $importance = "info"
}

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": [
#       {
#           "ip": "192.0.2.11",
#           "port": 21,
#           "status": "Close"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "Open"
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 23,
#           "status": "Close"
#       }
#   ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   ...
#}
#
# Конечное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule": []
#}
#
# Корреляционное событие:
#{

```

```
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "4fd15123-c2d4-4fe7-9dc9-2a3e416a3438"
#   ],
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   "correlation_name": "Example",
#   "object": "client",
#   "action": "check",
#   ...
#}
```

Табличный список Tabular_List_CorrelationRule в формате YAML:

```
name: Tabular_List_CorrelationRule
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ip
    - port
- ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
- port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
```



```

nullable: false
primaryKey: false
type: String

```

Табличный список Tabular_List_CorrelationRule в формате JSON:

```

"Tabular_List_CorrelationRule": {
  "description": "Tabular list for correlation rule - main",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "complex_key": {
      "compositeFields": [
        "ip",
        "port"
      ],
      "index": false,
      "nullable": false,
      "primaryKey": true,
      "type": "composite",
      "unique": true
    },
    "ip": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "String"
    },
    "port": {
      "index": false,
      "nullable": false,
      "primaryKey": false,
      "type": "Number"
    },
    "status": {
      "index": true,

```

```

        "nullable": false,
        "primaryKey": false,
        "type": "String"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",
    "ip",
    "port",
    "status",
    "_last_changed",
    "_id"
],
"ttl": 259200,
"type": 1,
"typicalSize": 80000
}

```

10.7. Директива emit. Корреляционное событие

Директива `emit` предназначена для заполнения полей корреляционного события (названия полей нужно начинать со знака доллара). Директива выполняется при выполнении условия правила корреляции.

В блоке операторов директивы `emit` вы можете использовать переменные, объявленные в инструкциях `init` и `on` (названия нужно начинать со знака доллара), функции и операторы языка XPL или литералы с типом данных, соответствующим заполняемому полю.

Внимание! Не используйте в блоке операторов директивы `emit` поля подлежащих корреляции событий.

Примечание. Если требуется удалить уже собранную цепочку событий из памяти MaxPatrol SIEM без регистрации корреляционного события, вы можете использовать в директиве `emit` команду `close`.

Формат вызова:

```

emit {
    <Блок операторов emit>
}

```

Пример

```

emit {
    $correlation_type = "incident"
    $category.generic = "Attacks & Recon"
}

```

```

$category.high = "Attack"
$category.low = "Bruteforce"
$subject = "account"
$action = "initiate"
$object = "attack"
$status = "success"
$object.name = "Bruteforce"
$id = "PT_SIEM_Bruteforce_success_by_user"
$importance = "medium"
}

```

10.8. Команда `copy_fields`

Команда `copy_fields` используется для копирования значений из группы полей события с одним префиксом в группу полей с другим префиксом.

Названия полей должны отличаться только префиксами, указанными в аргументах. Префикс полей, в которые нужно вставить значения, нужно указать в первом аргументе. Префикс полей, из которых нужно скопировать значения, — во втором.

Формат вызова:

```
copy_fields(f1, f2)
```

Примечание. Для префиксов второго порядка и выше (например, `object.account` или `object.application.account`) выполняется дополнительная проверка: одна группа полей должна быть полным подмножеством другой либо состав обеих групп полей должен совпадать.

Команду `copy_fields` можно использовать для следующих действий:

- копирование значений полей исходного события в поля корреляционного события в инструкции `on` директивы `rule`:

```
copy_fields($object.account, object.account)
copy_fields($object.account, subject.application.account)
```
- копирование значений полей корреляционного события в другие поля корреляционного события в инструкциях `init` и `on` директивы `rule` и в директиве `emit`:

```
copy_fields($object.account, $subject.account)
copy_fields($object.account, $subject.application.account)
copy_fields($subject.application.account, $object.account)
```

Пример

```

rule Bruteforce_success_by_user: (Login_failure[5,] -> Login_success) within 5m
  init{
    $count.subevents = 0
  }
  on Login_failure {
    $count.subevents = $count.subevents + 1
  }

```

```

    }
    on Login_success {
        # $subject.account.name = subject.account.name
        # $subject.account.domain = subject.account.domain
        # $subject.account.id = subject.account.id
        # ...
        copy_fields($subject.account, subject.account)
        $count.subevents = $count.subevents + 1
    }
    emit {
        $correlation_type = "incident"
        $category.generic = "Attacks & Recon"
        $category.high = "Attack"
        $category.low = "Bruteforce"
        $subject = "account"
        $action = "initiate"
        $object = "attack"
        $status = "success"
        $object.name = "Bruteforce"
        $id = "PT_SIEM_Bruteforce_success_by_user"
        $importance = "medium"
        copy_fields($subject.application.account, $subject.account)
    }
}

```

10.9. Директива query. Объявление запроса к табличному списку

Директива `query` может использоваться для объявления запроса к табличному списку. В директиве нужно указать:

- название запроса, уникальное в рамках правила корреляции. В одном правиле корреляции можно объявить несколько запросов с разными названиями к одному или нескольким табличным спискам. Обращение к запросу выполняется по его названию;
- название табличного списка, к которому выполняется запрос (должно начинаться с прописной буквы латинского алфавита);
- аргументы запроса — переменные, используемые при составлении условия запроса. Значения переменных передаются в запрос при выполнении запроса. Запрос можно выполнить с помощью функций языка XP `exec_query` или `select_query_first`;
- условие, проверяемое при выполнении запроса. В условии запроса можно использовать значения переменных, переданные в аргументах запроса, имена колонок табличного списка (с префиксом `column:`), логические операторы и функции языка XP `in_subnet`, `regex_match`.

Примечание. При использовании функции `in_subnet` в условии запроса `query` к табличному списку в первом аргументе функции нужно указать имя переменной, переданной в запрос, во втором — имя колонки табличного списка (с префиксом `column::`).

Формат вызова:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
    <Условие запроса>
}
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список `Tabular_List_CorrelationRule_Query` (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{ "time": "2017-10-18T12:00:00.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 22, "protocol": "17", "object.process.id": "32571", "action": "open", "subject": "account", "object": "connection", "status": "success", "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11" }
```

Правило корреляции:

```
query TestQuery ($check_ip, $check_port) from Tabular_List_CorrelationRule_Query {
    in_subnet($check_ip, column::ipadr) and # Условие запроса возвращает True, если
    переданный в запросе узел находится в подсети 192.0.2.11/24, а переданный порт есть в
    табличном списке в состоянии Open
    column::port == $check_port and
    column::status == "Open"
}
```

event Event: # Выбор событий от источника Best Company, удовлетворяющих условию запроса

```
key:
    event_src.vendor
filter {
    event_src.vendor == "Best Company" and
    exec_query("TestQuery", [dst.ip, dst.port])
}
```

rule Example: Event

```
on Event {
    $tm = select_query_first("TestQuery", [dst.ip, dst.port], "dur") # Выполнение
    запроса TestQuery
```

```

        $datafield1 = "IP_" + string(dst.ip)
        $datafield2 = "Port_" + string(dst.port)
        $datafield3 = "Open_time_" + string($tm)
    }

    emit {
        $correlation_type = "event"
        $object = "client"
        $action = "check"
        $status = "success"
        $importance = "info"
    }
}

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule_Query": [
#       {
#           "ipadr": "192.0.2.0/24",
#           "port": 21,
#           "status": "Close",
#           "dur": 2
#       },
#       {
#           "ipadr": "192.0.2.0/24",
#           "port": 22,
#           "status": "Open",
#           "dur": 36
#       },
#       {
#           "ipadr": "192.0.2.0/24",
#           "port": 23,
#           "status": "Close",
#           "dur": 82
#       }
#   ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",
#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "protocol": "17",

```

```
#   "object.process.id": "32571",
#   "action": "open",
#   "subject": "account",
#   ...
#}
#
# Корреляционное событие:
#{
#   "correlation_type": "event",
#   "count": 1,
#   "datafield1": "IP_192.0.2.11",
#   "datafield2": "Port_22",
#   "datafield3": "Open_time_36",
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "306137b2-d540-437d-844c-184cc3a2ee27"
#   ],
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   ...
#}
#}
```

Табличный список Tabular_List_CorrelationRule_Query в формате YAML:

```
name: Tabular_List_CorrelationRule_Query
fillType: CorrelationRule
type: 2
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite
    unique: true
    compositeFields:
    - ipadr
    - dur
- ipadr:
    index: false
    nullable: false
    primaryKey: false
```

```

    type: String
- port:
    index: true
    nullable: false
    primaryKey: false
    type: Number
- status:
    index: true
    nullable: false
    primaryKey: false
    type: String
- dur:
    index: true
    nullable: false
    primaryKey: false
    type: Number

```

Табличный список Tabular_List_CorrelationRule_Query в формате JSON:

```

"Tabular_List_CorrelationRule_Query": {
  "description": "Tabular list for query",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "complex_key": {
      "compositeFields": [
        "ipadr",
        "dur"
      ],
      "index": false,
      "nullable": false,
      "primaryKey": true,
      "type": "composite",
      "unique": true
    },
    "dur": {

```



```

        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "ipadr": {
        "index": false,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    },
    "port": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "Number"
    },
    "status": {
        "index": true,
        "nullable": false,
        "primaryKey": false,
        "type": "String"
    }
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"objectId": "LOC-TL-1",
"order": [
    "complex_key",
    "ipadr",
    "port",
    "status",
    "dur",
    "_last_changed",
    "_id"
],
"ttl": 259200,
"type": 2,
"typicalSize": 80000
}

```

В этом разделе

[Инструкции qhandler, limit, skip \(см. раздел 10.9.1\)](#)

[Агрегатные функции для результатов запроса к табличному списку \(см. раздел 10.9.2\)](#)

См. также

[in_subnet](#) (см. раздел 4.8.3)

Функция [regex_match](#), проверка строки по регулярному выражению из табличного списка (см. раздел 4.9.5)

Функция [exes_query](#), запрос к табличному списку (см. раздел 4.9.2)

Функция [select_query_first](#), запрос значения из табличного списка (см. раздел 4.9.4)

10.9.1. Инструкции qhandler, limit, skip

Инструкция `qhandler` используется для выполнения арифметических операций над результатом запроса к табличному списку. При выполнении запроса, с инструкцией `qhandler`, функция `exes_query` возвращает ассоциативный массив (с типом данных `KeyValue`), содержащий результаты вычислений (с типом данных `Number`) для указанных в инструкции агрегатных функций. Для использования инструкции нужно указать:

- агрегатную функцию для выполнения арифметической операции;
- имя колонки табличного списка (с префиксом `column:`), к элементам которого применяется агрегатная функция (поддерживаются операции над элементами колонок с типом данных `Number`);
- ключ для получения результата применения агрегатной функции из ассоциативного массива, возвращаемого функцией `exes_query`.

Инструкция `limit` используется для указания количества первых строк в результате запроса, к которым применяются агрегатные функции, указанные в инструкции `qhandler`.

Инструкция `skip` используется для исключения указанного количества первых строк из результата запроса, к оставшимся строками применяются агрегатные функции, указанные в инструкции `qhandler`.

Примечание. При использовании инструкций `limit` и `skip` нужно учитывать, что сортировка строк в результате запроса не определена.

Формат вызова:

```
query <Название запроса>(<Аргументы запроса>) from <Название табличного списка> {
  <Условие запроса>
}
qhandler {
  <Ключ 1> = <Агрегатная функция 1>(column:<Имя колонки 1>)
  <Ключ 2> = <Агрегатная функция 2>(column:<Имя колонки 2>)
  ...
}
limit <Максимальное число возвращаемых строк>
skip <Количество строк, исключаемых из результата>
```

Пример

Вы можете использовать код из примера, чтобы проверить работу правила с помощью утилиты PTSIEMSDK GUI. Для этого необходимо скопировать код из блока с исходным событием в поле **Исходное событие** утилиты, а код из блока с правилом корреляции — в поле **Формула**, после чего нажать **Запустить**. Коррелированное событие отобразится в поле **Результат**. Для работы правила необходим табличный список Tabular_List_CorrelationRule_Duration (примеры табличного списка в форматах YAML и JSON см. ниже).

Исходное событие:

```
{ "time": "2017-10-18T12:00:00.000Z", "event_src.ip": "10.0.12.45", "object.account.domain": "super_admin", "dst.ip": "192.0.2.11", "dst.port": 22, "protocol": "17", "object.process.id": "32571", "action": "open", "subject": "account", "object": "connection", "status": "success", "importance": "info", "event_src.vendor": "Best Company", "event_src.title": "NG Session Controller", "event_src.category": "Other", "id": "ID_normalized_event_example", "event_src.host": "10.0.12.45", "dst.host": "192.0.2.11" }
```

Правило корреляции:

```
# Запрос TestQuery к табличному списку Tabular_List_CorrelationRule_Duration
query TestQuery ($check_ip) from Tabular_List_CorrelationRule_Duration {
  column::ip == $check_ip and
  column::port == 22 and
  column::status == "Open"
}

qhandler {
  $get_max_duration = max(column::duration)
  $get_min_duration = min(column::duration)
  $get_avg_duration = avg(column::duration)
  $get_count = count()
} # Использование арифметических функций при получении записей из табличного списка
```

```
event Event:
  key:
    event_src.vendor
  filter {
    event_src.vendor == "Best Company"
  }
```

```
rule Example: Event
  on Event {
    $get_duration = exec_query("TestQuery", [dst.ip])
    # Присваивание переменным максимального, минимального и среднего времени
    # открытия порта 22 для переданного в запросе IP-адреса узла
    $max_duration = $get_duration["$get_max_duration"]
    $min_duration = $get_duration["$get_min_duration"]
    $avg_duration = $get_duration["$get_avg_duration"]
```

```

$counter = $get_duration["$get_count"]
$datafield1 = "IP_" + string(dst.ip)
$datafield2 = "Max_duration_" + string($max_duration)
$datafield3 = "Min_duration_" + string($min_duration)
$datafield4 = "Average_duration_" + string($avg_duration)
$datafield5 = "Result_count_" + string($counter)
}

emit {
    $correlation_type = "event"
    $object = "client"
    $action = "check"
    $status = "success"
    $importance = "info"
}

```

Результат работы правила:

```

# Исходное состояние табличного списка:
#{
#   "Tabular_List_CorrelationRule_Duration": [
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "Open",
#           "duration": 41
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "Open",
#           "duration": 33
#       },
#       {
#           "ip": "192.0.2.11",
#           "port": 22,
#           "status": "Open",
#           "duration": 67
#       }
#   ]
#}
#
# Исходное событие:
#{
#   "time": "2017-10-18T12:00:00.000Z",
#   "event_src.ip": "10.0.12.45",
#   "object.account.domain": "super_admin",

```

```

#   "dst.ip": "192.0.2.11",
#   "dst.port": 22,
#   "action": "open",
#   "subject": "account",
#   "object": "connection",
#   "status": "success",
#   "importance": "info",
#   "event_src.vendor": "Best Company",
#   ...
#}
#
# Корреляционное событие:
#{
#   "datafield1": "IP_192.0.2.11",
#   "datafield2": "Max_duration_67",
#   "datafield3": "Min_duration_33",
#   "datafield4": "Average_duration_47",
#   "datafield5": "Result_count_3",
#   "correlation_type": "event",
#   "count": 1,
#   "importance": "info",
#   "subevents.time": [
#       "2017-10-18T12:00:00.000Z"
#   ],
#   "subevents": [
#       "ed212d91-627e-4596-85bd-ef24a3bb3f9d"
#   ],
#   "_rule": "Example",
#   "time": "2017-10-18T12:00:00.000Z",
#   "correlation_name": "Example",
#   ...
#}

```

Табличный список Tabular_List_CorrelationRule_Duration в формате YAML:

```

name: Tabular_List_CorrelationRule_Duration
fillType: CorrelationRule
type: 1
maxSize: 100000
ttl: 259200
typicalSize: 80000
fields:
- complex_key:
    index: false
    nullable: false
    primaryKey: true
    type: composite

```

```

    unique: true
    compositeFields:
      - ip
      - port
      - duration
  - ip:
    index: false
    nullable: false
    primaryKey: false
    type: String
  - port:
    index: false
    nullable: false
    primaryKey: false
    type: Number
  - status:
    index: true
    nullable: false
    primaryKey: false
    type: String
  - duration:
    index: true
    nullable: false
    primaryKey: false
    type: Number

```

Табличный список Tabular_List_CorrelationRule_Duration в формате JSON:

```

"Tabular_List_CorrelationRule_Duration": {
  "description": "Tabular list for correlation rule - duration",
  "fields": {
    "_id": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Number",
      "unique": true
    },
    "_last_changed": {
      "index": true,
      "nullable": false,
      "primaryKey": false,
      "type": "Datetime"
    },
    "complex_key": {
      "compositeFields": [
        "ip",

```

```

        "port",
        "duration"
    ],
    "index": false,
    "nullable": false,
    "primaryKey": true,
    "type": "composite",
    "unique": true
},
"duration": {
    "index": true,
    "nullable": false,
    "primaryKey": false,
    "type": "Number"
},
"ip": {
    "index": false,
    "nullable": false,
    "primaryKey": false,
    "type": "String"
},
"port": {
    "index": false,
    "nullable": false,
    "primaryKey": false,
    "type": "Number"
},
"status": {
    "index": true,
    "nullable": false,
    "primaryKey": false,
    "type": "String"
}
},
"fillType": "CorrelationRule",
"maxSize": 100000,
"order": [
    "complex_key",
    "ip",
    "port",
    "status",
    "duration",
    "_last_changed",
    "_id"
],
"ttl": 259200,

```

```

    "type": 1,
    "typicalSize": 80000
  }

```

10.9.2. Агрегатные функции для результатов запроса к табличному списку

Указанные в таблице агрегатные функции используются только в инструкции `qhandler` для выполнения арифметических операций над результатами запроса к табличному списку.

Таблица 97. Агрегатные функции для выполнения арифметических операций над результатами запроса к табличному списку

Агрегатная функция	Возвращаемое значение
<code>min(column::<Имя колонки>)</code>	Наименьший элемент в результатах запроса
<code>max(column::<Имя колонки>)</code>	Наибольший элемент в результатах запроса
<code>avg(column::<Имя колонки>)</code>	Среднее значение элементов, округленное до целого
<code>median(column::<Имя колонки>)</code>	Значение медианного элемента. При четном числе элементов выбирается нижнее медианное значение
<code>sum(column::<Имя колонки>)</code>	Сумма всех элементов результатов запроса
<code>count()</code>	Количество элементов в результатах запроса
<code>column::<Имя колонки></code>	Первый элемент из результатов запроса

11. Создание файла расширения аудита

Для управления активами в MaxPatrol SIEM используется доменная модель, подробно описывающая активы для регулярного анализа защищенности инфраструктуры. Модель представлена в виде классов с набором атрибутов. Полный модельный тип актива можно посмотреть в веб-интерфейсе MaxPatrol SIEM.

Вершиной модели является корневая сущность, поддерживающая наследование. Одна из корневых сущностей модели актива Host с пространством имен Core используется для расширения аудита. Например, сервер с установленным программным обеспечением описывается классом Core.Host (или его наследником), который в качестве одного из атрибутов содержит коллекцию программного обеспечения класса Core.Software (или его наследника).

Для возможности расширения аудита в классы Core.Host и Core.Software добавлен атрибут `AdditionalProperties` с типом данных Dictionary, который позволяет сохранять в формате коллекции пар «ключ — значение» информацию, которой нет в модели. В отличие от остальной модели, содержимое `AdditionalProperties` не привязано к какой-то схеме данных.

Для задач на сбор данных с модулем Audit вы можете расширить область сбора данных с помощью файла в формате YAML. При выполнении задачи будет проведен аудит данных в соответствии с конфигурацией файла расширения.

Внимание! YAML-файл расширения аудита включает либо секции `$extractors`, `$transformers` и `$loaders` (при этом заполнение секции `$transformers` не является обязательным), либо секции `$converters` и `$loaders`.

Описание секций представлено в разделах ниже. Если секции заполнены некорректно, возможны следующие ошибки валидации:

- `String: False` — некорректный тип данных параметра `root`.
- `Allow: False` — некорректный тип данных для поля в схеме данных.
- `Correct type: False` — указанный тип данных поля не соответствует типу данных, которые возвращает XML-функция.
- `AllParamsExist: False` — не заполнены все обязательные параметры поля.

Примечание. В названии файла не рекомендуется использовать точки и пробелы.

В этом разделе

[Секция `\$converters` \(см. раздел 11.1\)](#)

[Секция `\$extractors` \(см. раздел 11.2\)](#)

[Секция `\$transformers` \(см. раздел 11.3\)](#)

[Секция `\$loaders` \(см. раздел 11.4\)](#)

Примеры заполнения файлов расширения аудита с секциями `$converters` и `$loaders` (см. раздел 11.5)

Примеры заполнения файлов расширения аудита с секциями `$extractors`, `$transformers` и `$loaders` (см. раздел 11.6)

11.1. Секция `$converters`

Секция `$converters` описывает сбор и преобразование данных с помощью Python-скриптов и функций API для последующей передачи в модель актива.

В секции `$converters` необходимо заполнить следующие поля:

- `<Имя конвертера>` — определяет название конвертера. Обязательное поле.
- `$template` — объявляет используемый плагин (для конвертера всегда `converter`). Обязательное поле.
- `$origins` — определяет список дополнительных источников данных (результат других конвертеров). Опциональное поле.
- `$params` — определяет параметры, которые могут быть переданы из ладеров или других конвертеров. Как правило, представлено в виде `<Имя параметра>: <Тип данных>`. Опциональное поле.
- `$schema` — определяет схему вывода Python-скрипта. Обязательное поле.
- `$script` — содержит Python-скрипт. Вы можете либо ввести текст скрипта, либо использовать команду `!include <Имя файла скрипта>.py` (в этом случае файл скрипта необходимо разместить в той же папке, где сохранен YAML-файл расширения аудита). Обязательное поле.

В полях `$schema` и `$params` используются базовые типы данных `String`, `Int`, `Decimal`, `Binary`, `Boolean`, `DateTime`, а также списки (`Array`) и словари (`Dictionary`) из перечисленных типов данных.

Пример секции `$converters`:

```
$converters:
  Process: # Имя конвертера
    $template: converter # Плагин
    $origins: # Передаваемые конвертеры
    - TestConverter
    $params: # Передаваемые параметры
      some_param: String
    $schema: # Схема вывода Python-скрипта
      ProcessJSON: String
    $script: |
      print('Hello World!')
```

В таблице приведены примеры ошибок, выявляемых при валидации полей секции `$converters`.

Таблица 98. Возможные ошибки (секция \$converters)

Ошибка	Описание
<pre>\$Converter: - \$Kind: ConverterTemplate - \$Origins: - \$Name: WrongConverterName - ?Exists: false</pre>	Имя конвертера указано некорректно (не существует)
<pre>\$Converter: - \$Kind: ConverterTemplate - \$Origins: - ?IsArray: false</pre>	В поле \$origins необходимо передавать только список
<pre>\$Converter: - \$Kind: ConverterTemplate - \$Params: - \$Name: some_name - ?CorrectType: false</pre>	Тип параметра указан некорректно
<pre>\$Converter: - \$Kind: ConverterTemplate - \$Origins: - \$Name: SomeName - ?DuplicatesAllowed: false</pre>	В поле \$origins не допускается передавать дублирующие имена
<pre>\$Converter: - \$Kind: ConverterTemplate - \$Script: - ?WithoutSyntaxError: false</pre>	В Python-скрипте обнаружена синтаксическая ошибка

В этом разделе

[Audit Extension SDK \(см. раздел 11.1.1\)](#)

[Функции API \(см. раздел 11.1.2\)](#)

11.1.1. Audit Extension SDK

Audit Extension SDK предназначен для разработки и отладки пользовательских Python-скриптов расширенного сбора данных для задач, использующих модуль Audit. В соответствии с инструкцией по расширению области сбора данных для задач с модулем Audit (см. раздел «Расширение области сбора данных для задач с модулем Audit» Руководства оператора) необходимо [создать YAML-файл расширения аудита \(см. раздел 11\)](#), в котором указать файл Python-скрипта или непосредственно текст скрипта.

Для добавления пользовательских скриптов необходимо поместить подготовленные YAML-файлы расширения аудита и файлы Python-скриптов в папку `sdk\AuditExtensions`. Примеры доступны в папке `examples`.

Для настройки профиля подключения необходимо отредактировать файл `profile.json`. По умолчанию используется профиль `Windows localhost`. Примеры конфигураций профилей размещены в папке `profiles`.

Для запуска скрипта необходимо выполнить команду `.\run_extension.cmd <Имя YAML-файла расширения аудита> <Имя конвертера, содержащего скрипт>`. Пример команды: `.\run_extension.cmd hello_world HelloWorld`.

Результат выполнения скрипта отображается в терминале и в файле `.scandata\run_extension\cache.yaml` и включает:

- имя вызванного скрипта;
- `id` — порядковый номер вызова;
- `$request` — переданные параметры (при наличии);
- `$response` — результат выполнения скрипта в виде списка в соответствии с заданной [схемой вывода \(см. раздел 11.1\)](#).

Примечание. Если требуется выполнить полное сканирование, вы можете использовать команду `.\run_scan.cmd`. Результаты сканирования сохраняются в папку `.scandata\run_scan\`. Журнал сканирования сохраняется в файл `sdk\scanner-sdk\XPython.log`.

11.1.2. Функции API

В этом разделе приведено описание функций API, которые вы можете использовать в Python-скриптах расширения аудита.

mpx.data

Отображает список словарей, в которые записывается результат выполнения скрипта (для полей, которые указаны [в схеме вывода \(см. раздел 11.1\)](#)).

Пример для одной записи:

```
mpx.data.append(
    Name='Name 1',
    Version=1
)
```

Пример результата:

```
MP9.Capture: Result of operation
MP9.Capture: 1: ds_extension.test.Test(Name='Name 1', Version=1)
```

Пример для нескольких записей:

```
for name, version in (('Name 1', 1), ('Name 2', 2), ('Name 3', 3)):
    mpx.data.append(
        Name=name,
```

```

        Version=version
    )

```

Пример результата:

```

MP9.Capture: Result of operation
MP9.Capture: 1: ds_extension.test.Test(Name='Name 1', Version=1)
MP9.Capture: 2: ds_extension.test.Test(Name='Name 2', Version=2)
MP9.Capture: 3: ds_extension.test.Test(Name='Name 3', Version=3)

```

mpx.log

Выводит сообщение в терминале и сохраняет его в журнал.

Уровни журналирования:

- info: `mpx.log.info('Info log');`
- debug: `mpx.log.debug('Debug log');`
- warn: `mpx.log.warn('Warn log');`
- warning: `mpx.log.warning('Warning log');`
- error: `mpx.log.error('Error log');`

mpx.origin

Отображает коллекцию источников, переданных из других конвертеров.

Пример для источника, содержащего одну запись:

```
processes = mpx.origin.GetProcess().try_one().Data
```

Пример для источника, содержащего несколько записей:

```

uninstall_data = mpx.origin.UninstallData()
for item in uninstall_data:
    print(item)

```

Вы можете отфильтровать записи, используя булево значение: `uninstall_data = mpx.origin.UninstallData().filter(lambda item: item.InstallPath is not None)`. Вы также можете отфильтровать их по полям схемы вывода: `uninstall_data = mpx.origin.UninstallData().filter_by(Publisher='GitHub, Inc.', Architecture='x86')`.

mpx.params

Отображает параметры, переданные в скрипт.

```

path = mpx.params['path']
version = mpx.windows.shell_exec(command=f'{path} --version')
parsed_pg_data = mpx.origin.PgData(pg_data=mpx.params['pg_data'])

```

Примечание. При отладке скрипта через SDK и запуске скрипта по команде `run_extension.cmd` (неполное сканирование), когда параметры передаются [из ладера \(см. раздел 11.4\)](#), необходимо ввести значения параметров в терминале.

mpx.stop

Останавливает выполнение скрипта, выводит сообщение в журнал (если сообщение указано).

Варианты использования:

- `mpx.stop()` — останавливает выполнение скрипта (по умолчанию — без условий);
- `mpx.stop(True)` — останавливает выполнение скрипта;
- `mpx.stop(False)` — продолжает выполнение скрипта;
- `mpx.stop(True, 'some reason')` — останавливает выполнение скрипта, выводит сообщение в журнал.

Пример 1:

```
test_var = 'Stop script'
mpx.stop('Stop' in test_var)
```

Пример 2:

```
test_int_var = 10
mpx.stop(test_int_var != 10, reason='var is not 10')
```

Пример 3:

```
processes = mpx.windows.powershell_exec(command='Get-Process')
mpx.stop(not processes)
```

В этом разделе

[mpx.ldap_exec](#) (см. раздел 11.1.2.1)

[mpx.odbc.mssql_exec](#) (см. раздел 11.1.2.2)

[mpx.odbc.mysql_exec](#) (см. раздел 11.1.2.3)

[mpx.odbc.oracle_exec](#) (см. раздел 11.1.2.4)

[mpx.odbc.postgre_exec](#) (см. раздел 11.1.2.5)

[mpx.scanner.check_port](#) (см. раздел 11.1.2.6)

[mpx.scanner.dns_lookup](#) (см. раздел 11.1.2.7)

[mpx.scanner.dns_reverse_lookup](#) (см. раздел 11.1.2.8)

[mpx.scanner.get_profile_data](#) (см. раздел 11.1.2.9)

[mpx.scanner.get_scanning_ip](#) (см. раздел 11.1.2.10)

[mpx.unix.get_file_by_path](#) (см. раздел 11.1.2.11)

[mpx.unix.ssh_exec](#) (см. раздел 11.1.2.12)
[mpx.windows.delete_file](#) (см. раздел 11.1.2.13)
[mpx.windows.dir_exist](#) (см. раздел 11.1.2.14)
[mpx.windows.eight_dot_three_to_fullpath](#) (см. раздел 11.1.2.15)
[mpx.windows.file_exist](#) (см. раздел 11.1.2.16)
[mpx.windows.find_files](#) (см. раздел 11.1.2.17)
[mpx.windows.get_account_name_by_sid](#) (см. раздел 11.1.2.18)
[mpx.windows.get_audit_policy](#) (см. раздел 11.1.2.19)
[mpx.windows.get_binary_file_by_path](#) (см. раздел 11.1.2.20)
[mpx.windows.get_dirs](#) (см. раздел 11.1.2.21)
[mpx.windows.get_file_by_path](#) (см. раздел 11.1.2.22)
[mpx.windows.get_file_info_by_path](#) (см. раздел 11.1.2.23)
[mpx.windows.get_file_info_recursive](#) (см. раздел 11.1.2.24)
[mpx.windows.get_filetime](#) (см. раздел 11.1.2.25)
[mpx.windows.get_lockout_policy](#) (см. раздел 11.1.2.26)
[mpx.windows.get_registry_values](#) (см. раздел 11.1.2.27)
[mpx.windows.get_security_info](#) (см. раздел 11.1.2.28)
[mpx.windows.get_sid_string_by_account_name](#) (см. раздел 11.1.2.29)
[mpx.windows.get_system_temp_directory](#) (см. раздел 11.1.2.30)
[mpx.windows.get_user_sids_by_privilege](#) (см. раздел 11.1.2.31)
[mpx.windows.get_users_by_privilege](#) (см. раздел 11.1.2.32)
[mpx.windows.normalize_path](#) (см. раздел 11.1.2.33)
[mpx.windows.powershell_exec](#) (см. раздел 11.1.2.34)
[mpx.windows.shell_exec](#) (см. раздел 11.1.2.35)
[mpx.windows.wmi_exec](#) (см. раздел 11.1.2.36)
[mpx.windows.wmi_query](#) (см. раздел 11.1.2.37)
[mpx.windows.write_file](#) (см. раздел 11.1.2.38)

11.1.2.1. mpx.ldap_exec

Выполняет рекурсивный поиск по дереву LDAP.

Использует следующие параметры:

- `attributes` — список атрибутов (строка);
- `distinguished_name` — отличительное имя учетной записи (строка);
- `page_size` — количество сущностей, отображаемых на странице поиска (целое число). Значение по умолчанию — 0 (означает бесстраничный поиск, при котором отображаются все найденные сущности);
- `search_scope` — область поиска (строка). Возможные значения: 'base', 'onelevel', 'subtree';
- `where` — поисковый фильтр LDAP (строка).

Пример:

```
root_dse = mpx.ldap_exec(
    search_scope='base',
    where='(ObjectClass=*)',
    attributes=['ConfigurationNamingContext', 'DefaultNamingContext', 'DnsHostName',
'DomainFunctionality', 'ForestFunctionality']
)
domain_dns = mpx.ldap_exec(
    distinguished_name=f'CN=Enterprise Configuration,CN=Partitions,
{root_dse.configurationNamingContext}',
    search_scope='base',
    where='(ObjectClass=*)',
    attributes=['DnsRoot']
)
mpx.data.append(
    DomainDNS=domain_dns.dnsRoot
)
```

11.1.2.2. mpx.odbc.mssql_exec

Выполняет запрос к базе данных MS SQL.

Использует следующие параметры:

- `instance_name` — имя экземпляра, в котором выполняется запрос (строка). Значение по умолчанию — None (используется экземпляр, указанный в профиле);
- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из SQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из SQL-запроса, значение — новое имя столбца. Значение по умолчанию — None;
- `port` — порт (целое число). Значение по умолчанию — None (используется порт, указанный в профиле);
- `query` — SQL-запрос (строка).

Пример:

```
config_value = mpx.odbc.mssql_exec(
    query="exec xp_loginconfig 'audit level';",
    instance_name="some_instanse_name",
    mapping={'config_value': 'ConfigValue'}
).ConfigValue
mpx.data.append(Value=config_value)
```

11.1.2.3. mpx.odbc.mysql_exec

Выполняет запрос к базе данных MySQL.

Использует следующие параметры:

- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из SQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из SQL-запроса, значение — новое имя столбца. Значение по умолчанию — `None`;
- `port` — порт (целое число). Значение по умолчанию — `None` (используется порт, указанный в профиле);
- `query` — SQL-запрос (строка).

Пример:

```
users = mpx.odbc.mysql_exec(query='SELECT * FROM mysql.user')
result = list()
for user in users:
    name = user.Name
    host = user.Host
    password_hash = user.PasswordHash
    ssl_type = user.SSLType
    # some python code
    # some python code
    # some python code
    result.append(dict(
        Name=name,
        Host=host,
        PasswordHash=password_hash,
        SSLType=ssl_type,
    ))
mpx.data.append(UsersJSON=json.dumps(result, indent=4))
```

11.1.2.4. mpx.odbc.oracle_exec

Выполняет запрос к базе данных Oracle.

Использует следующие параметры:

- `instance` — экземпляр (строка);
- `instance_type` — тип экземпляра (строка). Значение по умолчанию — `None`;
- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из SQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из SQL-запроса, значение — новое имя столбца. Значение по умолчанию — `None`;
- `port` — порт (целое число). Значение по умолчанию — `None` (используется порт, указанный в профиле);
- `query` — SQL-запрос (строка).

Пример:

```
sql = '''
    SELECT rp.GRANTEE AS Name
           , rp.GRANTED_ROLE AS Role
           , rp.ADMIN_OPTION AS AdminOption
           , rp.DEFAULT_ROLE AS DefaultRole
    FROM dba_role_privs rp
'''

dba_roles = mpx.odbc.oracle_exec(query=sql)
for item in dba_roles:
    if item.Role != 'Admin':
        continue
    mpx.data.append(
        Name=item.Name,
        Role=item.Role,
        AdminOption=item.AdminOption,
        DefaultRole=item.DefaultRole
    )
```

11.1.2.5. mpx.odbc.postgre_exec

Выполняет запрос к базе данных PostgreSQL.

Использует следующие параметры:

- `database` — база данных, к которой выполняется запрос (строка). Значение по умолчанию — `None` (используется база данных, указанная в профиле);
- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из SQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из SQL-запроса, значение — новое имя столбца. Значение по умолчанию — `None`;

- `port` — порт (целое число). Значение по умолчанию — `None` (используется порт, указанный в профиле);
- `query` — SQL-запрос (строка).

Пример:

```
sql = '''
    SELECT
        ps.name,
        ps.setting
    FROM
        pg_catalog.pg_settings ps
    WHERE
        name in ('server_version', 'data_directory')
    ORDER BY
        ps.name;
'''

pg_settings = mpx.odbc.postgresql_exec(query=sql)
version = list(filter(lambda item: item.name == 'server_version', pg_settings))
[0].setting
directory = list(filter(lambda item: item.name == 'data_directory', pg_settings))
[0].setting
mpx.data.append(
    Version=version,
    Directory=directory
)
```

11.1.2.6. `mpx.scanner.check_port`

Проверяет доступность порта.

Использует следующие параметры:

- `address` — адрес (строка);
- `port` — порт (целое число).

Пример:

```
port_is_available = mpx.scanner.check_port(address=target_ip, port=5432)
print(port_is_available)
```

Пример результата:

```
root.stdout: True
```

11.1.2.7. `mpx.scanner.dns_lookup`

Возвращает IP-адрес по доменному имени.

Использует параметр `hostname` — имя узла (строка).

Пример:

```
ip_by_name = mpx.scanner.dns_lookup('Some Name')
print(ip_by_name)
```

Пример результата:

```
root.stdout: 127.0.0.1
```

11.1.2.8. mpx.scanner.dns_reverse_lookup

Возвращает доменное имя, связанное с IP-адресом.

Использует параметр `ip_address` — IP-адрес (строка).

Пример:

```
name_by_ip = mpx.scanner.dns_reverse_lookup('127.0.0.1')
print(name_by_ip)
```

Пример результата:

```
root.stdout: Some Name
```

11.1.2.9. mpx.scanner.get_profile_data

Возвращает параметр из профиля.

Использует параметр `parameter` — путь к параметру (строка).

Пример:

```
database = mpx.scanner.get_profile_data(parameter='transports/odbc_postgres/
connection/database')
print(database)
```

Пример результата:

```
root.stdout: Some Database
```

11.1.2.10. mpx.scanner.get_scanning_ip

Возвращает IP-адрес целевого узла.

Пример:

```
target_ip = mpx.scanner.get_scanning_ip()
print(target_ip)
```

Пример результата:

```
root.stdout: 127.0.0.1
```

11.1.2.11. mpx.unix.get_file_by_path

Возвращает содержимое файла.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```
etc_hosts = mpx.unix.get_file_by_path('/etc/hosts')
print(etc_hosts)
```

Пример результата:

```
root.stdout: 127.0.0.1 localhost
root.stdout: 127.0.1.1 ubuntu-20-04
root.stdout:
root.stdout: # The following lines are desirable for IPv6 capable hosts
root.stdout: ::1      ip6-localhost ip6-loopback
root.stdout: fe00::0 ip6-localnet
root.stdout: ff00::0 ip6-mcastprefix
root.stdout: ff02::1 ip6-allnodes
root.stdout: ff02::2 ip6-allrouters
```

11.1.2.12. mpx.unix.ssh_exec

Выполняет SSH-запрос.

Использует следующие параметры:

- `check_error` — булево значение: если во время выполнения запроса была обнаружена ошибка, то при значении `True` выводится `None`, при значении `False` — выводится ошибка. Значение по умолчанию — `True`;
- `command` — команда (строка).

Пример 1:

```
uname = mpx.unix.ssh_exec('uname -v')
print(uname)
```

Пример результата:

```
root.stdout: #224-Ubuntu SMP Thu Dec 5 13:38:28 UTC 2024
```

Пример 2:

```
raw_file_permissions = mpx.unix.ssh_exec(
    'ls -lZd /etc/at.deny /etc/cron.allow /etc/cron.deny boot/grub/grub.cfg /etc/
cron.d',
    check_error=False,
)
print(raw_file_permissions)
```

Пример результата:

```
root.stdout: ls: cannot access '/etc/at.deny': No such file or directory
root.stdout: ls: cannot access '/etc/cron.allow': No such file or directory
root.stdout: ls: cannot access '/etc/cron.deny': No such file or directory
root.stdout: -r--r--r-- 1 root root    ? 5813 Jun 19  2017 /boot/grub/grub.cfg
root.stdout: drwxr-xr-x 2 root root    ? 4096 May 29  2019 /etc/cron.d
```

11.1.2.13. mpx.windows.delete_file

Удаляет файл.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```
file_is_deleted = mpx.windows.delete_file(r'C:\ProgramData\SomePath\SomeFile.txt')
print(file_is_deleted)
```

Пример результата:

```
root.stdout: True
```

11.1.2.14. mpx.windows.dir_exist

Проверяет существование папки. Поддерживает переменные среды.

Использует параметр `path` — полный путь к папке (строка).

Пример:

```
git_dir_exist = mpx.windows.dir_exist(r'C:\Program Files\Git')
print(git_dir_exist)
```

Пример результата:

```
root.stdout: True
```

11.1.2.15. mpx.windows.eight_dot_three_to_fullpath

Восстанавливает путь к файлу или папке из короткого формата (8.3) в полный.

Использует параметр `path` — путь в формате 8.3 (строка).

Пример:

```
program_files = mpx.windows.eight_dot_three_to_fullpath(r'C:\PROGRA~1')
print(program_files)
```

Пример результата:

```
root.stdout: C:\Program Files
```

11.1.2.16. mpx.windows.file_exist

Проверяет существование файла. Поддерживает переменные среды.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```
git_bash_exist = mpx.windows.file_exist(r'C:\Program Files\Git\git-bash.exe')
print(git_bash_exist)
```

Пример результата:

```
root.stdout: True
```

11.1.2.17. mpx.windows.find_files

Выполняет поиск файлов в заданной папке по маске. Поддерживает рекурсивный поиск и символьные ссылки.

Использует следующие параметры:

- `file_mask` — маска для поиска файлов в заданной папке (поддерживаются регулярные выражения) (строка);
- `recursive` — булево значение: включение (`True`) или отключение (`False`) рекурсивного поиска на любую глубину;
- `root_dir` — корневая папка (строка).

Пример:

```
files = mpx.windows.find_files(
    root_dir=r'C:\Program Files\Git',
    file_mask='*.exe',
    recursive=True
)
for item in files:
    print(item)
```

Пример результата:

```
root.stdout: C:\Program Files\Git\git-bash.exe
root.stdout: C:\Program Files\Git\git-cmd.exe
root.stdout: C:\Program Files\Git\unins001.exe
root.stdout: C:\Program Files\Git\bin\bash.exe
root.stdout: C:\Program Files\Git\bin\git.exe
root.stdout: C:\Program Files\Git\bin\sh.exe
root.stdout: C:\Program Files\Git\cmd\git-gui.exe
root.stdout: C:\Program Files\Git\cmd\git-lfs.exe
root.stdout: C:\Program Files\Git\cmd\git-receive-pack.exe
root.stdout: C:\Program Files\Git\cmd\git-upload-pack.exe
```

11.1.2.18. mpx.windows.get_account_name_by_sid

Возвращает имя по заданному SID: для пользователя или группы — sAMAccountName, для домена — NetBiosName.

Использует параметр `sid` — SID (строка).

Пример:

```
username = mpx.windows.get_account_name_by_sid('S-1-5-21-123123123-123123123-123123123-12345')
print(username)
```

Пример результата:

```
root.stdout: username
```

11.1.2.19. mpx.windows.get_audit_policy

Возвращает состояние политик аудита.

Пример:

```
audit_policy = mpx.windows.get_audit_policy()
print(audit_policy)
```

Пример результата:

```
root.stdout: {
  "Account Logon": {
    "Credential Validation": "Success",
    "Kerberos Authentication Service": "Success",
    "Kerberos Service Ticket Operations": "Success",
    "Other Account Logon Events": "No auditing"
  },
  "Account Management": {
    "Application Group Management": "No auditing",
    "Computer Account Management": "Success",
    "Distribution Group Management": "No auditing",
    "Other Account Management Events": "No auditing",
    "Security Group Management": "Success",
    "User Account Management": "Success"
  },
  "Audit account logon events": "No auditing",
  "Audit account management": "No auditing",
  "Audit directory service access": "No auditing",
  "Audit logon events": "No auditing",
  "Audit object access": "No auditing",
  "Audit policy change": "No auditing",
  "Audit privilege use": "No auditing",
  "Audit process tracking": "No auditing",
```



```

"Audit system events": "No auditing",
"Auditing mode": "True",
"DS Access": {
  "Detailed Directory Service Replication": "No auditing",
  "Directory Service Access": "Success",
  "Directory Service Changes": "No auditing",
  "Directory Service Replication": "No auditing"
},
"Detailed Tracking": {
  "DPAPI Activity": "No auditing",
  "Plug and Play Events": "No auditing",
  "Process Creation": "No auditing",
  "Process Termination": "No auditing",
  "RPC Events": "No auditing",
  "Token Right Adjusted Events": "No auditing"
},
"Logon/Logoff": {
  "Access Rights": "No auditing",
  "Account Lockout": "Success",
  "Group Membership": "No auditing",
  "IPsec Extended Mode": "No auditing",
  "IPsec Main Mode": "No auditing",
  "IPsec Quick Mode": "No auditing",
  "Logoff": "Success",
  "Logon": "Success, Failure",
  "Network Policy Server": "Success, Failure",
  "Other Logon/Logoff Events": "No auditing",
  "Special Logon": "Success",
  "User / Device Claims": "No auditing"
},
"Object Access": {
  "Application Generated": "No auditing",
  "Central Policy Staging": "No auditing",
  "Certification Services": "No auditing",
  "Detailed File Share": "No auditing",
  "File Share": "No auditing",
  "File System": "No auditing",
  "Filtering Platform Connection": "No auditing",
  "Filtering Platform Packet Drop": "No auditing",
  "Handle Manipulation": "No auditing",
  "Kernel Object": "No auditing",
  "Other Object Access Events": "No auditing",
  "Registry": "No auditing",
  "Removable Storage": "No auditing",
  "SAM": "No auditing"
},

```

```

"Policy Change": {
    "Audit Policy Change": "Success",
    "Authentication Policy Change": "Success",
    "Authorization Policy Change": "No auditing",
    "Filtering Platform Policy Change": "No auditing",
    "MPSSVC Rule-Level Policy Change": "No auditing",
    "Other Policy Change Events": "No auditing"
},
"Privilege Use": {
    "Non Sensitive Privilege Use": "No auditing",
    "Other Privilege Use Events": "No auditing",
    "Sensitive Privilege Use": "No auditing"
},
"System": {
    "IPsec Driver": "No auditing",
    "Other System Events": "Success, Failure",
    "Security State Change": "Success",
    "Security System Extension": "No auditing",
    "System Integrity": "Success, Failure"
}
}

```

11.1.2.20. mpx.windows.get_binary_file_by_path

Возвращает содержимое бинарного файла. Поддерживает переменные среды.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```

binary_data = mpx.windows.get_binary_file_by_path(r'C:\ProgramData\Kaspersky
Lab\AVP15.0.1\Data\u1313g.xml')
print(binary_data)

```

Пример результата:

```

root.stdout: b'<Update\n Date="11022015 0952"\n CompID="CORE"\n
ObjectType="2=Core;3=BIN"\n ....'

```

11.1.2.21. mpx.windows.get_dirs

Возвращает список папок.

Использует следующие параметры:

- `path` — полный путь к папке (строка);
- `recursion` — булево значение: включение (`True`) или отключение (`False`) рекурсивного поиска на любую глубину.

Пример:

```
dirs = mpx.windows.get_dirs(path=r'C:\Program Files\Git', recursion=True)
for item in dirs:
    print(item)
```

Пример результата:

```
root.stdout: C:\Program Files\Git\bin
root.stdout: C:\Program Files\Git\cmd
root.stdout: C:\Program Files\Git\dev
root.stdout: C:\Program Files\Git\dev\mqueue
root.stdout: C:\Program Files\Git\dev\shm
root.stdout: C:\Program Files\Git\etc
root.stdout: C:\Program Files\Git\etc\pkcs11
root.stdout: C:\Program Files\Git\etc\pki
root.stdout: C:\Program Files\Git\etc\pki\ca-trust
root.stdout: C:\Program Files\Git\etc\pki\ca-trust\extracted
```

11.1.2.22. mpx.windows.get_file_by_path

Возвращает содержимое файла. Поддерживает переменные среды.

Использует параметр `path` — полный путь к файлу (строка).

Примечание. Для файлов в формате UTF-8 необходимо использовать функцию `mpx.windows.get_file_by_path_utf8`.

Пример:

```
some_config = mpx.windows.get_file_by_path(r'C:\Users\SomePath\SomeConfig.txt')
print(some_config)
```

Пример результата:

```
root.stdout: SomeConfigText
```

11.1.2.23. mpx.windows.get_file_info_by_path

Возвращает информацию о файле. Поддерживает переменные среды.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```
telegram = mpx.windows.get_file_info_by_path(r'%AppData%\Telegram
Desktop\Telegram.exe')
print(telegram)
```

Пример результата:

```
root.stdout: FileInfoScheme(CompanyName='Telegram FZ-LLC', FileDescription='Telegram
Desktop', FileVersion='5.12.3.0', FileVersionEx='5.12.3.0', InternalName='',
LegalCopyright='Copyright (C) 2014-2025', OriginalFilename='', Path='C:\\Users\\
\\SomePath\\Appdata\\Roaming\\Telegram desktop\\Telegram.exe', ProductName='Telegram
Desktop', ProductVersion='5.12.3.0', ProductVersionEx='5.12.3.0')
```

11.1.2.24. mpx.windows.get_file_info_recursive

Возвращает информацию о файлах (рекурсивно).

Использует следующие параметры:

- `dirs` — список корневых папок для рекурсивного обхода;
- `filemasks` — список regex-масок имен файлов для поиска;
- `ignorlist` — список regex-масок имен папок, которые необходимо исключить из результатов поиска (по умолчанию пуст);
- `usesymlinks` — булево значение: включение (`True`) или отключение (`False`) поиска по символическим ссылкам на папки. Значение по умолчанию — `False`.

Списки корневых папок и regex-масок регистронезависимы.

11.1.2.25. mpx.windows.get_filetime

Возвращает время создания файла (`Created`), последнего изменения файла (`Changed`) и последнего открытия файла (`Opened`) в формате целого числа.

Использует параметр `path` — полный путь к файлу (строка).

Пример:

```
file_time = mpx.windows.get_filetime(r'C:\Program Files\Git\git-bash.exe')
print(file_time)
```

Пример результата:

```
root.stdout: ds~Filesystem.GetFileTime(Changed=1722266288, Created=1663067730,
Opened=1742835658)
```

11.1.2.26. mpx.windows.get_lockout_policy

Возвращает политики блокировки аккаунта.

Пример:

```
lockout_policy = mpx.windows.get_lockout_policy()
print(lockout_policy)
```

Пример результата:

```
root.stdout: {
  'Account lockout duration': '30',
  'Account lockout threshold': '0',
  'Reset lockout account counter after': '30'
}
```

11.1.2.27. mpx.windows.get_registry_values

Получает значения из реестра.

Использует следующие параметры:

- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из SQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из SQL-запроса, значение — новое имя столбца. Значение по умолчанию — `None`;
- `*patterns` — шаблоны путей реестра (строка).

Пример 1 (с одним путем реестра):

```
torrent = mpx.windows.get_registry_values(r'HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\utorrent')
print('Torrent: ', torrent)
print('Torrent DisplayName: ', torrent.DisplayName)
```

Пример результата:

```
root.stdout: Torrent: SafeNamespace(
  DisplayIcon='"C:\\Some\\Path\\utorrent\\utorrent.exe"',
  DisplayName='utorrent',
  DisplayVersion='3.6.0.47132',
  InstallLocation='C:\\Users\\SomePath\\AppData\\Roaming\\utorrent',
  MajorVersion=3,
  MinorVersion=6,
  NoModify=1,
  NoRepair=1,
  Publisher='BitTorrent Limited',
  SourceRegistryKey=RegistryKey(
    Arch='x64',
    Path='HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Uninstall\\utorrent',
    SysArch='x64'
  ),
  URLInfoAbout='http://www.utorrent.com',
  UninstallString='"C:\\Users\\SomePath\\AppData\\Roaming\\utorrent\\utorrent.exe" / UNINSTALL',
  VersionMajor=3,
  VersionMinor=6)
```

```
)
root.stdout: Torrent DisplayName: µTorrent
```

Пример 2 (с несколькими путями реестра):

```
sql_server = mpx.windows.get_registry_values(
    r'HKLM\SOFTWARE\Microsoft\Microsoft SQL Server\Instance Names\SQL',
    r'HKLM\SOFTWARE\Wow6432Node\Microsoft\Microsoft SQL Server\Instance Names\SQL'
)
if isinstance(sql_server, list):
    for item in sql_server:
        print(item)
    else:
        print(sql_server)
```

Пример результата:

```
root.stdout: SafeNamespace(MSSQLSERVER='MSSQL14.MSSQLSERVER',
    SQLEXPRESS='MSSQL14.SQLEXPRESS', SourceRegistryKey=RegistryKey(Arch='x64',
    Path='HKLM\SOFTWARE\Microsoft\Microsoft SQL Server\Instance Names\SQL',
    SysArch='x64'))
```

Пример 3 (с использованием регулярного выражения):

```
uninstall = mpx.windows.get_registry_values(r'HKEY_CURRENT_USER\SOFTWARE\Microsoft\Wi
ndows\CurrentVersion\Uninstall\{{{.*}}}')
for item in uninstall:
    print(item)
```

Пример результата:

```
root.stdout: SafeNamespace(InstallLocation='C:\Users\SomePath\AppData\Local\
\DBeaver', SourceRegistryKey=RegistryKey(Arch='x64', Path='HKCU\SOFTWARE\Microsoft\
\Windows\CurrentVersion\Uninstall\DBeaver', SysArch='x64'))
root.stdout: SafeNamespace(DisplayIcon='D:\Programs\GIMP 2\bin\gimp-2.10.exe',
    DisplayName='GIMP 2.10.36-1', DisplayVersion='2.10.36', ... VersionMajor=2,
    VersionMinor=10)
root.stdout: SafeNamespace(DisplayIcon='C:\Users\SomePath\AppData\Local\Mozilla
Firefox\firefox.exe,0', DisplayName='Mozilla Firefox (x64 ru)', ...
    DisplayVersion='127.0')
....
```

Пример 4 (с использованием параметра mapping):

```
uninstall = mpx.windows.get_registry_values(
    r'HKLM\SOFTWARE\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKU\{{{S[-\d]+}}}\SOFTWARE\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKU\{{{S[-\d]+}}}\SOFTWARE\Wow6432Node\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    mapping={'Inno Setup: App Path': 'AppPath'})
)
```

```
for item in uninstall:
    print(item)
```

Пример результата:

```
root.stdout: SafeNamespace(InstallLocation='C:\\Users\\SomePath\\AppData\\Local\\
\\DBeaver', SourceRegistryKey=RegistryKey(Arch='x64', Path='HKCU\\SOFTWARE\\Microsoft\\
\\Windows\\CurrentVersion\\Uninstall\\DBeaver', SysArch='x64'))
root.stdout: SafeNamespace(DisplayIcon='D:\\Programs\\GIMP 2\\bin\\gimp-2.10.exe',
DisplayVersion='2.10.36-1', DisplayName='GIMP 2.10.36', .... VersionMajor=2,
VersionMinor=10)
root.stdout: SafeNamespace(DisplayIcon='C:\\Users\\SomePath\\AppData\\Local\\Mozilla
Firefox\\firefox.exe,0', DisplayName='Mozilla Firefox (x64 ru)', ....
DisplayVersion='127.0')
root.stdout: SafeNamespace(DisplayName='Ditto', .... AppPath='C:\\Program files\\
\\Ditto\\')
....
```

11.1.2.28. mpx.windows.get_security_info

Возвращает SDDL-строку для заданного объекта.

Формат SDDL-строки: O:BAG:SYD:<Флаги DACL и SACL>(<Тип ACE>;<Флаги ACE>;<Права доступа>;<GUID объекта>;<GUID наследуемого объекта>;<SID>).

Использует следующие параметры:

- `inherit` — булево значение: отображение (True) или неотображение (False) в возвращаемой SDDL-строке флагов DACL и SACL;
- `object` — путь к объекту (строка). Возможные значения параметра `object` определяются значениями параметра `object_type` (см. таблицу ниже);
- `object_type` — тип объекта (строка). Возможные значения параметра приведены в таблице ниже.

Таблица 99. Значения параметров `object_type` и `object`

Значение параметра <code>object_type</code>	Значение параметра <code>object</code>	Пример значения параметра <code>object</code>
file	Полный путь к объекту	Для файла: "C:\\Windows\\regedit.exe", для папки: "C:\\Windows\\"
service	Имя сервиса	"LanmanServer"
printer	Имя принтера, открытого на сканируемом компьютере для общего доступа	"SecDevP2055"
registry	Полное имя ключа реестра.	"HKLM\\Software\\KasperskyLab\\"

Значение параметра object_type	Значение параметра object	Пример значения параметра object
	Имя ключа реестра начинается с имени корневой папки ("HKEY_CLASSES_ROOT", "HKEY_CURRENT_USER", "HKEY_LOCAL_MACHINE", "HKEY_USERS"), при этом могут быть использованы следующие сокращения: — HKEY_CLASSES_ROOT = CLASSES_ROOT = ROOT = HKCR; — HKEY_LOCAL_MACHINE = MACHINE = HKLM; — HKEY_CURRENT_USER = CURRENT_USER = USER = HKCU; — HKEY_USERS = USERS = USERS = HKU	
share	Имя ресурса, открытого на сканируемом компьютере для общего доступа	"Share"
ds_object	Полное уникальное имя объекта (Distinguished Name, DN) в формате X.500	"CN=SomeObject,OU=ou2,OU=ou1,DC=DomainName,DC=CompanyName,DC=com,O=internet"

Пример:

```
sddl_from_registry = mpx.windows.get_security_info(
    object_type='registry',
    object_path='HKCU',
    inherit=True
)
sddl_from_service = mpx.windows.get_security_info(
    object_type='service',
    object_path='W32Time',
    inherit=False
)
```

11.1.2.29. mpx.windows.get_sid_string_by_account_name

Возвращает SID по имени объекта.

Использует параметр `account_name` — имя (строка). Для пользователей и групп это могут быть имена в формате pre-Windows 2000, user principal name или в виде `domain\name`, для доменов — NetBios- и DNS-имена. Имена регистронезависимы.

Пример:

```
sid = mpx.windows.get_sid_string_by_account_name('username')
print(sid)
```

Пример результата:

```
root.stdout: S-1-5-21-123123123-123123123-123123123-12345
```

11.1.2.30. `mpx.windows.get_system_temp_directory`

Возвращает путь к системной папке Temp.

Пример:

```
system_temp = mpx.windows.get_system_temp_directory()
print(system_temp)
```

Пример результата:

```
root.stdout: C:\windows\TEMP\
```

11.1.2.31. `mpx.windows.get_user_sids_by_privilege`

Возвращает SID пользователей, у которых есть заданная привилегия.

Использует параметр `privilege` — привилегия (строка), который может принимать следующие значения:

- `SeAssignPrimaryTokenPrivilege;`
- `SeAuditPrivilege;`
- `SeBackupPrivilege;`
- `SeBatchLogonRight;`
- `SeChangeNotifyPrivilege;`
- `SeCreateGlobalPrivilege;`
- `SeCreatePagefilePrivilege;`
- `SeCreatePermanentPrivilege;`
- `SeCreateTokenPrivilege;`
- `SeDebugPrivilege;`
- `SeDenyBatchLogonRight;`
- `SeDenyInteractiveLogonRight;`

- SeDenyNetworkLogonRight;
- SeDenyServiceLogonRight;
- SeEnableDelegationPrivilege;
- SeImpersonatePrivilege;
- SeIncreaseBasePriorityPrivilege;
- SeIncreaseQuotaPrivilege;
- SeInteractiveLogonRight;
- SeLoadDriverPrivilege;
- SeLockMemoryPrivilege;
- SeMachineAccountPrivilege;
- SeManageVolumePrivilege;
- SeNetworkLogonRight;
- SeProfileSingleProcessPrivilege;
- SeRemoteShutdownPrivilege;
- SeRestorePrivilege;
- SeSecurityPrivilege;
- SeServiceLogonRight;
- SeShutdownPrivilege;
- SeSyncAgentPrivilege;
- SeSystemEnvironment;
- SeSystemProfilePrivilege;
- SeSystemtimePrivilege;
- SeTakeOwnershipPrivilege;
- SeTcbPrivilege;
- SeUndockPrivilege;
- SeUnsolicitedInputPrivilege.

Пример:

```
sids = mpx.windows.get_user_sids_by_privilege('SeAssignPrimaryTokenPrivilege')
print(sids)
```

Пример результата:

```
root.stdout: ('S-1-5-20', 'S-1-5-19')
```

11.1.2.32. mpx.windows.get_users_by_privilege

Возвращает список имен пользователей, у которых есть заданная привилегия.

Использует параметр `privilege` — привилегия (строка), который может принимать следующие значения:

- `SeAssignPrimaryTokenPrivilege;`
- `SeAuditPrivilege;`
- `SeBackupPrivilege;`
- `SeBatchLogonRight;`
- `SeChangeNotifyPrivilege;`
- `SeCreateGlobalPrivilege;`
- `SeCreatePagefilePrivilege;`
- `SeCreatePermanentPrivilege;`
- `SeCreateTokenPrivilege;`
- `SeDebugPrivilege;`
- `SeDenyBatchLogonRight;`
- `SeDenyInteractiveLogonRight;`
- `SeDenyNetworkLogonRight;`
- `SeDenyServiceLogonRight;`
- `SeEnableDelegationPrivilege;`
- `SeImpersonatePrivilege;`
- `SeIncreaseBasePriorityPrivilege;`
- `SeIncreaseQuotaPrivilege;`
- `SeInteractiveLogonRight;`
- `SeLoadDriverPrivilege;`
- `SeLockMemoryPrivilege;`
- `SeMachineAccountPrivilege;`
- `SeManageVolumePrivilege;`
- `SeNetworkLogonRight;`
- `SeProfileSingleProcessPrivilege;`
- `SeRemoteShutdownPrivilege;`
- `SeRestorePrivilege;`

- SeSecurityPrivilege;
- SeServiceLogonRight;
- SeShutdownPrivilege;
- SeSyncAgentPrivilege;
- SeSystemEnvironment;
- SeSystemProfilePrivilege;
- SeSystemtimePrivilege;
- SeTakeOwnershipPrivilege;
- SeTcbPrivilege;
- SeUndockPrivilege;
- SeUnsolicitedInputPrivilege.

Пример:

```
users = mpx.windows.get_users_by_privilege('SeAssignPrimaryTokenPrivilege')
print(users)
```

Пример результата:

```
root.stdout: ('NETWORK SERVICE', 'LOCAL SERVICE')
```

11.1.2.33. mpx.windows.normalize_path

Нормализует путь, а именно:

- корректирует регистр в имени;
- если указаны переменные среды, то добавляет их;
- если путь к файлу указан в коротком формате (8.3), то дополняет его;
- если это папка, то добавляет символ / (слеш) в конце пути.

Использует параметр `path` — полный путь к файлу или папке (строка).

Пример:

```
normalized_path = mpx.windows.normalize_path(r'%AppData%\tElegram dESKtop')
print(normalized_path)
```

Пример результата:

```
root.stdout: C:\Users\User\AppData\Roaming\Telegram desktop\
```

11.1.2.34. mpx.windows.powershell_exec

Выполняет PowerShell-запрос.

Использует следующие параметры:

- `scheduler` — булево значение: разрешает (True) или запрещает (False) запуск команды через планировщик задач. Значение по умолчанию — False;
- `script` — PowerShell-скрипт (строка).

Пример:

```
processes = mpx.windows.powershell_exec(script='Get-Process')
print(processes)
```

Пример результата:

```
root.stdout: Handles NPM(K) PM(K) WS(K) CPU(s) Id SI ProcessName
root.stdout: -----
root.stdout: 133 8 1896 6592 7764 0 armsvc
root.stdout: 220 17 8504 14512 32,36 16320 0 audiodg
root.stdout: 196 13 2672 11668 0,16 7236 1 AutoModeDetect
root.stdout: 4162 159 322776 235340 7896 0 avp
root.stdout: 455 27 9708 15032 12004 0 avp
root.stdout: 460 36 13920 3272 14728 0 avpsus
....
```

11.1.2.35. mpx.windows.shell_exec

Выполняет запрос Windows Shell.

Использует следующие параметры:

- `code_page` — кодировка (строка). Значение по умолчанию — None (используется системная кодировка);
- `command` — Shell-команда (строка).

Пример 1:

```
routes = mpx.windows.shell_exec(command='route print')
print(routes)
```

Пример результата:

```
root.stdout:
=====
root.stdout: Список интерфейсов
root.stdout: 25.....Wintun Userspace Tunnel
root.stdout: 13...84 a9 38 e0 b9 86 .....Realtek PCIe GbE Family Controller
root.stdout: 9...00 ff 4b 19 a0 fc .....TAP-Windows Adapter V9
root.stdout: 12...00 ff 68 88 a6 e8 .....TAP-Windows Adapter V9 #2
root.stdout: 22...f8 b5 4d 3f 57 b4 .....Intel(R) Wi-Fi 6 AX201 160MHz
root.stdout: 10...f8 b5 4d 3f 57 b5 .....Microsoft Wi-Fi Direct Virtual Adapter
root.stdout: 4...fa b5 4d 3f 57 b4 .....Microsoft Wi-Fi Direct Virtual Adapter #2
...
```

Пример 2 (с кодировкой):

```
wecutil = mpx.windows.shell_exec(command='wecutil es', code_page='utf-8-sig')
print(wecutil)
```

Пример 3 (с кодировкой):

```
license = mpx.windows.shell_exec(command='java ru.CryptoPro.JCP.tools.License',
code_page='CP1251')
print(license)
```

11.1.2.36. mpx.windows.wmi_exec

Выполняет команду на удаленной системе. Результаты не возвращаются в функции, поэтому необходимо перенаправлять их в файл.

Примечание. Процесс на удаленной системе создается через метод `Create` класса `Win32_Process` в WMI.

Использует следующие параметры:

- `command` — выполняемая команда (строка);
- `wait` — булево значение: для ожидания окончания выполнения команды — `True`, в противном случае — `False`. Значение по умолчанию — `True`.

По умолчанию использует системную кодировку. Для смены кодировки необходимо применить команду `chcp`.

Пример 1:

```
exec_ = mpx.windows.wmi_exec(command=r'cmd.exe /c netstat -ano > C:\Windows\temp\netstat_result.txt 2>&1')
```

Пример 2:

```
cmd.exe /c chcp 65001 & netstat -ano > C:\Temp\netstat_result.txt 2>&1
```

11.1.2.37. mpx.windows.wmi_query

Выполняет WQL-запрос.

Использует следующие параметры:

- `mapping` — словарь в формате «ключ — значение», который используется в случаях, когда необходимо преобразовать имя столбца из WQL-запроса, например, если в нем есть пробелы, точки или сложно читаемые имена. Ключ — это имя столбца из WQL-запроса, значение — это новое имя столбца. Значение по умолчанию — `None`;
- `query` — WQL-запрос (строка);
- `wmi_ns` — пространство имен WMI, в котором выполняется запрос (строка). Значение по умолчанию — `root\cimv2`.

Пример 1:

```
win32_share = mpx.windows.wmi_query(query='SELECT * FROM Win32_Share')
for item in win32_share:
    print(item)
```

Пример результата:

```
root.stdout: SafeNamespace(AccessMask=None, AllowMaximum=True, Caption='Remote
Admin', Description='Remote Admin', InstallDate=None, MaximumAllowed=None,
Name='ADMIN$', Path='C:\\Windows', Status='OK', Type=2147483648)
root.stdout: SafeNamespace(AccessMask=None, AllowMaximum=True, Caption='Default
share', Description='Default share', InstallDate=None, MaximumAllowed=None,
Name='C$', Path='C:\\', Status='OK', Type=2147483648)
root.stdout: SafeNamespace(AccessMask=None, AllowMaximum=True, Caption='Remote IPC',
Description='Remote IPC', InstallDate=None, MaximumAllowed=None, Name='IPC$',
Path='', Status='OK', Type=2147483651)
```

Пример 2:

```
msft_volume = mpx.windows.wmi_query(
    query='SELECT * FROM MSFT_Volume',
    wmi_ns=r'Root\Microsoft\Windows\Storage',
    mapping={'Path': 'DeviceID', 'UniqueId': 'DeviceID'}
)
```

11.1.2.38. mpx.windows.write_file

Записывает текстовый файл на сканируемый компьютер: если такого файла нет — он создается, если есть — его содержимое перезаписывается.

Использует следующие параметры:

- `content` — текст для записи в файл (строка);
- `path` — полный путь к файлу (строка).

Пример:

```
write_file_is_success = mpx.windows.write_file(path=r'C:\Windows\Temp\TESTFILE.txt',
    content='some_text')
print(write_file_is_success)
```

Пример результата:

```
root.stdout: True
```

11.2. Секция \$extractors

Секция `$extractors` описывает способ получения данных со сканируемого узла.

В разделе описаны экстракторы, которые могут быть использованы в секции `$extractors`.

В этом разделе

[Экстракторы классов File и FileInfo \(см. раздел 11.2.1\)](#)

[ODBC-экстракторы \(см. раздел 11.2.2\)](#)

[Registry Extractor \(см. раздел 11.2.3\)](#)

[Shell adapters \(см. раздел 11.2.4\)](#)

[Экстракторы WMI \(см. раздел 11.2.5\)](#)

11.2.1. Экстракторы классов File и FileInfo

Экстракторы класса File

Экстракторы класса File позволяют получить содержимое файлов.

Экстрактор Unix file adapter должен иметь следующую структуру:

```
$extractors:
  <Имя экстрактора>
    $template: <Имя плагина>
    $filepath: <Путь к файлу>
    $schema: <Схема получаемых данных>
      Path: String
      Data: String
```

Имя экстрактора, имя плагина, путь к файлу, а также схема данных являются обязательными полями.

Пример

```
$extractors:
  UnixFile:
    $template: unix_file
    $filepath: /etc/testfile
    $schema:
      Path: String
      Data: String
```

Экстрактор Windows file adapter должен иметь следующую структуру:

```
$extractors:
  <Имя экстрактора>
    $template: <Имя плагина>
    $filepath: <Путь к файлу>
    $encoding: <Кодировка для чтения файла>
    $schema: <Схема получаемых данных>
      Path: String
      Data: String
```


Имя экстрактора, имя плагина, путь к файлу, а также схема данных являются обязательными полями.

Пример

```
$extractors:
  WindowsFile:
    $template: windows_file
    $filepath: C:\Users\File.txt
    $encoding: utf-8
    $schema:
      Path: String
      Data: String
```

Экстраторы Unix file adapter и Windows file adapter recursive возвращают значения следующих полей:

- Data — содержимое файла. Обязательное поле;
- Path — путь к файлу. Необязательное поле.

Экстракторы класса FileInfo

Экстракторы класса FileInfo позволяют получить данные о файле.

Экстратор Windows file info adapter должен иметь следующую структуру:

```
$extractors:
  <Имя экстрактора>
    $template: <Имя плагина>
    $filepath: <Путь к файлу>
```

Имя экстрактора, имя плагина и путь к файлу являются обязательными полями.

Пример

```
$extractors:
  WindowsFileInfo:
    $template: windows_file_info
    $filepath: C:\Users\File.txt
```

Экстрактор Windows file info recursive adapter должен иметь следующую структуру:

```
$extractors:
  <Имя экстрактора>
    $template: <Имя плагина>
    $filemasks: <Список масок регулярных выражений в именах файлов для поиска>
    $ignorlist: <Список масок регулярных выражений в именах каталогов,
исключаемых из результатов поиска>
    $dirs: <Список корневых каталогов для рекурсивного обхода>
    $usesymlinks: <Искать ли по символическим ссылкам>
```

Имя экстрактора, имя плагина, список масок регулярных выражений в именах файлов и список корневых каталогов для рекурсивного обхода являются обязательными полями.

Пример

```
$extractors:
  WindowsFileInfoRecursive
    $template: windows_file_info_recursive
    $filemasks:
      - File\.txt
      - File2\.txt
    $ignorlist:
      - Backup
      - Updates
    $dirs:
      - C:\Users
    $usesymlinks: true
```

Экстракторы Windows file info adapter и Windows file info recursive adapter возвращают следующие данные:

- `CompanyName` — название компании, создавшей файл;
- `FileDescription` — описание файла;
- `FileVersion` — версия файла;
- `FileVersionEx` — расширенная информация о версии файла;
- `InternalName` — внутреннее имя файла при наличии;
- `LegalCopyright` — копирайт, относящийся к файлу;
- `OriginalFilename` — имя файла;
- `ProductName` — название продукта, для которого создан файл;
- `ProductVersion` — версия продукта;
- `ProductVersionEx` — расширенная информация о версии продукта;
- `Path` — путь к файлу.

11.2.2. ODBC-экстракторы

ODBC-экстракторы позволяют делать SQL-запросы в СУБД.

Экстрактор Select Extractor

Экстрактор Select Extractor должен иметь следующую структуру:

```
$extractors:
  <Имя экстрактора>:
```

```

$template: <Имя плагина>
$table: <Имя таблицы, из которой будут собираться данные>
$mapping: <Отображение параметров системы в параметрах источника данных>
    <Параметр источника данных>: <Поле в $schema>
$schema: <Схема получаемых данных>
    <Поле>: <Тип данных>

```

Имена экстрактора, плагина, таблицы, а также схема данных являются обязательными полями.

Пример

```

$extractors:
  MSSQLSelect:
    $template: mssql_select
    $table: ?db_name.system_table
    $mapping:
      table_catalog: TableCatalog
      table_schema: TableSchema
      table_name: TableName
    $schema:
      TableCatalog: String
      TableSchema: String
      TableName: String

```

Экстрактор Query Extractor

Экстрактор Query Extractor должен иметь следующую структуру:

```

$extractors:
  <Имя экстрактора>
    $template: <Имя плагина>
    $query: | <Запрос к СУБД, из которой будут собираться данные>
      USE [sys]
      SELECT table_catalog, table_schema, table_name
      FROM [information_schema].[?table_name]
    $mapping: <Отображение параметров системы в параметрах источника данных>
      <Параметр источника данных>: <Поле в $schema>
    $schema: <Схема получаемых данных>
      <Поле>: <Тип данных>

```

Пример

```

$extractors:
  MSSQLQuery:
    $template: mssql_query
    $query:
      USE [sys]
      SELECT table_catalog, table_schema, table_name

```

```

FROM [information_schema].[?table_name]
$mapping:
  table_catalog: TableCatalog
  table_schema: TableSchema
  table_name: TableName
$schema:
  TableCatalog: String
  TableSchema: String
  TableName: String

```

Имя экстрактора, имя плагина, запрос к СУБД, а также схема данных являются обязательными полями.

Особенности заполнения полей ODBC-экстракторов:

- Допустимые значения `$template`:
 - `mssql_select`;
 - `mssql_query`;
 - `mysql_select`;
 - `mysql_query`;
 - `oracle_select`;
 - `oracle_query`;
 - `postgresql_select`;
 - `postgresql_query`.
- Поля `$table` и `$query` используют тип данных `String`.
- Значения полей `$mapping` должны соответствовать полям в `$schema`.
- Поля `$schema` используют типы данных `String` и `Int`.

11.2.3. Registry Extractor

Экстрактор Registry Extractor позволяет делать запросы WMI по ключам реестра Windows.

Структура экстрактора:

```

$extractors:
  Registry: <Имя экстрактора>
    $template: <Имя плагина>
    $paths: <Пути к ключам реестра>
      - HKEY_LOCAL_MACHINE\SOFTWARE\Oracle\VirtualBox
      - HKEY_LOCAL_MACHINE\SOFTWARE\Sun\VM VirtualBox
      - HKLM\SOFTWARE\{{{VirtualBox\d+\.\d+ \x64\}}}}
    $mapping: <Отображение параметров системы в параметрах источника данных>
      <Параметр источника данных>: <Поле в $scheme>

```

```

$schema: <Схема получаемых данных>
  <Поле>: <Тип данных>
  SourceRegistryKey: RegistryKey

```

Имя экстрактора, имя плагина, пути до ключей реестра, а также схема данных являются обязательными полями.

Пример

```

$extractors:
  Registry:
    $template: registry_select
    $paths:
      - HKEY_LOCAL_MACHINE\SOFTWARE\Oracle\VirtualBox
      - HKEY_LOCAL_MACHINE\SOFTWARE\Sun\VM VirtualBox
      - HKLM\SOFTWARE\{{{VirtualBox\d+\.\d+ \ (x64\)}}}
    $mapping:
      InstallDir: InstallDirectory
    $schema:
      InstallDirectory: String
      Version: String
      VersionExt: String
      SourceRegistryKey: RegistryKey

```

Особенности заполнения полей экстракторов Registry Extractor:

- Допустимое значение `$template` — `registry_select`.
- Поля `$paths` используют тип данных `String`, передаются в виде списка и поддерживают регулярные выражения.
- Значения полей `$mapping` должны соответствовать полям в `$schema`.
- В `$schema` поле `SourceRegistryKey` с типом `RegistryKey` является обязательным.

11.2.4. Shell adapters

Экстрактор `shell adapters` используется для реализации пользовательских схем с помощью плагинов.

Структура экстрактора:

```

$extractors:
  <Имя экстрактора>:
    $template: <Имя плагина>
    $command: <Команда>
    $params:
      <Имя параметра>: <Тип данных>

```

Все поля, кроме поля `$params`, являются обязательными. Поле `$params` позволяет динамически задавать параметры для транспортного запроса во время выполнения аудита.

Допустимые имена плагинов: `windows_shell`, `unix_shell`, `device_shell`, `powershell`. Для плагина `powershell` может использоваться необязательное поле `$scheduler` со следующими значениями:

- `True` — запуск команды через планировщик задач;
- `False` — прямой запуск через процесс Windows.

Пример с использованием плагина `windows_shell`:

```
$extractors:
  RawReverseDNSLookup:
    $template: windows_shell
    $command: ping -n 1 -a ?host_field
```

Пример с использованием плагина `unix_shell`:

```
$extractors:
  ExtName:
    $template: unix_shell
    $command: 'tail -n ?n_param /var/log/some.log'
    $params:
      n_param: int
```

11.2.5. Экстракторы WMI

Экстрактор `wmi_select_adapter`

Экстрактор `wmi_select_adapter` предназначен для WMI-запросов формата `SELECT [%field%,] + FROM [%wmi_class%]`.

Структура экстрактора:

```
$extractors:
  <Имя экстрактора>:
    $template: wmi_select
    $query: <Класс WMI, объекты которого необходимо получить>
    $ns: <Пространство WMI для подключения>
    $mapping: <Отображение параметров системы в параметрах источника данных>
      <Свойство класса WMI>: <Поле в $schema>
    $schema: <Схема получаемых данных>
      <Поле>: <Тип данных>
```

Имя экстрактора, класс WMI и схема данных являются обязательными полями.

Пример

Экстрактор для запроса `SELECT Name, Domain, TotalPhysicalMemory, Manufacturer FROM Win32_ComputerSystem` с преобразованием поля `Manufacturer` в поле `Vendor`.

```
$extractors:
  Win32ComputerSystem:
    $template: wmi_select
    $query: Win32_ComputerSystem
    $mapping:
      Manufacturer: Vendor
    $schema:
      Name: String
      Domain: String
      TotalPhysicalMemory: Int
      Vendor: String
```

Экстрактор wmi_query_adapter

Экстрактор `wmi_query_adapter` предназначен для произвольных WMI-запросов.

Структура экстрактора:

```
$extractors:
  <Имя экстрактора>:
    $template: wmi_query
    $query: <Шаблон выполняемого запроса с обязательными операторами SELECT и
FROM>
    $ns: <Пространство WMI для подключения>
    $mapping: <Отображение параметров системы в параметрах источника данных>
      <Свойство из WMI-запроса>: <Поле в $schema>
    $schema: <Схема получаемых данных>
      <Поле>: <Тип данных>
```

Имя экстрактора, шаблон запроса и схема данных являются обязательными полями.

Пример

Экстрактор для запроса `Get-WmiObject -namespace "root\cimv2" -query "SELECT * FROM Win32_Group WHERE LocalAccount = TRUE"`

```
$extractors:
  Win32LocalGroup:
    $template: wmi_query
    $query: SELECT * FROM Win32_Group WHERE LocalAccount = TRUE
    $ns: root\cimv2
    $schema:
      Name: String
      Domain: String
      SID: String
```

Особенности заполнения полей экстракторов WMI:

- В названии класса WMI в поле `$query` можно использовать латинские буквы, цифры или символы подчеркивания. Название класса должно начинаться с буквы и не должно заканчиваться символом подчеркивания.
- Если поле `$ns` не заполнено, запрос будет выполнен в пространство `root\cimv2`.

11.3. Секция \$transformers

Секция `$transformers` описывает способ преобразования данных.

В разделе описаны трансформеры, которые могут быть использованы в секции `$transformers`.

В этом разделе

[DictValue \(JSON\) Parser \(см. раздел 11.3.1\)](#)

[KeyValue Parsers \(см. раздел 11.3.2\)](#)

[Search Parser \(см. раздел 11.3.3\)](#)

[XML-парсеры классов XMLFindone и XMLFinditer \(см. раздел 11.3.4\)](#)

[Resolvers \(см. раздел 11.3.5\)](#)

11.3.1. DictValue (JSON) Parser

Структура трансформера:

```
$transformers:
  <Имя трансформера>
    $template: <Имя плагина>
    $prefix: <Строка ключей, разделяется точкой>
    $ignorecase: <Игнорировать ли регистр в полях>
    $schema: <Схема получаемых данных>
    <Поле>:
      type: <Тип данных>
      attribute: <Значение>
```

Имя трансформера, имя плагина и схема данных являются обязательными полями.

Пример

```
$transformers:
  FindOneParser:
    $template: json_find_one_parser
    $prefix: test_prefix
    $ignorecase: true
    $schema:
```



```
Field_one:
  type: String
  attribute: field_one
Field_two:
  type: String
  attribute: field_two
```

Пример

```
$transformers:
  FinditerParser:
    $template: json_finditer_parser
    $prefix: test_prefix
    $ignorecase: true
    $schema:
      Field_one:
        type: String
        attribute: field_one
      Field_two:
        type: String
        attribute: field_two
```

11.3.2. KeyValue Parsers

Структура трансформера:

```
$transformers:
  <Имя трансформера>:
    $template: <Имя плагина>
    $separator: <Разделительный символ>
    $strip_chars: <Символ для удаления>
    $ignorecase: <Игнорировать ли регистр в полях>
    $schema: <Схема получаемых данных>
      <Поле>:
        type: <Тип данных>
        attribute: <Значение>
```

Имя трансформера, имя плагина и схема данных являются обязательными полями.

Пример

```
$transformers:
  KeyValueParser:
    $template: key_value_parser
    $separator: test_separator
    $strip_chars: test_strip_chars
```

```

$skip_empty: true
$ignorecase: true
$schema:
  Field_one:
    type: String
    attribute: STRING_FIELD
  Field_two:
    type: Int
    attribute: INT_FIELD
  Field_three:
    type: Array(Int)
    attribute: ARRAY_INT_FIELD

```

11.3.3. Search Parser

Для передачи данных в секцию `$loaders` из трансформера Search Parser используются функции.

Трансформер Search Parser ищет текст по заданному регулярному выражению, в котором можно определить именованные группы. В результате поиска в группы попадает текст, соответствующий описанному шаблону. Чтобы получить текст из именованной группы, необходимо применить функцию, которая извлекает значение только из этой группы.

Структура трансформера:

```

$transformers:
  <Имя трансформера>
    $template: <Имя плагина>
    $pattern: <Регулярное выражение>
    $schema: <Схема получаемых данных>
      <Поле>: <Тип данных>
    <Название функции>:
      $template: function
      $apply: <Имя трансформера>
      $output: <Поле регулярного выражения>

```

Имя трансформера, имя плагина и схема данных являются обязательными полями. Функции используются в секции `$loaders` для получения результата работы трансформера.

Пример

```

$transformers:
  NameVersionParser:
    $template: regexp_search
    $pattern: '^.*(?:P<Name>\S+)\s+(?:P<Version>\d+\.\d+\.\d+).*'
    $schema:
      Name: String
      Version: String

```

```

get_name:
  $template: function
  $apply: NameVersionParser
  $output: Name
get_version:
  $template: function
  $apply: NameVersionParser
  $output: Version

```

11.3.4. XML-парсеры классов XMLFindone и XMLFinditer

Структура трансформера:

```

$transformers:
  <Имя пользовательского трансформера>
    $template: <Имя плагина>
    $schema: <Схема получаемых данных>
      <Поле>:
        data_type: <Тип данных>
        xml_function: <Название функции поиска>
        xml_path: <Путь до элемента XML>
    $root: null
    $prefix: '*'
  <Имя исходного трансформера>:
    $template: <Имя плагина>
    $schema: <Схема получаемых данных>
      <Поле>:
        data_type: <Тип данных>
        xml_function: <Название функции поиска>
        xml_path: <Путь до элемента XML>
    $root: null
    $prefix: '*'

```

Пример

```

$transformers:
  RuleTermParser:
    $template: xml_findone_parser
    $schema:
      Name:
        data_type: String
        xml_function: text
        xml_path: name
      Source:
        data_type: String
        xml_function: text
        xml_path: from/source-address/name

```

```

    Gateway:
      data_type: String
      xml_function: text
      xml_path: then/backup-remote-gateway
  $root: 'root'
  $prefix: '*'
AddressFamilyFinditer:
  $template: xml_finditer_parser
  $schema:
    AddressFamily:
      data_type: String
      xml_function: text
      xml_path: ns:address-family-name
    InterfaceID:
      data_type: String
      xml_function: text
      xml_path: ../ns:name
    FamilyFlags:
      data_type: Array(String)
      xml_function: tags
      xml_path: ns:address-family-flags
  $root: null
  $prefix: '*'

```

Тип данных, название функции поиска, путь до элемента XML в `$schema` являются обязательными полями.

Пример заполнения секции `$loaders`:

```

$loaders:
  VirtualBox:
    - $template: mapping
      $target: Core.Software
      $kind: detect
      $parent: OperatingSystem.Windows.WindowsHost
      $origin:
        $template: get_extractor_path
        $name: VirtualBoxRegistry
      $field_map:
        Name: VirtualBox
        Vendor: Oracle
        OsFamily: Windows
        Architecture: $row.SourceRegistryKey.Arch
        InstallPath: $row.InstallDirectory
      RuleTerm:
        $template: apply_transformer
        $name: RuleTermParser
        $source: $row.InstallDirectory

```

```
AddressFamily:
  $template: apply_transformer
  $name: AddressFamilyFinder
  $source: $row.InstallDirectory
```

11.3.5. Resolvers

Трансформер Resolvers позволяет отображать полученные значения параметров системы в нормализованном или измененном виде. Трансформер представляет собой расширенный словарь, ключом которого являются данные из экстракторов или других трансформеров.

Структура трансформера:

```
$transformers:
  <Имя трансформера>
    $template: <Имя плагина>
    $default: <Выходное значение, если не найден ключ>
    $undefined: <Выходное значение, если на вход получено значение null>
    $mapping: <Отображение параметров системы в параметрах источника данных>
      <Ключ>: <Значение>
      <Ключ>: <Значение>
```

Имя трансформера и имя плагина являются обязательными полями.

Особенности заполнения полей трансформера:

— Допустимые значения `$template`:

- `boolean_to_string_resolver;`
- `int_to_boolean_resolver;`
- `int_to_int_array_resolver;`
- `int_to_string_resolver;`
- `int_to_string_array_resolver;`
- `major_version_to_string_resolver;`
- `string_to_boolean_resolver;`
- `string_to_int_resolver;`
- `string_to_int_array_resolver;`
- `string_to_string_resolver;`
- `string_to_string_array_resolver;`
- `strip_string_to_string_resolver.`

— Строковые поля `$mapping` необходимо указывать в одинарных кавычках. Для обозначения списка в `$mapping` необходимо использовать квадратные скобки.

— Значения `$default` и `$undefined` должны соответствовать типу выходных данных.

Пример

```
$transformers:
  Resolver:
    $template: boolean_to_string_resolver
    $default: '100'
    $undefined: '200'
    $mapping:
      false: '10'
      true: '1'
```

Пример

```
$transformers:
  Resolver:
    $template: int_to_boolean_resolver
    $mapping:
      12: true
      14: false
```

Пример

```
$transformers:
  Resolver:
    $template: int_to_int_array_resolver
    $mapping:
      12: [10, 2]
      13: [1]
```

Пример

```
$transformers:
  Resolver:
    $template: int_to_string_resolver
    $mapping:
      12: '10'
      13: '1'
```

Пример

```
$transformers:
  Resolver:
    $template: int_to_string_array_resolver
    $mapping:
      12: ['10', '2']
      13: ['1']
```

Пример

```
$transformers:
  Resolver:
    $template: major_version_to_string_resolver
    $mapping:
      '2.2': '2'
      '3.10': '3'
```

Пример

```
$transformers:
  Resolver:
    $template: string_to_boolean_resolver
    $mapping:
      '12': true
      'true': false
```

Пример

```
$transformers:
  Resolver:
    $template: string_to_int_resolver
    $mapping:
      '12': 10
      'true': 1
```

Пример

```
$transformers:
  Resolver:
    $template: string_to_int_array_resolver
    $mapping:
      '12': [10, 2]
      'true': [1]
```

Пример

```
$transformers:
  Resolver:
    $template: string_to_string_resolver
    $mapping:
      '12': '10'
      'true': '1'
```

Пример

```
$transformers:
  Resolver:
    $template: string_to_string_array_resolver
    $mapping:
      '12': ['true', '2']
      'true': ['1']
```

Пример

```
$transformers:
  Resolver:
    $template: strip_string_to_string_resolver
    $mapping:
      '12': 'str1'
      'true': 'str2'
```

11.4. Секция \$loaders

Секция `$loaders` определяет порядок размещения данных, полученных либо из секции `$converters`, либо из секций `$transformers` и `$extractors`. Конвертеры и экстракторы возвращают список, в котором может быть любое количество элементов. Каждый элемент списка представляет собой структуру, которую декларирует конвертер или экстрактор, или структуру, встроенную в движок процесса расширения аудита.

В секции `$loaders` необходимо заполнить следующие поля:

- `<Имя_лоадера>` — определяет название ладера. Обязательное поле.
- `$template` — объявляет используемый плагин (для ладера всегда `mapping`). Все последующие использования `$template` объявляются внутри других полей и нужны для указания плагина, с помощью которого будет заполнено поле. Обязательное поле.
- `$target` — объявляет целевую сущность, для которой собираются данные. Обязательное поле.
- `$kind` — определяет метод заполнения сущности: создание новой (`detect`) или дополнение существующей (`scan`). Обязательное поле.
- `$parent` — указывает родительскую сущность для заполняемой. Опциональное поле.

Примечание. Для `Core.Software` родительской сущностью является `OS`, для `OS` родительской сущности нет.

- `$guard` — описывает условие заполнения сущности (см. ниже). Опциональное поле.
- `$origin` — указывает источник данных (конвертер или экстрактор), который содержит данные для передачи в поля сущности. Как правило, представлен в виде `$template: get_origin_path $name: <Имя конвертера из секции $converters или экстрактора из секции $extractors>`. Опциональное поле.

- `$params` — указывает источник для подстановки параметра в раздел `$origin`. Опциональное поле.
- `$field_map` — указывает необходимые поля модели актива для заполняемой сущности (для передачи значений используются либо константы, либо команда `$row.<Имя поля>`). Обязательное поле.
- `AdditionalProperties` — позволяет загружать в модель актива дополнительные поля, не предусмотренные в сущностях. Опциональное поле. Обычно применяется в пользовательских правилах MaxPatrol HCC. В поле используется тип данных Dictionary, а в создаваемых в нем полях — String.

Примечание. Если в MaxPatrol HCC необходимо передать данные, отличные от строки, рекомендуется сохранить их в файл формата JSON, YAML или XML и использовать в скриптах один из следующих методов: `extractJson`, `extractXml`, `extractYaml`, `extractJsonCollection`, `extractXmlCollection`, `extractYamlCollection`.

Вы можете описать условие заполнения сущности с помощью необязательного блока `$guard`. Например, условие для наследников класса `Core.Software`, у которых в поле `Name` указано значение `MS Office VBA`, а в поле `Version` — `'2019'`, будет выглядеть следующим образом:

```
$target: Core.Software
$kind: scan
$parent: OperatingSystem.Windows.WindowsHost
$guard: !eq-str
    $target.Name: MS Office VBA
    $target.Version: '2019'
```

Если условие ложно, метод заполнения сущности не будет определен.

В блоке `$guard` вы можете использовать:

- операторы сравнения:
 - `!eq-str` — проверяет равенство строк: если все пары равны, возвращает `True`;
 - `!neq-str` — проверяет неравенство строк: если все пары не равны, возвращает `True`;
 - `!str-startswith` и `!str-endswith` — проверяют, что значение поля начинается или заканчивается с определенной строки. Если условие выполнено, возвращают `True`;
 - `!str-in-str` — проверяет, содержится ли одна строка внутри другой строки;
- логические операторы:
 - `$and` — проверяет, выполняются ли все условия внутри него: если условия выполняются, возвращает `True`;
 - `$or` — проверяет, выполняется ли хотя бы одно условие внутри него: если условие выполняется, возвращает `True`;
 - `$not` — инвертирует результаты проверки: возвращает `True`, если условие не выполняется, и `False`, если условие выполняется.

Например, если первое условие — наличие в полях `$target.Name` и `$target.Version` значений `MS Office VBA` и `'2019'` соответственно, а второе условие — наличие в поле `$target.Architecture` значения, отличного от `x86`, то блок будет выглядеть следующим образом:

```
$guard:
  $or:
    - $and:
      - !eq-str
        $target.Name: MS Office VBA
      - !eq-str
        $target.Version: '2019'
    - $not: !eq-str
      $target.Architecture: x86
```

Секция `$loaders` в YAML-файле, содержащем секции `$converters` и `$loaders`

Пример секции `$loaders`:

```
$loaders:
  Sysmon: # Имя лоадера
  - $template: mapping # Плагин
    $target: Core.Software # Целевая сущность для заполнения
    $kind: detect # Метод заполнения сущности
    $parent: OperatingSystem.Windows.WindowsHost # Родительская сущность
    $origin:
      $template: get_origin_path # Плагин вызова конвертера
      $name: SysmonDetect # Имя конвертера
    $field_map: # Структура заполняемых полей
      Name: Sysmon
      Vendor: Sysinternals
      OsFamily: Windows
      InstallPath: $row.InstallPath
      Version: $row.Version
      AdditionalProperties:
        State: $row.State
        ConfigPath: $row.ConfigPath
        ConfigHashAlgorithm: $row.ConfigHashAlgorithm
        ConfigHash: $row.ConfigHash
        RegistryConfigPath: $row.RegistryConfigPath
```

В таблице приведены примеры ошибок, выявляемых при валидации полей секции `$loaders`.

Таблица 100. Возможные ошибки (секция \$loaders)

Ошибка	Описание
<pre>\$Mapping: - \$Kind: MappingLoaderTemplate - \$FieldMap: - \$AdditionalProperties: - ?Exists: false - \$Field: SomeField - ?Allowed: false</pre>	Дополнительные поля необходимо указывать только в поле <code>AdditionalProperties</code>
<pre>\$Mapping: - \$Kind: MappingLoaderTemplate - \$Target: - \$Entity: Core.Host - ?AllowedEntity: false</pre>	Для сущности <code>Core.Host</code> доступен только метод заполнения <code>scan</code>
<pre>\$Mapping: - \$Kind: MappingLoaderTemplate - \$KindMethod: - \$Method: SomeWrongMethod - ?CorrectMethod: false</pre>	Метод заполнения сущности указан некорректно
<pre>\$Mapping: - \$Kind: MappingLoaderTemplate - \$Target: - \$Entity: Some.Wrong.Entity - ?AllowedEntity: false</pre>	Сущность указана некорректно (не существует)
<pre>\$GetOriginPath: - \$Kind: GetOriginPathTemplate - Name: - \$Value: WrongExtractorName - ?Exists: false</pre>	Имя конвертера указано некорректно (не существует)

Секция \$loaders в YAML-файле, содержащем секции \$extractors, \$transformers и \$loaders

Структура секции \$loaders:

```
$loaders:
  <Имя лоадера>:
    - $template: <Имя плагина>
      $target: <Сущность, в которую запишутся данные>
      $kind: <Режим работы экстрактора>
      $parent: <Родительский узел для заполняемой сущности>
      $origin: <Источник, который содержит данные для передачи их в поля скана или
в функцию обработки>
        $template: get_extractor_path
        $name: <Имя экстрактора>
```

```

    $params: <Имя параметра, объявленного в экстракторе>: <Данные,
передаваемые в параметр>
    $field_map: <Поля модели актива для заполняемой сущности и специальное поле
AdditionalProperties>
        <Поле>: <Значение>
    AdditionalProperties: <Поле для добавления в скан полей, отсутствующих в
модели актива>
        <Поле>:
            $serialize: <Источник данных>
            $template: <Имя плагина>
            $to: <Формат файла>

```

В секции `$loaders` могут быть использованы следующие плагины:

- `mapping` — определяет, что будет генерироваться XML-файл с информацией об активе;
- `apply_transformer` — позволяет применить функцию, созданную в секции `$transformers`, указать входные данные и записать результат в поле модели;
- `apply_resolver` — позволяет применить резолвер, созданный в секции `$transformers`, указать входные данные с помощью поля `$value` и записать результат в поле модели;
- `first_not_empty_string` — позволяет получить первое значение из списка (пустые значения не учитываются);
- `get_extractor_path` — объявляет источник данных для раздела `$origin` с помощью экстрактора из секции `$extractors`.

Пример 1:

```

$loaders:
  VirtualBox:
    - $template: mapping
      $target: Core.Software
      $kind: detect
      $parent: OperatingSystem.UNIX.Linux.LinuxHost
      $origin:
        $template: get_extractor_path
        $name: VirtualBoxUnix
      $field_map:
        Name: VirtualBox
        Vendor: Oracle
        OsFamily: $parent.OsFamily
        Version: 6.1.5
        AdditionalProperties:
          SomeField1:
            $serialize:
              $template: apply_transformer
              $name: get_virtualbox_version
              $source: $row.Data
            $to: json

```

```

SomeField2: $parent.OsName
SomeField3:
  $serialize: $parent.IpAddress
  $to: yaml
SomeField4:
  $template: apply_transformer
  $name: get_virtualbox_version
  $source: $row.Data

```

Пример 2:

```

$loaders:
  VirtualBox:
  - $template: mapping
    $target: Software.Oracle.VirtualBox
    $kind: detect
    $parent: OperatingSystem.UNIX.Linux.LinuxHost
    $origin:
      $template: get_extractor_path
      $name: VirtualBoxUnix
    $field_map:
      AdditionalProperties:
        SomeUserField1: $row.StringData
        SomeUserField2:
          $serialize: $row.UserSource
          $to: json
        SomeUserField3:
          $serialize:
            $template: apply_transformer
            $name: get_virtualbox_version
            $source: $row.Data
          $to: json

```

11.5. Примеры заполнения файлов расширения аудита с секциями \$converters и \$loaders

В этом разделе приведены примеры заполнения YAML-файлов расширения аудита и Python-скриптов.

В этом разделе

[Сбор данных о процессах в Windows \(см. раздел 11.5.1\)](#)

[Сбор данных о процессах в ОС семейства Unix \(см. раздел 11.5.2\)](#)

[Сбор данных о Microsoft Sysmon \(см. раздел 11.5.3\)](#)

[Сбор данных о ПО из Uninstall-веток реестра \(см. раздел 11.5.4\)](#)

[Сбор данных о БД под управлением Microsoft SQL Server \(см. раздел 11.5.5\)](#)

[Дополнительный сбор данных о Microsoft Exchange \(см. раздел 11.5.6\)](#)

11.5.1. Сбор данных о процессах в Windows

YAML-файл расширения аудита

```
$converters:
  Process:
    $template: converter
    $schema:
      ProcessJSON: String
    $script: !include process.py

$loaders:
  ProcessAP:
  - $template: mapping
    $target: OperatingSystem.Windows.WindowsHost
    $kind: scan
    $origin:
      $template: get_origin_path
      $name: Process
    $field_map:
      AdditionalProperties:
        ProcessJSON: $row.ProcessJSON
```

Python-скрипт process.py

```
import json
import re

processes = mpx.windows.powershell_exec(script='Get-Process')
mpx.stop(not processes)

pattern = re.compile(
    r'\s*(?P<Handles>\d+)\s+'
    r'(?P<NPM>\d+)\s+'
    r'(?P<PM>\d+)\s+'
    r'(?P<WS>\d+)\s+'
    r'(?P<CPU>\d+, \d+\s+)?'
    r'(?P<ID>\d+)\s+'
    r'(?P<SI>\d+)\s+'
    r'(?P<Name>\S+)'
)
```

```

result = list()
for process in processes.strip().splitlines()[2:]:
    match = pattern.search(process)

    if match:
        result.append({
            'Handles': int(match.group('Handles')),
            'NPM': int(match.group('NPM')),
            'PM': int(match.group('PM')),
            'WS': int(match.group('WS')),
            'CPU': (
                float(match.group('CPU').replace(',', ' '))
                if match.group('CPU')
                else None
            ),
            'ID': int(match.group('ID')),
            'SI': int(match.group('SI')),
            'ProcessName': match.group('Name')
        })

mpx.data.append(ProcessJSON=json.dumps(result, indent=4))

```

11.5.2. Сбор данных о процессах в ОС семейства Unix

YAML-файл расширения аудита

```

$converters:
  Process:
    $template: converter
    $schema:
      ProcessJSON: String
    $script: !include process.py

$loaders:
  ProcessAP:
    - $template: mapping
      $target: OperatingSystem.UNIX.Linux.LinuxHost
      $kind: scan
      $origin:
        $template: get_origin_path
        $name: Process
      $field_map:
        AdditionalProperties:
          ProcessJSON: $row.ProcessJSON

```

Python-скрипт process.py

```
import json
import re

from datetime import datetime

def parse_linux_processes(config):
    if not config:
        return

    normalized_config = re.sub(r'\n+(\s+?;.*(?:\n|\Z))', r'\1', config)

    pattern = re.compile(
        r'^(?P<start>.{24}).(?P<pid>.{14}).(?P<ppid>.{14}).(?P<user>.{35}).'
        r'(?P<tty>.{10}).(?P<group>.{35}).(?P<comm>.{20}).(?P<label>.{80}).'
        r'(?P<args>.*)$'
    )

    result = list()
    for line in normalized_config.splitlines()[1:]:
        if not line:
            continue

        match = pattern.match(line)

        if not match:
            continue

        start = match.group('start').rstrip()
        pid = match.group('pid').rstrip()
        ppid = match.group('ppid').rstrip()
        user = match.group('user').rstrip()
        tty = match.group('tty').rstrip()
        group = match.group('group').rstrip()
        comm = match.group('comm').rstrip()
        label = match.group('label').rstrip()
        args = match.group('args').rstrip()

        result.append(dict(
            Start=str(datetime.strptime(start, '%a %b %d %H:%M:%S %Y')),
            PID=int(pid),
            PPID=int(ppid),
            User=user,
            TTY=tty if tty != '?' else None,
            Group=group,
        ))
```



```

        Command=comm,
        Label=label,
        Args=args
    ))

    return result

raw_processes = mpx.unix.ssh_exec(
    'ps -e -o
"lstart:24,pid:14,ppid:14,user:35,ttty:10,group:35,comm:20,label:80,args" 2>/dev/null
| cat')
processes = parse_linux_processes(raw_processes)

mpx.data.append(ProcessJSON=json.dumps(processes, indent=4))

```

11.5.3. Сбор данных о Microsoft Sysmon

YAML-файл расширения аудита

```

$converters:
  SysmonDetect:
    $template: converter
    $schema:
      State: String
      InstallPath: String
      Version: String
      ConfigPath: String
      ConfigHashAlgorithm: String
      ConfigHash: String
      RegistryConfigPath: String
    $script: !include sysmon.py

$loaders:
  Sysmon:
    - $template: mapping
    $target: Core.Software
    $kind: detect
    $parent: OperatingSystem.Windows.WindowsHost
    $origin:
      $template: get_origin_path
      $name: SysmonDetect
    $field_map:
      Name: 'Sysmon'
      Vendor: 'Sysinternals'
      OsFamily: 'Windows'

```

```

InstallPath: $row.InstallPath
Version: $row.Version

AdditionalProperties:
  State: $row.State
  ConfigPath: $row.ConfigPath
  ConfigHashAlgorithm: $row.ConfigHashAlgorithm
  ConfigHash: $row.ConfigHash
  RegistryConfigPath: $row.RegistryConfigPath

```

Python-скрипт sysmon.py

```

state = path = None

for item in mpx.windows.wmi_query('Select * from Win32_Service'):
    if not item.Name:
        continue

    if 'sysmon' not in item.Name.lower():
        continue

    state = item.State
    path = item.PathName

    break

mpx.stop(not path)

file_info = mpx.windows.get_file_info_by_path(path)
version = file_info.ProductVersion
mpx.stop(not version)

sysmon_registry_params = mpx.windows.get_registry_values(
    r'HKLM\SYSTEM\CurrentControlSet\Services\SysmonDrv\Parameters')

config_hash = sysmon_registry_params.ConfigHash
hash_algorithm, _, hash_data = (config_hash or '').lower().partition('=')

mpx.data.append(
    State=state,
    InstallPath=path,
    Version=version,
    ConfigPath=sysmon_registry_params.ConfigFile or '',
    ConfigHashAlgorithm=hash_algorithm,
    ConfigHash=hash_data,

```

```
RegistryConfigPath=str(sysmon_registry_params.SourceRegistryKey or '')
)
```

11.5.4. Сбор данных о ПО из Uninstall-веток реестра

YAML-файл расширения аудита

```
$converters:
  UninstallData:
    $template: converter
    $schema:
      Name: String
      DisplayIcon: String
      Version: String
      InstallPath: String
      InstallSource: String
      InstallDate: String
      Publisher: String
      Vendor: String
      Architecture: String
      UninstallString: String
      Readme: String
      RegistryKeyName: String
      RegistryKeyPath: String
    $script: !include uninstall_data.py

  Softs:
    $template: converter
    $origins:
      - UninstallData
    $schema:
      Name: String
      Version: String
      InstallPath: String
      Vendor: String
      Architecture: String
      UninstallString: String
    $script: !include softs.py

$loaders:
  Soft:
    - $template: mapping
      $target: Core.Software
      $kind: detect
      $parent: OperatingSystem.Windows.WindowsHost
      $origin:
```

```

$template: get_origin_path
$name: Softs
$field_map:
  Name: $row.Name
  Version: $row.Version
  InstallPath: $row.InstallPath
  Vendor: $row.Vendor
  Architecture: $row.Architecture
  OsFamily: 'Windows'

AdditionalProperties:
  UninstallString: $row.UninstallString

```

Python-скрипт uninstall_data.py

```

import re

def strip(arg, chars=None):
    if not arg:
        return

    return arg.strip(chars)

uninstall = mpx.windows.get_registry_values(
    r'HKLM\SOFTWARE\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKU\{{{S[-\d]+}}}\SOFTWARE\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    r'HKU\{{{S[-\d]+}}}\SOFTWARE\Wow6432Node\Microsoft\Windows'
    r'\CurrentVersion\Uninstall\{{{(!InstallWIX_).*}}}',
    mapping={'Inno Setup: App Path': 'AppPath'}
)

kb_filter = re.compile(r'\(KB\d+\)').search

result = list()

for data in uninstall:
    if kb_filter(data.DisplayName or ''):
        continue

    display_icon = data.DisplayIcon

```

```

if display_icon:
    icon_path, icon_index = display_icon.partition(',')[::2]
    icon_path = mpx.windows.normalize_path(path=icon_path)
    display_icon = ','.join([icon_path, icon_index])

install_path = strip(data.InstallLocation or data.AppPath)

mpx.data.append(
    Name=(
        strip(data.DisplayName) or
        strip(data.SourceRegistryKey.Name)
    ),
    DisplayIcon=display_icon,
    Version=data.DisplayVersion or data.Version or None,
    InstallPath=install_path and mpx.windows.normalize_path(path=install_path),
    InstallSource=mpx.windows.normalize_path(path=data.InstallSource),
    InstallDate=data.InstallDate,
    Publisher=strip(data.Publisher) or None,
    Vendor=(
        strip(data.Publisher) or
        strip(data.DisplayName) or
        None
    ),
    Architecture=data.SourceRegistryKey.Arch,
    UninstallString=data.UninstallString,
    Readme=data.Readme or None,
    RegistryKeyName=data.SourceRegistryKey.Name,
    RegistryKeyPath=data.SourceRegistryKey.Path,
)

```

Python-скрипт softs.py

```

for item in mpx.origin.UninstallData():
    if not all((item.Name, item.Version, item.InstallPath)):
        continue

    mpx.data.append(
        Name=item.Name,
        Version=item.Version,
        InstallPath=item.InstallPath,
        Vendor=item.Vendor,
        Architecture=item.Architecture,
        UninstallString=item.UninstallString,
    )

```

11.5.5. Сбор данных о БД под управлением Microsoft SQL Server

YAML-файл расширения аудита

```
$converters:
  Databases:
    $template: converter
    $params:
      sql_version: String
      instance_name: String
    $schema:
      Databases: String
    $script: !include databases.py

$loaders:
  MSSQLDatabases:
  - $template: mapping
    $target: Software.MSSQL.SQLInstance
    $kind: scan
    $parent: Core.Host
    $origin:
      $template: get_origin_path
      $name: Databases
    $params:
      sql_version: $target.Version
      instance_name: $target.InstanceName
    $field_map:
      AdditionalProperties:
        Databases: $row.Databases
```

Python-скрипт databases.py

```
import json

def parse_product_version(version, part=None):
    if not version:
        return

    parts = version.split('.')
    parsed_version = {
        'Major': int(parts[0]),
        'Minor': int(parts[1]),
```

```

        'Build': int(parts[2]),
        'PrivateBuild': int(parts[3]) if len(parts) == 4 else None,
    }

    return part and parsed_version[part] or parsed_version

major_version = parse_product_version(
    version=mpx.params['sql_version'], part='Major')
mpx.stop(not major_version)

instance_name = mpx.params['instance_name']

if major_version >= 11:
    query = '''
        SELECT database_id, name
        ,      master.dbo.fn_varbintohexstr(owner_sid) AS owner_sid
        ,      create_date, compatibility_level
        ,      user_access, state, state_desc, is_in_standby, recovery_model
        ,      recovery_model_desc, is_trustworthy_on, is_db_chaining_on
        ,      is_master_key_encrypted_by_server, is_published, is_merge_published
        ,      is_distributor, is_encrypted, replica_id, group_database_id
        ,      default_language_lcid, containment, containment_desc
        FROM master.sys.databases
    ...
elif major_version >= 10:
    query = '''
        SELECT database_id, name
        ,      master.dbo.fn_varbintohexstr(owner_sid) AS owner_sid
        ,      create_date, compatibility_level
        ,      user_access, state, state_desc, is_in_standby, recovery_model
        ,      recovery_model_desc, is_trustworthy_on, is_db_chaining_on
        ,      is_master_key_encrypted_by_server, is_published
        ,      is_merge_published, is_distributor, is_encrypted
        FROM master.sys.databases
    ...
elif major_version >= 9:
    query = '''
        SELECT database_id, name
        ,      master.dbo.fn_varbintohexstr(owner_sid) AS owner_sid
        ,      create_date, compatibility_level
        ,      user_access, state, state_desc, is_in_standby, recovery_model
        ,      recovery_model_desc, is_trustworthy_on, is_db_chaining_on
        ,      is_master_key_encrypted_by_server, is_published
        ,      is_merge_published, is_distributor
        FROM master.sys.databases
    ...

```

```

result = list()
for db in mpx.odbc.mssql_exec(query=query, instance_name=instance_name):
    is_accessible = not any([
        db.user_access != 0,
        db.state != 0,
        db.is_in_standby == 1
    ])

    result.append(dict(
        ID=db.database_id,
        Name=db.name,
        OwnerSID=db.owner_sid,
        CreateDate=db.create_date,
        CompatibilityLevel=db.compatibility_level,
        IsAccessible=is_accessible,
        IsTrustworthyOn=db.is_trustworthy_on,
        IsDbChainingOn=db.is_db_chaining_on,
        IsMasterKeyEncryptedByServer=db.is_master_key_encrypted_by_server,
        IsPublished=db.is_published,
        IsMergePublished=db.is_merge_published,
        IsDistributor=db.is_distributor,
        IsEncrypted=db.is_encrypted,
        ReplicaId=db.replica_id,
        GroupDatabaseId=db.group_database_id,
        DefaultLanguageLCID=db.default_language_lcid,
    ))

mpx.data.append(Databases=json.dumps(result, indent=4))

```

11.5.6. Дополнительный сбор данных о Microsoft Exchange

YAML-файл расширения аудита

```

$converters:
  ExchangeAP:
    $template: converter
    $schema:
      Quotas: String
      PickupPath: String
      SenderReputation: String
      SenderFilter: String
      SenderID: String
      AttachFiltering: String

```



```

    $script: !include exchange_ap.py

$loaders:
  ExchangeAP:
  - $template: mapping
    $target: Software.Microsoft.Exchange
    $kind: scan
    $parent: OperatingSystem.Windows.WindowsHost
    $origin:
      $template: get_origin_path
      $name: ExchangeAP
    $field_map:
      AdditionalProperties:
        Quotas: $row.Quotas
        PickupPath: $row.PickupPath
        SenderReputation: $row.SenderReputation
        SenderFilter: $row.SenderFilter
        SenderID: $row.SenderID
        AttachFiltering: $row.AttachFiltering

```

Python-скрипт exchange_ap.py

```

import json
import re

import xml.etree.ElementTree as ET

LOAD_CMDLET_SCRIPT = '''
    if (!(Get-PSSnapin | Where-Object {$_.Name -eq
"Microsoft.Exchange.Management.PowerShell.E2010"}))
    {
        try
        {
            Add-PSSnapin Microsoft.Exchange.Management.PowerShell.E2010 -ErrorAction
STOP
        }
        catch
        {
            #Snapin was not loaded
            EXIT
        }
    }
    %s
'''

# Quotas
mailboxes = mpx.windows.powershell_exec(

```

```

        LOAD_CMDLET_SCRIPT % 'Get-MailboxDatabase -Server $env:computername | select Name
| ConvertTo-Xml -NoTypeInfoInformation -As String',
        scheduler=True
    )

mailbox_names = [item.text for item in ET.fromstring(mailboxes).findall('.//
Property[@Name="Name"]')]

mailbox_quotas = list()
for name in mailbox_names:
    quotas = mpx.windows.powershell_exec(
        LOAD_CMDLET_SCRIPT % f'Get-MailboxDatabase "{name}" | select
IssueWarningQuota, ProhibitSendReceiveQuota, ProhibitSendQuota | ConvertTo-Xml
-NoTypeInfoInformation -As String',
        scheduler=True
    )

    quotas_root = ET.fromstring(quotas)
    mailbox_quotas.append(dict(
        Name=name,
        **{
            field: quotas_root.find(f'//Property[@Name="{field}"]').text
            for field in ['IssueWarningQuota', 'ProhibitSendReceiveQuota',
'ProhibitSendQuota']
        }
    ))

quotas = json.dumps(mailbox_quotas, indent=4)

# PickupPath
pickup = mpx.windows.powershell_exec(
    LOAD_CMDLET_SCRIPT % 'Get-TransportService | select Identity, PickupDirectoryPath
| ConvertTo-Xml -NoTypeInfoInformation -As String',
    scheduler=True
)

pickup_path = ET.fromstring(pickup).find(f'//
Property[@Name="PickupDirectoryPath"]').text

ENABLED = re.compile(r'Enabled\s+:\s+(?P<Enabled>\S+)').search

# SenderReputation
sender_reputation_config = mpx.windows.powershell_exec(
    LOAD_CMDLET_SCRIPT % 'Get-SenderReputationConfig | Format-List Enabled',
    scheduler=True
)

sender_reputation = ENABLED(sender_reputation_config)

```

```

# SenderFilter
sender_filter_config = mpx.windows.powershell_exec(
    LOAD_CMDLET_SCRIPT % 'Get-SenderFilterConfig | Format-List Enabled',
    scheduler=True
)

sender_filter = ENABLED(sender_filter_config)

# SenderID
sender_id_config = mpx.windows.powershell_exec(
    LOAD_CMDLET_SCRIPT % 'Get-SenderIDConfig | Format-List Enabled',
    scheduler=True
)

sender_id = ENABLED(sender_id_config)

# AttachFiltering
attach_filtering_config = mpx.windows.powershell_exec(
    LOAD_CMDLET_SCRIPT % 'Get-TransportAgent "Attachment Filtering Agent"',
    scheduler=True
)

attach_filtering = ENABLED(attach_filtering_config)

mpx.data.append(
    Quotas=quotas,
    PickupPath=pickup_path,
    SenderReputation=sender_reputation.group('Enabled') if sender_reputation else
None,
    SenderFilter=sender_filter.group('Enabled') if sender_filter else None,
    SenderID=sender_id.group('Enabled') if sender_id else None,
    AttachFiltering=attach_filtering.group('Enabled') if attach_filtering else None,
)

```

11.6. Примеры заполнения файлов расширения аудита с секциями \$extractors, \$transformers и \$loaders

Пример заполнения файла расширения аудита для поиска приложения Telegram с сохранением только первых двух секций версии

```

$extractors:
    GetTelegramExeInfo:
        $template: windows_file_info

```

```

    $filepath: C:\Users\aiivanov\AppData\Roaming\Telegram Desktop\Telegram.exe
$transformers:
  TelegramTwoSections:
    $template: regexp_search
    $pattern: '(?P<Value>^\d+\.\d+).*'
    $schema:
      Value: String
  FirstTwoSections:
    $template: function
    $apply: TelegramTwoSections
    $output: Value
$loaders:
  TelegramExeInfo:
  - $template: mapping
    $target: Core.Software
    $kind: detect
    $parent: OperatingSystem.Windows.WindowsHost
    $origin:
      $template: get_extractor_path
      $name: GetTelegramExeInfo
    $field_map:
      Vendor: $row.CompanyName
      Name: $row.ProductName
      InstallPath: $row.Path
      Version:
        $template: apply_transformer
        $name: FirstTwoSections
        $source: $row.FileVersion

```

Пример заполнения файла расширения аудита для приложения VirtualBox

```

$extractors:
  VirtualBoxRegistry:
    $template: registry_select
    $paths:
      - HKEY_LOCAL_MACHINE\SOFTWARE\Oracle\VirtualBox
      - HKEY_LOCAL_MACHINE\SOFTWARE\Sun\XVM VirtualBox
    $mapping:
      InstallDir: InstallDirectory
    $schema:
      InstallDirectory: String
      Version: String
      VersionExt: String
      SourceRegistryKey: RegistryKey
$transformers:

```

```

VirtualBoxVersionParser:
  $template: regexp_search
  $pattern: '^.*(P<Value>\d+\. \d+\. \d+).* $'
  $schema:
    Value: String
get_virtualbox_version:
  $template: function
  $apply: VirtualBoxVersionParser
  $output: Value
$loaders:
  VirtualBox:
  - $template: mapping
    $target: Core.Software
    $kind: detect
    $parent: OperatingSystem.Windows.WindowsHost
    $origin:
      $template: get_extractor_path
      $name: VirtualBoxRegistry
    $field_map:
      Name: VirtualBox
      Vendor: Oracle
      OsFamily: Windows
      Architecture: $row.SourceRegistryKey.Arch
      InstallPath: $row.InstallDirectory
      Version:
        $template: apply_transformer
        $name: get_virtualbox_version
        $source:
          $template: first_not_empty_string
          $source:
            - $row.Version
            - $row.VersionExt

```

Пример заполнения файла расширения аудита для поиска приложения Telegram в папке C:\Users

```

$extractors:
  TelegramFileInfo:
    $template: windows_file_info_recursive
    $filemasks:
      - telegram.exe
      - телеграм.exe
    $usesymlinks: false
    $dirs:
      - C:\Users
$loaders:

```

Telegram:

```
- $template: mapping
  $target: Core.Software
  $kind: detect
  $parent: OperatingSystem.Windows.WindowsHost
  $origin:
    $template: get_extractor_path
    $name: TelegramFileInfo
  $field_map:
    Name: Telegram
    Vendor: Telegram
    OsFamily: Windows
    InstallPath: $row.Path
    Version:
      $template: first_not_empty_string
      $source:
        - $row.ProductVersion
        - $row.ProductVersionEx
```

12. Настройка регистрации инцидентов

Инциденты автоматически регистрируются компонентом MP 10 Core на основании одного или нескольких событий ИБ, зарегистрированных на MP SIEM Server. Инциденты регистрируются только для событий, у которых в поле `correlation_type` указано значение `incident`.

Для корреляционных событий значение поля `correlation_type` заполняется в правиле корреляции и может быть изменено с помощью правил обогащения. Также с помощью правил корреляции и обогащения можно указать значения некоторых полей регистрируемого инцидента и настроить агрегацию инцидентов.

При агрегации в один инцидент объединяются данные нескольких событий, имеющих одинаковые ключи агрегации. Ключ может быть неизменным для всех событий, регистрируемых или обогащаемых по одному правилу, или формироваться динамически для каждого события. Способ формирования ключа агрегации указывается в правиле корреляции или обогащения в поле `incident.aggregation.key`.

Для настройки регистрации инцидентов в правиле корреляции или правиле обогащения нужно:

1. Для поля `correlation_type` указать значение `incident`.
2. Настроить поля инцидента.
3. Если требуется, настроить агрегацию инцидентов.

Настройка регистрации инцидентов при регистрации корреляционных событий выполняется в правиле корреляции в директиве `emit`. При заполнении полей инцидента (названия полей нужно начинать со знака доллара) вы можете использовать переменные, объявленные в инструкциях `init` и `on` (названия нужно начинать со знака доллара), функции и операторы языка XPL или литералы с типом данных, соответствующим заполняемому полю.

Внимание! Не используйте в блоке операторов директивы `emit` поля подлежащих корреляции событий.

Для настройки регистрации инцидентов при обогащении корреляционных событий нужно создать правило обогащения для этих событий (или изменить уже существующие). Настройка регистрации инцидентов выполняется в директиве `enrich` в инструкции `enrich_fields`. При заполнении полей инцидента вы можете использовать переменные (названия нужно начинать со знака доллара), поля указанного в инструкции `enrich_fields` события, функции и операторы языка XPL или литералы с типом данных, соответствующим заполняемому полю.

В этом разделе

[Алгоритм регистрации инцидента \(см. раздел 12.1\)](#)

[Поля для заполнения информации об инцидентах \(см. раздел 12.2\)](#)

[Поля для настройки агрегации инцидентов \(см. раздел 12.3\)](#)

См. также

[Структура правила корреляции \(см. раздел 10.2\)](#)

[Структура правила обогащения \(см. раздел 9.1\)](#)

12.1. Алгоритм регистрации инцидента

При появлении на MP SIEM Server события, в котором для поля `correlation_type` указано значение `incident`, данные этого события передаются в MP 10 Core, в том числе время регистрации события и ключ агрегации, указанные в правиле поля инцидента и параметры агрегации. В зависимости от значения ключа агрегации в MP 10 Core выполняется одно из следующих действий:

- Если ключ агрегации не указан, регистрируется новый инцидент (агрегация для этого инцидента не выполняется).
- Если ключ агрегации указан и в базе инцидентов отсутствуют инциденты с таким же ключом, регистрируется новый инцидент с новым ключом.
- Если ключ агрегации указан и в базе инцидентов присутствуют инциденты с таким же ключом, выполняется агрегация инцидентов, данные события добавляются в инцидент. Если инцидентов с таким ключом несколько, данные события добавляются в инцидент с более поздней датой изменения.

Примечание. Если существующий инцидент имеет статус «Закрыт», в зависимости от параметров агрегации инцидентов по событию может быть зарегистрирован новый инцидент, данные события могут быть добавлены в существующий закрытый инцидент или отброшены.

12.2. Поля для заполнения информации об инцидентах

В этом разделе приведен список полей событий для заполнения полей регистрируемых по ним инцидентов (см. документ Синтаксис языка запроса PDQL). Эти поля событий не отображаются при просмотре событий через веб-интерфейс MaxPatrol SIEM. Указанные значения полей инцидента отображаются в карточке инцидента.

incident.name

Название инцидента для заполнения поля инцидента `name`. Если значение не указано в правиле, поле инцидента будет заполнено значением из поля события `correlation_name`; если в событии значение не указано, формируется название вида `INC-〈Порядковый номер〉`. При агрегации инцидентов значение не меняется.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Параметры** в поле **Название**.

Тип данных: String

Чтобы заполнить имя инцидента значениями полей нормализованного события, на которое сработало правило корреляции, необходимо использовать блок операторов инструкции `on` правила корреляции для соответствующего события.

Пример:

```
on <Название события> {
    $incident.name = "Выход пользователя " + subject.name + " в выходной " + time
}
```

Если в правиле корреляции переопределяются значения как `subject.name`, так и `time`, необходимо использовать директиву `emit` с переменными корреляционного события.

Пример:

```
emit {
    $correlation_type = "incident" $incident.name = "Выход пользователя " +
    $subject.name + " в выходной " + $time
}
```

incident.description

Описание инцидента для заполнения поля инцидента `description`. Если значение не указано в правиле, поле инцидента будет заполнено по правилу локализации события, если правило локализации не создано, формируется описание вида `Incident <Порядковый номер>, <Категория>`. При агрегации инцидентов значение не меняется.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Параметры** в поле **Описание**.

Тип данных: String

Пример:

```
$incident.description = "Обнаружена попытка подбора пароля для учетной записи"
```

incident.category

Тип инцидента для заполнения поля инцидента `type`. Поле нужно заполнять в соответствии с [классификацией инцидентов \(см. приложение Е\)](#) значениями из колонки `type`. По указанному значению автоматически определяется категория инцидента и заполняется поле инцидента `category` (значениями из колонки `category`). Если тип инцидента не указан в правиле, в полях инцидента будет `undefined`. При агрегации инцидентов значение поля не меняется.

При регистрации инцидента вручную тип инцидента выбирается в блоке параметров **Параметры** в раскрывающемся списке **Категория и тип**.

Тип данных: Enum

Пример:

```
$incident.category = "DataLeakage"
```

Примечание. Для этого примера поля инцидента примут значения `type="DataLeakage"`, `category="DataSecurity"`, в карточке инцидента будут указаны тип инцидента «Утечка данных» и категория «Безопасность данных».

incident.severity

Уровень опасности инцидента для заполнения поля инцидента `severity`. Возможные значения:

- `high` — высокая;
- `medium` — средняя (по умолчанию);
- `low` — низкая.

При регистрации инцидента вручную значение поля выбирается в блоке параметров **Статус** в раскрывающемся списке **Опасность**.

Тип данных: Enum

Пример:

```
$incident.severity = "medium"
```

incident.severity_behavior

Выбор уровня опасности инцидента при агрегации. Возможные способы заполнения поля инцидента `severity`:

- `first` — заполняется значением из поля `importance` события, по которому зарегистрирован инцидент;
- `highest` — заполняется самым высоким значением из полей `importance` всех событий, данные которых добавлены в инцидент (по умолчанию).

Тип данных: Enum

Пример:

```
$incident.severity_behavior = "first"
```

incident.assigned_to_user_id

Идентификатор пользователя MaxPatrol SIEM, который назначается ответственным за расследование инцидента. Имя и фамилия пользователя указываются в полях инцидента `assigned.FirstName` и `assigned.LastName`. При агрегации инцидентов поле заполняется значением из первого события с заполненным значением этого поля и в дальнейшем не изменяется.

При регистрации инцидента вручную значения полей выбираются в блоке параметров **Статус** в раскрывающемся списке **Ответственный**.

Примечание. Для автоматического назначения ответственного вы можете создать табличный список, содержащий категории или типы инцидентов, идентификаторы и имена пользователей, и написать правило обогащения, которое будет автоматически заполнять поле события `incident.assigned_to_user_id` в зависимости от категории или типа инцидента.

Тип данных: String

Пример:

```
$incident.assigned_to_user_id = "07075c57-1b0e-481d-9017-bcb6260481f3"
```

incident.related_addresses

Названия активов, на которые направлена атака, для заполнения поля инцидента `Targets.Addresses.Name`. При агрегации инцидентов названия добавляются в список.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Активы и сети** в поле **Сетевые адреса**.

Тип данных: StringList

Пример:

```
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event
    $related_addresses = src.host
emit {
  $correlation_type = "incident"
  $incident.related_addresses = append($incident.related_addresses,
  $related_addresses)
}
```

incident.related_assets

Цели атаки, которые не могут быть отнесены к активам, группам активов, сетям и сетевым адресам, для заполнения поля инцидента `Targets.Others.Name`. При агрегации инцидентов адреса добавляются в список.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Активы и сети** в поле **Прочие активы и сети**.

Тип данных: StringList

Пример:

```
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event
    $related_assets = src.asset
emit {
  $correlation_type = "incident"
  $incident.related_assets = append($incident.related_assets, $related_assets)
}
```

incident.attacking_addresses

Сетевые адреса — источники атаки, для заполнения поля инцидента

`Attackers.Addresses.Name`. При агрегации инцидентов адреса добавляются в список.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Атакующие активы** в поле **Сетевые адреса**.

Тип данных: StringList

Пример:

```
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event
    $attacking_addresses = dst.host
emit {
  $correlation_type = "incident"
  $incident.attacking_addresses = append($incident.attacking_addresses,
  $attacking_addresses)
}
```

incident.attacking_assets

Источники атаки, которые не могут быть отнесены к активам, группам активов, сетям и сетевым адресам, для заполнения поля инцидента `Attackers.Others.Name`. При агрегации инцидентов адреса добавляются в список.

При регистрации инцидента вручную значение поля вводится в блоке параметров **Атакующие активы** в поле **Прочие активы и сети**.

Тип данных: StringList

Пример:

```
event Event:
  key:
    event_src.title
  filter {
    event_src.title == "test"
  }
rule Example: Event
  on Event
    $attacking_asset = dst.asset
emit {
  $correlation_type = "incident"
  $incident.attacking_assets = append($incident.attacking_assets, $attacking_asset)
}
```

12.3. Поля для настройки агрегации инцидентов

В этом разделе приведен список полей событий для настройки агрегации регистрируемых по ним инцидентов. Эти поля событий не отображаются при просмотре событий через веб-интерфейс MaxPatrol SIEM.

incident.aggregation.key

Ключ для агрегации инцидентов (регистронезависимый). При выполнении условий агрегации для событий с одинаковыми ключами регистрируется один инцидент. Если значение поля не указано в правиле, агрегация инцидентов не выполняется и для каждого события регистрируется новый инцидент. Если ранее были зарегистрированы несколько инцидентов с одинаковыми ключами, эти события добавляются в инцидент с более поздней датой изменения.

Ключ агрегации может быть неизменным для всех регистрируемых или обогащаемых по одному правилу событий или формироваться динамически для каждого события. Динамический ключ представляет собой строку, сформированную из значений указанных полей события. Вы можете использовать функцию `join` для [формирования динамических ключей](#) (см. [раздел 4.5.7](#)).

Внимание! Для корректной агрегации инцидентов по событиям одного правила при формировании ключа агрегации необходимо использовать значение поля события `correlation_name`.

Тип данных: String

Пример:

```
$incident.aggregation.key = join([$correlation_name, $event_src.host], "|")
```

incident.aggregation.time_window

Выбор способа отсчета времени агрегации инцидентов, указанного в поле `incident.aggregation.timeout`. Время регистрации инцидента определяется по значению поля `time` события. Если значение поля не указано в правиле, за него принимается значение `max_aggregation_time`. Возможные значения:

- `infinite` — время агрегации не ограничено, инцидент регистрируется с ключом первого полученного события;
- `max_aggregation_time` — время агрегации отсчитывается с момента получения первого события, инцидент регистрируется при превышении времени, указанного в параметре `incident.aggregation.timeout`, и содержит все события с таким же ключом, зарегистрированные за это время (по умолчанию);
- `no_alerts_time` — время отсчитывается с момента регистрации последнего события. Инцидент регистрируется, если время между моментами регистрации событий с одинаковыми ключами превышает указанное время, — и содержит все события с момента регистрации первого события с этим ключом.

Тип данных: Enum

Пример:

```
$incident.aggregation.time_window = "infinite"
```

incident.aggregation.timeout

Время агрегации инцидентов в секундах. Для ввода времени можно использовать сокращения: `s` — секунды, `m` — минуты, `h` — часы, `d` — дни. Если значение поля не указано в правиле, за него принимается значение 2 часа.

Тип данных: Number

Пример:

```
$incident.aggregation.timeout = 1h
```

incident.aggregation.closed_behavior

Выбор действия, если зарегистрированный ранее инцидент с тем же ключом имеет статус «Закрит». Если значение поля не указано в правиле, регистрируется новый инцидент, если при этом событие было зарегистрировано при проверке устаревших событий — данные события добавятся в зарегистрированный ранее инцидент. Возможные значения:

- `new` — регистрировать новый инцидент (по умолчанию);
- `append` — добавить новый инцидент в зарегистрированный ранее инцидент со статусом «Закрит» (статус не изменяется). Если при этом изменяются данные инцидента, обновляется время изменения инцидента;
- `skip` — пропустить инцидент.

Тип данных: Enum

Пример:

```
$incident.aggregation.closed_behavior = "new"
```

13. Утилиты SDK

В этом разделе описаны утилиты MaxPatrol SIEM SDK, используемые для отладки правил нормализации, обогащения, корреляции и агрегации, даны примеры команд для запуска утилит, указаны необходимые для работы утилит файлы. Утилиты SDK входят в комплект поставки MaxPatrol SIEM.

Для работы с утилитами MaxPatrol SIEM SDK нужно выполнить экспорт всех объектов из Knowledge Base (для установки в SIEM Lite). При экспорте вы получите архив `knowledgebase_<Дата экспорта>.zip`, содержащий файлы правил нормализации, обогащения, корреляции и агрегации MaxPatrol SIEM, а также дополнительные файлы, необходимые для отладки новых правил.

В этом разделе

[Экспорт событий из БД. Утилита `export\_data` \(см. раздел 13.1\)](#)

[Отладка правил нормализации \(см. раздел 13.2\)](#)

[Отладка правил агрегации \(см. раздел 13.3\)](#)

[Отладка правил обогащения \(см. раздел 13.4\)](#)

[Отладка правил корреляции \(см. раздел 13.5\)](#)

[Структура файла `schema.json` \(см. раздел 13.6\)](#)

13.1. Экспорт событий из БД. Утилита `export_data`

Утилита `export_data` предназначена для экспорта необработанных и нормализованных событий из Elasticsearch и LogSpace.

Утилита поставляется в составе пакета `siem-sdk-linux_<Версия пакета>` приложения MaxPatrol 10 в дистрибутиве MaxPatrol SIEM.

Для работы утилиты требуется, чтобы на узле с компонентом MP 10 Core были запущены службы Core Events и SIEM Querying. Если в IT-инфраструктуре организации используются межсетевой экран или другие средства для контроля сетевого трафика, необходимо настроить в них правила, разрешающие трафик между узлом, на котором запускается утилита, и узлом, на котором установлен компонент MP 10 Core, через порт 443/TCP.

Формат вызова для экспорта необработанных событий:

```
./export_data
--pem=<Путь к файлу закрытого ключа сертификата для аутентификации>
--time_from=<Дата и время начала интервала>
--time_to=<Дата и время конца интервала>
--output=<Путь к файлу событий>
raw
<PDQL-фильтр>
```


Формат вызова для экспорта нормализованных событий:

```
./export_data
  --pem=<Путь к файлу закрытого ключа сертификата для аутентификации>
  --time_from=<Дата и время начала интервала>
  --time_to=<Дата и время конца интервала>
  --output=<Путь к файлу событий>
  norm
  <PDQL-фильтр>
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных stdout) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных stdout) номер версии утилиты.

— `raw`

Выполнить экспорт необработанных событий (нормализованных и ненормализованных).

Примечание. Для экспорта только ненормализованных событий вы можете запустить утилиту с ключом `raw` и PDQL-фильтром `normalized=null` or `normalized=false`.

— `norm`

Выполнить экспорт нормализованных событий (выполняется по умолчанию, если не указан тип событий для экспорта).

— `--pem` или `-p`

Путь к файлу закрытого ключа сертификата для аутентификации MP SIEM Server в PT MC. По умолчанию при запуске с сервера MP SIEM Server это каталог `/var/lib/deployed-roles/<Название экземпляра приложения MaxPatrol 10>/siemserver-<Идентификатор экземпляра роли>/certs/authKey.pem`, а при запуске с сервера MP 10 Core — каталог `/var/lib/deployed-roles/<Название экземпляра приложения MaxPatrol 10>/core-<Идентификатор экземпляра роли>/certs/authCert.pem`. Обязательный ключ.

Параметры хранилища событий:

— `--host=<IP-адрес узла>`

Узел, на котором установлен компонент MP 10 Core.

— `--siem_id=<Идентификатор конвейера>` или `-s=<Идентификатор конвейера>`

Выполнить экспорт событий с указанного в ключе конвейера. Если ключ не указан, события будут экспортированы с главного конвейера.

— `--port=<Порт>`

Порт подключения к серверу с MP 10 Core (по умолчанию 443/TCP).

Параметры запроса:

- `--time_from=<Дата и время начала интервала>` или `-f=<Дата и время начала интервала>`

Выполнить экспорт событий, зарегистрированных позднее указанных даты и времени. Дату и время нужно указывать в формате ISO 8601 (пример без поправки на часовой пояс: 2015-05-13T12:13:12Z).

Примечание. Для ввода текущих даты и времени вы можете использовать переменную `now`, для ввода более раннего времени — конструкцию `now-<Дни>d<Часы>h`.

- `--time_to=<Дата и время конца интервала>` или `-t=<Дата и время конца интервала>`

Выполнить экспорт событий, зарегистрированных ранее указанных даты и времени. Дату и время нужно указывать в формате ISO 8601 (пример для московского времени: 2015-05-13T12:13:12+03:00).

- `<PDQL-запрос>`

Применить к отобранным для экспорта событиям указанный PDQL-запрос (см. документ Синтаксис языка запроса PDQL). В запросе вы можете использовать поля событий, математические и логические операторы языка XP.

Примечание. Вы можете использовать в PDQL-запросе одинарные кавычки (') или экранировать двойные кавычки обратной косой чертой (\").

Выходные данные:

- `--output=<Путь к файлу событий>` или `-o <Путь к файлу событий>`

Сохранить события в указанном файле формата JSON. Каждое событие сохраняется в одной строке.

- `--zip` или `-z`

Добавить файл с событиями в архив.

- `--limit=<Значение>`

Ограничить количество событий в файле указанным значением (по умолчанию 10 000).

- `--count`

Вывести на экран количество найденных для экспорта событий.

Параметры отладки:

- `--quiet` или `-q`

Не выводить на экран информацию о работе утилиты.

- `--debug`

Вывести на экран запрос, отправляемый в хранилище событий.

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок stderr) в указанном файле. Если параметр не указан, данные выводятся на экран.

Пример экспорта необработанных событий

```
./export_data --pem="/certs/authKey.pem" -f now-100d -t now --
output=outfile_raw.json raw
```

Пример экспорта нормализованных событий

```
./export_data --pem="/certs/authKey.pem" -f 2024-01-01T00:00:00Z -t now --
output=outfile_raw.json norm
```

Пример экспорта нормализованных событий с сервера MP SIEM Server

```
export_data --host 192.169.1.1 --port 443 -f 2023-01-30T17:30:00Z -t
2023-06-30T18:30:00Z -p /var/lib/deployed-roles/application-id/core/certs/authKey.pem
-o outfile_norm.json norm uuid="00000005-f9b9-0f68-f000-0000abbac077"
```

Пример экспорта нормализованных событий с сервера MP 10 Core

```
export_data -f 2023-01-30T17:30:00Z -t 2023-06-30T18:30:00Z -p /var/lib/deployed-
roles/application-id/core/certs/authCert.pem -o outfile_norm.json norm
uuid="00000005-f9b9-0f68-f000-0000abbac077"
```

Пример экспорта нормализованных событий с сервера MP 10 Core с PDQL-фильтром

```
export_data -f 2023-01-30T17:30:00Z -t 2023-06-30T18:30:00Z -p /var/lib/deployed-
roles/application-id/core/certs/authCert.pem -o outfile_norm.json norm action =
\"login\" and dst.host = \"PC-One\" AND subject.account.name = \"User\"
```

13.2. Отладка правил нормализации

Каждое правило нормализации сохраняется в отдельном файле `formula.xp`. Файлы правил нормализации MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `normalization-formulas\formulas`. Для отладки правил нормализации вы можете использовать утилиту `normalizer-cli`. При отладке вам могут потребоваться дополнительные файлы.

appendix.xp

Файл формата JSON, содержит правила заполнения полей нормализованного события `event_src.host`, `src.host`, `dst.host`. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip` в папке `normalization-formulas\xp_appendix`.

formulas_graph.json

Граф нормализации — файл формата JSON, описывающий совокупность всех используемых правил нормализации.

Для создания графа нормализации вы можете использовать утилиту `rcc`.

taxonomy.json

Файл формата JSON, содержит схему полей событий MaxPatrol SIEM. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip`.

В этом разделе

[rcc](#) (см. раздел 13.2.1)

[normalizer-cli](#) (см. раздел 13.2.2)

13.2.1. rcc

Утилита `rcc` может использоваться для создания графа нормализации.

Формат запуска:

```
rcc
  --lang=normalization
  --taxonomy=<Путь к файлу taxonomy.json>
  --schema=<Путь к файлу schema.json>
  --formula-appendix=<Путь к файлу appendix.xpr>
  <Путь к папке правил нормализации>
  --output=<Путь к файлу графа>
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

— `--lang=<Тип графа>` или `-l=<Тип графа>`

Выбор типа правила, для которого создается граф. Для правил нормализации нужно указать `normalization` или `n`.

— `<Путь к папке правил нормализации>`

Путь к папке с файлами правил нормализации, для которых нужно создать граф. Из папки отбираются файлы с расширением `xr`. Вместо папки вы можете указать названия файлов с правилами.

- `--taxonomy=<Путь к файлу taxonomy.json>` или `-t <Путь к файлу taxonomy.json>`

Использовать указанный файл `taxonomy.json` со схемой полей событий.

- `--schema=<Путь к файлу schema.json>` или `-s <Путь к файлу schema.json>`

Использовать указанный файл `schema.json` со схемой БД табличных списков.

- `--lib-path <Путь к папке макросов>` или `-p <Путь к папке макросов>`

Путь к папке с файлами макросов. Вы можете указать несколько папок, используя параметр несколько раз в одной команде. Файлы макросов MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `filters`.

- `--formula-appendix=<Путь к файлу appendix.xr>`

Использовать указанный файл `appendix.xr` с правилами заполнения полей нормализованного события `event_src.host`, `src.host` и `dst.host`.

Выходные данные:

- `--output=<Путь к файлу графа>` или `-o <Путь к файлу графа>`

Сохранить граф нормализации в указанном файле. Если параметр не указан, граф выводится на экран (в стандартный поток вывода `stdout`).

- `--pretty`

Использовать для формата JSON представление с переносом строк и отступами при выводе на экран и сохранении в файл, указанный в параметре `--output`.

Параметры отладки:

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок `stderr`) в указанном файле. Если параметр не указан, данные выводятся на экран.

- `--mre`

Выводить данные отладки в формате JSON. Если параметр не указан, данные выводятся в текстовом формате.

Пример

```
./rcc --lang=normalization --taxonomy=/var/knowledgebase_2020_11_01/taxonomy.json --
schema=/var/knowledgebase_2020_11_01/schema.json --formula-appendix=/var/
knowledgebase_2020_11_01/normalization-formulas/xr_appendix/appendix.xr /var/
knowledgebase_2020_11_01/normalization-formulas/formulas --output=/var/rules/output/
formulas_graph.json
```

13.2.2. normalizer-cli

Внимание! Для работы утилиты требуются файлы библиотек `icudt55.dll`, `icuuc55.dll`.

Утилита `normalizer-cli` предназначена для нормализации потока необработанных событий. Через стандартный поток ввода данных (`stdin`) утилита получает необработанные события, нормализует их, используя граф нормализации, и направляет в стандартный поток вывода данных (`stdout`).

Формат запуска:

```
normalizer-cli
  --raw
  --not-norm=<Файл для ненормализованных событий>
  --mime=<Формат события>
  <Путь к файлу formulas_graph.json>
  < <Файл с необработанными событиями>
  > <Файл для нормализованных событий>
```

Ключи запуска утилиты:

- `--help` или `-h`
Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.
- `--version` или `-v`
Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.
- `--verbose-version`
Вывести на экран (в стандартный поток вывода данных `stdout`) номера версий утилиты и используемых ею компонентов.

Входные данные:

- `< <Путь к файлу с событиями>`
Использовать указанный текстовый файл (стандартный поток ввода данных `stdin`) с необработанными событиями или, если указан параметр `--raw`, со значениями полей необработанных событий `body`. Каждое событие в файле должно быть записано в отдельной строке.
- `<Путь к файлу formulas_graph.json>`
Использовать указанный файл `formulas_graph.json` с графом нормализации.

Выходные данные:

- `> <Путь к файлу с нормализованными событиями>`

Сохранить нормализованные события (из стандартного потока вывода данных stdout) в указанном файле формата JSON.

- `--not-norm=<Путь к файлу с ненормализованными событиями>` или `-u <Путь к файлу с ненормализованными событиями>`

Сохранить в указанном файле события, которые не удалось нормализовать.

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок stderr) в указанном файле. Если параметр не указан, данные выводятся на экран.

Параметры необработанного события:

- `--raw` или `-r`

В файле с событиями содержатся значения полей необработанных событий `body`. Формат событий нужно указать в параметре `-mime`, название модуля MP 10 Collector можно указать в параметре `--tag`:

- `--mime=<Формат>`

Формат необработанного события:

`text/plain` — простой текст;

`application/json` — формат JSON;

`application/x-pt-eventlog` — событие журнала Windows;

`text/csv` — ассоциативный массив в формате JSON;

`text/xml` — XML.

- `--tag <tag>`

Название модуля MP 10 Collector, получившего необработанное событие.

Параметры отладки:

- `--stat` или `-s`

Вывести на экран (в стандартный поток ошибок stderr) статистику обработанных событий, например:

```
Normalized: 151 (83.43%)
```

```
Not normalized: 30 (16.57%)
```

```
Total: 181
```

- `--trace`

Вывести на экран (в стандартный поток ошибок stderr) причину неудачной нормализации.

Возможные причины: не удалось найти правило нормализации; для найденных правил нормализации не выполнено условие, объявленное после ключевого слова `COND`; возникла ошибка при распознавании текста или обработке данных события.

Примечание. Параметр можно использовать только для `--mime=text/plain`. Параметр несовместим с параметром `--benchmark`.

— `--benchmark=<Точность>` или `-b <Точность>`

Вывести в файл (в стандартный поток вывода данных `stdout`) отчет по измерению скорости нормализации каждого события в EPS. Рекомендуется указывать точность измерения от 100 до 10000.

Примечание. Параметр несовместим с параметром `--trace`.

Для события выводится:

```
{ "eps" : <Скорость нормализации: Number>, "raw":<Текст исходного события>, "normalized":<Текст нормализованного события> }
```

Пример

```
./normalizer-cli --raw --stat --not-norm=/var/knowledgebase_2020_11_01/not_norm.json
--mime=application/json /var/rules/output/formulas_graph.json < /var/
knowledgebase_2020_11_01/raw.txt > /var/knowledgebase_2020_11_01/norm.json
```

13.3. Отладка правил агрегации

Каждое правило агрегации сохраняется в отдельном файле `<Название правила>.agr`. Файлы правил агрегации MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `aggregation-rules`. Для отладки правил агрегации вы можете использовать утилиту `aggregator-cli`. При отладке вам могут потребоваться дополнительные файлы.

aggfilters.json

Граф агрегации — файл формата JSON, описывающий совокупность всех условий отбора событий для агрегации, указанных во всех используемых правилах агрегации.

Для создания графа агрегации вы можете использовать утилиту `rcc`.

schema.json

Файл формата JSON содержит схему для создания БД табличных списков. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip` или по аналогии [создать собственный файл \(см. раздел 13.6\)](#).

taxonomy.json

Файл формата JSON, содержит схему полей событий MaxPatrol SIEM. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip`.

В этом разделе

`rcc` (см. раздел 13.3.1)

`aggregator-cli` (см. раздел 13.3.2)

13.3.1. `rcc`

Утилита `rcc` может использоваться для создания графа агрегации.

Формат запуска:

```
rcc
  --lang=aggregation
  --taxonomy=<Путь к файлу taxonomy.json>
  --schema=<Путь к файлу schema.json>
  <Путь к папке правил агрегации>
  --output=<Путь к файлу графа>
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

— `--lang=<Тип графа>` или `-l=<Тип графа>`

Выбор типа правила, для которого создается граф. Для правил агрегации нужно указать `aggregation` или `a`.

— `<Путь к папке правил агрегации>`

Путь к папке с файлами правил агрегации, для которых нужно создать граф. Из папки отбираются файлы с расширением `agr`. Вместо папки вы можете указать названия файлов с правилами.

— `--taxonomy=<Путь к файлу taxonomy.json>` или `-t <Путь к файлу taxonomy.json>`

Использовать указанный файл `taxonomy.json` со схемой полей событий.

— `--schema=<Путь к файлу schema.json>` или `-s <Путь к файлу schema.json>`

Использовать указанный файл `schema.json` со схемой БД табличных списков.

— `--lib-path <Путь к папке макросов>` или `-p <Путь к папке макросов>`

Путь к папке с файлами макросов. Вы можете указать несколько папок, используя параметр несколько раз в одной команде. Файлы макросов MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `filters`.

Выходные данные:

- `--output=<Путь к файлу графа>` или `-o <Путь к файлу графа>`

Сохранить граф агрегации в указанном файле. Если параметр не указан, граф выводится на экран (в стандартный поток вывода stdout).

- `--pretty`

Использовать для формата JSON представление с переносом строк и отступами при выводе на экран и сохранении в файл, указанный в параметре `--output`.

Параметры отладки:

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок stderr) в указанном файле. Если параметр не указан, данные выводятся на экран.

- `--mre`

Выводить данные отладки в формате JSON. Если параметр не указан, данные выводятся в текстовом формате.

Пример

```
./rcc --lang=aggregation --taxonomy=/var/knowledgebase_2020_11_01/taxonomy.json --
schema=/var/knowledgebase_2020_11_01/schema.json /var/knowledgebase_2020_11_01/
aggregation-rules --output=/var/rules/output/aggregfilters.json
```

13.3.2. aggregator-cli

Утилита `aggregator-cli` предназначена для агрегации потока нормализованных (и корреляционных) событий, поступающих через стандартный поток ввода данных (stdin). После агрегации события направляются в стандартный поток вывода данных (stdout).

Формат запуска:

```
aggregator-cli
  --rules <Путь к файлу aggregfilters.json>
  --input <Путь к файлу с событиями>
  > <Путь к файлу агрегированных событий>
```

Ключи запуска утилиты:

- `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных stdout) справку утилиты.

- `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных stdout) номер версии утилиты.

- `--mode=<Режим>` или `-m <Режим>`

Выбрать режим работы утилиты. В зависимости от выбранного режима в файл с выходными данными сохраняются:

- 0 — режим Normal: агрегированные и не подвергшиеся агрегации события (выбран по умолчанию);
- 1 — режим AggregationOnly: агрегированные события;
- 2 — режим SkippedOnly: события, удовлетворяющие условию правила агрегации, но не объединенные в агрегированные события;
- 3 — режим UnfilteredOnly: события, не подвергшиеся агрегации.

Входные данные:

- `--rules=<Путь к файлу aggfilters.json>` или `-r <Путь к файлу aggfilters.json>`

Использовать граф агрегации из указанного файла `aggfilters.json`.

- `--input=<Путь к файлу с событиями>` или `-i <Путь к файлу с событиями>`

Выполнить агрегацию нормализованных (и корреляционных) событий из указанного файла (из стандартного потока ввода данных `stdin`) формата JSON. Одно событие может занимать только одну строку файла и должно быть ограничено фигурными скобками, разделители между событиями не используются.

Выходные данные:

- `> <Путь к файлу с выходными данными>`

Сохранять события после агрегации (из стандартного потока вывода данных `stdout`) в указанном файле. Если параметр не указан, события выводятся на экран.

Параметры отладки:

- `--log=<Путь к файлу журнала>` или `-l <Путь к файлу журнала>`

Сохранять журнал агрегации в указанном файле `aggregator_cli.log`.

Пример

```
./aggregator-cli --rules=/var/knowledgebase_2020_11_01/rules/aggfilters.json --
input /var/knowledgebase_2020_11_01/norm_events.json > /var/rules/output/
agg_events.json --log /var/knowledgebase_2020_11_01/aggregator-cli.log
```

13.4. Отладка правил обогащения

Каждое правило обогащения сохраняется в отдельном файле `<Название правила>.en`. Файлы правил обогащения MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `enrichment-rules`. Для отладки правил обогащения вы можете использовать утилиту `enricher-cli`. При отладке вам могут потребоваться дополнительные файлы.

enrules_graph.json

Граф обогащения — файл формата JSON, описывающий совокупность всех условий отбора событий, указанных во всех используемых правилах обогащения.

Для создания графа обогащения вы можете использовать утилиту `rcc`.

fpta_db.db

Файл содержит БД табличных списков. Для создания и изменения БД вы можете использовать утилиту `fpta_filler`.

schema.json

Файл формата JSON содержит схему для создания БД табличных списков. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip` или по аналогии [создать собственный файл \(см. раздел 13.6\)](#).

taxonomy.json

Файл формата JSON, содержит схему полей событий MaxPatrol SIEM. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip`.

В этом разделе

[fpta_filler \(см. раздел 13.4.1\)](#)

[rcc \(см. раздел 13.4.2\)](#)

[enricher-cli \(см. раздел 13.4.3\)](#)

13.4.1. fpta_filler

Утилита `fpta_filler` предназначена для создания и изменения БД табличных списков.

Формат запуска:

```
fpta_filler
  --schema <Путь к файлу schema.json>
  --database <Путь к файлу БД>
  --fillType=All
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

- `--schema <Путь к файлу schema.json>`

Использовать указанный файл `schema.json` со схемой БД табличных списков.

- `--database=<Путь к файлу БД>`

Использовать указанный файл БД табличных списков `fpta_db.db`. Если файл БД отсутствует, то будет создана новая БД в соответствии со схемой, указанной в параметре `--schema`.

- `--filesize=<Размер файла>`

Установить максимальный размер файла БД в мегабайтах (по умолчанию 10 МБ).

Параметры фильтрации при действиях с табличными списками:

- `--table=<Название табличного списка>`

Выполнять действие с указанным табличным списком. Параметр не используется при добавлении записей из файла, указанного в параметре `--input`.

- `--fillType=<Назначение табличного списка>`

Примечание. Если параметр `fillType` не указан, по умолчанию выбрано значение `CorrelationRule`.

Выполнять действия с табличными списками указанного назначения: `Registry` — справочник, `CorrelationRule` — для правил корреляции, `EnrichmentRule` — для правил обогащения, `AssetGrid` — для данных об активах, `CybsiGrid` — для репутационных списков, `All` — с любыми табличными списками.

- `--filter=<PDQL-запрос>`

Выполнить действия с записями, удовлетворяющими PDQL-запросу (см. документ Синтаксис языка запроса PDQL).

Параметры записи в табличные списки:

- `--input=<Путь к файлу с записями>`

Добавить в табличные списки записи из указанного файла формата JSON (см. ниже). Если параметр не указан, табличные списки будут выведены на экран (в стандартный поток вывода данных `stdout`).

- `--create_tables`

Добавить новый табличный список в существующую БД.

Параметры удаления записей из табличных списков:

- `--remove`

Удалить все записи из всех табличных списков в БД. Дополнительные параметры:

- `--remove --filter=<PDQL-запрос>`

Удалить записи, удовлетворяющие запросу, из всех табличных списков.

- `--remove --table=<Название табличного списка>`

Удалить все записи из указанного табличного списка.

- `--remove --filter=<PDQL-запрос> --table=<Название табличного списка>`

Удалить записи, удовлетворяющие запросу, из указанного табличного списка.

Выходные данные:

- `--output=<Путь к файлу>` или `-o <Путь к файлу>`

Сохранить табличные списки (из стандартного потока вывода данных stdout) в указанном файле формата JSON. Если параметр не указан, табличные списки будут выведены на экран.

- `--pretty`

Использовать представление с отступами при сохранении в файл формата JSON, указанный в параметре `--output`.

Параметры отладки:

- `--nofail`

Не прекращать работу утилиты при возникновении одной ошибки.

Формат файла с записями табличного списка

В файле вы можете указать записи для нескольких табличных списков. Для каждого табличного списка нужно указать название, значение ключевой колонки и значения колонок, которые не могут быть пустыми (null). Обязательные для заполнения колонки `_id` и `_last_changed` будут заполнены автоматически (в колонке `_last_changed` вы можете указать дату и время в Unix-формате).

```
{
  <Название табличного списка>:[
    {<Имя ключевой колонки>:<Значение ключа 1>, <Имя колонки 1>:<Значение 11>,
    <Имя колонки 2>:<Значение 12>, ...}
    {<Имя ключевой колонки>:<Значение ключа 2>, <Имя колонки 2>:<Значение 21>,
    <Имя колонки 2>:<Значение 22>, ...}
    ...
  ]
}
```

Пример:

```
{
  "Potentially_unwanted_software":[
    {"key": 6, "filename": "first_example.exe"},
    {"key": 7, "filename": "second_example.exe"}
  ],
}
```

```
"LSASS_openers_whitelist": [
  {"process": "first_example.exe"},
  {"process": "second_example.exe"}
]
```

Пример просмотра записей табличного списка

Просмотр в БД `fpta_db.db` записей табличного списка `Potentially_unwanted_software`.

```
./fpta_filler --schema=/var/knowledgebase_2020_11_01/schema.json --database=/var/
knowledgebase_2020_11_01/dataSource/fpta_db.db --table=Potentially_unwanted_software
```

Пример добавления записей в табличные списки

Добавить в БД `fpta_db.db` записи из файла `input.json`.

```
./fpta_filler --schema=/var/knowledgebase_2020_11_01/schema.json --database=/var/
knowledgebase_2020_11_01/dataSource/fpta_db.db --input=/var/knowledgebase_2020_11_01/
input.json
```

Пример создания новой БД и добавления в нее записей

Создать БД `fpta_db.db` в соответствии со схемой `schema.json` и добавить в нее записи из файла `input.json`.

```
./fpta_filler --schema=/var/knowledgebase_2020_11_01/schema.json --database=/var/
knowledgebase_2020_11_01/dataSource/fpta_db.db --input=/var/knowledgebase_2020_11_01/
input.json --fillType=All
```

13.4.2. rcc

Утилита `rcc` может использоваться для создания графа обогащения.

Формат запуска:

```
rcc
  --lang=enrichment
  --taxonomy=<Путь к файлу taxonomy.json>
  --schema=<Путь к файлу schema.json>
  <Путь к папке правил обогащения>
  --output=<Путь к файлу графа>
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

- `--lang=<Тип графа>` или `-l=<Тип графа>`

Выбор типа правила, для которого создается граф. Для правил обогащения нужно указать `enrichment` или `e`.

- `<Путь к папке правил обогащения>`

Путь к папке с файлами правил обогащения, для которых нужно создать граф. Из папки отбираются файлы с расширением `en`. Вместо папки вы можете указать названия файлов с правилами.

- `--taxonomy=<Путь к файлу taxonomy.json>` или `-t <Путь к файлу taxonomy.json>`

Использовать указанный файл `taxonomy.json` со схемой полей событий.

- `--schema=<Путь к файлу schema.json>` или `-s <Путь к файлу schema.json>`

Использовать указанный файл `schema.json` со схемой БД табличных списков.

- `--lib-path <Путь к папке макросов>` или `-p <Путь к папке макросов>`

Путь к папке с файлами макросов. Вы можете указать несколько папок, используя параметр несколько раз в одной команде. Файлы макросов MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `filters`.

Выходные данные:

- `--output=<Путь к файлу графа>` или `-o <Путь к файлу графа>`

Сохранить граф обогащения в указанном файле. Если параметр не указан, граф выводится на экран (в стандартный поток вывода `stdout`).

- `--pretty`

Использовать для формата JSON представление с переносом строк и отступами при выводе на экран и сохранении в файл, указанный в параметре `--output`.

Параметры отладки:

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок `stderr`) в указанном файле. Если параметр не указан, данные выводятся на экран.

- `--mre`

Выводить данные отладки в формате JSON. Если параметр не указан, данные выводятся в текстовом формате.

Пример

```
./rcc --lang=enrichment --taxonomy=/var/knowledgebase_2020_11_01/taxonomy.json --
schema=/var/knowledgebase_2020_11_01/schema.json /var/knowledgebase_2020_11_01/
enrichment-rules --output=/var/rules/output/enrules_graph.json
```


13.4.3. enricher-cli

Внимание! Для работы утилиты требуются файлы библиотек `icudt55.dll`, `icuuc55.dll`.

Утилита `enricher-cli` предназначена для обогащения данными потока нормализованных и корреляционных событий.

Через стандартный поток ввода данных (`stdin`) утилита получает нормализованные события, обогащает их данными, используя граф обогащения, и направляет в стандартный поток вывода данных (`stdout`).

Формат запуска:

```
enricher-cli
  -r <Путь к файлу enrules_graph.json>
  -b <Путь к БД табличных списков>
  < <Путь к файлу с событиями>
  > <Путь к файлу обогащенных событий>
```

Ключи запуска утилиты:

— `-h` или `-?`

Вывести на экран (в стандартный поток ошибок `stderr`) справку утилиты.

— `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

— `-r <Путь к файлу enrules_graph.json>`

Использовать указанный файл `enrules_graph.json` с графом обогащения.

— `< <Путь к файлу с событиями>`

Выполнить обогащение нормализованных событий из указанного файла (из стандартного потока ввода данных `stdin`) формата JSON. Одно событие может занимать только одну строку файла и должно быть ограничено фигурными скобками, разделители между событиями не используются.

— `-b <Путь к fpta_db.db>`

Использовать указанный файл `fpta_db.db` с БД табличных списков для правил обогащения.

Выходные данные:

— `> <Путь к файлу обогащенных событий>`

Сохранять события после обогащения данными (из стандартного поток вывода данных `stdout`) в указанном файле. Если параметр не указан, события выводятся на экран.

Параметры отладки:

- `-l <Путь к файлу журнала>`
Сохранять журнал обогащения в указанном файле `enricher-cli.log`.
- `-e <Уровень журналирования>`
Выбрать уровень журналирования `trace`, `debug`, `info` или `fatal`.

Пример

```
./enricher-cli -r /var/rules/output/enrules_graph.json -b /var/
knowledgebase_2020_11_01/fpta_db.db < /var/knowledgebase_2020_11_01/norm_events.json
> /var/knowledgebase_2020_11_01/en_events.json -e trace -l /var/
knowledgebase_2020_11_01/enricher-cli.log
```

13.5. Отладка правил корреляции

Каждое правило корреляции сохраняется в отдельном файле `<Название правила>.co`. Файлы правил корреляции MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `correlation-rules`. Для отладки правил корреляции вы можете использовать утилиты `router-cli` и `correlator-cli`. При отладке вам могут потребоваться дополнительные файлы.

fpta_db.db

Файл содержит БД табличных списков. Для создания и изменения БД вы можете использовать утилиту `fpta_filler`.

rules_graph.json

Граф корреляции — файл формата JSON, описывающий совокупность всех последовательностей событий, приводящих к регистрации корреляционного события, указанных во всех условиях используемых правил корреляции.

Для создания графа корреляции вы можете использовать утилиту `rcc`.

schema.json

Файл формата JSON содержит схему для создания БД табличных списков. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip` или по аналогии [создать собственный файл \(см. раздел 13.6\)](#).

taxonomy.json

Файл формата JSON, содержит схему полей событий MaxPatrol SIEM. Вы можете использовать файл, расположенный в архиве `knowledgebase_<Дата экспорта>.zip`.

В этом разделе

[rcc \(см. раздел 13.5.1\)](#)

[router-cli \(см. раздел 13.5.2\)](#)

[correlator-cli \(см. раздел 13.5.3\)](#)

13.5.1. rcc

Утилита `rcc` может использоваться для создания графа корреляции.

Формат запуска:

```
rcc
  --lang=correlation
  --taxonomy=<Путь к файлу taxonomy.json>
  --schema=<Путь к файлу schema.json>
  <Путь к папке правил корреляции>
  --output=<Путь к файлу графа>
```

Ключи запуска утилиты:

— `--help` или `-h`

Вывести на экран (в стандартный поток вывода данных `stdout`) справку утилиты.

— `--version` или `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

— `--lang=<Тип графа>` или `-l=<Тип графа>`

Выбор типа правила, для которого создается граф. Для правил корреляции нужно указать `correlation` или `c`.

— `<Путь к папке правил корреляции>`

Путь к папке с файлами правил корреляции, для которых нужно создать граф. Из папки отбираются файлы с расширением `co`. Вместо папки вы можете указать названия файлов с правилами.

— `--taxonomy=<Путь к файлу taxonomy.json>` или `-t <Путь к файлу taxonomy.json>`

Использовать указанный файл `taxonomy.json` со схемой полей событий.

— `--schema=<Путь к файлу schema.json>` или `-s <Путь к файлу schema.json>`

Использовать указанный файл `schema.json` со схемой БД табличных списков.

— `--lib-path <Путь к папке макросов>` или `-p <Путь к папке макросов>`

Путь к папке с файлами макросов. Вы можете указать несколько папок, используя параметр несколько раз в одной команде. Файлы макросов MaxPatrol SIEM вы можете найти в архиве `knowledgebase_<Дата экспорта>.zip` в папке `filters`.

Выходные данные:

- `--output=<Путь к файлу графа>` или `-o <Путь к файлу графа>`

Сохранить граф корреляции в указанном файле. Если параметр не указан, граф выводится на экран (в стандартный поток вывода `stdout`).

- `--pretty`

Использовать для формата JSON представление с переносом строк и отступами при выводе на экран и сохранении в файл, указанный в параметре `--output`.

Параметры отладки:

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок `stderr`) в указанном файле. Если параметр не указан, данные выводятся на экран.

- `--mre`

Выводить данные отладки в формате JSON. Если параметр не указан, данные выводятся в текстовом формате.

Пример

```
./rcc --lang=correlation --taxonomy=/var/knowledgebase_2020_11_01/taxonomy.json --
schema=/var/knowledgebase_2020_11_01/schema.json /var/knowledgebase_2020_11_01/
correlation-rules --output=/var/rules/output/rules_graph.json
```

13.5.2. router-cli

Внимание! Для работы утилиты требуются файлы библиотек `icudt55.dll`, `icuuc55.dll` и `libejdb.dll`.

Утилита `router-cli` предназначена для поиска нормализованных событий, удовлетворяющих условиям отбора событий, указанным в правилах корреляции. Через стандартный поток ввода данных (`stdin`) утилита получает нормализованные события, используя граф корреляции, выполняет поиск событий и направляет найденные события в стандартный поток вывода данных (`stdout`).

Формат запуска:

```
router-cli
-r <Путь к файлу rules_graph.json>
< <Путь к файлу с событиями>
> <Путь к файлу найденных событий>
```

Ключи запуска утилиты:

- `-h` или `-?`

Вывести на экран (в стандартный поток ошибок `stderr`) справку утилиты.

- `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

- `-r` <Путь к файлу `rules_graph.json`>

Использовать указанный файл `rules_graph.json` с графом корреляции.

- `<` <Путь к файлу с событиями>

Выполнить поиск событий в указанном файле (в стандартном потоке ввода данных `stdin`) формата JSON. Одно событие может занимать только одну строку файла и должно быть ограничено фигурными скобками, разделители между событиями не используются.

- `-b` <Путь к файлу `fpta_db.db`>

Использовать указанный файл `fpta_db.db` с БД табличных списков для правил корреляции.

Выходные данные:

- `>` <Путь к файлу найденных событий>

Сохраняют найденные события (из стандартного потока вывода данных `stdout`) в указанном файле. Если параметр не указан, события выводятся на экран.

- `2>` <Путь к файлу ошибок>

Сохранять данные отладки (из стандартного потока ошибок `stderr`) в указанном файле. Если параметр не указан, данные выводятся на экран.

Параметры поиска событий:

- `-k` <Поле события 1>, <Поле события 2>, ...

Искать события, относящиеся к потоку нормализованных событий, выделенному по совокупности указанных полей.

- `-f` <Название правила 1>, <Название правила 2>, ...

Искать события, удовлетворяющие условиям отбора событий в перечисленных правилах.

Параметры отладки:

- `-s`

Выводить на экран (в стандартный поток ошибок `stderr`) найденные события.

- `-l` <Путь к файлу журнала>

Сохранять журнал поиска в указанном файле `router-cli.log`.

- `-e <Уровень журналирования>`

Выбрать уровень журналирования `trace`, `debug`, `info` или `fatal`.

- `-t <Делитель>`

Изменить временные интервалы в условиях правил корреляции в указанное число раз. Если делитель меньше единицы (но больше нуля), интервал будет увеличен, если больше единицы — уменьшен. Например, если в условии правила корреляции с помощью оператора отсчета времени `timer` указан интервал 10 секунд, при делителе 0,5, корреляционное событие регистрируется при составлении всей последовательности событий в течение 20 секунд.

Пример

```
./router-cli -r /var/knowledgebase_2020_11_01/rules_graph.json < /var/
knowledgebase_2020_11_01/norm_events.json > /var/knowledgebase_2020_11_01/events.json
```

13.5.3. correlator-cli

Внимание! Для работы утилиты `correlator-cli` требуется файл библиотеки `libejdb.dll`.

Утилита `correlator-cli` предназначена для корреляции потока нормализованных событий. Через стандартный поток ввода данных (`stdin`) утилита получает нормализованные события, используя граф корреляции, регистрирует корреляционные события и направляет их в стандартный поток вывода данных (`stdout`).

Формат запуска:

```
correlator-cli
  -r <Путь к файлу rules_graph.json>
  < <Путь к файлу с событиями>
  > <Путь к файлу корреляционных событий>
```

Ключи запуска утилиты:

- `-h` или `-?`

Вывести на экран (в стандартный поток ошибок `stderr`) справку утилиты.

- `-v`

Вывести на экран (в стандартный поток вывода данных `stdout`) номер версии утилиты.

Входные данные:

- `-r <Путь к файлу rules_graph.json>`

Использовать указанный файл `rules_graph.json` с графом корреляции.

- `< <Путь к файлу с событиями>`

Выполнить корреляцию нормализованных событий из указанного файла (из стандартного потока ввода данных `stdin`) формата JSON. Одно событие может занимать только одну строку файла и должно быть ограничено фигурными скобками, разделители между событиями не используются.

- `-b <Путь к файлу fpta_db.db>`

Использовать указанный файл `fpta_db.db` с БД табличных списков для правил корреляции.

Выходные данные:

- `> <Путь к файлу корреляционных событий>`

Сохранять корреляционные события (из стандартного потока вывода данных `stdout`) в указанном файле. Если параметр не указан, события выводятся на экран.

- `2> <Путь к файлу ошибок>`

Сохранять данные отладки (из стандартного потока ошибок `stderr`) в указанном файле. Если параметр не указан, данные выводятся на экран.

Параметры корреляции:

- `-k <Поле события 1>, <Поле события 2>, ...`

Выполнять корреляцию только для правил, содержащих в инструкции `key` директивы `event` все указанные поля события.

- `-f <Название правила 1>, <Название правила 2>, ...`

Выполнять корреляцию только для перечисленных правил.

- `-t <Делитель>`

Изменить временные интервалы в условиях правил корреляции в указанное число раз. Если делитель меньше единицы (но больше нуля), интервал будет увеличен, если больше единицы — уменьшен. Например, если в условии правила корреляции с помощью оператора отсчета времени `timer` указан интервал 10 секунд, при делителе 0,5, корреляционное событие регистрируется при составлении всей последовательности событий в течение 20 секунд.

Параметры отладки:

- `-s`

Выводить на экран (в стандартный поток ошибок `stderr`) статистику регистрации корреляционных событий.

- `-l <Путь к файлу журнала>`

Сохранять журнал корреляции в указанном файле `correlator_cli.log`.

- `-e <Уровень журналирования>`

Выбрать уровень журналирования `trace`, `debug`, `info` или `fatal`.

Пример

```
./correlator-cli -r /var/knowledgebase_2020_11_01/rules_graph.json < /var/
knowledgebase_2020_11_01/norm_events.json > /var/knowledgebase_2020_11_01/
corr_events.json -s 2> /var/knowledgebase_2020_11_01/stderr.json -e trace -l /var/
knowledgebase_2020_11_01/correlator_cli.log
```

13.6. Структура файла schema.json

Файл формата JSON содержит схему для создания БД табличных списков. Для каждого табличного списка нужно указать общие параметры, в зависимости от назначения табличного списка, и параметры каждой колонки. Фрагмент файла `schema.json` с параметрами табличного списка `Success_bruteforce_on_host_table`:

```
"Success_bruteforce_on_host_table": {
  "fields": {
    "host": {
      "type": "String",
      "primaryKey": true,
      "index": true,
      "unique": true,
      "nullable": false
    },
    "user": {
      "type": "String",
      "primaryKey": false,
      "index": false,
      "unique": false,
      "nullable": false
    },
    "_last_changed": {
      "type": "Datetime",
      "primaryKey": false,
      "index": true,
      "unique": false,
      "nullable": false
    },
    "_id": {
      "type": "Number",
      "primaryKey": false,
      "index": true,
      "unique": true,
      "nullable": false
    }
  },
  "fillType": "CorrelationRule",
```



```

    "description": "Table with list of users that logged in to the Cisco Firewall in
past 24 hours",
    "type": 1,
    "ptkbId": "942ca0ec-c076-41b0-a71e-f2eba34817e8",
    "objectId": "PT-TL-1",
    "ttl": 86400,
    "maxSize": 100000,
    "typicalSize": 80000
  },

```

Параметры табличного списка типа «справочник»

Для табличного списка типа «справочник» нужно указать:

- "fillType": "Registry"

Назначение табличного списка. Для табличного списка типа «справочник» параметр имеет значение Registry.

- "description": "<Значение>"

Текст с описанием табличного списка.

- "type": <Значение>

Тип табличного списка: 1 — стандартный, 2 — пользовательский.

- "ptkbId": "<Системный идентификатор>"

Системный идентификатор табличного списка.

- "objectId": "<Идентификатор>"

Идентификатор табличного списка.

- "userCanEditContent": <Значение>

Пользователь может добавлять, изменять и удалять записи в табличном списке — true, иначе — false.

Параметры табличного списка для правил корреляции и обогащения

Для табличного списка для правил корреляции и обогащения нужно указать:

- "fillType": "<Назначение табличного списка>"

Назначение табличного списка. Для табличного списка для правил корреляции параметр имеет значение CorrelationRule, для правил обогащения — EnrichmentRule.

- "description": "<Значение>"

Текст с описанием табличного списка.

- "type": <Значение>

Тип табличного списка: 1 — стандартный, 2 — пользовательский.

- "ptkbId": "<Системный идентификатор>"

Системный идентификатор табличного списка.

- "objectId": "<Идентификатор>"

Идентификатор табличного списка.

- "ttl": <Значение>

Время жизни записи табличного списка в секундах; 0 — время жизни не ограничено.

- "maxSize": <Значение>

Максимальное число записей.

- "typicalSize": <Значение>

Рекомендуемое число записей.

Параметры табличного списка для данных об активах

Для табличного списка для данных об активах нужно указать:

- "fillType": "AssetGrid"

Назначение табличного списка. Для табличного списка для данных об активах параметр имеет значение AssetGrid.

- "description": "<Значение>"

Текст с описанием табличного списка.

- "type": <Значение>

Тип табличного списка: 1 — стандартный, 2 — пользовательский.

- "ptkbId": "<Системный идентификатор>"

Системный идентификатор табличного списка.

- "objectId": "<Идентификатор>"

Идентификатор табличного списка.

- "pdql": "<PDQL-запрос>"

PDQL-запрос для заполнения табличного списка.

- "pdqlMappings": {}

Объект для указания колонок табличного списка, в которые будут записываться данные об активах, полученные из PDQL-запроса. Названия полей актива и колонок табличного списка должны совпадать с указанными в PDQL-запросе. Колонки нужно указывать через запятую в виде: "<Название поля актива>": {"tableColumn": "<Название колонки>"}.

Примечание. Для колонок `_id` и `_last_changed`, `complex_key` описание не требуется.

- `"groupsFilter": {"groupIds": [], "includeNested": <Значение>}`

Дополнительный фильтр по группам активов содержит объекты:

- `"groupIds": ["ID группы 1", "ID группы 2", ...]`

Список системных идентификаторов групп активов. PDQL-запрос выполняется к данным активов в указанных группах.

- `"includeNested": <Значение>`

Выполнять PDQL-запрос к данным активов во вложенных группах — `true`, иначе — `false` (по умолчанию).

Параметры репутационного табличного списка

Для репутационного табличного списка нужно указать:

- `"fillType": "CybsiGrid"`

Назначение табличного списка. Для репутационного табличного списка имеет значение `CybsiGrid`.

- `"description": "<Значение>"`

Текст с описанием табличного списка.

- `"type": <Значение>`

Тип табличного списка: 1 — стандартный, 2 — пользовательский.

- `"ptkbId": "<Системный идентификатор>"`

Системный идентификатор табличного списка.

- `"objectId": "<Идентификатор>"`

Идентификатор табличного списка.

- `"minScore": 90`

Параметр для ввода значения интегральной уязвимости (целое число от 0 до 100), при превышении которого объект попадет в табличный список. Если параметр не указан, в табличный список попадут все объекты.

- `"itemType": "<Значение>"`

Параметр для выбора объектов репутационного списка: `ip` — IP-адрес, `domain` — имя домена, `hash` — значение контрольной суммы;

- `"mappings": {}`

Объект для указания колонок табличного списка, в которые будут записываться объекты репутационного списка. Колонки нужно указывать через запятую в виде: `"<Название объекта репутационного списка>": {"tableColumn": "<Название колонки>"}`.

Параметры колонок

Для каждой колонки табличного списка нужно указать:

— `fields: {}`

Примечание. Каждый табличный список должен содержать колонки `_id` и `_last_changed` для идентификатора записи и времени ее изменения. Эти колонки заполняются автоматически при добавлении записи.

Объект содержит параметры колонок:

- `"<Название колонки>"`

Название колонки.

- `"type": "<Тип данных>"`

Тип данных колонки может принимать значения: `Datetime`, `Number`, `String`. Для хранения регулярных выражений нужно указать `Regex` (только для табличного списка типа «справочник»).

- `"primary_key": <Значение>`

Колонка является ключевой — `true`, иначе — `false`. В табличном списке может быть одна ключевая колонка.

Примечание. Для создания составного ключа, нужно добавить служебную колонку `complex_key` и указать в ней названия колонок, составляющих ключ. При этом для всех колонок табличного списка в параметре `primary_key` нужно указать `false`.

- `"index": <Значение>`

Данные в колонке индексируются — `true`, иначе — `false`.

- `"unique": <Значение>`

Значения в колонке должны быть уникальными — `true`, иначе — `false`. Для ключевой колонки должно быть `true`.

- `"nullable": <Значение>`

Колонка может содержать `null` — `true`, иначе — `false`.

— Предусмотрена возможность создания служебной колонки `complex_key` для хранения параметров составного ключа. Для колонки нужно указать:

- `"type": "composite"`
- `"primary_key": true`
- `"unique": true`
- `"composite_fields": [<Название колонки 1>, <Название колонки 2>, ...]`

Список названий колонок (не менее двух), значения которых образуют составной ключ записи.

14. Утилита PTSIEMSDK GUI

Утилита предназначена для создания и отладки отдельных правил нормализации, агрегации, обогащения, корреляции и локализации, для создания табличных списков и макросов, а также для отладки их совместной работы. Добавление правил в MaxPatrol SIEM выполняется через веб-интерфейс Knowledge Base. Утилита входит в комплект поставки MaxPatrol SIEM.

Программные требования

Поддерживается работа утилиты в Windows 11, 10, 8.1, 7 и Windows Server 2019, 2016, 2012R2, 2008R2 с установленными обновлениями.

Для работы утилиты требуется следующее ПО:

- Microsoft .NET Framework v4.7.1,
- Microsoft Visual C++ для Visual Studio 2015, 2017, 2019 и 2022 (x86),
- Microsoft Visual C++ для Visual Studio 2015, 2017, 2019 и 2022 (x64);
- Microsoft .NET Runtime 6.0.

Примечание. Вы можете скачать это ПО с сайта support.microsoft.com.

В этом разделе

[Интерфейс \(см. раздел 14.1\)](#)

[Предварительная настройка \(см. раздел 14.2\)](#)

[Работа с пакетами \(см. раздел 14.3\)](#)

[Отладка отдельного правила \(см. раздел 14.4\)](#)

[Отладка совместной работы правил \(см. раздел 14.5\)](#)

14.1. Интерфейс

Этот раздел содержит описание интерфейса утилиты PTSIEMSDK GUI.

В окне программы расположены вкладки **Редактор объектов**, **Сценарии** и **Настройка** и кнопка **Управление пакетами**. В нижней части окна в строке состояния указаны версии используемых утилитой MaxPatrol SIEM SDK, набора дополнительных инструментов для работы с правилами и схемы полей событий, а также расположена ссылка для вызова справки, где содержится детальная информация о схеме полей событий и другие справочные данные.

В этом разделе

[Вкладка Редактор объектов \(см. раздел 14.1.1\)](#)

[Вкладка Сценарии \(см. раздел 14.1.2\)](#)

[Вкладка Настройка \(см. раздел 14.1.3\)](#)

[Управление пакетами \(см. раздел 14.1.4\)](#)

[Справка \(см. раздел 14.1.5\)](#)

14.1.1. Вкладка Редактор объектов

Вкладка предназначена для создания и отладки отдельных правил нормализации, агрегации, обогащения, корреляции и локализации, а также для создания макросов и табличных списков.

В панели инструментов вкладки расположены:

- Кнопка **Создать** для создания нового правила и набора данных для его отладки.
- Кнопка **Открыть** для изменения созданного ранее правила или набора данных для его отладки.
- Раскрывающийся список для быстрого выбора одного из открытых правил.

В центральной части вкладки расположены вкладки открытых объектов. В названии вкладки отображается название объекта, сокращение для типа объекта: **[N]** — для правил нормализации, **[A]** — для правил агрегации, **[C]** — для правил корреляции, **[E]** — для правил обогащения, **[M]** — для макросов, **[T]** — для табличного списка, а также звездочка — если изменения в объекте не были сохранены. Вы можете менять порядок вкладок, перетаскивая их мышью.

Примечание. В контекстном меню названия вкладки доступны пункты для копирования, переименования правила или закрытия вкладки.

В панели инструментов вкладки правила расположены:

- Меню **Файл**, содержит пункты для сохранения, копирования, переименования правила или закрытия вкладки правила.
- Меню **Правка**, содержит пункты для добавления шаблона кода правила, добавления исходных событий для тестов из файла и удаления всех тестов для правила.
- Меню **Вид**, содержит пункты для включения (отключения) переноса по словам и нумерации строк в полях вкладки.
- Кнопка  для быстрого сохранения правила.
- Кнопка **Метаданные** для добавления правил локализации регистрируемых по правилу событий и описания правила.
- Кнопка **Запуск** для запуска отладки правила. По стрелке справа от кнопки вы можете выбрать тесты для отладки правила.
- Кнопка **RegEx** для запуска отладки регулярных выражений. Кнопка доступна при редактировании правил корреляции и обогащения, а также макросов.
- Поле **Тип объекта**, в поле указан тип правила.

- Раскрывающийся список **MIME** для принудительного выбора формата исходного события (доступен только для правил нормализации).
- Кнопка **✕** для закрытия вкладки правила.

В рабочей области вкладки правила расположены:

- Поле **Правило (Формула, Макрос, Табличный список)** для ввода кода правила.

Примечание. В контекстном меню полей **Правило (Формула, Макрос, Табличный список)**, **Исходное событие (Тестовый сценарий)**, **Результат** доступны пункты для работы с текстом, в том числе поиск и замена. Также в меню доступен флажок для включения переноса по словам.

- Поля **Исходное событие** или **Тестовый сценарий** для ввода исходного события для правила нормализации или тестового сценария для правил агрегации, корреляции или обогащения.
- Поле **Результат**, в котором выводятся результат и сообщения о работе правила.
- Панель для работы с тестами. В верхней части панели расположены кнопки для добавления **+** и для удаления **—** тестов. Для каждого теста нужно указать исходное событие или тестовый сценарий.

14.1.2. Вкладка Сценарии

Вкладка предназначена для отладки совместной работы правил на этапах создания графов и потоковой обработки событий при нормализации, агрегации, обогащении, корреляции и локализации.

В панели инструментов вкладки расположены:

- Кнопка **Запуск** для запуска сценария.
- Поле для выбора сценария из списка.
- Кнопка **+** для создания нового сценария.
- Кнопка **⚙** для вызова окна **Управление сценариями**, в котором можно добавить, удалить, изменить название сценария, а также переключаться между сценариями.
- Кнопка **Папка с результатами** для просмотра папки с результатами выполнения сценария в проводнике Windows. По кнопке открывается папка, указанная на вкладке **Настройка → Пути** в блоке параметров **Прочее** в поле **Папка с результатами**, или папка, переопределенная пользователем с помощью кнопки выбора папки в панели инструментов.
- Поле для отображения папки с результатами выполнения сценария.
- Кнопка **...** для выбора папки с результатами выполнения сценария.
- Кнопка **✕** для сброса значения в поле **Папка с результатами**.

- Кнопка **Дополнительно**. По кнопке открывается меню с пунктом **Просмотр событий** для просмотра локализованных событий из указанного файла.
- Кнопка **Сброс параметров** для очистки блоков параметров, соответствующих этапам совместной работы правил.

В рабочей области вкладки расположена панель **Параметры BUILD**, которая содержит флажки с названиями этапов, связанных с созданием графов нормализации, агрегации, обогащения, корреляции и локализации, а также с созданием базы данных табличных списков. Установка того или иного флажка включает соответствующий этап в сценарий и делает доступным блок для настройки параметров выполнения этого этапа. В панели доступны следующие флажки:

- **BUILD FORMULAS** — для настройки создания графа нормализации. В блоке нужно указать расположение папки с правилами нормализации, для которых нужно создать граф. Из указанной и всех вложенных папок отбираются файлы с расширением `xr`. Для запуска тестов правил нормализации необходимо установить флажок **Выполнять тесты для формул нормализации**. Граф нормализации сохраняется в папке с результатами в файле `formulas_graph.json`.
- **BUILD AGGREGATION RULES** — для настройки создания графа агрегации. В блоке нужно указать расположение папки с правилами агрегации, для которых нужно создать граф. Из указанной и всех вложенных папок отбираются файлы с расширением `agr`. Граф агрегации сохраняется в папке с результатами в файле `aggfilters.json`.
- **BUILD TABLE LISTS SCHEMA** — для настройки создания схемы табличных списков, а также файла со значениями по умолчанию для табличных списков. В блоке необходимо указать расположение папки с табличными списками, для которых нужно создать схему. Из указанной и всех вложенных папок отбираются файлы с расширением `tl`. Схема табличных списков сохраняется в папке с результатами в файле `schema.json`, а значения по умолчанию для табличных списков в файле `correlation_defaults.json`.
- **BUILD TABLE LISTS DATABASE** — для настройки создания БД табличных списков. В блоке нужно указать расположение файла `schema.json` со схемой табличных списков и файла `correlation_defaults.json` с записями для заполнения табличных списков. БД сохраняется в папке с результатами в файле `fpta_db.db`.

Примечание. Если флажки **BUILD TABLE LISTS SCHEMA** и **BUILD TABLE LISTS DATABASE** установлены совместно, расположение файлов будет указано автоматически.

- **BUILD ENRICHMENT RULES** — для настройки создания графа обогащения. В блоке нужно указать расположение папки с правилами обогащения, для которых нужно создать граф. Из указанной и всех вложенных папок отбираются файлы с расширением `en`. Также нужно указать расположение файла `schema.json` со схемой табличных списков. Граф обогащения сохраняется в папке с результатами в файле `enrules_graph.json`.

- **BUILD CORRELATION RULES** — для настройки создания графа корреляции. В блоке нужно указать расположение папки с правилами корреляции, для которых нужно создать граф. Из указанной и всех вложенных папок отбираются файлы с расширением `co`. Также нужно указать расположение файла `schema.json` со схемой табличных списков. Граф корреляции сохраняется в папке с результатами в файле `corrules_graph.json`.
- **BUILD LOCALIZATION** — для настройки создания графов локализации нормализованных, агрегированных и корреляционных событий для русского и английского языков. В блоке нужно указать расположение папок с правилами локализации правил в формате YAML. Из указанных и всех вложенных папок отбираются файлы `i18n_ru.yaml`, `i18n_en.yaml`. Графы локализации сохраняются в папке с результатами в папке `langs` в файлах `ru.lang` и `en.lang`.

В рабочей области вкладки расположена панель **Параметры RUN**, которая содержит флажки с названиями этапов обработки событий: нормализации, агрегации, обогащения, корреляции и локализации. Установка флажка включает соответствующий этап в сценарий и делает доступными блок для настройки параметров выполнения этого этапа. В панели доступны следующие флажки:

- **NORMALIZE** — для настройки нормализации событий. В блоке нужно указать расположение файлов с графом нормализации `formulas_graph.json` и с исходными событиями. Для сохранения ненормализованных событий в файле `not_normalized.json` нужно установить флажок **Создать файл ненормализованных событий**, для вывода статистики нормализации — установить флажок **Показать статистику**. Нормализованные события сохраняются в папке с результатами в файле `norm_events.json`.

Примечание. В файле с исходными событиями можно указывать либо события в формате JSON (прошедшие предварительную обработку в MaxPatrol SIEM), либо необработанные события. Каждое событие нужно указывать в отдельной строке. Если в файле указаны необработанные события, нужно установить флажок **Исходные события без конвертов** и с помощью раскрывающегося списка **MIME** выбрать тип событий. Все необработанные события в файле должны быть одного типа.

- **AGGREGATE** — для настройки агрегации событий. В блоке нужно указать расположение файлов с графом агрегации `aggfilters.json` и с нормализованными событиями. Агрегированные события сохраняются в папке с результатами в файле `aggr_norm_events.json`.
- **ENRICH** — для настройки обогащения событий. В блоке нужно указать расположение файлов с графом обогащения `enrules_graph.json` и с нормализованными событиями. Если в правилах обогащения используются запросы к табличным спискам, нужно указать расположения файла `fpta_db.db` с БД табличных списков. После обогащения события сохраняются в папке с результатами в файле `enrich_events.json`.
- **CORRELATE** — для настройки корреляции событий. В блоке нужно указать расположение файлов с графом корреляции `corrules_graph.json` и с нормализованными событиями. Если в правилах корреляции используются запросы к табличным спискам, нужно указать расположения файла `fpta_db.db` с БД табличных списков. Корреляционные события сохраняются в папке с результатами в файле `corr_events.json`. Для вывода в окне

Сценарии статистики корреляции нужно установить флажок **Показать статистику**. Для создания отдельного файла с событиями, которые попали под фильтры правил корреляции, нужно установить флажок **Создать файл с отфильтрованными событиями**.

Примечание. Вы можете настроить фильтр правил, по которым будет выполняться корреляция, установив флажок **Дополнительные фильтры** и указав через запятую названия правил или полей событий. Для корреляции по правилу все указанные поля событий должны содержаться в правиле корреляции в инструкции key директивы event.

- **LOCALIZE** — для настройки локализации событий. В блоке нужно указать расположение папки с файлами графов локализации `ru.lang`, `en.lang` и файла с нормализованными событиями. Описания регистрируемых по правилам событий сохраняются в папке с результатами в файлах `ru_events.json` и `en_events.json`. Чтобы после завершения сценария просмотреть описания зарегистрированных событий, нужно установить флажок **Автоматически запускать просмотр событий**.

14.1.3. Вкладка Настройка

Вкладка предназначена для настройки параметров работы утилиты и содержит вкладки **Пути**, **Редактор объектов**, **Сценарии** и **Общее**. В верхней части вкладки расположена кнопка **О приложении** для просмотра информации об утилите.

Вкладка Пути

Вкладка **Пути** содержит следующие блоки параметров:

- **Исходный код объектов** — для выбора папок с правилами для отладки их совместной работы:
 - **Папки с объектами (по умолчанию)** — поле для выбора папок с правилами или табличных списков.
 - **Папка с макросами** — поле для выбора папки с файлами фильтров для правил обогащения и корреляции.
 - **Файл «appendix.xpr»** — поле для выбора файла `appendix.xpr`, содержащего правила заполнения полей нормализованного события `event_src.host`, `src.host`, `dst.host`.
- **Инструменты и файлы** — для выбора инструментов и файлов, необходимых для работы утилиты:
 - **Папка «ptsiem-sdk»** — поле для выбора папки с утилитами MaxPatrol SIEM SDK (входит в комплект поставки утилиты).
 - **Папка «build-tools»** — поле для выбора папки с набором дополнительных инструментов для работы с правилами (входит в комплект поставки утилиты).
 - **Файл «taxonomy»** — поле для выбора файла `taxonomy.json` со схемой полей событий MaxPatrol SIEM (входит в комплект поставки утилиты).

- **Файл схемы табличных списков** — поле для выбора файла `schema.json` со схемой табличных списков.
- **Файл значений по умолчанию для табличных списков** — поле для выбора файла `correlation_defaults.json` с записями для заполнения табличных списков.
- **Прочее** — содержит поле **Папка с результатами** для выбора папки сохранения результатов совместной работы правил.

Вкладка Редактор объектов

Вкладка содержит следующие блоки параметров для настройки редактора объектов:

- **Цветовая тема** — для выбора светлого или темного фона полей.
- **Шрифт** — для изменения шрифта текста в полях.
- **Дополнительно** — для настройки дополнительных параметров отображения кода. В блоке расположены следующие флажки:
 - **Подсветка синтаксиса** — при установленном флажке операторы, функции, токены, ключевые слова, поля и значения полей событий выделяются в коде цветом.
 - **Подсветка совпадающих слов** — при установленном флажке во время ввода кода одинаковые слова выделяются серым.
 - **Сворачивание кода** — при установленном флажке появляется возможность сворачивать блоки кода инструкций `filter`, `init`, `on`, `emit`, функции `submessage` и конструкции `subformula ... endsubformula`.
 - **Автоотступ** — при установленном флажке по клавише Enter выполняется выравнивание строк кода.
 - **Автозаполнение** — при установленном флажке **Фрагменты кода** во время ввода кода выводятся подсказки для названий функций, при установленном флажке **Схема полей событий** — варианты автозаполнения полей событий, при установленном флажке **Описание функций** — краткие описания функций, при установленном флажке **Временные переменные** — варианты автозаполнения временных переменных, описанных ранее в правиле, при установленном флажке **Скобки и кавычки** — автозаполнение парных скобок и кавычек.
- **Образец** — для просмотра примера отображения кода правил.
- **Сеанс пользователя** — для настройки автоматического сохранения сеанса работы с утилитой и восстановления результата предыдущего сеанса.

- **Параметры по умолчанию** — для настройки параметров создания и сохранения правил:
 - **Скрыть вкладки** — при установленном флажке не отображаются вкладки открытых правил. Выбор правила осуществляется с помощью раскрывающегося списка в панели инструментов.
 - **Добавлять шаблоны метаданных для новых объектов** — при установленном флажке при создании набора данных для отладки правила в набор автоматически добавляется шаблон для метаданных.
 - **Проверять статусы тестов перед сохранением объектов** — при установленном флажке при сохранении тестов для отладки правил выполняется проверка корректности выполнения тестов. Если тесты не выполнялись или завершились с ошибкой, выводится предупреждение.
 - **Отключить отслеживание изменений файлов объекта** — при установленном флажке отключено отслеживание изменение файлов объекта внешними программами. Если отслеживание включено, при обнаружении изменений утилита предлагает обновить файлы объекта в редакторе.
- **Вывод результата** — для настройки сортировки полей событий в поле **Результат**.
- **Отображение названия объектов** — для настройки названий вкладок:
 - **Отображать тип объекта в названии** — при установленном флажке в названиях вкладок отображаются сокращения для типов правил.
 - **Помечать звездочкой (*) несохраненные объекты** — при установленном флажке для несохраненного правила в названии вкладки отображается звездочка (*).

Вкладка Сценарии

Вкладка **Сценарии** содержит блок параметров **Сеанс пользователя** с флажками для настройки автоматического сохранения сценария и восстановления результата предыдущего сеанса работы.

Вкладка Общее

Вкладка содержит следующие блоки параметров:

- **Язык** с раскрывающимся списком для выбора языка интерфейса утилиты.
- **Информация о поставщике для создания пакетов экспертизы**, в котором расположены:
 - Поле для отображения информации о поставщике.
 - Кнопка **Создать** для заполнения информации о поставщике. Необходимо указать псевдоним поставщика, его системное имя, а также отображаемое имя кириллицей и латиницей (все поля являются обязательными для заполнения). Идентификатор поставщика будет сгенерирован утилитой.

- Кнопка **Изменить** для изменения информации о поставщике. Вы можете изменить отображаемое имя поставщика.
- Кнопка **Удалить** для удаления информации о поставщике.
- Кнопка **Импорт** для импорта информации о поставщике.
- Кнопка **Экспорт** для экспорта информации о поставщике.

14.1.4. Управление пакетами

Кнопка **Управление пакетами** предназначена для распаковки архивов с объектами, экспортированными из Knowledge Base, а также создания архивов объектов для импорта в Knowledge Base. По кнопке раскрывается меню со следующими пунктами:

- **Распаковать .kb** — для распаковки архива пакета с объектами, экспортированными из Knowledge Base.
- **Запаковать .kb** — для формирования пакета с объектами из папок экспертной структуры.
- **Создать .kb с объектами** — для формирования пакета с объектами из произвольных папок.
- **Создать .kb с пакетом экспертизы** — для формирования пакета с пользовательским пакетом экспертизы.
- **Информация о поставщике** — для перехода в блок параметров **Информация о поставщике для создания пакетов экспертизы** на вкладке **Общие** вкладки **Настройка**.

14.1.5. Справка

Окно **Справка** содержит вкладки **Схема полей событий** и **Горячие клавиши**.

На вкладке **Схема полей событий** расположены:

- Вкладка **Редактируемые поля** с информацией о полях, которые вы можете заполнять при описании нормализованного события.
- Вкладка **Системные поля** с информацией о полях, заполняемых системой.
- Блок параметров **Допустимые значения поля Enum**.

При создании правила вы можете перетащить нужное значение поля из вкладки или блока параметров в редактор объектов.

Вкладка **Горячие клавиши** содержит информацию про использование горячих клавиш в утилите.

14.2. Предварительная настройка

Перед началом работы с утилитой рекомендуется выполнить экспорт всех объектов из Knowledge Base с вариантом формата экспорта **Для импорта в другую Knowledge Base**. При экспорте вы получите файл с расширением `.kb` с файлами правил нормализации, обогащения, агрегации, корреляции и локализации, а также с табличными списками и макросами, используемыми в MaxPatrol SIEM. После экспорта нужно распаковать файл в папку `knowledgebase_<Дата экспорта>` с помощью пункта меню **Распаковать .kb**, которое раскрывается по кнопке **Управление пакетами**.

► Чтобы настроить утилиту:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Настройка**.

3. Выберите вкладку **Пути**.

4. В блоке параметров **Прочее** справа от поля **Папка с результатами** нажмите  и выберите папку для сохранения результатов работы утилиты.

5. Если вы планируете выполнять отладку совместной работы правил, в поле **Папки с объектами** по кнопке **Добавить папку** добавьте папки с вашими правилами.

6. Если вы планируете выполнять отладку совместной работы правил с правилами MaxPatrol SIEM, по кнопке **Добавить папку** добавьте папки `knowledgebase_<Дата экспорта>\packages` и `knowledgebase_<Дата экспорта>\objects`.

7. Если вы планируете выполнять отладку правил нормализации, в блоке параметров **Исходный код объектов** справа от поля **Файл «appendix.xp»** нажмите  и выберите файл `appendix.xp`.

Вы можете выбрать файл, расположенный в папке `knowledgebase_<Дата экспорта>\contracts\xp_appendix`.

8. Если вы планируете выполнять отладку правил корреляции или обогащения и в них используются макросы, справа от поля **Папка с макросами** нажмите  и выберите папку с файлами макросов.

Если вы используете только макросы из MaxPatrol SIEM, вы можете выбрать папку `knowledgebase_<Дата экспорта>\common\rules_filters`.

Примечание. Папки с объектами, файл `appendix.xp` и папка с файлами макросов будут автоматически указаны в поле **Папки с объектами (по умолчанию)**, если при распаковке архива с объектами MaxPatrol SIEM в окне **Распаковать .kb** установлен флажок **Задать как папку с объектами (по умолчанию)** в разделе «Настройка».

Утилита настроена.

14.3. Работа с пакетами

Утилита позволяет распаковывать пакеты с объектами (правилами, макросами и табличными списками), экспортированные из Knowledge Base, и формировать пакеты с объектами или пользовательским пакетом экспертизы для импорта в Knowledge Base. Пакет представляет собой файл с расширением `.kb`.

В этом разделе

[Распаковка пакета \(см. раздел 14.3.1\)](#)

[Формирование пакета с объектами из папок экспертной структуры \(см. раздел 14.3.2\)](#)

[Формирование пакета с объектами из произвольных папок \(см. раздел 14.3.3\)](#)

[Формирование пакета с пользовательским пакетом экспертизы \(см. раздел 14.3.4\)](#)

14.3.1. Распаковка пакета

► Чтобы распаковать пакет:

1. Запустите файл `PTSIEMSDK_GUI.exe`.
Откроется окно утилиты PTSIEMSDK GUI.
2. В панели инструментов нажмите кнопку **Управление пакетами** и в раскрывшемся меню выберите пункт **Распаковать .kb**.
3. В открывшемся окне в поле **Пакет базы знаний (.kb)** выберите пакет для распаковки.
4. В поле **Папка** выберите папку для распаковки пакета.

Примечание. В названии папки для распаковки не рекомендуется использовать кириллицу и пробелы.

В выбранной папке утилита создаст папку `knowledgebase_<Дата экспорта>` со следующими вложенными папками: `common` для общих объектов, не подлежащих разделению на пакеты; `contracts` для файлов работы с контентом; `object` для объектов, размещаемых вне пакетов экспертизы; `packages` для пакетов экспертизы с объектами.

Чтобы сохранить выбранный путь к папке, вы можете установить флажок **Сохранить в разделе «Настройка» как папку с объектами (по умолчанию)**.

5. Нажмите кнопку **Распаковать**.

Начнется процесс распаковки. По его завершении появится сообщение `Knowledge base unpacking completed successfully`.

14.3.2. Формирование пакета с объектами из папок экспертной структуры

При распаковке пакета с объектами из Knowledge Base утилита создает папку `knowledgebase_<Дата экспорта>` с вложенными папками экспертной структуры `common`, `contracts`, `object` и `packages`. Вы можете сформировать пакет с объектами из папок экспертной структуры.

► Чтобы сформировать пакет с объектами из папок экспертной структуры:

1. Запустите файл `PTSIEMSDK_GUI.exe`.
Откроется окно утилиты PTSIEMSDK GUI.
2. В панели инструментов нажмите кнопку **Управление пакетами** и в раскрывшемся меню выберите пункт **Запаковать .kb**.
3. В поле **Папка с объектами** выберите папку, в которую был распакован пакет с объектами из Knowledge Base.
4. В поле **Обзор объектов** выберите объекты для формирования пакета.

Примечание. Чтобы исключить отображение объектов, созданных с префиксом PT, вы можете установить флажок **Исключить объекты PT**.

5. В поле **Папка** укажите папку, в которую будет помещен сформированный пакет.
В поле **Название** вы можете изменить название пакета, предложенное утилитой.
6. Если требуется валидация объектов, установите флажок **Выполнить валидацию правил и табличных списков**.

Примечание. Для формирования пакета, совместимого с MaxPatrol SIEM версий 24.0 и 24.1, необходимо установить флажок **Совместимость с релизом 24**.

7. Нажмите кнопку **Создать**.

Пакет сформирован.

14.3.3. Формирование пакета с объектами из произвольных папок

Вы можете сформировать пакет с объектами, созданными не в папках экспертной структуры, а в произвольных папках.

► Чтобы сформировать пакет с объектами из произвольных папок:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. В панели инструментов нажмите кнопку **Управление пакетами** и в раскрывшемся меню выберите пункт **Создать .kb с объектами**.
3. В открывшемся окне в поле **Папка с объектами** выберите произвольную папку с объектами.
4. В поле **Обзор объектов** выберите объекты для формирования пакета.

Примечание. Чтобы исключить отображение объектов, созданных с префиксом PT, вы можете установить флажок **Исключить объекты PT**.

5. В поле **Папка** укажите папку, в которую будет помещен сформированный пакет.
В поле **Название** вы можете изменить название пакета, предложенное утилитой.
6. Если требуется валидация объектов, установите флажок **Выполнить валидацию правил и табличных списков**.

Примечание. Для формирования пакета, совместимого с MaxPatrol SIEM версий 24.0 и 24.1, необходимо установить флажок **Совместимость с релизом 24**.

7. Нажмите кнопку **Создать**.

Пакет сформирован.

14.3.4. Формирование пакета с пользовательским пакетом экспертизы

Вы можете создать собственный пакет экспертизы, сформировать пакет (файл с расширением .kb) и импортировать его в Knowledge Base. Для этого предварительно необходимо ввести информацию о поставщике на вкладке **Общее** вкладки **Настройка**: псевдоним поставщика, его системное имя, а также отображаемое имя кириллицей и латиницей. Идентификатор поставщика будет сгенерирован утилитой при создании поставщика. Вы можете импортировать информацию о поставщике из файла knowledgebase_<Дата экспорта>\contracts\origins\origins.json (файл появится после распаковки пакета с объектами, экспортированного из Knowledge Base).

► Чтобы сформировать пакет с пользовательским пакетом экспертизы:

1. Запустите файл PTSIEMSDK_GUI.exe.
Откроется окно утилиты PTSIEMSDK GUI.
2. В панели инструментов нажмите кнопку **Управление пакетами** и в раскрывшемся меню выберите пункт **Создать .kb с пакетом экспертизы**.
3. В открывшемся окне в поле **Папка с объектами** выберите папку с объектами для формирования пакета.
4. В поле **Обзор объектов** выберите объекты для формирования пакета.

Примечание. Чтобы исключить отображение объектов, созданных с префиксом PT, вы можете установить флажок **Исключить объекты PT**.

5. На вкладке **Общее** укажите порядковый номер, версию и системное имя пакета экспертизы, а также его названия кириллицей и латиницей.

На вкладках **Описание (RU)** и **Описание (EN)** вы можете добавить описание пакета экспертизы.

На вкладке **Вложения** вы можете добавить файлы к пакету экспертизы.

6. В поле **Папка** укажите папку, в которую будет помещен сформированный пакет.

В полях **Название (RU)** и **Название (EN)** вы можете изменить название пакета, предложенное утилитой.

7. Если требуется валидация объектов, установите флажок **Выполнить валидацию правил и табличных списков**.

Примечание. Для формирования пакета, совместимого с MaxPatrol SIEM версий 24.0 и 24.1, необходимо установить флажок **Совместимость с релизом 24**.

8. Нажмите кнопку **Создать**.

Пакет с пакетом пользовательской экспертизы сформирован.

14.4. Отладка отдельного правила

Внимание! После отладки отдельного правила нормализации необходимо убедиться в отсутствии конфликтов с другими правилами MaxPatrol SIEM, после отладки отдельного правила агрегации, обогащения или корреляции — проверить работу правила с потоком событий. Для этого нужно выполнить отладку совместной работы правил.

Вы можете выполнять отладку отдельных правил нормализации, агрегации, обогащения, корреляции и локализации. После отладки правила вы можете сохранить набор данных для отладки в отдельной папке с названием правила. При этом сохраняются файлы с кодом правила, с исходными событиями, с описанием правила и с правилами локализации для различных языков. Для отладки отдельного правила нужно:

1. Если вы планируете выполнять отладку правил корреляции или обогащения и в них используются запросы к табличным спискам — создать схему табличных списков и файл со значениями по умолчанию для табличных списков.
2. Создать набор данных для отладки правила и добавить в него код правила.
3. Добавить в набор исходное событие (для правила нормализации) или тестовый сценарий (для правил обогащения, корреляции или агрегации).
4. Если требуется выполнить отладку правила для нескольких типов необработанных событий или нескольких сценариев — добавить в набор дополнительные тесты для каждого типа или сценария.

5. Если вы планируете выполнять отладку совместной работы правил локализации — добавить в набор метаданные для локализации.
6. Если в правилах корреляции и обогащения или в макросах вы планируете использовать регулярные выражения — проверить корректность этих регулярных выражений.
7. Запустить отладку правила.

В этом разделе

[Создание схемы табличных списков и файла со значениями по умолчанию \(см. раздел 14.4.1\)](#)

[Создание набора данных для отладки и отладка правила \(см. раздел 14.4.2\)](#)

[Составление тестового сценария \(см. раздел 14.4.3\)](#)

[Отладка правила с помощью тестов \(см. раздел 14.4.4\)](#)

[Метаданные для локализации \(см. раздел 14.4.5\)](#)

[Проверка регулярного выражения \(см. раздел 14.4.6\)](#)

14.4.1. Создание схемы табличных списков и файла со значениями по умолчанию

Схема табличных списков и файл со значениями по умолчанию для табличных списков используются для отладки правил корреляции или обогащения, если в правилах используются запросы к табличным спискам.

- Чтобы создать схему табличных списков и файл со значениями по умолчанию для табличных списков:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Сценарии** и в блоке параметров **BUILD** установите флажок **BUILD TABLE LISTS SCHEMA**.

Откроется блок параметров для настройки создания схемы табличных списков и файла со значениями по умолчанию для табличных списков.

3. Выберите вариант расположения папки с табличными списками, для которых нужно создать схему табличных списков и файл со значениями по умолчанию.
4. Если выбран вариант **Из выбранных папок**, нажмите кнопку **Добавить папку** и в открывшемся окне выберите папку с табличными списками.

Если выбран вариант **Из папок по умолчанию**, для создания схемы будут использованы табличные списки из папок, указанных в поле **Папки с объектами (по умолчанию)** блока параметров **Исходный код объектов** на вкладке **Пути** вкладки **Настройка**.

5. Установите флажок **Задать файлы схемы и значений по умолчанию в разделе «Настройка»**.

После создания файлов утилита заполнит поля **Файл схемы табличных списков** и **Файл значений по умолчанию для табличных списков** блока параметров **Инструменты и файлы** на вкладке **Пути** вкладки **Настройка**.

6. Нажмите кнопку **Запуск**.

Схема табличных списков и файл со значениями по умолчанию для табличных списков созданы.

14.4.2. Создание набора данных для отладки и отладка правила

- Чтобы создать набор данных для отладки и выполнить отладку правила:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Редактор объектов**.

3. В панели инструментов нажмите кнопку **Создать**.

Откроется окно **Создание объекта**.

4. В поле **Название** введите название набора данных для отладки правила.

Рекомендуется использовать это название также в качестве названия в коде правила для правил агрегации, обогащения и корреляции.

5. В раскрывающемся списке **Тип** выберите тип правила.

6. Нажмите кнопку **Создать**.

Появится вкладка **<Название правила>**.

7. В поле **Формула** или **Правило** введите код правила.

Примечание. Для ускорения ввода кода правила вы можете использовать шаблон, выбрав в меню **Правка** пункт **Вставить шаблон** → **<Название шаблона>**.

8. Введите данные для отладки правила:

- Если нужно выполнить отладку правила нормализации, в поле **Исходное событие** введите событие.
- Если нужно выполнить отладку правила агрегации, обогащения или корреляции, в поле **Тестовый сценарий** введите тестовый сценарий.

9. Нажмите кнопку **Запуск**.

При корректной работе правила в поле **Результат** появятся поля зарегистрированного события, при некорректной — описание ошибки.

10. В панели инструментов вкладки нажмите .

Откроется окно **Сохранить как**.

11. В поле **Папка** укажите папку для сохранения набора данных для отладки правила.

12. В поле **Название** введите название правила.

13. Нажмите кнопку **Сохранить**.

Набор данных для отладки создан.

14.4.3. Составление тестового сценария

Тестовые сценарии используются для отладки правил агрегации, обогащения и корреляции. Тестовый сценарий должен содержать хотя бы одну строку с нормализованным событием, хотя бы одну строку с критерием корректной работы правила и может содержать строку с записями для заполнения табличных списков, используемых в правиле.

Нормализованные события

Для отладки правила нужно указать нормализованные события — такие, чтобы их особенности, количество и порядок приводили хотя бы к одному срабатыванию правила. Каждое событие нужно вводить в отдельной строке в фигурных скобках (в формате JSON). Для каждого события нужно указать значения обязательных полей. Пример:

```
{ "dst.port": 22, "time": "2017-10-18T12:00:00Z", "id": "developer_guide",
  "importance": "info", "action": "start", "object": "task", "status": "success",
  "event_src.vendor": "Best Company", "event_src.title": "test", "event_src.category":
  "Other", "uuid": "00000005-9f0a-0e90-f000-0000abbac077" }
```

Ожидаемый результат, ключевое слово expect

Критерием корректной работы правила является количество событий, которые должны быть зарегистрированы на основе переданных в правило нормализованных событий. Для ввода критерия используется ключевое слово **expect**. Вы можете указать целое положительное число событий, **any** — если количество событий должно быть не менее одного и **not** — если ни одного события не должно быть зарегистрировано. Вы также можете указать шаблон регистрируемых по правилу событий (в формате JSON). В фигурных скобках нужно через запятую перечислить названия и значения полей, которые должны содержаться в событиях. Если вы хотите указать шаблоны для нескольких событий, вы можете добавить несколько строк с ключевым словом **expect**.

Формат ввода критерия:

```
expect <Количество событий 1> {<Шаблон события 1>}
expect <Количество событий 2> {<Шаблон события 2>}
...
```

Например, если по правилу агрегации `Connection_teardown_UDP` должно быть зарегистрировано одно событие, для проверки вы можете указать:

```
expect 1 {"aggregation_name": "Connection_teardown_UDP"}
```

Ожидаемое изменение табличных списков, ключевые слова `expect table_list`

Примечание. Все используемые в правиле табличные списки должны быть описаны в файле `schema.json`.

Если при срабатывании правила выполняется запись в табличный список, то критерием корректной работы правила также является изменение табличного списка. Для ввода ожидаемых результатов изменения табличного списка используются ключевые слова `expect table_list`. В отдельной строке в фигурных скобках в формате JSON для каждого табличного списка нужно указать записи, которые должны появиться в списке. Нужно указать значения для всех колонок табличного списка.

Формат ввода записей табличного списка:

```
"<Название табличного списка>": [{"<Название колонки 1>": "<Значение колонки 11>",
  "<Название колонки 2>": "<Значение колонки 21>, ..."}, {"<Название колонки 1>": "<Значение колонки 12>",
  "<Название колонки 2>": "<Значение колонки 22>, ..."}, ...]
```

Формат ввода критерия:

```
expect table_list {"<Название табличного списка 1>": [<Записи табличного списка 1>],
  {"<Название табличного списка 2>": [<Записи табличного списка 2>], ...}}
```

Например, если после выполнения правила в табличном списке `Incidents` должна появиться запись с данными об узле и имени пользователя, для проверки вы можете указать:

```
expect table_list {"Incidents": [{"host": "win10x64-130.example.com", "user_name": "test-admin"}]}
```

Наполнение табличных списков, ключевое слово `table_list`

Если в правиле используются табличные списки, вы можете указать записи, которыми нужно заполнить эти табличные списки для отладки правила. Для добавления записей используется ключевое слово `table_list`. Если в правиле нужно использовать записи, добавляемые в списки по умолчанию (из файла `correlation_defaults.json`), вы можете указать `table_list default`. Если для некоторых табличных списков нужно использовать другие записи, укажите эти записи для каждого табличного списка в формате, который описан выше.

Формат ввода критерия:

```
table_list default
table_list {"<Название табличного списка 1>": [<Записи табличного списка 1>],
  {"<Название табличного списка 2>": [<Записи табличного списка 2>], ...}}
```

Например, для заполнения табличных списков MITRE_ATTCK_whitelist и AD_Security_Administrators вы можете указать:

```
table_list {"MITRE_ATTCK_whitelist":[], "AD_Domain_Controllers":[{"hostname":
"win10x64-010.example.local", "ip": "192.0.2.10"}], "AD_Security_Administrators":
[{"username": "test-admin", "source_ip": "192.0.2.11"}]}
```

См. также

[Структура файла schema.json \(см. раздел 13.6\)](#)

14.4.4. Отладка правила с помощью тестов

► Чтобы выполнить отладку правила с помощью тестов:

1. Запустите файл PTSIEMSDK_GUI.exe.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Редактор объектов**.

3. В панели инструментов нажмите кнопку **Открыть**.

Откроется окно **Открытие объектов**.

4. В поле **Папка с объектами** введите путь к папке, которая содержит вложенные папки с правилами.

5. В блоке параметров **Обзор объектов** в раскрывающемся списке выберите тип правила.

Появится список папок с правилами.

Примечание. Вы можете использовать быстрый поиск по названию папки правила.

6. Выберите папку и нажмите кнопку **Открыть**.

Появится вкладка **<Название правила>** с кодом правила. Если в выбранной папке содержатся данные для отладки правила, они также появятся на вкладке.

7. В панели тестов нажмите **+**.

Примечание. Для правила нормализации вы можете добавить одновременно несколько тестов с исходными событиями из файла, используя пункт меню **Правка → Импортировать исходные события → Из одного файла**.

В панели появится строка со значком  и номером теста.

8. В поле **Исходное событие** или **Тестовый сценарий** введите необработанное событие или тестовый сценарий.

Примечание. Вы можете удалить тест выделив строку с его номером и нажав **—**.

9. Если нужно, аналогичным образом добавьте другие тесты.

10. Нажмите кнопку **Запуск**.

Последовательно будут запущены все добавленные тесты. Для просмотра результата в панели тестов выберите строку теста. Если тест пройден успешно, в строке теста появится значок , в поле **Результат** отобразятся поля зарегистрированного события; если нет — появится значок , в поле **Результат** отобразится описание ошибки.

11. В панели инструментов вкладки нажмите .

14.4.5. Метаданные для локализации

Вы можете добавлять метаданные для локализации в правила нормализации, обогащения, агрегации и корреляции, а также в макросы и табличные списки. Метаданные используются для хранения правил локализации регистрируемых по правилам событий и описаний правил. На основе правил локализации создается граф локализации, по которому в соответствии с указанным в правиле условием для каждого зарегистрированного в MaxPatrol SIEM события в интерфейсе отображается его описание.

Правила обогащения, в отличие от правил нормализации, агрегации и корреляции, не создают новых нормализованных событий. Для них предусмотрен упрощенный шаблон метаданных, содержащих только описание правила.

Для макросов используется собственный формат метаданных, который содержит детальное описание макроса.

Формат метаданных для табличных списков в базовом варианте для всех типов табличных списков содержит только описание табличного списка, а в расширенном варианте для табличного списка типа «справочник» предусматривает создание шаблонов исключений.

Вы можете вводить метаданные для правила в окне **Редактор метаданных**, используя разметку YAML. В папке каждого правила для хранения метаданных создаются: файл `metainfo.yaml` для хранения правил локализации и папка `i18n` с файлами `i18n_ru.yaml` и `i18n_en.yaml` для хранения описания правила и описаний событий.

Поле Метаданные

Поле **Метаданные** используется для ввода условий правил локализации в следующем виде:

```
EventDescriptions:
  - Criteria: <Условие правила локализации 1>
    LocalizationId: <Ключ правила локализации 1>
  - Criteria: <Условие правила локализации 2>
    LocalizationId: <Ключ правила локализации 2>
...
```


Секция `Criteria` предназначена для ввода условия правила локализации. Условие может состоять из одного или нескольких предикатов вида: <Поле события><Оператор><Значение> или <Поле 1 события><Оператор><Поле 2 события>. В предикате вы можете использовать логические операторы равенства (=) и неравенства (!=). Между предикатами можно использовать операторы `and` или `or`. В условии можно использовать круглые скобки.

Примечание. В условии вы также можете использовать предикаты вида: <Поле события> и `not <Поле события>` (альтернативная запись `!<Поле события>`). Значение первого предиката будет `true`, если значение поля события отличается от `null`, второго — если значение равно `null` или `false`.

Секция `LocalizationId` предназначена для ввода идентификатора (ключа) для сопоставления условия правила локализации с одним описанием событий, указанным в поле **RU**, и одним описанием, указанным в поле **EN**.

Пример:

```
EventDescriptions:
  - Criteria: correlation_name = "Bruteforce_attempt_from_src" and correlation_type
    = "incident"
    LocalizationId: Bruteforce_attempt_from_src_incident
  - Criteria: correlation_name = "Bruteforce_attempt_from_src" and correlation_type
    = "event"
    LocalizationId: Bruteforce_attempt_from_src_incident_event
```

Поля RU и EN

Поля **RU** и **EN** используются для ввода описания правила и описаний регистрируемых по правилу событий в следующем виде:

```
Description: <Описание правила>
EventDescriptions:
  - LocalizationId: <Ключ правила локализации 1>
    EventDescription: <Описание события 1>
  - LocalizationId: <Ключ правила локализации 2>
    EventDescription: <Описание события 2>
  ...
```

Секция `Description` предназначена для ввода описания правила.

Примечание. При использовании в описании двоеточия все описание нужно заключать в кавычки.

Секция `EventDescriptions` предназначена для ввода описаний регистрируемых по правилу событий для разных условий локализации.

Секция `EventDescription` предназначена для ввода описания регистрируемых по правилу событий.

Секция `LocalizationId` предназначена для ввода идентификатора (ключа) для сопоставления описания событий с условием правила локализации, указанным в поле **Метаданные**.

Пример для поля **RU**:

Description: Обнаружение большого количества ошибок при попытках войти в систему с одного узла и попытке подбора пароля

EventDescriptions:

- LocalizationId: Bruteforce_attempt_from_src_incident
EventDescription: Обнаружена попытка подбора пароля с узла {src.host}
- LocalizationId: Bruteforce_attempt_from_src_incident_event
EventDescription: Обнаружено большое количество ошибок при попытках войти в систему с узла {src.host}

Пример для поля **EN**:

Description: Large number of errors during attempt to log in from host and password bruteforce detection

EventDescriptions:

- LocalizationId: Bruteforce_attempt_from_src_incident
EventDescription: An attempt of host {src.host} to bruteforce a password is detected
- LocalizationId: Bruteforce_attempt_from_src_incident_event
EventDescription: Large number of errors is detected during attempts to log in to the system from host {src.host}

Добавление метаданных

► Чтобы добавить метаданные в набор данных для отладки правила:

1. Запустите файл PTSIEMSDK_GUI.exe.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Редактор объектов**.

3. В панели инструментов нажмите кнопку **Открыть**.

Откроется окно **Открытие объектов**.

4. В поле **Папка с объектами** укажите путь к папке с правилом.

Появится вкладка **<Название правила>** с кодом правила. Если в выбранной папке содержатся данные для отладки правила, они также появятся на вкладке.

5. В панели инструментов вкладки нажмите кнопку **Метаданные**.

Откроется окно **Редактор метаданных**.

6. В нижней правой части окна в меню **Параметры** выберите пункт **Вставить шаблон**.

7. В поле **Метаданные** введите условие правила локализации событий.

8. В поле **RU** введите описание правила и описание событий на русском языке.

9. В поле **EN** введите описание правила и описание событий на английском языке.

10. В панели инструментов вкладки нажмите .

Метаданные добавлены в наборы данных для отладки правила.

14.4.6. Проверка регулярного выражения

Вы можете выполнить проверку регулярного выражения для его дальнейшего использования в правилах корреляции и обогащения, а также в макросах. Утилита проверяет на тестовом правиле, корректно ли будет использоваться регулярное выражение.

► Чтобы проверить регулярное выражение:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. На вкладке **Редактор объектов** выберите правило корреляции, правило обогащения или макрос и нажмите кнопку **RegEx**.

3. В блоке параметров **Регулярное выражение** выберите вариант для проверки регулярного выражения в правиле или в табличном списке.

4. В поле **Регулярное выражение** введите регулярное выражение для проверки.

5. В поле **Текстовые значения поля события** введите строку для проверки регулярного выражения.

6. В блоке параметров **Формат текстовых данных** выберите вариант представления данных в поле **Текстовые значения поля события**.

Примечание. Текст из нормализованного события в формате JSON содержит экранированные специальные символы. Это вид событий после работы консольных утилит SDK. Например, значение поля `object.process.path` будет отображаться как `"c:\\windows\\system32\\"` (символ `\` экранирован дополнительным символом `\` в соответствии с правилами экранирования специальных символов в JSON). Обычный текст не содержит экранированных символов. Это вид событий в интерфейсе просмотра событий SIEM. Например, значение поля `object.process.path` будет отображаться как `"c:\windows\system32\"`.

7. Для возвращения определенной группы в регулярном выражении установите номер группы в поле **Возвращаемая группа**.

Чтобы показать предыдущие группы, вы можете установить флажок **Показать предыдущие**.

8. Нажмите кнопку **Проверить (F5)**.

В поле **Результат** отразится результат проверки регулярного выражения. Если строка, подходящая под регулярное выражение, найдена (обозначается префиксом *Match*), она появится в поле. Если перед запуском в поле **Возвращаемая группа** был выбран номер группы, отличный от 0, также будет указана часть текста, которая попала в целевую группу регулярного выражения (обозначается префиксом *Group*).

Регулярное выражение проверено.

Вы можете скопировать проверенное регулярное выражение для использования в правиле с помощью кнопки **Копировать рег. выражение (F6)**.

14.5. Отладка совместной работы правил

После создания и отладки отдельных правил перед добавлением их в MaxPatrol SIEM необходимо выполнить отладку совместной работы правил. Это позволит проверить работу правила с потоком событий и убедиться в отсутствии конфликтов с другими правилами. Вы можете гибко настроить сценарий отладки, выбрав нужные этапы, или проверить совместную работу всех доступных правил на всех этапах обработки событий. Файлы с результатами совместной работы правил сохраняются в указанной вами папке для дальнейшего анализа.

Если вы создали правила нормализации, необходимо выполнить отладку совместной работы ваших правил с правилами MaxPatrol SIEM, чтобы убедиться в отсутствии конфликтов. Для этого нужно настроить сценарий для создания графа нормализации и нормализации событий по вашим правилам и правилам MaxPatrol SIEM. Если результат нормализации совпадает с тем, который вы получили для тех же исходных событий при отладке отдельного правила, — конфликта нет.

Если вы создали правила для других этапов обработки событий, необходимо для этих этапов выполнить отладку совместной работы ваших правил с правилами MaxPatrol SIEM, чтобы проверить работу правил с потоком событий. Например, если вы создали правила обогащения и корреляции, нужно настроить сценарий для создания базы данных табличных списков, графов обогащения и корреляции, обогащения и корреляции событий по вашим правилам и правилам MaxPatrol SIEM. Если результат корреляции совпадает с ожидаемым — правила корректно работают с потоком событий.

В этом разделе

[Сценарий для отладки правил нормализации и локализации \(см. раздел 14.5.1\)](#)

[Сценарий для отладки правил для всех этапов обработки событий \(см. раздел 14.5.2\)](#)

14.5.1. Сценарий для отладки правил нормализации и локализации

- Чтобы настроить сценарий для отладки совместной работы правил нормализации и локализации:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Сценарии**.
3. В панели **Параметры BUILD** установите флажок **BUILD FORMULAS**.
4. В панели **Параметры RUN** установите флажок **NORMALIZE**.
5. Справа от поля **Файл исходных событий** нажмите  и выберите файл с исходными событиями.
6. Если исходные события не были предварительно обработаны в MaxPatrol SIEM и преобразованы в формат JSON, установите флажок **Исходные события без конвертов** и в раскрывающемся списке **MIME** выберите тип событий.
7. Установите флажок **Показать статистику**.
8. Установите флажок **Создать файл ненормализованных событий**.
9. В панели **Параметры BUILD** установите флажок **BUILD LOCALIZATION**.
10. В панели **Параметры RUN** установите флажок **LOCALIZE**.
11. Установите флажок **Автоматически запускать просмотр событий**.
12. В панели инструментов нажмите кнопку **Запуск**.

Откроется окно **Сценарии**, начнутся создание графов нормализации и локализации, нормализация и локализация исходных событий. По окончании нормализации в окне появится статистика по событиям.

13. В окне **Сценарии** нажмите кнопку **Продолжить**.

Откроется окно **Просмотр событий** с нормализованными событиями.

14. Закройте окно.

15. В панели инструментов нажмите **Папка с результатами**.

В открывшейся папке содержатся созданные файлы: `formulas_graph.json` — граф нормализации; `ru_events.json` и `en_events.json` — графы локализации; `not_normalized.json` — события, которые не удалось нормализовать; `norm_events.json` — нормализованные события; `ru_events.json` и `en_events.json` — нормализованные и локализованные события.

14.5.2. Сценарий для отладки правил для всех этапов обработки событий

- Чтобы настроить сценарий для отладки совместной работы правил нормализации, агрегации, обогащения и корреляции событий:

1. Запустите файл `PTSIEMSDK_GUI.exe`.

Откроется окно утилиты PTSIEMSDK GUI.

2. Выберите вкладку **Сценарии**.
3. В панели **Параметры BUILD** установите флажок **BUILD FORMULAS**.
4. Установите флажок **BUILD AGGREGATION RULES**.
5. Установите флажок **BUILD TABLE LISTS SCHEMA**.
6. Установите флажок **BUILD TABLE LISTS DATABASE**.
7. В открывшемся блоке параметров установите флажок **Использовать файлы из раздела «Настройка»**.
8. Установите флажок **BUILD ENRICHMENT RULES**.
9. В открывшемся блоке параметров установите флажок **Использовать файл из раздела «Настройка»**.
10. Установите флажок **BUILD CORRELATION RULES**.
11. В открывшемся блоке параметров установите флажок **Использовать файл из раздела «Настройка»**.
12. В панели **Параметры RUN** установите флажок **NORMALIZE**.
13. В открывшемся блоке параметров справа от поля **Файл исходных событий** нажмите  и выберите файл с исходными событиями.
14. Если исходные события не были предварительно обработаны и преобразованы в формат JSON, установите флажок **Исходные события без конвертов** и в раскрывающемся списке **MIME** выберите тип событий.
15. Установите флажок **Показать статистику**.
16. Установите флажок **AGGREGATE**.
17. Установите флажок **ENRICH**.
18. В открывшемся блоке параметров установите флажок **База данных с табличными списками**.
19. Установите флажок **CORRELATE**.
20. В открывшемся блоке параметров установите флажок **База данных с табличными списками**.
21. Установите флажок **Создать файл с отфильтрованными событиями**.
22. Установите флажок **Показать статистику**.
23. В панели инструментов нажмите кнопку **Запуск**.

Откроется окно **Сценарии**, начнется создание графов нормализации и агрегации, базы данных табличных списков, графов обогащения и корреляции, затем нормализация исходных событий, агрегация, обогащение и корреляция нормализованных событий. По окончании в окне появится статистика по событиям.

24. В окне **Сценарии** нажмите кнопку **Заккрыть**.

25. В панели инструментов нажмите **Папка с результатами**.

В открывшейся папке содержатся созданные файлы: `formulas_graph.json` — граф нормализации; `aggfilters.json` — граф агрегации; `enrules_graph.json` — граф обогащения; `corrules_graph.json` — граф корреляции; `fpta_db.db` — база данных табличных списков; `norm_events.json` — нормализованные события; `aggr_norm_events.json` — агрегированные события; `enrich_events.json` — обогащенные данными события; `corr_events.json` — корреляционные события, `filtered_events.json` — события, к которым применены фильтры из правил корреляции.

15. Утилита kbtools

Утилита `kbtools` предназначена для экспорта и импорта объектов из БД Knowledge Base, создания и удаления БД. Каталог `tools` с утилитой расположен в каталоге установки Knowledge Base.

При запуске утилиты без ключей (или с ключами `help`, `version`) на экран выводятся версия утилиты и список ключей, позволяющих использовать утилиту для выполнения тех или иных действий. При запуске утилиты с ключом для выбора действия и ключом `help` на экран выводятся список дополнительных ключей для выполнения этого действия.

В этом разделе

`exportPackage`, экспорт объектов из БД (см. раздел 15.1)

`importPackage`, импорт объектов в БД (см. раздел 15.2)

`unpack`, распаковка пакета (см. раздел 15.3)

`pack`, формирование пакета для импорта (см. раздел 15.4)

`createRootDB`, создание БД (см. раздел 15.5)

`dropRootDB`, удаление БД (см. раздел 15.6)

15.1. exportPackage, экспорт объектов из БД

Ключ `exportPackage` позволяет использовать утилиту для экспорта всех объектов из указанной БД Knowledge Base или из набора для установки.

Примечание. Для работы с объектами нужно распаковать полученный при экспорте пакет (файл с расширением `.kb`), запустив утилиту с ключом `unpack`.

Формат запуска:

```
kbtools
  exportPackage
  --db <Идентификатор БД>
  --mode DeploySet
  --deploySet all/<Системное название набора>
  --output <Путь к файлу с расширением .kb>
  --kbHost <IP-адрес>
  --login <Логин>
  --password <Пароль>
```

Ключи для настройки экспорта:

- `--db`
Идентификатор БД (обязательный ключ).
- `--mode`

Варианты экспорта объектов: `All` — все объекты, `DeploySet` — объекты из набора для установки (обязательный ключ).

— `--deploySet`

Название набора для установки. Необходимо указывать в формате `all/<Системное название набора>`, так как все наборы входят в корневой набор **Все объекты** с системным названием `all`. Для указания вложенных наборов вы можете использовать косую черту (`/`).

— `--output`

Путь и название файла с расширением `.kb` для сохранения объектов (обязательный ключ).

Ключи для настройки подключения к Knowledge Base:

— `--kbHost`

Адрес узла установки Knowledge Base (обязательный ключ).

— `--kbPort`

Порт подключения к Knowledge Base (по умолчанию 8091).

— `--iamHost`

Адрес узла установки PT MC (по умолчанию совпадает с узлом установки Knowledge Base).

— `--iamPort`

Порт подключения к PT MC (по умолчанию 3334).

— `--login`

Логин для подключения к Knowledge Base (обязательный ключ).

Примечание. Создавать, изменять и удалять пользовательские БД могут только пользователи с правами администратора в Knowledge Base.

— `--password`

Пароль для подключения к Knowledge Base (обязательный ключ).

Пример для экспорта всех объектов из БД `Test_DB`:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools exportPackage --db
Test_DB --mode All --output /var/lib/deployed-roles/kb-application/kb/tools/
Test_DB_content.kb --kbHost kb.company.com --login <Логин> --password <Пароль>
```

Пример для экспорта объектов из набора для установки `Set_1`, вложенного в набор `RootSet`:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools exportPackage --db
Test_DB --mode DeploySet --deploySet all/RootSet/Set_1 --output /var/lib/deployed-
roles/kb-application/kb/tools/Set_1_content.kb --kbHost kb.company.com --login
<Логин> --password <Пароль>
```

15.2. importPackage, импорт объектов в БД

Ключ `importPackage` позволяет использовать утилиту для импорта объектов в указанную БД Knowledge Base.

Примечание. Для импорта объектов нужно сформировать пакет (файл с расширением `.kb`), запустив утилиту с ключом `pack`.

Формат запуска:

```
kbtools
  importPackage
  --db <Идентификатор БД>
  --source <Путь к файлу с расширением .kb>
  --mode <Режим импорта>
  --kbHost <IP-адрес>
  --login <Логин>
  --password <Пароль>
```

Ключи для настройки импорта:

— `--db`

Идентификатор БД (обязательный ключ).

— `--mode`

Режим импорта объектов (обязательный ключ):

- `Upsert` — все объекты импортируются в БД как пользовательские. Новые объекты добавляются в БД, уже существующие в БД пользовательские объекты заменяются объектами из файла (в том числе — записи табличных списков).
- `UpsertOrigin` — импортируются только объекты, разработанные сторонними поставщиками. Все объекты импортируются в БД как стандартные. Новые объекты добавляются в БД, уже существующие в БД объекты этого поставщика заменяются объектами из файла.
- `ReplaceOrigin` — импортируются только объекты, разработанные сторонними поставщиками. Все объекты импортируются в БД как стандартные. Если в БД и в файле есть объекты, разработанные одним и тем же поставщиком, все такие объекты заменяются в БД объектами из файла. Таким образом, новые объекты добавляются в БД, уже существующие в БД объекты заменяются объектами из файла, отсутствующие в файле объекты удаляются из БД.

— `--source`

Путь и название файла с расширением `.kb` с объектами для импорта (обязательный ключ).

— `--ImportPtOrigin`

Импортировать объекты разработанные Positive Technologies: `true` — да, `false` — нет (по умолчанию).

— `--ImportMacros`

Импортировать макросы: `true` — да, `false` — нет (по умолчанию).

— `--ImportTaxonomy`

Импортировать схему полей событий: `true` — да, `false` — нет (по умолчанию).

Ключи для настройки подключения к Knowledge Base:

— `--kbHost`

Адрес узла установки Knowledge Base (обязательный ключ).

— `--kbPort`

Порт подключения к Knowledge Base (по умолчанию 8091).

— `--iamHost`

Адрес узла установки PT MC (по умолчанию совпадает с узлом установки Knowledge Base).

— `--iamPort`

Порт подключения к PT MC (по умолчанию 3334).

— `--login`

Логин для подключения к Knowledge Base (обязательный ключ).

Примечание. Создавать, изменять и удалять пользовательские БД могут только пользователи с правами администратора в Knowledge Base.

— `--password`

Пароль для подключения к Knowledge Base (обязательный ключ).

Пример:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools importPackage --db
TestRootDB --mode Upsert --source /var/lib/deployed-roles/kb-application/kb/tools/
Set_1_content_mod.kb --kbHost kb.company.ru --login <Логин> --password <Пароль>
```

15.3. unpack, распаковка пакета

Ключ `unpack` позволяет использовать утилиту для распаковки пакетов с объектами, экспортированных из Knowledge Base.

Формат запуска:

```
kbtools
  unpack
  --source <Путь к файлу с расширением .kb>
  --output <Путь к папке с объектами>
```

Ключи для настройки распаковки пакета:

— `--source` или `-s`

Путь и название файла с импортированными объектами (обязательный ключ).

— `--output` или `-o`

Путь к папке для сохранения объектов (обязательный ключ).

Пример:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools unpack --source /var/
lib/deployed-roles/kb-application/kb/tools/Set_1_content.kb --output /var/lib/
deployed-roles/kb-application/kb/tools/Set_1
```

15.4. pack, формирование пакета для импорта

Ключ `pack` позволяет использовать утилиту для формирования пакетов с объектами для импорта в Knowledge Base.

Формат запуска:

```
kbtools
  pack
  --source <Путь к папке с объектами>
  --output <Путь к файлу с расширением .kb>
```

Ключи для настройки формирования пакета:

— `--source` или `-s`

Путь к папке с объектами (обязательный ключ).

— `--output` или `-o`

Путь и название для сохранения файла для импорта (обязательный ключ).

Пример:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools pack --source /var/
lib/deployed-roles/kb-application/kb/tools/Set_1 --output /var/lib/deployed-roles/kb-
application/kb/tools/Set_1_content_mod.kb
```

15.5. createRootDB, создание БД

Ключ `createRootDB` позволяет использовать утилиту для создания в Knowledge Base пустой БД.

Формат запуска:

```
kbtools
  createRootdb
  --db <Идентификатор БД>
  --kbHost <IP-адрес>
  --login <Логин>
  --password <Пароль>
```

Ключи для настройки параметров новой БД:

- `--db`
Идентификатор БД (обязательный ключ).
- `--db_name`
Название БД для отображения в интерфейсе.

Ключи для настройки подключения к Knowledge Base:

- `--kbHost`
Адрес узла установки Knowledge Base (обязательный ключ).
 - `--kbPort`
Порт подключения к Knowledge Base (по умолчанию 8091).
 - `--iamHost`
Адрес узла установки РТ МС (по умолчанию совпадает с узлом установки Knowledge Base).
 - `--iamPort`
Порт подключения к РТ МС (по умолчанию 3334).
 - `--login`
Логин для подключения к Knowledge Base (обязательный ключ).
- Примечание.** Создавать, изменять и удалять пользовательские БД могут только пользователи с правами администратора в Knowledge Base.
- `--password`
Пароль для подключения к Knowledge Base (обязательный ключ).

Пример:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools createRootdb --db
TestRootDB --db_name "Тестовая база" --kbHost kb.company.ru --login <Логин> --
password <Пароль>
```

15.6. dropRootDB, удаление БД

Ключ `dropRootDB` позволяет использовать утилиту для удаления БД Knowledge Base.

Формат запуска:

```
kbtools
  dropRootdb
  --db <Идентификатор БД>
  --kbHost <IP-адрес>
  --login <Логин>
  --password <Пароль>
```

Ключ для удаления БД:

- `--db`

Идентификатор БД (обязательный ключ).

Ключи для настройки подключения к Knowledge Base:

- `--kbHost`

Адрес узла установки Knowledge Base (обязательный ключ).

- `--kbPort`

Порт подключения к Knowledge Base (по умолчанию 8091).

- `--iamHost`

Адрес узла установки РТ МС (по умолчанию совпадает с узлом установки Knowledge Base).

- `--iamPort`

Порт подключения к РТ МС (по умолчанию 3334).

- `--login`

Логин для подключения к Knowledge Base (обязательный ключ).

Примечание. Создавать, изменять и удалять пользовательские БД могут только пользователи с правами администратора в Knowledge Base.

- `--password`

Пароль для подключения к Knowledge Base (обязательный ключ).

Пример:

```
/var/lib/deployed-roles/kb-application/kb/tools/kbtools/kbtools dropRootdb --db  
TestRootDB --kbHost kb.company.ru --login <Логин> --password <Пароль>
```

16. Виды запросов к API

Раздел содержит информацию о протоколе обмена данными между сторонними приложениями и MaxPatrol SIEM. Обмен данными производится через программный интерфейс продукта (REST API) по протоколу HTTPS.

Для выполнения некоторых запросов необходимо использовать токен доступа. Персональный токен доступа формата `reference` вы можете [создать](#) (см. раздел 16.1) в веб-интерфейсе PT MC.

Внимание! Если в ответ на запрос к API сервис возвращает ошибку 500 (Internal Server Error), необходимо [назначить](#) пользователю дополнительную роль приложения Management and Configuration и создать новый токен доступа. Вы можете назначить роль **Пользователь** или создать новую роль с минимальными привилегиями.

В этом разделе

[Создание токена доступа](#) (см. раздел 16.1)

[Работа с группами активов](#) (см. раздел 16.2)

[Получение токена PDQL-запросов](#) (см. раздел 16.3)

[Импорт активов из CSV](#) (см. раздел 16.4)

[Работа с табличными списками в MaxPatrol SIEM](#) (см. раздел 16.5)

[Работа с табличными списками в Knowledge Base](#) (см. раздел 16.6)

[Работа с инцидентами](#) (см. раздел 16.7)

[Работа с событиями](#) (см. раздел 16.8)

[Управление табличными списками для получения информации об изменении активов](#) (см. раздел 16.9)

[Работа с учетными записями сканирования](#) (см. раздел 16.10)

[Работа со сканами](#) (см. раздел 16.11)

[Работа с профилями сканирования](#) (см. раздел 16.12)

[Управление задачами сканирования](#) (см. раздел 16.13)

[Мониторинг задач сканирования](#) (см. раздел 16.14)

[Журналирование действий пользователя](#) (см. раздел 16.15)

[Работа с активами](#) (см. раздел 16.16)

[Получение метаданных актива](#) (см. раздел 16.17)

[Экспорт таблицы активов в CSV](#) (см. раздел 16.18)

[Массовые операции над задачами](#) (см. раздел 16.19)

[Работа с выбором задач](#) (см. раздел 16.20)

16.1. Создание токена доступа

Для выполнения запросов к API, требующих аутентификации пользователя, необходимо создать персональный токен доступа формата reference. Токен доступа создается в интерфейсе сервиса управления пользователями и доступом Positive Technologies Management and Configuration.

Вы можете создать не более 100 токенов. Список всех созданных токенов пользователя доступен на странице **Токены доступа**.

► Чтобы создать токен доступа:

1. В главном меню MaxPatrol SIEM перейдите в приложение Positive Technologies Management and Configuration.
2. В главном меню PT MC выберите  → **Токены доступа**.
3. Нажмите **Создать**.
4. Введите название токена.
5. Выберите приложение.
6. Нажмите **Создать**.
7. Сохраните токен, чтобы он был доступен только вам.

Внимание! Скопируйте и сохраните токен сразу: после перехода на другую страницу скопировать созданный токен будет невозможно. Потребуется создавать новый.

Вы также можете изменить токен доступа по кнопке  и удалить токен доступа по кнопке .

Внимание! Если в ответ на запрос к API сервис возвращает ошибку 500 (Internal Server Error), необходимо [назначить](#) пользователю дополнительную роль приложения Management and Configuration и создать новый токен доступа. Вы можете назначить роль **Пользователь** или создать новую роль с минимальными привилегиями.

16.2. Работа с группами активов

С помощью запросов к API вы можете получить идентификаторы созданных в MaxPatrol SIEM пользовательских инфраструктур и групп активов. Также вы можете создавать и удалять группы активов.

В этом разделе

[Получение идентификатора инфраструктуры \(см. раздел 16.2.1\)](#)

[Получение идентификатора группы активов \(см. раздел 16.2.2\)](#)

[Создание группы активов \(см. раздел 16.2.3\)](#)

[Проверка статуса создания группы \(см. раздел 16.2.4\)](#)

[Удаление группы активов \(см. раздел 16.2.5\)](#)

16.2.1. Получение идентификатора инфраструктуры

Запрос для получения идентификаторов созданных инфраструктур.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scopes/v2/scopes
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 101. Параметры в теле запроса /api/scopes/v2/scopes

Параметр	Обязательный	Тип данных	Описание
limit	Нет	Number	Максимальное количество элементов инфраструктуры в ответе
offset	Нет	Number	Количество отбрасываемых элементов

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 102. Поля ответа на запрос /api/scopes/v2/scopes

Поле	Тип данных	Описание
id	UUID	Идентификатор инфраструктуры
name	String	Название инфраструктуры
tenantId	UUID	Идентификатор дочерней организации

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/scopes/v2/scopes
```

Ответ:

```
[
  {
    "id": "00000000-0000-0000-0000-000000000005",
    "name": "Инфраструктура по умолчанию",
    "tenantId": "32b17948-5755-4905-b0f3-17db2402256a"
  }
]
```

16.2.2. Получение идентификатора группы активов

Запрос для получения идентификаторов групп активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_temporal_readmodel/v2/groups/hierarchy

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 103. Параметры в теле запроса /api/assets_temporal_readmodel/v2/groups/hierarchy

Параметр	Обязательный	Тип данных	Описание
type	Нет	Enum	Тип группы: — <code>all</code> — любой; — <code>static</code> — статическая; — <code>dynamic</code> — динамическая
groupId	Нет	UUID	Идентификатор группы

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 104. Поля ответа на запрос /api/assets_temporal_readmodel/v2/groups/hierarchy

Поле	Тип данных	Описание
id	UUID	Идентификатор группы
name	String	Имя группы для отображения в интерфейсе

Поле	Тип данных	Описание
groupType	Enum	Тип группы: — <code>all</code> — любой; — <code>static</code> — статическая; — <code>dynamic</code> — динамическая
isReadOnly	Bool	Возможно ли изменение группы
isRemovable	Bool	Возможно ли удаление группы
isRoot	Bool	Является ли группа корневой
isInvalidPredicate	Bool	—
isSlow	Bool	—
treePath	String	Полный путь к группе в списке
children	Array	Список вложенных групп

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_temporal_readmodel/v2/groups/hierarchy
```

Ответ:

```
[
  {
    "id": "00000000-0000-0000-0000-000000000002",
    "name": "Root",
    "groupType": "static",
    "isReadOnly": true,
    "isRemovable": false,
    "isRoot": true,
    "isInvalidPredicate": false,
    "isSlow": false,
    "treePath": "Root",
    "children": [
      {
        "id": "1685ce7c-e440-0001-0000-000000000005",
        "name": "Import",
        "groupType": "static",
        "isReadOnly": false,
        "isRemovable": true,
        "isRoot": false,
        "isInvalidPredicate": false,
        "isSlow": false,
        "treePath": "Import",
        "children": []
      },
      {
        "id": "1685cf07-70c0-0001-0000-000000000009",
        "name": "Windows",
        "groupType": "dynamic",
        "isReadOnly": false,
        "isRemovable": true,
        "isRoot": false,
        "isInvalidPredicate": false,
        "isSlow": false,
        "treePath": "Windows",
        "children": []
      },
      {
        "id": "00000000-0000-0000-0000-000000000003",
        "name": "Unmanaged hosts",
        "groupType": "dynamic",
        "isReadOnly": true,
        "isRemovable": false,
        "isRoot": false,
        "isInvalidPredicate": false,
        "isSlow": false,
```

```

        "treePath": "Unmanaged hosts",
        "children": []
    }
]
}
]

```

16.2.3. Создание группы активов

Запрос для создания группы активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/assets_processing/v2/groups

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 105. Параметры в теле запроса api/assets_processing/v2/groups

Параметр	Обязательный	Тип данных	Описание
description	Нет	String	Описание группы
groupType	Да	Enum	Тип группы: — <code>undefined</code> — не определен; — <code>static</code> — статическая; — <code>dynamic</code> — динамическая
metadata	Да	Array of objects	Метаданные группы активов
metadata → sourceId	Да	String	Идентификатор источника метаданных. Допустимые значения: — <code>TextInput</code>
metadata → typeId	Да	String	Тип данных. Допустимые значения: — <code>Cvss</code>
metadata → payload	Нет	String	Контекстные метрики CVSS v2: — <code>ar</code> — требования к доступности: • <code>unknown</code> — неизвестны; • <code>ND</code> — не определены; • <code>L</code> — низкие;

Параметр	Обязательный	Тип данных	Описание
			• M — средние; • H — высокие; — cdp — вероятность нанесения косвенного ущерба: • unknown — неизвестна; • ND — не определена; • N — отсутствует; • L — низкая; • LM — средняя пониженная; • MH — средняя повышенная; • H — высокая; — cr — требования к конфиденциальности: • unknown — неизвестны; • ND — не определены; • L — низкие; • M — средние; • H — высокие; — ir — требования к целостности: • unknown — неизвестны; • ND — не определены; • L — низкие; • M — средние; • H — высокие; — td — плотность целей: • unknown — неизвестна; • ND — не определена; • N — отсутствует; • L — низкая;

Параметр	Обязательный	Тип данных	Описание
			- М — средняя; - Н — высокая
name	Да	String	Название группы
organizationInformation	Нет	Array of objects	Информация об организации
organizationInformation → contactUserId	Нет	String	Идентификатор пользователя, являющегося контактным лицом
organizationInformation → address	Нет	String	Адрес
organizationInfrastructure	Нет	Array of objects	Информация об инфраструктуре
organizationInfrastructure → internetProviders	Нет	String	Провайдеры интернета
organizationInfrastructure → numberOfNodes	Нет	String	Количество узлов на периметре
organizationInfrastructure → registeredDomains	Нет	String	Зарегистрированные домены
organizationInfrastructure → usedNetworkApplications	Нет	String	Используемые веб-приложения
organizationInfrastructure → usedNetworks	Нет	String	Используемые сети
parentId	Да	UUID	Идентификатор родительской группы

Параметр	Обязательный	Тип данных	Описание
predicate	Да для динамических групп	String	PDQL-фильтр активов для динамической группы

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 106. Поля ответа на запрос `api/assets_processing/v2/groups`

Поле	Тип данных	Описание
operationId	UUID	Идентификатор операции создания группы

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 500 (Internal Server Error) — синтаксическая ошибка в запросе.

Пример

Запрос:

```
POST https://localhost/api/assets_processing/v2/groups
{
  "name": "Group name",
  "parentId": "00000000-0000-0000-0000-000000000002",
  "description": null,
  "groupType": "dynamic",
  "predicate": "WindowsHost",
  "metadata": [
    {
      "sourceId": "UserInput",
      "typeId": "Cvss",
      "payload": "{\\"td\\": \\"L\\", \\"cr\\": \\"L\\", \\"cdp\\": \\"L\\", \\"ar\\": \\"L\\", \\"ir\\": \\"L\\"}"
    }
  ],
  "organizationInformation": {
    "contactUserId": null,
    "address": null
  }
}
```



```

    },
    "organizationInfrastructure": {
      "usedNetworks": null,
      "numberOfNodes": null,
      "usedNetworkApplications": null,
      "internetProviders": null,
      "registeredDomains": null
    }
  }
}

```

Ответ:

```

{
  "operationId": "14a5ca6b-f780-a001-0000-00000000003f"
}

```

16.2.4. Проверка статуса создания группы

Запрос для получения статуса операции по созданию группы активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_processing/v2/groups/operations/<Идентификатор операции>

URL запроса должен содержать идентификатор операции по созданию группы активов.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 107. Поля ответа на запрос api/assets_processing/v2/groups/operations/<Идентификатор операции>

Поле	Тип данных	Описание
<Идентификатор группы>	UUID	Идентификатор группы

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция по созданию группы выполняется;
- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_processing/v2/groups/operations/168600a8-8f00-a001-0000-000000000008
```

Ответ:

```
"168600a9-8dc0-0001-0000-00000000000d"
```

16.2.5. Удаление группы активов

Запрос для удаления группы активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/assets_processing/v2/groups/removeOperation
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 108. Параметры в теле запроса /api/assets_processing/v2/groups/removeOperation

Параметр	Обязательный	Тип данных	Описание
groupIds	Да	Array [UUID]	Идентификаторы групп

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 109. Поля ответа на запрос /api/assets_processing/v2/groups/removeOperation

Поле	Тип данных	Описание
operationId	UUID	Идентификатор операции удаления группы

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/assets_processing/v2/groups/removeOperation
{
  "groupIds": [
    "14a57a85-8b00-0001-0000-000000000015"
  ]
}
```

Ответ:

```
{
  "operationId": "14cdba4b-c680-a001-0000-00000000384e"
}
```

16.3. Получение токена PDQL-запросов

Запрос для получения токена PDQL-запросов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Время действия токена 30 минут.

Метод и URL запроса:

```
POST <Корневой URL API>/api/assets_temporal_readmodel/v1/assets_grid
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 110. Параметры в теле запроса /api/assets_temporal_readmodel/v1/assets_grid

Параметр	Обязательный	Тип данных	Описание
previousToken	Нет	String	Предыдущий PDQL-токен сеанса пользователя
pdql	Да	String	PDQL-запрос
utcOffset	Нет	String	Сдвиг временной зоны действия от UTC
additionalFilterParameters	Нет	Object	Дополнительные параметры фильтра
selectedGroupIds	Нет	Array	Идентификаторы выбранных групп активов
includeNestedGroups	Да	Bool	Включать ли вложенные группы
executionTime	Нет	String (date-time)	Время выполнения запроса

Параметр	Обязательный	Тип данных	Описание
Дополнительные параметры фильтра			
groupIds	Нет	Array [String]	Список групп, в которых будет происходить выборка
assetIds	Нет	Array	Список активов, по которым будет выполняться запрос

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 111. Поля ответа на запрос `/api/assets_temporal_readmodel/v1/assets_grid`

Поле	Тип данных	Описание
token	String	PDQL-токен для запроса данных
isPotentiallySlow	Bool	Является ли запрос потенциально медленным
hasTimepointPipe	Bool	Есть ли в запросе <code>timepoint</code>
hasTimeseriesPipe	Bool	Есть ли в запросе <code>timeseries</code>
fields	Array [Object]	Описание полей
Описание полей		
name	String	Имя поля
localizedName	String	Локализованное имя поля
type	String (Enum)	Тип поля
isArray	Bool	Является ли коллекцией
origin	String (Enum)	Примененная агрегация

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 500 (Internal Server Error) — внутренняя ошибка сервера;
- 503 (Service Unavailable) — ошибка доступа.

16.4. Импорт активов из CSV

С помощью запросов к API вы можете импортировать в MaxPatrol SIEM список активов из CSV-файла. Для этого необходимо:

1. Если в модель активов были добавлены пользовательские поля — получить данные этих полей.
2. Создать CSV-файл в соответствии с требуемым форматом.
3. Загрузить CSV-файл со списком активов.
4. Запустить процедуру импорта активов.

В этом разделе

[Получение пользовательских полей \(см. раздел 16.4.1\)](#)

[Запрос формата CSV-файла \(см. раздел 16.4.2\)](#)

[Загрузка CSV-файла \(см. раздел 16.4.3\)](#)

[Выгрузка файла ошибок \(см. раздел 16.4.4\)](#)

[Запуск импорта активов \(см. раздел 16.4.5\)](#)

[Проверка статуса импорта \(см. раздел 16.4.6\)](#)

16.4.1. Получение пользовательских полей

Запрос для получения данных пользовательских полей в модели активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/assets_processing/v2/csv/metadata
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 112. Поля ответа на запрос /api/assets_processing/v2/csv/metadata

Поле	Тип	Описание
modelName	String	Название корневой группы параметров в пользовательской модели активов

Поле	Тип	Описание
propertyDeclarations	Array	Свойства пользовательского поля
propertyName	String	Название пользовательского поля
propertyType	String	Тип данных пользовательского поля

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_processing/v2/csv/metadata
```

Ответ:

```
[
  {
    "propertyName": "uf_assetnumber",
    "propertyDeclarations": [
      {
        "modelName": "Core.Host",
        "propertyType": "int"
      }
    ]
  },
  {
    "propertyName": "uf_assetowner",
    "propertyDeclarations": [
      {
        "modelName": "Core.Host",
        "propertyType": "string"
      }
    ]
  },
  {
    "propertyName": "uf_assetrevisiondate",
    "propertyDeclarations": [
      {
        "modelName": "Core.Host",
        "propertyType": "datetime"
      }
    ]
  }
]
```

16.4.2. Запрос формата CSV-файла

Запрос для получения формата CSV-файла.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/assets_processing/v2/csv/example
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Тело ответа содержит пример содержимого CSV-файла с данными актива.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_processing/v2/csv/example
```

Ответ:

```
"typealias";"Fqdn";"Hostname";"Ip";"Mac";"IsVirtual"
"";"dns.somedomain.ru";"w2k3sp2x86s5";"192.168.0.4|182.168.10.1";"00:50:56:A6:0C:36|
00:60:56:A6:0C:36";"false"
```

16.4.3. Загрузка CSV-файла

Запрос для загрузки CSV-файла с данными активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/assets_processing/v2/csv/import_operation
```

Параметры строки запроса описаны в таблице ниже.

Таблица 113. Параметры строки запроса /api/assets_processing/v2/csv/import_operation

Параметр	Обязательный	Тип данных	Описание
scopeId	Да	UUID	Идентификатор инфраструктуры

В запрос необходимо добавить заголовок `Content-Disposition` с описанием CSV-файла, а в тело запроса — CSV-файл с данными активов.

Формат CSV-файла

Файл должен быть представлен в кодировке UTF-8 с BOM. Первая строка файла должна содержать названия полей актива. Вторая и последующие строки — значения полей импортируемых активов (одна строка соответствует одному активу). Значения полей должны быть разделены точкой с запятой. Значения текстовых полей должны быть заключены в кавычки.

Таблица 114. Колонки CSV-файла

Колонка	Тип данных	Описание
Fqdn	String	Полное доменное имя
Hostname	String	Имя узла
Ip	Array [String]	IP-адрес. Поле может содержать несколько значений, которые должны быть разделены вертикальной чертой
IsVirtual	Bool	Является ли виртуальным
Mac	String [String]	MAC-адрес. Поле может содержать несколько значений, которые должны быть разделены вертикальной чертой
typealias	Enum	Тип актива: — Alcatel OmniSwitch (под управлением AOS) — <code>omniswitch</code>; — BSD, FreeBSD и macOS — <code>bsd</code>; — Check Point GAiA OS (межсетевой экран) — <code>checkpoint</code>; — Check Point SPLAT (межсетевой экран) — <code>checkpoint</code>; — Cisco ACS — <code>acs</code>; — Cisco ASA (межсетевой экран) — <code>asa</code>; — Cisco FWSM (межсетевой экран) — <code>fws</code>; — Cisco IOS — <code>ios</code>; — Cisco IOS XE — <code>ios</code>; — Cisco ISE — <code>ise</code>; — Cisco Nexus — <code>nexus</code>; — Cisco PIX (межсетевой экран) — <code>pix</code>; — FortiNet FortiGate (межсетевой экран) — <code>fortigate</code>; — HPE HP-UX — <code>hp_ux</code>; — Huawei VRP — <code>vrp</code>; — IBM AIX — <code>aix</code>; — Juniper JunOS — <code>junos</code>; — Linux (все семейство ОС) — <code>linux</code>; — Oracle Solaris — <code>solaris</code>;

Колонка	Тип данных	Описание
		— Palo Alto Networks PAN-OS (межсетевые экраны) — <code>pan_os</code>; — VMware vSphere Hypervisor (ESXi) — <code>esxi</code>; — Windows — <code>windows</code>
<Пользовательские поля>	—	Добавленные пользователем поля актива, не являющиеся стандартными

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 115. Поля ответа на запрос `/api/assets_processing/v2/csv/import_operation`

Поле	Тип данных	Описание
<code>id</code>	UUID	Идентификатор операции импорта
<code>isLogFileCreated</code>	Bool	Создан ли файл с ошибками
<code>rowCountExceeded</code>	Bool	Превышает ли количество строк в импортируемом файле установленное ограничение
<code>totalRowCount</code>	Number	Общее количество строк в импортируемом файле
<code>validRowCount</code>	Number	Количество строк с валидными данными активов в импортируемом файле

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/assets_processing/v2/csv/import_operation?
scopeId=00000000-0000-0000-0000-000000000005
```

Заголовок запроса:

```
Content-Disposition:form-data; name="upfile"; filename="Assets_list.csv"
```

В тело запроса (в представлении `form-data`) необходимо добавить CSV-файл `Assets_list.csv` с ключом `upfile`.

Ответ:

```
{
  "id": "0e372968-e18b-471e-b3b7-ad0d927c2dd9",
  "isLogFileCreated": true,
  "rowCountExceeded": false,
  "validRowCount": 2,
  "totalRowCount": 3
}
```

16.4.4. Выгрузка файла ошибок

Запрос для выгрузки списка ошибок, обнаруженных при импорте CSV-файла со списком активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/assets_processing/v2/csv/import_operation/<Идентификатор операции>/logfile
```

URL запроса должен содержать идентификатор операции импорта CSV-файла.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Заголовок ответа *Content-Disposition* содержит информацию о выгруженном файле с ошибками. Тело ответа содержит список ошибок с указанием номера строки в импортированном CSV-файле. Если файл с ошибками не создан, сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_processing/v2/csv/import_operation/0e372968-e18b-471e-b3b7-ad0d927c2dd9/logfile
```

Ответ:

```
"3";"Значение [version] некорректно для псевдонима"
```

16.4.5. Запуск импорта активов

Запрос для запуска импорта активов из загруженного CSV-файла.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/assets_processing/v2/csv/import_operation/<Идентификатор операции>/start
```

URL запроса должен содержать идентификатор операции импорта CSV-файла.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 116. Параметры в теле запроса `api/assets_processing/v2/csv/import_operation/<Идентификатор операции>/start`

Параметр	Обязательный	Тип данных	Описание
groupsId	Да	Array [UUID]	Идентификаторы групп, в которые нужно добавить активы

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/assets_processing/v2/csv/import_operation/0e372968-e18b-471e-b3b7-ad0d927c2dd9/start
{
  "groupsId": [
    "1499141b-4c00-0001-0000-00000000000d"
  ]
}
```

16.4.6. Проверка статуса импорта

Запрос для получения статуса импорта активов из CSV-файла.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_processing/v2/csv/import_operation/<Идентификатор операции>/state

URL запроса должен содержать идентификатор операции импорта CSV-файла.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 117. Поля ответа на запрос api/assets_processing/v2/csv/import_operation/<Идентификатор файла>/state

Поле	Тип данных	Описание
errorModel	Array	Данные ошибок
errorModel → code	Number (int32)	Код ошибки
errorModel → errors	Array	Название ошибки
errorModel → message	String	Текст ошибки
state	Enum	Статус импорта: — <code>completed</code> — завершен; — <code>failed</code> — не выполнен; — <code>inprogress</code> — выполняется; — <code>validated</code> — на проверке
succeedCount	Number	Количество импортированных активов
updatedGroups	Array	Данные обновленной группы
updatedGroups → displayName	String	Название группы
updatedGroups → id	UUID	Идентификатор группы

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/assets_processing/v2/csv/import_operation/0e372968-e18b-471e-b3b7-ad0d927c2dd9/state
```

Ответ:

```
{
  "state": "completed",
  "succeedCount": 1,
  "updatedGroups": [
    {
      "id": "149f37fa-c2c0-0001-0000-000000000011",
      "displayName": "Import"
    }
  ],
  "errorModel": null
}
```

16.5. Работа с табличными списками в MaxPatrol SIEM

С помощью запросов к API вы можете экспортировать и импортировать записи табличных списков, установленных в MaxPatrol SIEM, обновлять, очищать, искать табличные списки, а также получать информацию о табличном списке.

В этом разделе

[Поиск табличного списка \(см. раздел 16.5.1\)](#)

[Получение информации о табличном списке \(см. раздел 16.5.2\)](#)

[Экспорт табличного списка \(см. раздел 16.5.3\)](#)

[Импорт табличного списка \(см. раздел 16.5.4\)](#)

[Очистка табличных списков \(см. раздел 16.5.5\)](#)

[Обновление табличного списка \(см. раздел 16.5.6\)](#)

16.5.1. Поиск табличного списка

Запрос для поиска табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/events/v2/table_lists

Параметры строки запроса описаны в таблице ниже.

Таблица 118. Параметры строки запроса /api/events/v2/table_lists

Параметр	Обязательный	Тип данных	Описание
kind	Нет	Enum	Назначение табличного списка: — <code>registry</code> — справочник; — <code>correlationRule</code> — для правил корреляции; — <code>enrichmentRule</code> — для правил обогащения; — <code>assetGrid</code> — для данных об активах; — <code>cybsiGrid</code> — репутационный табличный список
siemId	Нет	String	Идентификатор конвейера обработки событий

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 119. Поля ответа на запрос /api/events/v2/table_lists

Поле	Тип данных	Описание
name	String	Название
editable	Bool	Возможно ли изменение пользователем
token	String	Токен доступа к списку
ttlEnabled	Bool	Ограничено ли время жизни записей
fillType	Enum	Назначение
notifications	Array [String]	Список ошибок

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 503 (SIEM недоступен) — ошибка доступа к конвейеру обработки событий.

Пример

Запрос:

```
GET https://localhost/api/events/v2/table_lists?kind=assetGrid
```

Ответ:

```
[
  {
    "name": "AssetGrid_AD_Domain_Controllers",
    "editable": false,
    "token": "8ea181098d4f6225750c98b17d110d5e68664c753573d6c2c24c9c44fc606c06",
    "ttlEnabled": false,
    "fillType": "assetGrid",
    "notifications": null
  },
  {
    "name": "Critical_hosts",
    "editable": false,
    "token": "65f39698a6c5a75bf14d061089926f897f4de2740e8855930b13736798a15460",
    "ttlEnabled": false,
    "fillType": "assetGrid",
    "notifications": null
  }
]
```

16.5.2. Получение информации о табличном списке

Запрос для получения информации о табличном списке.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/events/v2/table_lists/<Токен доступа>
```

URL запроса должен содержать токен доступа к табличному списку.

Параметры строки запроса описаны в таблице ниже.

Таблица 120. Параметры строки запроса /api/events/v2/table_lists/<Токен доступа>

Параметр	Обязательный	Тип данных	Описание
siemId	Нет	String	Идентификатор конвейера обработки событий

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 121. Поля ответа на запрос /api/events/v2/table_lists/<Токен доступа>

Поле	Тип данных	Описание
Поля с информацией о табличном списке		
type	Enum	Назначение
name	String	Название
editable	Bool	Возможно ли изменение пользователем
description	String	Описание
notifications	Array	Список ошибок
created	DateTime	Дата и время создания
lastUpdated	DateTime	Дата и время последнего изменения
currentSize	Number	Количество записей
fields	Array	Описание полей
Параметры описания полей (fields)		
name	String	Название
type	Enum	Тип данных: — <code>datetime</code> — дата и время; — <code>number</code> — число; — <code>string</code> — строка; — <code>regex</code> — регулярное выражение (только для табличного списка типа «справочник»)
index	Bool	Является ли индексируемой
primaryKey	Bool	Является ли ключевой

Поле	Тип данных	Описание
nullable	Bool	Может ли содержать null

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 503 (SIEM недоступен) — ошибка доступа к конвейеру обработки событий.

Пример

Запрос:

```
GET https://localhost/api/events/v2/table_lists/  
0f3b138df02dd518f7a11f0bd7b85dde6f408c6e41b959838ed1ee7d2c9bbdc2
```

Ответ:

```
{
  "type": "Registry",
  "name": "Allowed_firefox_settings_accessers",
  "editable": false,
  "description": "Исполняемые файлы, которым разрешен доступ к параметрам браузера Mozilla Firefox",
  "created": "2021-03-15T17:17:25.000000Z",
  "lastUpdated": "2021-04-30T08:55:35.000000Z",
  "currentSize": 2,
  "fields": [
    {
      "name": "_id",
      "type": "number",
      "index": true,
      "primaryKey": false,
      "nullable": true
    },
    {
      "name": "_last_changed",
      "type": "datetime",
      "index": true,
      "primaryKey": false,
      "nullable": true
    },
    {
      "name": "filepath",
      "type": "string",
      "index": true,
      "primaryKey": true,
      "nullable": false
    }
  ]
}
```

16.5.3. Экспорт табличного списка

Запрос для экспорта записей из табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/events/v1/table_lists/<Название>/export

URL запроса должен содержать название табличного списка.

Параметры строки запроса описаны в таблице ниже.

Таблица 122. Параметры строки запроса `/api/events/v1/table_lists/<Название>/export`

Параметр	Обязательный	Тип данных	Описание
<code>siemId</code>	Нет	String	Идентификатор конвейера обработки событий

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 123. Параметры в теле запроса `api/events/v1/table_lists/<Название>/export`

Параметр	Обязательный	Тип данных	Описание
<code>filter</code>	Да	String	PDQL-запрос
<code>filter → select</code>	Да	Array [String]	Предикат PDQL-запроса
<code>filter → where</code>	Нет	String	Предикат PDQL-запроса
<code>filter → timeZone</code>	Нет	Number	Часовой пояс в минутах
<code>limit</code>	Нет	Number	Максимальное количество записей в ответе (по умолчанию 50)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Тело ответа содержит CSV-файл с записями табличного списка.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/events/v1/table_lists/Allowed_firefox_settings_accessers/
export
{
```

```

"filter": {
  "select": [
    "_id",
    "_last_changed",
    "filepath"
  ],
  "where": "",
  "timeZone": 180
}
}

```

Ответ:

```

"_id";"_last_changed";"filepath"
"1";"30.04.2021 08:55:35";"c:\program files (x86)\mozilla firefox\firefox.exe"
"0";"30.04.2021 08:55:35";"c:\program files\mozilla firefox\firefox.exe"

```

16.5.4. Импорт табличного списка

Запрос для импорта записей в табличный список.

Внимание! Вы можете импортировать записи только в табличные списки для правил обогащения или корреляции.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/events/v1/table_lists/<Название>/import
```

URL запроса должен содержать название табличного списка.

Параметры строки запроса описаны в таблице ниже.

Таблица 124. Параметры строки запроса /api/events/v1/table_lists/<Название>/import

Параметр	Обязательный	Тип данных	Описание
update	Нет	Bool	Признак выполнения функции upsert
siemId	Нет	String	Идентификатор конвейера обработки событий

В заголовке запроса Content-Type необходимо указать тип данных text/csv; charset=utf-8.

В теле запроса (в представлении row text) необходимо указать записи для импорта. Первая строка должна содержать названия колонок, вторая и последующие строки — значения колонок. Одна строка должна соответствовать одной записи. Названия и значения колонок должны быть заключены в кавычки и разделены точкой с запятой.

Примечание. При отправке запроса в командной строке Windows или терминале Linux необходимо использовать символы `\r\n` в качестве разделителя строк.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 125. Поля ответа на запрос `api/events/v1/table_lists/<Название>/import`

Поле	Тип данных	Описание
<code>recordsNum</code>	Number	Общее количество записей
<code>importedNum</code>	Number	Количество импортированных записей
<code>badRecordsNum</code>	Number	Количество неимпортированных записей
<code>skippedRecordsNum</code>	Number	Количество записей, отброшенных из-за превышения установленного для табличного списка максимального количества записей

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/events/v1/table_lists/VPN_Sessions_PaloAlto/import
Content-Type: text/csv; charset=utf-8
"user";"active_sessions"
"user1";"8"
"user2";"14"
"user3";"2"
```

Ответ:

```
{
  "recordsNum": 3,
  "importedNum": 3,
  "badRecordsNum": 0,
  "skippedRecordsNum": 0
}
```

16.5.5. Очистка табличных списков

Запрос для очистки табличных списков.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

DELETE <Корневой URL API>/api/events/v2/table_lists/<Токен доступа>/content

URL запроса должен содержать токен доступа к табличному списку.

Параметры строки запроса описаны в таблице ниже.

Таблица 126. Параметры строки запроса /api/events/v2/table_lists/<Токен доступа>/content

Параметр	Обязательный	Тип данных	Описание
siemId	Нет	String	Идентификатор конвейера обработки событий

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 127. Поля ответа на запрос /api/events/v2/table_lists/<Токен доступа>/content

Поле	Тип данных	Описание
result	ClearResult	Результат операции очистки данных таблицы

Возможные коды ошибок и их значения:

- 400 (Некорректный параметр) — синтаксическая ошибка в запросе;
- 401 (Ошибка доступа) — ошибка аутентификации;
- 503 (SIEM недоступен) — ошибка доступа к конвейеру обработки событий.

16.5.6. Обновление табличного списка

Запрос для обновления табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/events/v2/table_lists/<Токен доступа>/content

URL запроса должен содержать токен доступа к табличному списку — идентификатор табличного списка.

Параметры строки запроса описаны в таблице ниже.

Таблица 128. Параметры строки запроса `/api/events/v2/table_lists/<Токен доступа>/content`

Параметр	Обязательный	Тип данных	Описание
<code>siemId</code>	Нет	String	Идентификатор конвейера обработки событий

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 129. Параметры в теле запроса `api/events/v2/table_lists/<Токен доступа>/content`

Параметр	Обязательный	Тип данных	Описание
<code>payload</code>	Да	UpdateTableContentPayload	Команды

Параметры команды

<code>add</code>	Нет	Object	Добавленные записи. Каждая строка представлена массивом значений столбцов в порядке, объявленном в схеме
<code>remove</code>	Нет	Object	Удаленные записи. Каждая строка представлена массивом значений столбцов в порядке, объявленном в схеме

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 130. Поля ответа на запрос `/api/events/v2/table_lists/<Токен доступа>/content`

Поле	Тип данных	Описание
<code>result</code>	UpdateResult	Результат операции обновления данных таблицы

Возможные коды ошибок и их значения:

- 400 (Некорректный параметр) — синтаксическая ошибка в запросе;
- 401(Ошибка доступа) — ошибка аутентификации;
- 503 (Возвращается в случае реинициализации SIEM при обновлении данных из Knowledge Base) — ошибка доступа.

16.6. Работа с табличными списками в Knowledge Base

С помощью запросов к API вы можете создавать табличные списки, экспортировать и импортировать записи табличных списков. Также вы можете устанавливать табличные списки в MaxPatrol SIEM.

Для импорта записей в табличный список необходимо:

1. Открыть сессию импорта записей.
2. Загрузить CSV-файл с записями.
3. Запустить импорт записей.

В этом разделе

[Создание табличного списка \(см. раздел 16.6.1\)](#)

[Поиск объекта БД \(см. раздел 16.6.2\)](#)

[Получение информации о табличном списке \(см. раздел 16.6.3\)](#)

[Экспорт табличного списка \(см. раздел 16.6.4\)](#)

[Открытие сессии импорта табличного списка \(см. раздел 16.6.5\)](#)

[Загрузка CSV-файла \(см. раздел 16.6.6\)](#)

[Запуск импорта табличного списка \(см. раздел 16.6.7\)](#)

[Установка объектов в MaxPatrol SIEM \(см. раздел 16.6.8\)](#)

16.6.1. Создание табличного списка

Запрос для создания табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>:8091/api-studio/siem/tabular-lists
```

Параметры строки запроса описаны в таблице ниже.

Таблица 131. Параметры строки запроса /api-studio/siem/tabular-lists

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 132. Параметры в теле запроса /api-studio/siem/tabular-lists

Параметр	Обязательный	Тип данных	Описание
Параметры табличного списка			
assetGroupsToSave	Нет	Array [UUID]	—
description	Нет	String	Описание
fields	Да	Array	Параметры колонок (см. ниже)
fillType	Да	Enum (String)	Назначение: — <code>registry</code> — справочник; — <code>correlationRule</code> — для правил корреляции; — <code>enrichmentRule</code> — для правил обогащения; — <code>assetGrid</code> — для данных об активах; — <code>cybsiGrid</code> — репутационный табличный список
folderId	Нет	UUID	Список системных идентификаторов групп активов. PDQL-запрос выполняется к данным активов в указанных группах. Для табличного списка с данными об активах
groupsToSave	Нет	Array [UUID]	—
includeNested	Нет	Bool	Выполнять ли PDQL-запрос к данным активов во вложенных группах (по умолчанию <code>false</code>). Для табличного списка с данными об активах
itemType	Да	String	Параметр для выбора объектов репутационного списка: <code>ip</code> — IP-адрес, <code>domain</code> — имя домена, <code>hash</code> — значение контрольной суммы. Для репутационного табличного списка

Параметр	Обязательный	Тип данных	Описание
maxSize	Да	Number (int32)	Максимальное число записей. Для табличного списка для правил корреляции и обогащения
minScore	Да	Number (int32)	Параметр для ввода значения интегральной уязвимости (целое число от 0 до 100), при превышении которого объект попадет в табличный список. Если параметр не указан, в табличный список попадут все объекты. Для репутационного табличного списка
pdqlQuery	Да	String	PDQL-запрос для заполнения табличного списка. Для табличного списка с данными об активах
systemName	Да	String	Название
ttl	Да	Number (int32)	Время жизни записи табличного списка в секундах; 0 — время жизни не ограничено. Для табличного списка для правил корреляции и обогащения
ttlEnabled	Нет	Bool	Ограничено ли время жизни записи табличного списка. Для табличного списка для правил корреляции и обогащения
typicalSize	Да	Number (int32)	Рекомендуемое число записей. Для табличного списка для правил корреляции и обогащения
userCanEditContent	Да	Bool	Может ли пользователь добавлять, изменять и удалять записи в табличном списке
Параметры колонок (fields)			
isIndex	Нет	Bool	Является ли индексируемой
isNullable	Нет	Bool	Может ли содержать null
isPrimaryKey	Да	Bool	Является ли ключевой
mapping	Нет	String	—
name	Да	String	Название
typeId	Да	Number (int32)	Тип данных

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 133. Поля ответа на запрос /api-studio/siem/tabular-lists

Поле	Тип данных	Описание
id	UUID	Идентификатор объекта БД

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/tabular-lists?
contentDatabase=siem_content_for_test
{
  "systemName": "TableLiat_Example",
  "description": {},
  "typicalSize": 80000,
  "maxSize": 100000,
  "ttl": 86400,
  "ttlEnabled": true,
  "minScore": 70,
  "itemType": "Ip",
  "includeNested": true,
  "fillType": "Registry",
  "userCanEditContent": false,
  "groupsToSave": [],
  "assetGroupsToSave": [],
  "fields": [
    {
      "name": "First_Column",
      "typeId": 1,
      "isPrimaryKey": true,
      "isIndex": true
    },
    {
      "name": "Second_Column",
      "typeId": 2,
      "isPrimaryKey": false,
      "isNullable": true
    }
  ]
}
```

Ответ:

```
{
  "Id": "3fd59caf-80eb-4963-b0ca-4ab87b19b988"
}
```

16.6.2. Поиск объекта БД

Запрос для поиска объекта в БД.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>:8091/api-studio/siem/objects/list

Параметры строки запроса описаны в таблице ниже.

Таблица 134. Параметры строки запроса /api-studio/siem/objects/list

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 135. Параметры в теле запроса /api-studio/siem/objects/list

Параметр	Обязательный	Тип данных	Описание
filters	Нет	Array	Параметры фильтрации
filters → contentType	Нет	String	Тип объекта: — Aggregation — правило агрегации; — Correlation — правило корреляции; — Enrichment — правило обогащения; — Normalization — правило нормализации; — TabularList — табличный список
filters → siemObjectType	Нет	String	Происхождение объекта: — System — стандартный объект; — User — пользовательский объект

Параметр	Обязательный	Тип данных	Описание
<code>filters → deploymentStatus</code>	Нет	Number (int32)	Статус установки: — 3 — частично установлено; — 2 — установлено неактуальное; — 1 — установлено; — 0 — не установлено
<code>filters → compilationStatus</code>	Нет	Number (int32)	Статус валидации: — 4 — есть предупреждения; — 3 — требует валидации; — 2 — ошибка валидации; — 1 — нет ошибок
<code>filters → siemObjectRegex</code>	Нет	String	Регулярное выражение для поиска по тексту правила
<code>folderId</code>	Нет	UUID	Идентификатор каталога, по которому осуществляется фильтрация
<code>groupId</code>	Нет	UUID	Идентификатор группы, по которой осуществляется фильтрация
<code>recursive</code>	Нет	Bool	Признак рекурсивности
<code>search</code>	Нет	String	Значение для поиска
<code>skip</code>	Нет	Number (int32)	—
<code>sort</code>	Нет	Array	Параметры сортировки
<code>sort → name</code>	Нет	String	Название
<code>sort → order</code>	Нет	Number (int32)	Порядковый номер
<code>sort → type</code>	Нет	Number (int32)	Тип
<code>take</code>	Нет	Number (int32)	—
<code>withoutGroups</code>	Нет	Bool	—

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 136. Поля ответа на запрос `/api-studio/siem/objects/list`

Поле	Тип данных	Описание
Count	Number	Количество найденных объектов
Rows	Array	Параметры объектов БД
Поля объектов БД		
Rows → Actions	Array	—
Rows → Actions → CanDelete	Bool	—
Rows → Actions → CanEdit	Bool	—
Rows → CompilationStatus	Array	—
Rows → CompilationStatus → CompilationStatusId	Number (int32)	—
Rows → CompilationStatus → SdkVersion	String	—
Rows → CompilationStatus → SdkVersionGreater	Bool	—
Rows → CompilationStatus → SdkVersionLess	Bool	—

Поле	Тип данных	Описание
Rows → CompilationStatus → SiemProductVersion	String	—
Rows → CompilationStatus → TaxonomyVersion	String	—
Rows → CopyOf →	Array	Данные об исходном объекте, если объект является копией
Rows → CopyOf → Id	UUID	Идентификатор
Rows → CopyOf → Name	String	Название
Rows → DeploymentStatus	String	Статус установки в MaxPatrol SIEM: — <code>NotInstalled</code> — не установлен; — <code>Installed</code> — установлен; — <code>Outdated</code> — устарел
Rows → Description	String	Описание
Rows → FolderId	UUID	Идентификатор папки с объектом
Rows → FolderPath	String	Путь к папке с объектом
Rows → id	UUID	Идентификатор
Rows → ObjectId	String	Идентификатор Knowledge Base
Rows → ObjectKind	Enum (String)	Тип объекта БД: — <code>AggregationRule</code> — правило агрегации; — <code>CorrelationRule</code> — правило корреляции; — <code>EnrichmentRule</code> — правило обогащения; — <code>NormalizationRule</code> — правило нормализации; — <code>TabularList</code> — табличный список
Rows → Origin →	Array	—

Поле	Тип данных	Описание
Rows → Origin → Id	UUID	—
Rows → Origin → Name	String	—
Rows → Origin → Type	String	—
Rows → OriginId	UUID	Исходный идентификатор
Rows → SystemName	String	Название

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/objects/list?
contentDatabase=siem_content_for_test
{
  "search": "Auditd_processes_with_system_API_whitelist",
  "folderId": null,
  "recursive": true,
  "groupId": null,
  "withoutGroups": false,
  "skip": 0,
  "take": 50,
  "filters": {
    "SiemObjectType": [
      "TabularList"
    ]
  },
  "sort": [
    {
      "name": "objectId",
      "order": 0,
      "type": 0
    }
  ]
}
```

Ответ:

```
{
  "Rows": [
    {
      "Id": "e157f9c4-a67e-49ca-96d3-943fcbcfb026",
      "ObjectId": "PT-TL-150",
      "SystemName": "Auditd_processes_with_system_API_whitelist",
      "Description": "Исключения для правила
      \Detect_system_API_calls_from_suspicious_dir\"",
      "ObjectKind": "TabularList",
      "FolderId": "039818f1-4606-41e6-a811-da435606a604",
      "FolderPath": "Linux. Подозрительные действия пользователей\\tabular_lists",
      "OriginId": "95a1cca9-40b5-4dae-98a2-26aa36948c3c",
      "Origin": {
        "Type": "system",
        "Name": "Positive Technologies",
        "Id": "00000000-0000-0000-0000-000000000000"
      },
      "CopyOf": null,
      "CompilationStatus": {
        "TaxonomyVersion": null,
        "SdkVersion": null,
        "SiemProductVersion": "24.0.0",
        "SdkVersionGreater": false,
        "SdkVersionLess": true,
        "CompilationStatusId": 3
      },
      "DeploymentStatus": "Installed",
      "Actions": {
        "CanEdit": false,
        "CanDelete": false
      }
    }
  ],
  "Count": 1
}
```

16.6.3. Получение информации о табличном списке

Запрос для получения информации о табличном списке.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>:8091/api-studio/siem/tabular-lists/<Идентификатор списка>

URL запроса должен содержать идентификатор табличного списка (UUID).

Параметры строки запроса описаны в таблице ниже.

Таблица 137. Параметры строки запроса `/api-studio/siem/tabular-lists/<Идентификатор списка>`

Параметр	Обязательный	Тип данных	Описание
<code>contentDatabase</code>	Да	String	Название БД

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 138. Поля ответа на запрос `/api-studio/siem/tabular-lists/<Идентификатор списка>`

Поле	Тип данных	Описание
<code>Actions</code>	Array	Разрешения на изменение табличного списка
<code>Actions → CanDelete</code>	Bool	—
<code>Actions → CanEdit</code>	Bool	—
<code>Actions → CanEditContent</code>	Bool	—
<code>AssetGroups</code>	Array	—
<code>AssetGroups → AssetGroups</code>	Array	—
<code>AssetGroups → HasError</code>	Bool	—
<code>CompilationStatus</code>	Array	Статус валидации (см. ниже)
<code>CopyOf</code>	Array	—
<code>DeploymentStatus</code>	Enum (String)	Статус установки в MaxPatrol SIEM: — <code>NotInstalled</code> — не установлен; — <code>Installed</code> — установлен; — <code>Outdated</code> — устарел

Поле	Тип данных	Описание
Description	String	Описание
Fields	Array	Поля колонок (см. ниже)
FillType	Enum	Назначение: — <code>registry</code> — справочник; — <code>correlationRule</code> — для правил корреляции; — <code>enrichmentRule</code> — для правил обогащения; — <code>assetGrid</code> — для данных об активах; — <code>cybsiGrid</code> — репутационный табличный список
Filters	Array	—
Groups	Array	—
Id	UUID	Идентификатор
IncludeNested	Bool	—
ItemType	String	—
MaxSize	Number (int32)	—
MinScore	Number (int32)	—
ObjectId	String	—
Origin	Array	Информация о поставщике
Origin → Name	String	Название поставщика
Origin → Type	String	—
OriginId	String	—
PdqlQuery	String	—
SystemName	String	Название
TotalRows	Number (int32)	—
Ttl	Number (int32)	—
TtlEnabled	Bool	—
TypicalSize	Number (int32)	—

Поле	Тип данных	Описание
UserCanEditContent	Bool	—
Статус валидации (CompilationStatus)		
CompilationStatusId	Number (int32)	—
SdkVersion	String	—
SdkVersionGreater	Bool	—
SdkVersionLess	Bool	—
SiemProductVersion	String	—
TaxonomyVersion	String	—
Поля колонок (Fields)		
Id	UUID	Идентификатор
IsIndex	Bool	Является ли индексируемой
IsNullable	Bool	Может ли содержать null
IsPrimaryKey	Bool	Является ли ключевой
Mapping	String	—
Name	String	Название
TypeId	Number (int32)	Тип данных

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost:8091/api-studio/siem/tabular-lists/3fd59caf-80eb-4963-b0ca-4ab87b19b988?contentDatabase=siem_content_for_test
```

Ответ:

```
{
  "Id": "3fd59caf-80eb-4963-b0ca-4ab87b19b988",
  "ObjectId": "LOC-TL-9",
  "OriginId": "d75c0806-ab0c-45e3-aff0-fd2e723ed878",
  "Folder": null,
  "Origin": {
    "Type": "user",
    "Name": "Локальная система"
  },
  "CopyOf": null,
  "SystemName": "TableLiat_Example",
  "Description": null,
  "MaxSize": null,
  "TypicalSize": null,
  "Ttl": null,
  "TtlEnabled": null,
  "CompilationStatus": {
    "TaxonomyVersion": null,
    "SdkVersion": null,
    "SiemProductVersion": "24.0.0",
    "SdkVersionGreater": false,
    "SdkVersionLess": true,
    "CompilationStatusId": 3
  },
  "TotalRows": 3,
  "Fields": [
    {
      "Id": "9d2dd4ce-3c0e-4da4-ba0c-32e483a643e5",
      "Name": "First_Column",
      "Mapping": null,
      "TypeId": 1,
      "IsPrimaryKey": true,
      "IsIndex": true,
      "IsNullable": false
    },
    {
      "Id": "dea0273e-0e1d-4478-89b6-c0bdf4c00606",
      "Name": "Second_Column",
      "Mapping": null,
      "TypeId": 2,
      "IsPrimaryKey": false,
      "IsIndex": false,
      "IsNullable": true
    }
  ],
}
```

```

"Filters": [
  {
    "Id": "First_Column",
    "Name": "First_Column",
    "Type": "String",
    "MultiSelect": false,
    "Placeholder": ""
  },
  {
    "Id": "Second_Column",
    "Name": "Second_Column",
    "Type": "String",
    "MultiSelect": false,
    "Placeholder": "1-5, 8, 11-13"
  },
  {
    "Id": "ContentType",
    "Name": "Источник",
    "Type": "Select",
    "MultiSelect": true,
    "Placeholder": ""
  },
  {
    "Id": "Activity",
    "Name": "АКТИВНОСТЬ",
    "Type": "Select",
    "MultiSelect": true,
    "Placeholder": ""
  }
],
"Actions": {
  "CanEdit": true,
  "CanEditContent": true,
  "CanDelete": true
},
"UserCanEditContent": false,
"ItemType": null,
"MinScore": null,
"FillType": "Registry",
"PdqlQuery": null,
"Groups": [],
"AssetGroups": null,
"IncludeNested": null,
"DeploymentStatus": "NotInstalled"
}

```

16.6.4. Экспорт табличного списка

Запрос для экспорта записей из табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>:8091/api-studio/siem/tabular-lists/<Идентификатор списка>/export
```

URL запроса должен содержать идентификатор табличного списка.

Параметры строки запроса описаны в таблице ниже.

Таблица 139. Параметры строки запроса /api-studio/siem/tabular-lists/<Идентификатор списка>/export

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 140. Параметры в теле запроса /api-studio/siem/tabular-lists/<Идентификатор списка>/export

Параметр	Обязательный	Тип данных	Описание
sort	Нет	Array	Сортировка записей
sort → Name	Нет	String	Название колонки
sort → Type	Нет	Number (int32)	Тип сортировки в колонке: 0 — по убыванию значений, 1 — по возрастанию
sort → Order	Нет	Number (int32)	Порядковый номер при сортировке по нескольким колонкам
filters	Нет	Array	Фильтр записей
filters → <Название колонки>	Нет	Array [String]	Значения в указанной колонке
rowIds	Нет	Array [UUID]	Идентификаторы записей

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Тело ответа содержит CSV-файл с записями табличного списка.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/tabular-lists/e157f9c4-
a67e-49ca-96d3-943fcbcfb026/export?contentDatabase=siem_content_for_test
{
  "sort": null,
  "filters": {},
  "rowIds": []
}
```

Ответ:

```
"host";"user";"action";"exe_regex";"proctitle_regex"
"*";"*";"xattr change";".+";"^/var/tmp/mkinitramfs$"
"*";"*";"xattr change";".+";"^\\(d-logind\\)$"
```

16.6.5. Открытие сессии импорта табличного списка

Запрос для открытия сессии импорта записей в табличный список.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>:8091/api-studio/siem/tabular-lists/<Идентификатор списка>/
import
```

URL запроса должен содержать идентификатор табличного списка.

Параметры строки запроса описаны в таблице ниже.

Таблица 141. Параметры строки запроса api-studio/siem/tabular-lists/<Идентификатор списка>/import

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 142. Параметры в теле запроса `api-studio/siem/tabular-lists/<Идентификатор списка>/import`

Параметр	Обязательный	Тип данных	Описание
<code>fileSize</code>	Да	Number (int64)	Размер CSV-файла с записями в байтах

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 143. Поля ответа на запрос `api-studio/siem/tabular-lists/<Идентификатор списка>/import`

Поле	Тип данных	Описание
<code>sessionId</code>	UUID	Идентификатор сессии

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/tabular-lists/3fd59caf-80eb-4963-b0ca-4ab87b19b988/import?contentDatabase=siem_content_for_test
{
  "fileSize":88
}
```

Ответ:

```
{
  "sessionId": "d0c2f332-1903-4471-a1cf-5ebc0c9140cd"
}
```

16.6.6. Загрузка CSV-файла

Запрос для загрузки CSV-файла с записями для импорта в табличный список.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>:8091/api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>/upload
```

URL запроса должен содержать идентификатор табличного списка и идентификатор сессии импорта записей.

Параметры строки запроса описаны в таблице ниже.

Таблица 144. Параметры строки запроса api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>/upload

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД
isLastPart	Да	Bool	Признак того, что загружаемая часть данных является последней в последовательности. Допустимые значения: — <code>true</code> — загружаемая часть данных является последней; — <code>false</code> — загружаемая часть данных не является последней и необходимо ожидать последующие части

В теле запроса (в представлении `row text`) необходимо указать записи для импорта. Первая строка должна содержать названия колонок, вторая и последующие строки — значения колонок. Одна строка должна соответствовать одной записи. Названия и значения колонок должны быть заключены в кавычки и разделены точкой с запятой.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/tabular-lists/3fd59caf-80eb-4963-b0ca-4ab87b19b988/import/d0c2f332-1903-4471-a1cf-5ebc0c9140cd/upload?contentDatabase=siem_content_for_test&isLastPart=true
"First_column";"Second_column"
```

```
"Row 1";"212"
"Row 2";"85"
"Row 3";"06"
```

16.6.7. Запуск импорта табличного списка

Запрос для запуска импорта записей в табличный список.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>:8091/api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>
```

URL запроса должен содержать идентификатор табличного списка и идентификатор сессии импорта записей.

Параметры строки запроса описаны в таблице ниже.

Таблица 145. Параметры строки запроса `api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>`

Параметр	Обязательный	Тип данных	Описание
<code>contentDatabase</code>	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 146. Параметры в теле запроса `api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>`

Параметр	Обязательный	Тип данных	Описание
<code>matchingRowsActivation</code>	Да	Enum (String)	Выбор действия с записями при совпадении значений ключевых колонок (для табличного списка типа «справочник»): — <code>Activate</code> — активировать неактивную запись и перезаписать данные в ней, если импортируемые данные отличаются от данных в табличном списке; — <code>LeaveAsIs</code> — перезаписать существующую запись

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 147. Поля ответа на запрос `api-studio/siem/tabular-lists/<Идентификатор списка>/import/<Идентификатор сессии>`

Поле	Тип данных	Описание
Total	Number (int64)	Количество записей в файле
Successful	Number (int64)	Количество импортированных записей
Failed	Number (int64)	Количество пропущенных записей

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/tabular-lists/3fd59caf-80eb-4963-b0ca-4ab87b19b988/import/d0c2f332-1903-4471-a1cf-5ebc0c9140cd?
contentDatabase=siem_content_for_test
{
  "matchingRowsActivation": "LeaveAsIs"
}
```

Ответ:

```
{
  "Total": 3,
  "Successful": 3,
  "Failed": 0
}
```

16.6.8. Установка объектов в MaxPatrol SIEM

Запрос для установки объектов БД в MaxPatrol SIEM.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>:8091/api-studio/siem/deploy

Параметры строки запроса описаны в таблице ниже.

Таблица 148. Параметры строки запроса /api-studio/siem/deploy

Параметр	Обязательный	Тип данных	Описание
contentDatabase	Да	String	Название БД

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 149. Параметры в теле запроса /api-studio/siem/deploy

Параметр	Обязательный	Тип данных	Описание
deployParams	Да	String	Идентификаторы конвейеров, в которые устанавливаются компоненты

В ответ на успешный запрос сервис возвращает код 200 (ОК). Ответ может содержать поля, описанные в таблице ниже.

Таблица 150. Поля ответа на запрос /api-studio/siem/deploy

Поле	Тип данных	Описание
Id	UUID	Идентификатор установки
PipelineId	UUID	Идентификатор конвейера

Возможные коды ошибок и их значения:

Пример

Запрос:

```
POST https://localhost:8091/api-studio/siem/deploy?
contentDatabase=siem_content_for_test
Body: ["c9956036-fd79-4e2b-a178-e9c40f5acc68"]
```

Ответ:

```
{
  "Id": "10bcd49-d7d4-41ed-953c-e50a413eee42"
  "PipelineId": "c9956036-fd79-4e2b-a178-e9c40f5acc68"
}
```

Получение идентификаторов конвейеров

- Чтобы получить идентификаторы конвейеров,

выполните запрос:

```
curl -k -X GET --location 'https://<FQDN или IP-адрес Knowledge Base>:8091/api-studio/siem/pipelines?contentDatabase=<Название БД>' \
--header 'Accept: application/json' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--header 'Authorization: Bearer <Токен>'
```

В выводе запроса в параметрах Id будут содержаться идентификаторы конвейеров обработки событий.

16.7. Работа с инцидентами

С помощью запросов к API вы можете регистрировать инциденты, обновлять информацию об инциденте, удалять инциденты, удалять связь события или актива с инцидентом, изменять меры инцидента, обновлять группы инцидентов, создавать связь события с инцидентом, изменять статус инцидента, создавать и обновлять задачи, а также получать данные о событии.

Для импорта инцидентов из файла необходимо:

1. Открыть сессию импорта инцидентов.
2. Объявить загрузку JSON-файла с инцидентами.
3. Загрузить файл с инцидентами.
4. Запустить импорт инцидентов.

В этом разделе

[Регистрация инцидентов \(см. раздел 16.7.1\)](#)

[Обновление информации об инциденте \(см. раздел 16.7.2\)](#)

[Удаление инцидентов \(см. раздел 16.7.3\)](#)

[Получение списка инцидентов \(см. раздел 16.7.4\)](#)

[Получение данных инцидента \(см. раздел 16.7.5\)](#)

[Получение событий по инциденту \(см. раздел 16.7.6\)](#)

[Экспорт инцидентов \(см. раздел 16.7.7\)](#)

[Открытие сессии импорта инцидентов \(см. раздел 16.7.8\)](#)

[Объявление загрузки файла с инцидентами \(см. раздел 16.7.9\)](#)

[Загрузка файла с инцидентами \(см. раздел 16.7.10\)](#)

[Импорт инцидентов \(см. раздел 16.7.11\)](#)

[Удаление инцидентов по фильтру \(см. раздел 16.7.12\)](#)
[Удаление связи события с инцидентом \(см. раздел 16.7.13\)](#)
[Удаление связи актива с инцидентом \(см. раздел 16.7.14\)](#)
[Изменение мер инцидента \(см. раздел 16.7.15\)](#)
[Обновление групп инцидентов \(см. раздел 16.7.16\)](#)
[Привязка событий к инциденту \(см. раздел 16.7.17\)](#)
[Изменение статуса инцидента \(см. раздел 16.7.18\)](#)
[Создание задачи \(см. раздел 16.7.19\)](#)
[Обновление задачи \(см. раздел 16.7.20\)](#)
[Получение данных о событии \(см. раздел 16.7.21\)](#)

16.7.1. Регистрация инцидентов

Запрос для регистрации инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/incidents

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 151. Параметры в теле запроса /api/incidents

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Название инцидента
description	Нет	String	Текстовый комментарий
assigned	Нет	String	Данные ответственного за расследование инцидента
type	Да	String	Тип инцидента (см. приложение E). Примечание. Вы можете получить перечень типов по запросу <code>api/incident_dictionaries/types</code>
severity	Да	String	Уровень опасности инцидента. Возможные значения: — info — информация; — low — низкая;

Параметр	Обязательный	Тип данных	Описание
			— <code>medium</code> — средняя; — <code>high</code> — высокая. Примечание. Вы можете получить перечень уровней опасности по запросу <code>/api/incident_dictionaries/severities</code>
<code>influence</code>	Нет	String	Влияние инцидента. Возможные значения: — <code>null</code> — инцидент не имеет последствий; — <code>Influence1</code> — потенциальное воздействие на не критически важный актив; — <code>Influence2</code> — воздействие на не критически важный актив с последующим нарушением целостности; — <code>Influence3</code> — воздействие на не критически важный актив с последующим нарушением конфиденциальности; — <code>Influence4</code> — воздействие на критически важный актив, приводящее к финансовым или иным потерям
<code>detected</code>	Да	String	Дата и время регистрации
<code>groups</code>	Нет	String (UUID)	Идентификаторы групп, в которые будет добавлен инцидент: — <code>items</code> — элемент массива
<code>parameters</code>	Нет	Array	Параметры: — <code>id</code> — идентификатор параметра; — <code>value</code> — значение поля
<code>targets</code>	Да	Array	Атакованные активы: — <code>groups</code> — идентификаторы групп активов; — <code>assets</code> — идентификаторы активов; — <code>networks</code> — идентификаторы сетей; — <code>addresses</code> — IP-адреса; — <code>others</code> — прочие активы и сети

Параметр	Обязательный	Тип данных	Описание
attackers	Да	Array	Атакующие активы: — <code>groups</code> — идентификаторы групп активов; — <code>assets</code> — идентификаторы активов; — <code>networks</code> — идентификаторы сетей; — <code>addresses</code> — IP-адреса; — <code>others</code> — прочие активы и сети
linkedObjects	Нет	Array	Связанные объекты: — <code>netFor</code>: • <code>instance</code> — экземпляр; • <code>query</code> — запрос; • <code>uIContextRelativePath</code> — относительный путь в интерфейсе; • <code>sessions</code> — сессии; • <code>alerts</code> — оповещения
source	Нет	Array	Источник данных об инциденте: — <code>IncidentSourceType</code> — тип источника инцидента Возможные значения: — <code>Import</code> — импорт; — <code>NetFor</code> — PT NAD; — <code>SiemScript</code> — MaxPatrol SIEM; — <code>Unknown</code> — неизвестен; — <code>User</code> — оператор
events	Нет	Array	События: — <code>id</code> — идентификатор события; — <code>timestamp</code> — отметка времени
Параметры запроса (query)			
startDate	Нет	String	Дата начала
endDate	Нет	String	Дата окончания
filter	Нет	String	Фильтр

Параметр	Обязательный	Тип данных	Описание
assetGroups	Нет	Array	Группы активов: — <code>items</code> — элемент массива
sources	Нет	Array	Источник данных: — <code>items</code> — элемент массива
Параметры сессий (sessions)			
uuid	Нет	String	Универсальный уникальный идентификатор
startDate	Да	String	Дата начала
endDate	Да	String	Дата окончания
proto	Нет	String	Протокол
appProto	Нет	String	Прикладной протокол
source	Нет	Array	Источник данных: — <code>ipv4</code> — адрес IPv4; — <code>ipv6</code> — адрес IPv6; — <code>port</code> — порт; — <code>dns</code> — DNS-сервер; — <code>geo</code> — геолокация; — <code>asset</code> — актив
destination	Нет	Array	Назначение: — <code>ipv4</code> — адрес IPv4; — <code>ipv6</code> — адрес IPv6; — <code>port</code> — порт; — <code>dns</code> — DNS-сервер; — <code>geo</code> — геолокация
tags	Нет	Array	Метки: — <code>items</code> — элемент массива
Параметры оповещений (alerts)			
title	Нет	String	Заголовок
className	Нет	String	Наименование класса
criticality	Нет	String	Опасность инцидента

Параметр	Обязательный	Тип данных	Описание
tags	Нет	Array	Метки: — items — элемент массива
assetGroups	Нет	Array	Группы активов: — items — элемент массива
snortRule	Да	Array	Правило snort: — gid — генератор идентификатора; — sid — идентификатор правила; — rev — значение версии правила

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 152. Поля ответа на запрос /api/incidents

Поле	Тип данных	Описание
id	String	Идентификатор задачи
key	String	Ключ задачи

400 (Bad request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 153. Поля ответа на запрос /api/incidents

Поле	Тип данных	Описание
message	String	Сообщение
modelState	Array	Состояние модели: — additionalProp1; — additionalProp2; — additionalProp3

См. также

[Правила заполнения полей инцидента category и type \(см. приложение E\)](#)

16.7.2. Обновление информации об инциденте

Запрос для обновления информации об инциденте.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/incidents/<Идентификатор инцидента>

URL запроса должен содержать идентификатор инцидента (UUID).

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 154. Параметры в теле запроса /api/incidents/<Идентификатор инцидента>

Параметр	Обязательный	Тип данных	Описание
assigned	Нет	UUID	Ответственный за расследование инцидента
attackers	Да	Array	Атакующие активы (см. ниже)
description	Нет	String	Текстовый комментарий
detected	Да	DateTime	Дата и время регистрации
groups	Нет	Array [UUID]	Идентификаторы групп, в которые будет добавлен инцидент
influence	Нет	Enum (String)	Влияние инцидента: — <code>null</code> — инцидент не имеет последствий; — <code>Influence1</code> — потенциальное воздействие на не критически важный актив; — <code>Influence2</code> — воздействие на не критически важный актив с последующим нарушением целостности; — <code>Influence3</code> — воздействие на не критически важный актив с последующим нарушением конфиденциальности; — <code>Influence4</code> — воздействие на критически важный актив, приводящее к финансовым или иным потерям
name	Да	String	Название инцидента
parameters	Нет	Array	Дополнительные поля (см. ниже)

Параметр	Обязательный	Тип данных	Описание
severity	Да	Enum	Уровень опасности инцидента: — low — низкая; — medium — средняя; — high — высокая. Примечание. Вы можете получить перечень уровней опасности по запросу <code>/api/incident_dictionaries/severities</code>
source	Да	Enum (String)	Источник данных об инциденте: — Import — импорт; — NetFor — PT NAD; — SiemScript — MaxPatrol SIEM; — Unknown — неизвестен; — User — оператор
targets	Да	Array	Атакованные активы (см. ниже)
type	Да	Enum	Тип инцидента (см. приложение E). Примечание. Вы можете получить перечень типов по запросу <code>api/incident_dictionaries/types</code>
Атакующие активы (attackers)			
addresses	Нет	Array [String]	IP-адреса
assets	Нет	Array [UUID]	Идентификаторы активов
groups	Нет	Array [UUID]	Идентификаторы групп активов
networks	Нет	Array [UUID]	Идентификаторы сетей
others	Нет	Array [String]	Прочие активы и сети
Дополнительные поля инцидента (parameters)			
id	Да	String	Идентификатор поля
value	Нет	String	Значение поля

Параметр	Обязательный	Тип данных	Описание
Атакованные активы (targets)			
addresses	Нет	Array [String]	IP-адреса
assets	Нет	Array [UUID]	Идентификаторы активов
groups	Нет	Array [UUID]	Идентификаторы групп активов
networks	Нет	Array [UUID]	Идентификаторы сетей
others	Нет	Array [String]	Прочие активы и сети

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Пример

Запрос:

```
PUT https://localhost/api/incidents/cc9e3cbd-06be-48f1-8092-363afb95ee19
{
```

```

"name": "Название Simple444",
"source": "user",
"detected": "2021-07-12T21:00:34.000Z",
"type": "PatchPolicyViolation",
"severity": "Low",
"influence": "Influence2",
"targets": {
  "groups": [],
  "assets": [],
  "networks": [],
  "addresses": ["10.0.11.112"],
  "others": ["10.2.12.44"]
},
"attackers": {
  "groups": [],
  "assets": [],
  "networks": [],
  "addresses": ["10.5.13.108"],
  "others": ["10.0.14.1"]
}
}

```

16.7.3. Удаление инцидентов

Запрос для удаления инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v2/incidents/delete_by_ids

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 155. Параметры в теле запроса /api/v2/incidents/delete_by_ids

Параметр	Обязательный	Тип данных	Описание
incidents	Да	Array [UUID]	Идентификаторы инцидентов

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Пример

Запрос:

```
POST https://localhost/api/v2/incidents/delete_by_ids
{
  "incidents": [
    "a5dc33db-b96e-4117-be0f-6b6b0e2a74e0"
  ]
}
```

16.7.4. Получение списка инцидентов

Запрос для получения списка инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/v2/incidents
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 156. Параметры в теле запроса /api/v2/incidents

Параметр	Обязательный	Тип данных	Описание
filter	Да	Array	Фильтр (см. ниже)
filterTimeType	Да	Enum (String)	Свойство инцидента, попадающее во временной интервал: - Approval — дата утверждения; - Closing — дата закрытия; - Creation — дата регистрации; - Detection — дата обнаружения; - InProgress — дата взятия в работу; - LastModification — дата последнего изменения; - Resolving — дата разрешения
groups	Нет	Array	Группы инцидентов (или активов, с которыми связаны инциденты)
limit	Нет	Number	Максимальное количество инцидентов в ответе
offset	Нет	Number	Количество отбрасываемых инцидентов

Параметр	Обязательный	Тип данных	Описание
queryIds	Нет	Array	Идентификаторы фильтров инцидентов. Примечание. Список идентификаторов фильтров вы можете получить по GET-запросу <code>api/v1/incidents/filters_hierarchy</code>
timeFrom	Да	DateTime	Начало временного интервала (например <code>2020-12-31T21:00:00.000Z</code>)
timeTo	Нет	DateTime	Конец временного интервала
Фильтр инцидентов (filter)			
orderby	Нет	Array	Сортировка в списке
orderby → field	Нет	String	Поле, по которому нужно сортировать
orderby → sortOrder	Нет	Enum (String)	Порядок сортировки: — <code>ascending</code> — по возрастанию; — <code>descending</code> — по убыванию
select	Да	Array [String]	Список полей инцидента, из которых нужно получить данные
where	Нет	String	Условие фильтрации (предикат фильтра)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 157. Поля ответа на запрос `/api/v2/incidents`

Поле	Тип данных	Описание
totalItems	Number	Количество инцидентов, удовлетворяющих указанным условиям фильтрации
incidents	Array	Список инцидентов (см. ниже)
Данные инцидента (incidents)		
assigned	Array	Данные ответственного за расследование инцидента (см. ниже)
category	Enum	Категория инцидента
created	DateTime	Дата и время регистрации инцидента

Поле	Тип данных	Описание
id	UUID	Идентификатор инцидента
isConfirmed	Bool	Подтвержден ли инцидент
key	String	Идентификатор инцидента, например INC-24453
name	String	Название инцидента
severity	Enum	Уровень опасности инцидента: — low — низкая; — medium — средняя; — high — высокая. Примечание. Вы можете получить перечень уровней опасности по запросу <code>/api/incident_dictionaries/severities</code>
status	Enum (String)	Статус инцидента: — New — новый; — Approved — утвержден; — InProgress — в работе; — Resolved — разрешен; — Closed — закрыт
type	Enum	Тип инцидента (см. приложение E). Примечание. Вы можете получить перечень типов по запросу <code>api/incident_dictionaries/types</code>

Данные ответственного за расследование (incidents → assigned)

email	String	Адрес электронной почты
firstName	String	Имя
id	UUID	Идентификатор
lastName	String	Фамилия
phone	String	Телефон

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v2/incidents
{
  "offset": 0,
  "limit": 1,
  "groups": {
    "filterType": "no_filter"
  },
  "timeFrom": "2020-12-31T21:00:00.000Z",
  "timeTo": null,
  "filterTimeType": "creation",
  "filter": {
    "select": [
      "key",
      "name",
      "category",
      "type",
      "status",
      "created",
      "assigned"
    ],
    "where": "",
    "orderby": [
      {
        "field": "created",
        "sortOrder": "descending"
      },
      {
        "field": "status",
        "sortOrder": "ascending"
      },
      {
        "field": "severity",
        "sortOrder": "descending"
      }
    ]
  },
  "queryIds": [
    "all_incidents"
  ]
}
```

Ответ:

```
{
  "incidents": [
    {
      "id": "678123d6-c767-4983-89c2-2282e7e46b5b",
      "key": "INC-3",
      "name": "Пример",
      "isConfirmed": false,
      "status": "New",
      "category": "PolicyViolation",
      "type": "PatchPolicyViolation",
      "created": "2021-05-13T13:37:32.073000Z",
      "assigned": {
        "id": null,
        "firstName": null,
        "lastName": null,
        "email": null,
        "phone": null
      },
      "severity": "High"
    }
  ],
  "totalItems": 3
}
```

16.7.5. Получение данных инцидента

Запрос для получения данных инцидента.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/incidentsReadModel/incidents/<Идентификатор инцидента>

URL запроса должен содержать идентификатор инцидента (UUID).

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 158. Поля ответа на запрос /api/incidentsReadModel/incidents/<Идентификатор инцидента>

Поле	Тип данных	Описание
assigned	Array	Данные ответственного за расследование инцидента
attackers	Array	Атакующие активы (см. ниже)
category	Enum (String)	Категория инцидента
correlationRuleNames	Array [String]	Правила корреляции, по которым зарегистрирован инцидент
created	DateTime	Дата и время регистрации инцидента
description	String	Текстовый комментарий
detected	DateTime	Дата и время регистрации
groups	Array	Идентификаторы групп активов
id	UUID	Идентификатор инцидента
influence	String	Влияние инцидента: — <code>null</code> — инцидент не имеет последствий; — <code>Influence1</code> — потенциальное воздействие на не критически важный актив; — <code>Influence2</code> — воздействие на не критически важный актив с последующим нарушением целостности; — <code>Influence3</code> — воздействие на не критически важный актив с последующим нарушением конфиденциальности; — <code>Influence4</code> — воздействие на критически важный актив, приводящее к финансовым или иным потерям
isConfirmed	Bool	Подтвержден ли инцидент
key	String	Идентификатор инцидента, например INC-24453
measures	String	Меры, принятые для решения инцидента
modified	Array	История изменений данных инцидента
name	String	Название инцидента
parameters	Array	Дополнительные поля (см. ниже)
reporter	Array	Пользователь, зарегистрировавший инцидент

Поле	Тип данных	Описание
severity	String	Опасность: — low — низкая; — medium — средняя; — high — высокая. Примечание. Вы можете получить перечень уровней опасности по запросу <code>/api/incident_dictionaries/severities</code>
source	String	Источник данных об инциденте: — Import — импорт; — NetFor — PT NAD; — SiemScript — MaxPatrol SIEM; — Unknown — неизвестен; — User — оператор
status	Enum (String)	Статус инцидента: — New — новый; — Approved — утвержден; — InProgress — в работе; — Resolved — разрешен; — Closed — закрыт
targets	Array	Атакованные активы (см. ниже)
type	String	Тип инцидента (см. приложение E). Примечание. Вы можете получить перечень типов по запросу <code>api/incident_dictionaries/types</code>
unitedNotificationCount	Number (int32)	Количество объединенных уведомлений. Для автоинцидентов значение параметра отлично от null, для остальных — null
Атакующие активы (attackers)		
addresses	Array [String]	IP-адреса
assets	Array [UUID]	Идентификаторы активов

Поле	Тип данных	Описание
groups	Array [UUID]	Идентификаторы групп активов
networks	Array [UUID]	Идентификаторы сетей
others	Array [String]	Прочие активы и сети
Дополнительные поля инцидента (parameters)		
id	String	Идентификатор поля
value	String	Значение поля
Атакованные активы (targets)		
addresses	Array [String]	IP-адреса
assets	Array [UUID]	Идентификаторы активов
groups	Array [UUID]	Идентификаторы групп активов
networks	Array [UUID]	Идентификаторы сетей
others	Array [String]	Прочие активы и сети

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
GET https://localhost/api/incidentsReadModel/incidents/917490b0-1a4e-47f8-82d8-7d2031a1c518
```


Ответ:

```
{
  "id": "917490b0-1a4e-47f8-82d8-7d2031a1c518",
  "key": "INC-22",
  "name": "Пример инцидента",
  "source": "import",
  "description": "",
  "isConfirmed": false,
  "status": "New",
  "measures": "",
  "category": "Attack",
  "severity": "Medium",
  "type": "Spam",
  "created": "2021-03-23T08:43:11.0430000Z",
  "detected": "2021-03-15T21:43:00.0000000Z",
  "modified": {
    "date": "2021-05-13T14:31:04.1367790Z",
    "user": null,
    "properties": [
      {
        "name": "Name",
        "oldValue": "Test_inn22",
        "newValue": "Test_inn33"
      }
    ]
  },
  "reporter": null,
  "assigned": null,
  "influence": null,
  "groups": [
    {
      "type": "GroupInfo",
      "groupType": "static",
      "id": "1499141b-4c00-0001-0000-00000000000d",
      "name": "Import",
      "accessibility": "granted"
    }
  ],
  "targets": {
    "groups": [],
    "assets": [],
    "networks": [],
    "addresses": [],
    "others": []
  },
  "attackers": {
```

```

    "groups": [],
    "assets": [],
    "networks": [],
    "addresses": [],
    "others": []
  },
  "parameters": [
    {
      "id": "SpamMailCount",
      "value": ""
    }
  ],
  "unitedNotificationCount": null,
  "correlationRuleNames": []
}

```

См. также

[Правила заполнения полей инцидента category и type \(см. приложение E\)](#)

16.7.6. Получение событий по инциденту

Запрос для получения списка событий, связанных с инцидентом.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Примечание. В хранилищах событий Elasticsearch и LogSpace могут быть настроены ограничения на выгрузку большого количества событий. Необходимо учитывать такие ограничения при создании собственных правил корреляции.

Метод и URL запроса:

```
GET <Корневой URL API>/api/incidents/<Идентификатор инцидента>/events
```

URL запроса должен содержать идентификатор инцидента.

Параметры строки запроса описаны в таблице ниже.

Таблица 159. Параметры строки запроса /api/incidents/<Идентификатор инцидента>/events

Параметр	Обязательный	Тип данных	Описание
limit	Нет	Number	Максимальное число событий в ответе
offset	Нет	Number	Количество отбрасываемых событий

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 160. Поля ответа на запрос `/api/incidents/<Идентификатор инцидента>/events`

Поле	Тип данных	Описание
date	DateTime	Дата и время регистрации события из поля <code>time</code>
description	String	Текст события из поля <code>text</code>
id	UUID	Идентификатор события из поля <code>uuid</code>

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/incidents/1f6e44d8-4585-4c17-8104-fc2efc3a9216/events?
limit=5
```

Ответ:

```
[
  {
    "id": "00000006-0d99-00e2-f000-00009d368550",
    "date": "2021-06-28T09:05:37.000000Z",
    "description": "Kerberos-аутентификация пользователя ExampleUserName, запрошенная
с узла 10.0.110.10, прошла успешно. Служба Kerberos на узле example.host.ru выдала
билет проверки подлинности Kerberos (TGT)"
  },
  {
    "id": "00000006-0d99-00c7-f000-00019d368550",
    "date": "2021-06-28T09:05:09.000000Z",
    "description": "Пользователь ExampleUserName осуществил успешный вход в систему
на узле example.host.ru. Тип входа: Network"
  }
]
```

16.7.7. Экспорт инцидентов

Запрос для экспорта инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v1/incidents/export/export_by_filter

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 161. Параметры в теле запроса /api/v1/incidents/export/export_by_filter

Параметр	Обязательный	Тип данных	Описание
filter	Да	Array	Фильтр (см. ниже)
filterTimeType	Да	Enum (String)	Свойство инцидента, попадающее во временной интервал: - Approval — дата утверждения; - Closing — дата закрытия; - Creation — дата регистрации; - Detection — дата обнаружения; - InProgress — дата взятия в работу; - LastModification — дата последнего изменения; - Resolving — дата разрешения
groups	Нет	Array	Группы инцидентов (или активов, с которыми связаны инциденты)
queryIds	Нет	Array	Идентификаторы фильтров инцидентов. Примечание. Список идентификаторов фильтров вы можете получить по GET-запросу <code>api/v1/incidents/filters_hierarchy</code>
timeFrom	Да	DateTime	Начало временного интервала (например 2020-12-31T21:00:00.000Z)
timeTo	Нет	DateTime	Конец временного интервала
Фильтр инцидентов (filter)			
orderby	Нет	Array	Сортировка в списке
orderby → field	Нет	String	Поле, по которому нужно сортировать

Параметр	Обязательный	Тип данных	Описание
orderby → sortOrder	Нет	Enum (String)	Порядок сортировки: — <i>ascending</i> — по возрастанию; — <i>descending</i> — по убыванию
select	Да	Array [String]	Список полей инцидента, из которых нужно получить данные
where	Нет	String	Условие фильтрации (предикат фильтра)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Тело ответа содержит файл формата JSON с данными инцидентов. Файл может содержать поля, описанные в таблице ниже.

Таблица 162. Поля файла Инциденты_<Дата экспорта>.json

Поле	Тип данных	Описание
formatVersion	String	Версия формата экспорта
producer → type	String	Система, из которой импортированы инциденты
producer → id	UUID	Идентификатор системы
content	Array [String]	Тип содержимого
incidents	Array	Список инцидентов
incidents → identification	Array	Данные для идентификации
incidents → identification → source	Array	—
incidents → identification → id	String	Идентификатор
incidents → content	Array	Данные инцидента (см. ниже)

Поле	Тип данных	Описание
Данные инцидента (incidents → content)		
actionsTaken	String	Приняты меры по расследованию: — Yes — да; — No — нет; — Unknown — неизвестно
affectedEntities	Array	Затронутые объекты
assignedTo	Array	Данные ответственного за расследование инцидента
attackingEntities	Array	—
confirmed	Bool	Утвержден
createdAt	DateTime	—
createdBy	Array	—
description	String	Текстовый комментарий
influence	Array	Влияние инцидента: — null — инцидент не имеет последствий; — Influence1 — потенциальное воздействие на не критически важный актив; — Influence2 — воздействие на не критически важный актив с последующим нарушением целостности; — Influence3 — воздействие на не критически важный актив с последующим нарушением конфиденциальности; — Influence4 — воздействие на критически важный актив, приводящее к финансовым или иным потерям
modifiedAt	DateTime	Дата и время последнего изменения
name	String	Название
registeredAt	DateTime	Дата и время регистрации
severity	String	Опасность: — low — низкая; — medium — средняя; — high — высокая

Поле	Тип данных	Описание
state	Array	Статус инцидента: — New — новый; — Approved — утвержден; — InProgress — в работе; — Resolved — разрешен; — Closed — закрыт
type	Array	Тип инцидента
typeSpecificParams	Array	Дополнительные параметры инцидента

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v1/incidents/export/export_by_filter
{
```

```
"groups": {
  "filterType": "no_filter"
},
"timeFrom": "2020-12-31T21:00:00.000Z",
"timeTo": null,
"filterTimeType": "creation",
"filter": {
  "select": [
    "key",
    "name",
    "category",
    "type",
    "status",
    "created",
    "assigned"
  ],
  "where": "",
  "orderBy": [
    {
      "field": "created",
      "sortOrder": "descending"
    },
    {
      "field": "status",
      "sortOrder": "ascending"
    },
    {
      "field": "severity",
      "sortOrder": "descending"
    }
  ]
},
"queryIds": [
  "all_incidents"
]
}
```


Ответ:

```
{
  "formatVersion": "1.0",
  "producer": {
    "type": "PT.MPX",
    "id": "5a0a9d31-4e18-4b50-9ff0-502a2f4a071d"
  },
  "content": ["incidents"],
  "incidents": [ {
    "identification": {
      "source": {
        "type": "PT.MPX",
        "id": "5a0a9d31-4e18-4b50-9ff0-502a2f4a071d"
      },
      "id": "INC-2"
    },
    "content": {
      "name": "Название",
      "description": "",
      "state": {
        "id": "New"
      },
      "actionsTaken": "",
      "confirmed": false,
      "registeredAt": "2021-07-12T21:00:34Z",
      "createdAt": "2021-07-21T12:16:47.081Z",
      "modifiedAt": "2021-07-21T14:07:14.52149Z",
      "influence": {
        "id": "Influence2"
      },
      "severity": "Low",
      "type": {
        "id": "PatchPolicyViolation"
      },
      "typeSpecificParams": [],
      "affectedEntities": {
        "addresses": ["10.0.11.112"],
        "otherAssetsAndNetworks": ["10.2.12.44"]
      },
      "attackingEntities": {
        "addresses": ["10.5.13.108"],
        "otherAssetsAndNetworks": ["10.0.14.1"]
      },
      "createdBy": {
        "id": "bcfac8ac-3c0e-40d5-a13e-56252b4e2dd7"
      },
    },
  }
]
```

```

    "assignedTo": {
      "id": "bcfac8ac-3c0e-40d5-a13e-56252b4e2dd7"
    }
  }
}
]
}

```

16.7.8. Открытие сессии импорта инцидентов

Запрос для открытия сессии импорта инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/v1/incidents/import/sessions
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 163. Поля ответа на запрос /api/v1/incidents/import/sessions

Поле	Тип данных	Описание
sessionId	UUID	Идентификатор сессии

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v1/incidents/import/sessions
```

Ответ:

```

{
  "sessionId": "37b3fc33-e7ef-42e6-bd9b-8ee6b62b0c93"
}

```

16.7.9. Объявление загрузки файла с инцидентами

Запрос для объявления загрузки файла с инцидентами.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v1/incidents/import/sessions/<Идентификатор сессии>/files

URL запроса должен содержать идентификатор сессии импорта инцидентов.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 164. Параметры в теле запроса api/v1/incidents/import/sessions/<Идентификатор сессии>/files

Параметр	Обязательный	Тип данных	Описание
fileName	Да	String	Имя файла

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 165. Поля ответа на запрос /api/v1/incidents/import/sessions/<Идентификатор сессии>/files

Поле	Тип данных	Описание
fileId	UUID	Идентификатор файла

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v1/incidents/import/sessions/37b3fc33-e7ef-42e6-bd9b-8ee6b62b0c93/files
{
  "fileName": "My_Incident.json"
}
```

Ответ:

```
{
  "fileId": "54ca9e65-48b4-44c3-bd1f-2b56acf0fa7b"
}
```

16.7.10. Загрузка файла с инцидентами

Запрос для загрузки файла с инцидентами.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/v1/incidents/import/sessions/<Идентификатор сессии>/
uploads
```

URL запроса должен содержать идентификатор сессии импорта инцидентов (UUID).

Параметры строки запроса описаны в таблице ниже.

Таблица 166. Параметры строки запроса /api/v1/incidents/import/sessions/<Идентификатор сессии>/uploads

Параметр	Обязательный	Тип данных	Описание
chunkNumber	Да	Number	Номер фрагмента
fileId	Да	UUID	Идентификатор файла
format	Да	String	Формат данных инцидентов (по умолчанию unified)
totalChunks	Да	Number	Количество фрагментов

В теле запроса необходимо передать файл с инцидентами в формате JSON и описанные в таблице ниже параметры в представлении form-data.

Таблица 167. Параметры в теле запроса /api/v1/incidents/import/sessions/<Идентификатор сессии>/uploads

Параметр	Обязательный	Тип данных	Описание
flowChunkNumber	Да	Number	Номер фрагмента
flowChunkSize	Да	Number	Максимальный размер фрагмента
flowCurrentChunkSize	Да	Number	Размер передаваемого фрагмента
flowFilename	Да	String	Имя файла

Параметр	Обязательный	Тип данных	Описание
flowIdentifier	Да	String	Идентификатор файла
flowRelativePath	Да	String	Относительный путь к файлу
flowTotalChunks	Да	Number	Количество фрагментов

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 168. Поля ответа на запрос `/api/v1/incidents/import/sessions/<Идентификатор сессии>/uploads`

Поле	Тип данных	Описание
isValid	Bool	Удачно ли завершилась проверка файла

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v1/incidents/import/sessions/37b3fc33-e7ef-42e6-bd9b-8ee6b62b0c93/uploads?fileId=54ca9e65-48b4-44c3-bd1f-2b56acf0fa7b&chunkNumber=1&format=unified&totalChunks=1
flowChunkNumber:1
flowChunkSize:946
flowCurrentChunkSize:946
flowIdentifier:946-My_Incident.json
flowFilename:My_Incident.json
flowRelativePath:My_Incident.json
flowTotalChunks:1
file:My_Incident.json
```

Ответ:

```
{
  "isValid": true
}
```

16.7.11. Импорт инцидентов

Запрос для импорта инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v1/incidents/import/complete_import

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 169. Параметры в теле запроса api/v1/incidents/import/complete_import

Параметр	Обязательный	Тип данных	Описание
sessionId	Да	UUID	Идентификатор сессии
groups	Да	Array [UUID]	Идентификаторы групп в которые добавляются инциденты
mode	Да	Enum (String)	Действие при совпадении инцидентов: — Replace — заменять; — Skip — пропускать
format	Да	Enum	Формат данных инцидентов (по умолчанию unified)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 170. Поля ответа на запрос /api/v1/incidents/import/complete_import

Поле	Тип данных	Описание
importResult	Array	Результат импорта
importTime	DateTime	Дата и время начала импорта
totalImportedIncidentCount	Number	Количество импортированных инцидентов
totalIncidentCount	Number	Количество инцидентов в файле
totalSkippedIncidentCount	Number	Количество пропущенных инцидентов

Поле	Тип данных	Описание
Результат импорта (importResult)		
fileId	UUID	Идентификатор файла
importedIncidentCount	Number	Количество импортированных инцидентов
skippedIncidentCount	Number	Количество пропущенных инцидентов
success	Bool	Успешно ли выполнен импорт
totalIncidentCount	Number	Количество инцидентов в файле

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/v1/incidents/import/complete_import
{
  "sessionId": "37b3fc33-e7ef-42e6-bd9b-8ee6b62b0c93",
  "groups": ["1499141b-4c00-0001-0000-00000000000d"],
  "mode": "replace",
  "format": "unified"
}
```

Ответ:

```
{
  "importTime": "2021-05-18T13:44:45.2865836Z",
  "totalImportedIncidentCount": 1,
  "totalSkippedIncidentCount": 0,
  "totalIncidentCount": 1,
  "importResult": [
    {
      "fileId": "54ca9e65-48b4-44c3-bd1f-2b56acf0fa7b",
      "importedIncidentCount": 1,
      "skippedIncidentCount": 0,
      "totalIncidentCount": 1,
      "success": true
    }
  ]
}
```

16.7.12. Удаление инцидентов по фильтру

Запрос для удаления инцидентов по фильтру.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v2/incidents/delete_by_filter

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 171. Параметры в теле запроса /api/v2/incidents/delete_by_filter

Параметр	Обязательный	Тип данных	Описание
timeFrom	Да	String	Начало временного интервала
timeTo	Нет	String	Конец временного интервала
filterTimeType	Да	Enum (String)	Свойство инцидента, попадающее во временной интервал: — <code>creation</code> — дата регистрации; — <code>detection</code> — дата обнаружения; — <code>lastModification</code> — дата последнего изменения; — <code>approval</code> — дата утверждения; — <code>inProgress</code> — дата начала работы; — <code>resolving</code> — дата окончания работы; — <code>closing</code> — дата закрытия инцидента
filter	Да	Array	Фильтр: — <code>select</code> — список полей инцидента, из которых нужно получить данные; — <code>where</code> — условие фильтрации (предикат фильтра); — <code>orderby</code> — сортировка в списке: • <code>field</code> — поле; • <code>sortOrder</code> — порядок сортировки (<code>ascending</code> — по возрастанию, <code>descending</code> — по убыванию)

Параметр	Обязательный	Тип данных	Описание
<code>excludeIncidents</code>	Нет	Array	Инциденты, исключенные из фильтрации: — <code>items</code> — элемент массива
<code>queryIds</code>	Нет	Array	Идентификатор запроса: — <code>items</code> — элемент массива
<code>groups</code>	Да	Array	Группы: — <code>filterType</code> — тип фильтра

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 172. Поля ответа на запрос `/api/v2/incidents/delete_by_filter`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение
<code>modelState</code>	Array	Состояние модели: — <code>additionalProp1</code> ; — <code>additionalProp2</code> ; — <code>additionalProp3</code>

16.7.13. Удаление связи события с инцидентом

Запрос для удаления связи события с инцидентом.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/incidents/{incidentId}/events/{eventId}
```

URL запроса содержит path-параметры `incidentId` — идентификатор инцидента и `eventId` — идентификатор события.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 173. Поля ответа на запрос `/api/incidents/{incidentId}/events/{eventId}`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение
<code>modelState</code>	Array	Состояние модели: — <code>additionalProp1</code>; — <code>additionalProp2</code>; — <code>additionalProp3</code>

16.7.14. Удаление связи актива с инцидентом

Запрос для удаления связи актива с инцидентом.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/incidents/{incidentId}/assets/{assetType}/
{elementType}/{id}
```

URL запроса содержит path-параметры `incidentId` — идентификатор инцидента, `assetType` — тип актива, `elementType` — тип элемента и `id` — идентификатор.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 174. Поля ответа на запрос `/api/incidents/{incidentId}/assets/{assetType}/{elementType}/{id}`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение
<code>modelState</code>	Array	Состояние модели: — <code>additionalProp1</code>; — <code>additionalProp2</code>; — <code>additionalProp3</code>

16.7.15. Изменение мер инцидента

Запрос для изменения мер инцидента.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT `<Корневой URL API>/api/incidents/{incidentId}/measures`

URL запроса содержит path-параметр `incidentId` — идентификатор инцидента.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 175. Параметры в теле запроса `/api/incidents/{incidentId}/measures`

Параметр	Обязательный	Тип данных	Описание
<code>measures</code>	Нет	String	Меры инцидента

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 176. Поля ответа на запрос `/api/incidents/{incidentId}/measures`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение

Поле	Тип данных	Описание
modelState	Array	Состояние модели: — additionalProp1; — additionalProp2; — additionalProp3

16.7.16. Обновление групп инцидентов

Запрос для обновления групп инцидентов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/incidents/{incidentId}/groups

URL запроса содержит path-параметр incidentId — идентификатор инцидента.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 177. Параметры в теле запроса /api/incidents/{incidentId}/groups

Параметр	Обязательный	Тип данных	Описание
groupIds	Да	Array[String]	Идентификаторы групп

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

16.7.17. Привязка событий к инциденту

Запрос для привязки событий к инциденту.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/incidents/events

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 178. Параметры в теле запроса `/api/incidents/events`

Параметр	Обязательный	Тип данных	Описание
<code>linkInfo</code>	Да	Object	Информация о привязке события к инциденту: — <code>incidentKey</code> — ключ инцидента; — <code>events</code> — список событий: • <code>id</code> — системный идентификатор события; • <code>timestamp</code> — дата и время привязки к инциденту

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 179. Поля ответа на запрос `/api/incidents/events`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение
<code>modelState</code>	Array	Состояние модели: — <code>additionalProp1</code>; — <code>additionalProp2</code>; — <code>additionalProp3</code>

16.7.18. Изменение статуса инцидента

Запрос для изменения статуса инцидента.

Информация о возможных статусах инцидентов и переходах между этими статусами приведена в разделе «Работа с инцидентами» Руководства оператора.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT `<Корневой URL API>/api/incidents/{incidentId}/transitions`

URL запроса содержит path-параметр `incidentId` — идентификатор инцидента.

Тело запроса может содержать параметры, описанные в таблице ниже.

Примечание. Параметры в теле запроса должны быть представлены в формате JSON.

Таблица 180. Параметры в теле запроса `/api/incidents/{incidentId}/transitions`

Параметр	Обязательный	Тип данных	Описание
<code>id</code>	Нет	String	Статус инцидента: — <code>New</code> — новый; — <code>Approved</code> — утвержден; — <code>InProgress</code> — в работе; — <code>Resolved</code> — разрешен; — <code>Closed</code> — закрыт
<code>message</code>	Нет	String	Сообщение (комментарий об инциденте)
<code>measures</code>	Нет	String	Меры

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

400 (Bad request) — синтаксическая ошибка в запросе.

Пример

Запрос:

```
PUT https://localhost/api/incidents/4183e4d2-3ee4-4fc5-8b18-15dc23b58643/transitions
{
  "id": "Closed",
  "measures": "Источник заблокирован",
  "message": "Расследование окончено"
}
```

16.7.19. Создание задачи

Запрос для создания задачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/issues

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 181. Параметры в теле запроса /api/issues

Параметр	Обязательный	Тип данных	Описание
name	Нет	String	Название инцидента
type	Нет	String	Тип инцидента
estimatedTime	Нет	String	Расчетное время
assigned	Нет	String	Данные ответственного за расследование инцидента
description	Нет	String	Текстовый комментарий
incidentId	Да	String	Идентификатор инцидента

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 182. Поля ответа на запрос /api/issues

Поле	Обязательный	Описание
id	Да	Идентификатор задачи
status	Нет	Статус задачи
name	Нет	Имя задачи
assigned	Нет	Данные ответственного за расследование инцидента: — id — идентификатор; — firstName — имя; — lastName — фамилия; — email — адрес электронной почты; — phone — телефон
estimatedTime	Нет	Расчетное время

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.7.20. Обновление задачи

Запрос для обновления задачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/issues/{issueId}

URL запроса содержит path-параметр `issueId` — идентификатор задачи.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 183. Параметры в теле запроса `api/issues/{issueId}`

Параметр	Обязательный	Тип данных	Описание
<code>name</code>	Нет	String	Название инцидента
<code>type</code>	Нет	String	Тип инцидента
<code>estimatedTime</code>	Нет	String	Расчетное время
<code>assigned</code>	Нет	String	Данные ответственного за расследование инцидента
<code>description</code>	Нет	String	Текстовый комментарий
<code>incidentId</code>	Да	String	Идентификатор инцидента
<code>status</code>	Нет	String	Статус инцидента

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.7.21. Получение данных о событии

Запрос для получения данных о событии.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/incidentsReadModel/v2/incidents/{incidentId}/events

URL запроса содержит path-параметр `incidentId` — идентификатор инцидента.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 184. Поля ответа на запрос `/api/incidentsReadModel/v2/incidents/{incidentId}/events`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор события
<code>timestamp</code>	String	Дата и время события

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Ответ может содержать поля, описанные в таблице ниже.

Таблица 185. Поля ответа на запрос `/api/incidentsReadModel/v2/incidents/{incidentId}/events`

Поле	Тип данных	Описание
<code>message</code>	String	Сообщение
<code>modelState</code>	Array	Состояние модели: — <code>additionalProp1</code>; — <code>additionalProp2</code>; — <code>additionalProp3</code>

16.8. Работа с событиями

С помощью запросов к API вы можете получить список событий по фильтру или привязать события к зарегистрированному ранее инциденту.

В этом разделе

[Получение списка событий \(см. раздел 16.8.1\)](#)

[Получение списка агрегированных событий \(см. раздел 16.8.2\)](#)

[Привязка событий к инциденту \(см. раздел 16.8.3\)](#)

16.8.1. Получение списка событий

Запрос для получения списка событий.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/events/v2/events

Параметры строки запроса описаны в таблице ниже.

Таблица 186. Параметры строки запроса api/events/v2/events

Параметр	Обязательный	Тип данных	Описание
groupIds	Нет	Array [UUID]	Идентификаторы групп активов
incidentId	Нет	UUID	Идентификатор инцидента, с которым связаны события
limit	Нет	Number	Максимальное количество событий в списке
offset	Нет	Number (int32)	Интервал времени, за который зарегистрированы события
recursive	Нет	Bool	Включать ли вложенные группы активов
token	Нет	UUID	Токен представления

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 187. Параметры в теле запроса /api/events/v2/events

Параметр	Обязательный	Тип данных	Описание
checkpointFrom	Нет	Number (int64)	—
checkpointTo	Нет	Number (int64)	—
filter	Да	Object	Фильтр событий (см. ниже)
groupValues	Нет	Array [UUID]	—
timeFrom	Да	Number (int64)	Начало временного интервала, в который было зарегистрировано событие (Unix-время)
timeTo	Нет	Number (int64)	Конец временного интервала, в который было зарегистрировано событие (Unix-время)
Фильтр событий (filter)			
aliases	Нет	Array	—

Параметр	Обязательный	Тип данных	Описание
groupBy	Да	Array [String]	Список полей события, по которым нужно сгруппировать события
orderBy	Нет	Array	Сортировка в списке
orderby → field	Нет	String	Поле, по которому нужно сортировать
orderby → sortOrder	Нет	Enum (String)	Порядок сортировки: — <code>ascending</code> — по возрастанию; — <code>descending</code> — по убыванию
select	Да	Array [String]	Список полей события, из которых нужно получить данные
top	Нет	Number (int64)	—
where	Нет	String	Условие фильтрации (предикат фильтра)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 188. Поля ответа на запрос `/api/events/v2/events`

Поле	Тип данных	Описание
token	UUID	Токен представления
totalCount	Number (int64)	Количество событий, удовлетворяющих указанным условиям фильтрации
events	Array	Список событий

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/events/v2/events
{
```

```
"filter": {
  "aggregateBy": [],
  "aliases": null,
  "distributeBy": [],
  "groupBy": [],
  "orderBy": [
    {
      "field": "time",
      "sortOrder": "descending"
    }
  ],
  "select": [
    "time",
    "event_src.host",
    "text",
    "id",
    "uuid",
    "correlation_name",
    "object.type"
  ],
  "top": null,
  "where": ""
},
"groupValues": [],
"timeFrom": 1615826187,
"timeTo": 1615912587
}
```

Ответ:

```
{
  "events": [
    {
      "time": "2021-03-16T16:01:05.000000Z",
      "event_src.host": "example.host.ru",
      "text": "Пользователь qa-lisa$ осуществил успешный вход в систему на узле example.host.ru. Тип входа: Network",
      "id": "PT_Microsoft_Windows_eventlog_4624_An_account_was_successfully_logged_on",
      "uuid": "00000006-050d-0642-f000-0000255d3663",
      "correlation_name": null,
      "object.type": null,
      "_meta": {
        "id": "00000006-050d-0642-f000-0000255d3663",
        "time": "2021-03-16T16:01:05.000000Z",
        "assetIds": [
          "1493f164-2240-0001-0000-0000000000c6",
          "1493f163-9100-0001-0000-0000000000a7"
        ],
        "site_alias": "SITE",
        "site_name": "Площадка",
        "site_address": "example.site.ru",
        "site_is_deleted": false
      }
    },
    {
      "time": "2021-03-16T16:01:05.000000Z",
      "event_src.host": "example.host.ru",
      "text": "Kerberos-аутентификация пользователя healthmailbox93343d1, запрошенная с узла 10.0.211.23, прошла успешно. Служба Kerberos на узле example.host.ru выдала билет проверки подлинности Kerberos (TGT)",
      "id": "PT_Microsoft_Windows_eventlog_4768_A_Kerberos_authentication_ticket_was_requested",
      "uuid": "00000006-050d-0641-f000-0002255d3663",
      "correlation_name": null,
      "object.type": null,
      "_meta": {
        "id": "00000006-050d-0641-f000-0002255d3663",
        "time": "2021-03-16T16:01:05.000000Z",
        "assetIds": [
          "1493f1bf-41c0-0001-0000-000000000019f",
          "1493f163-9100-0001-0000-0000000000a7"
        ],
        "site_alias": "SITE",
        "site_name": "Площадка",
        "site_address": "example.site.ru",
      }
    }
  ]
}
```

```

        "site_is_deleted": false
    }
},
],
"totalCount": 125,
"token": "9176b52b-8e14-4a83-8e24-833af6a9d0b6"
}

```

16.8.2. Получение списка агрегированных событий

Запрос для получения списка агрегированных событий.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/events/v2/events/aggregation

Параметры строки запроса описаны в таблице ниже.

Таблица 189. Параметры строки запроса api/events/v2/events/aggregation

Параметр	Обязательный	Тип данных	Описание
limit	Нет	Integer	Количество запрашиваемых элементов выборки. Значение по умолчанию: 1000
groupIds	Нет	Array [String]	Идентификаторы групп активов
recursive	Нет	Bool	Включать ли вложенные группы активов. Значение по умолчанию: false
incidentId	Нет	String	Идентификатор инцидента, с которым связаны события
eventId	Нет	String	Идентификатор события
field	Нет	String	Поле для получения вложенных событий
time	Нет	Date (Date-time)	Время для получения вложенных событий

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 190. Параметры в теле запроса `/api/events/v2/events/aggregation`

Параметр	Обязательный	Тип данных	Описание
<code>timeFrom</code>	Да	Integer	Начало временного интервала, в который было зарегистрировано событие (Unix-время)
<code>timeTo</code>	Нет	Integer	Конец временного интервала, в который было зарегистрировано событие (Unix-время)
<code>filter</code>	Да	Object	Фильтр событий
<code>filter → where</code>	Нет	String	Условие фильтрации (предикат фильтра)
<code>filter → groupBy</code>	Да	Array [String]	Список полей, по которым необходимо группировать данные в выборке
<code>filter → showNullGroups</code>	Нет	Bool	Возвращать ли группы со значениями null
<code>filter → groupByOrder</code>	Нет	Array	Порядок сортировки столбцов для группировки
<code>filter → groupByOrder → field</code>	Нет	String	Поле, по которому выполняется сортировка
<code>filter → groupByOrder → sortOrder</code>	Да	String	Порядок сортировки: — <code>ascending</code> — по возрастанию; — <code>descending</code> — по убыванию
<code>filter → orderBy</code>	Нет	Array	Порядок сортировки в столбцах
<code>filter → orderBy → field</code>	Нет	String	Поле, по которому выполняется сортировка
<code>filter → orderBy → sortOrder</code>	Да	String	Порядок сортировки: — <code>ascending</code> — по возрастанию; — <code>descending</code> — по убыванию
<code>filter → aggregateBy</code>	Да	Array	Параметры агрегации
<code>filter → aggregateBy → function</code>	Да	String	Функция агрегации

Параметр	Обязательный	Тип данных	Описание
<code>filter → aggregateBy → unique</code>	Да	Bool	Признак уникальности значений. Допустимые значения: — <code>True</code> — только уникальные события; — <code>False</code> — все события
<code>filter → aggregateBy → field</code>	Да	String	Поле, по которому выполняется агрегация. Для обозначения любого поля вы можете использовать символ *
<code>filter → distributeBy</code>	Да	Array	Параметры распределения
<code>filter → distributeBy → field</code>	Да	String	Поле, по которому выполняется распределение
<code>filter → distributeBy → granularity</code>	Да	String	Гранулярность
<code>filter → top</code>	Нет	Long (int64)	Количество запрашиваемых элементов выборки
<code>filter → aliases</code>	Нет	Object	Словари связей полей с псевдонимами для выражений запроса <code>AggregateBy</code>
<code>filter → aliases → groupBy</code>	Нет	map [String, String]	Словарь связей полей с псевдонимами выражения <code>groupBy</code>
<code>filter → aliases → aggregateBy</code>	Нет	map [String, String]	Словарь связей полей с псевдонимами выражения <code>aggregateBy</code>
<code>filter → searchType</code>	Нет	String	Тип поиска. Возможные значения: — <code>local</code> — по локальному приложению; — <code>specific</code> — по указанным приложениям. Идентификаторы приложений необходимо указать в параметре <code>searchSources</code>; — <code>all</code> — по всем доступным приложениям
<code>filter → searchSources</code>	Нет	Array [UUID]	Идентификаторы приложений для поиска. Используется, если параметр <code>searchType</code> имеет значение <code>specific</code>
<code>filter → localSources</code>	Нет	Array [String]	Идентификаторы конвейеров обработки событий для поиска

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 191. Поля ответа на запрос `/api/events/v2/events/aggregation`

Поле	Тип данных	Описание
<code>errors</code>	Array [null]	Ошибки
<code>hasMoreResults</code>	Bool	Признак наличия дополнительных данных
<code>columns</code>	Array [String]	Столбцы выборки событий
<code>rows</code>	Array	Строки выборки событий (группы и значения)
<code>rows → groups</code>	Array [String]	Значения полей группировки
<code>rows → values</code>	Array [Double]	Значения полей агрегации

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 500 (Internal Server Error) — внутренняя ошибка сервера;
- 503 (Service Unavailable) — ошибка доступа.

Пример

Запрос:

```
POST https://localhost/api/events/v2/events/aggregation
{
  "filter": {
    "aggregateBy": [
      {
        "function": "COUNT",
        "field": "*",
        "unique": false
      }
    ],
    "aliases": null,
    "distributeBy": [],
    "groupBy": [
      "action"
    ],
    "orderBy": [
      {
        "field": "time",
        "sortOrder": "Descending"
      }
    ]
  }
}
```

```

    }
  ],
  "where": "normalized != false"
},
"timeFrom": 1754024952,
"timeTo": 1756652952
}

```

Ответ:

```

{
  "errors": null,
  "hasMoreResults": false,
  "columns": [
    "count"
  ],
  "rows": [
    {
      "groups": [
        "login"
      ],
      "values": [
        4.0
      ]
    },
    {
      "groups": [
        "alert"
      ],
      "values": [
        1.0
      ]
    }
  ]
}

```

16.8.3. Привязка событий к инциденту

Запрос для привязки событий к инциденту.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/incidents/events

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 192. Параметры в теле запроса /api/incidents/events

Параметр	Обязательный	Тип данных	Описание
incidentKey	Да	String	Ключ инцидента
events	Да	Array	Список событий
events → id	Да	UUID	Системный идентификатор события
events → timestamp	Нет	DateTime	Дата и время привязки к инциденту

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации.

Пример

Запрос:

```
POST https://localhost/api/incidents/events
{
  "incidentKey": "INC-1",
  "events": [
    {
      "id": "00000006-0d99-00e2-f000-00009d368550",
      "timestamp": "2021-06-28T09:05:37.000Z"
    },
    {
      "id": "00000006-0d99-00e1-f000-00019d368550",
      "timestamp": "2021-06-28T09:05:35.000Z"
    },
    {
      "id": "00000006-0d99-00e1-f000-00009d368550",
      "timestamp": "2021-06-28T09:05:35.000Z"
    }
  ]
}
```

16.9. Управление табличными списками для получения информации об изменении активов

Для получения информации об изменении активов используется подписка на изменения табличных списков для данных об активах. С помощью запросов к API вы можете получать информацию о подписчике и его табличных списках, регистрировать подписчика, создавать табличные списки, а также удалять подписчика и его табличные списки. Схема для описания формата табличного списка для данных по активам формируется с помощью PDQL-запроса для анализа активов и фильтра по группам.

В этом разделе

[Получение списка подписчиков \(см. раздел 16.9.1\)](#)

[Получение информации о подписчике \(см. раздел 16.9.2\)](#)

[Удаление подписчика и его табличных списков \(см. раздел 16.9.3\)](#)

[Регистрация подписчика \(см. раздел 16.9.4\)](#)

[Получение информации о табличном списке подписчика \(см. раздел 16.9.5\)](#)

[Создание или обновление табличного списка \(см. раздел 16.9.6\)](#)

[Удаление табличных списков подписчика \(см. раздел 16.9.7\)](#)

[Получение информации о табличных списках подписчика \(см. раздел 16.9.8\)](#)

[Удаление табличного списка подписчика \(см. раздел 16.9.9\)](#)

[Получение изменений табличного списка \(см. раздел 16.9.10\)](#)

16.9.1. Получение списка подписчиков

Запрос для получения списка подписчиков.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/event-tables/v2/subscribers/
```

Параметры строки запроса описаны в таблице ниже.

Таблица 193. Параметры строки запроса /api/event-tables/v2/subscribers/

Параметр	Обязательный	Тип данных	Описание
types	Нет	Array [String]	Фильтр по именам типов подписчиков

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 194. Поля ответа на запрос `/api/event-tables/v2/subscribers/`

Поле	Тип данных	Описание
<code>subscribers</code>	Array	Список зарегистрированных подписчиков на изменения табличных списков: — <code>id</code> — идентификатор подписчика. Обязательный параметр, тип String; — <code>name</code> — имя подписчика. Обязательный параметр, тип String; — <code>type</code> — имя типа подписчика. Обязательный параметр, тип String

16.9.2. Получение информации о подписчике

Запрос для получения информации о подписчике.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}
```

URL запроса содержит path-параметр `subscriberId` — уникальный идентификатор подписчика.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 195. Поля ответа на запрос `/api/event-tables/v2/subscribers/{subscriberId}`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор подписчика
<code>name</code>	String	Имя подписчика
<code>type</code>	String	Имя типа подписчика

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.9.3. Удаление подписчика и его табличных списков

Запрос на идемпотентное удаление подписчика и его табличных списков.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}
```

URL запроса содержит path-параметр `subscriberId` — идентификатор подписчика.

Ответ на запрос

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция по созданию группы выполняется.

16.9.4. Регистрация подписчика

Запрос для регистрации подписчика.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/event-tables/v2/subscribers/
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 196. Параметры в теле запроса `/api/event-tables/v2/subscribers/`

Параметр	Обязательный	Тип данных	Описание
<code>subscriberInfo</code>	Да	Object	Информация о подписчике: — <code>name</code> — имя подписчика, тип String; — <code>type</code> — имя типа подписчика, тип String

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 197. Поля ответа на запрос `/api/event-tables/v2/subscribers/`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор подписчика

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.9.5. Получение информации о табличном списке подписчика

Запрос для получения информации о табличном списке подписчика.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/{tableListId}
```

URL запроса содержит path-параметры `subscriberId` — уникальный идентификатор подписчика и `tableListId` — уникальный идентификатор табличного списка для указанного подписчика.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 198. Поля ответа на запрос `api/event-tables/v2/subscribers/{subscriberId}/tables/{tableListId}`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор табличного списка. Обязательный параметр
<code>name</code>	String	Имя подписчика. Обязательный параметр
<code>isValid</code>	Bool	Валиден ли табличный список. Обязательный параметр. Значение по умолчанию: <code>true</code>
<code>notifications</code>	Array	Перечень уведомлений о состоянии табличного списка: — <code>errorType</code> — тип ошибки. Обязательный параметр, тип String

Поле	Тип данных	Описание
ttl	String (\$time-span)	Время удаления табличного списка с момента создания или последнего получения данных. Пример значения: 90.00:00:00. Необязательный параметр

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.9.6. Создание или обновление табличного списка

Запрос для создания или обновления табличного списка.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/

URL запроса содержит path-параметр `subscriberId` — уникальный идентификатор подписчика. Подписчик является создаваемым через API компонентом для создания и отслеживания внутренних табличных списков с типом «данные об активах».

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 199. Параметры в теле запроса /api/event-tables/v2/subscribers/{subscriberId}/tables/

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Имя табличного списка подписчика для идентификации списка в рамках подписчика. Не зависит от регистра
schema	Да	Object	Схема табличного списка
schema → fields	Да	Array	Набор полей табличного списка
schema → fields → sourceFieldName	Да	String	Имя колонки исходного справочника
schema → fields → nullable	Да	Bool	Может ли содержать null
schema → schemaType	Да	String	Поле для передачи дискриминатора типа объекта

Параметр	Обязательный	Тип данных	Описание
ttl	Нет	String(\$time-span)	Время удаления табличного списка с момента создания или последнего получения данных. Пример значения: 90.00:00:00

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 200. Поля ответа на запрос `/api/event-tables/v2/subscribers/{subscriberId}/tables/`

Поле	Тип данных	Описание
id	String	Идентификатор табличного списка. Обязательный параметр

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.9.7. Удаление табличных списков подписчика

Запрос для идемпотентного удаления всех табличных списков подписчика.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

`DELETE <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/`

URL запроса содержит path-параметр `subscriberId` — уникальный идентификатор подписчика.

Ответ на запрос

Возможные коды ошибок и их значения:

— 202 (Accepted) — операция выполняется.

16.9.8. Получение информации о табличных списках подписчика

Запрос для получения информации о табличных списках подписчика.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/
```

URL запроса содержит path-параметр `subscriberId` — уникальный идентификатор подписчика.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 201. Поля ответа на запрос `/api/event-tables/v2/subscribers/{subscriberId}/tables/`

Поле	Тип данных	Описание
<code>lists</code>	Array	Информация о табличном списке: — <code>id</code> — идентификатор табличного списка, тип String; — <code>name</code> — имя табличного списка. Обязательный параметр, тип String; — <code>isValid</code> — валиден ли табличный список. Обязательный параметр, тип Bool. Значение по умолчанию: <code>true</code>; — <code>notifications</code> — перечень уведомлений о состоянии табличного списка: • <code>errorType</code> — тип ошибки. Обязательный параметр, тип String; — <code>ttl</code> — максимальное время жизни табличного списка, указанное при создании. Пример значения: <code>90.00:00:00</code>. Необязательный параметр, тип String (\$time-span)

16.9.9. Удаление табличного списка подписчика

Запрос для идиempотентного удаления табличного списка подписчика.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/{tableListId}
```

URL запроса содержит path-параметры `subscriberId` — уникальный идентификатор подписчика и `tableListId` — уникальный идентификатор табличного списка для указанного подписчика.

Ответ на запрос

Возможные коды ошибок и их значения:

— 202 (Accepted) — операция по созданию группы выполняется.

16.9.10. Получение изменений табличного списка

Запрос для получения изменений табличного списка. Если схема табличного списка стала невалидной, изменения табличного списка возможно получить только за период до инвалидации схемы.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/event-tables/v2/subscribers/{subscriberId}/tables/{tableListId}/changes
```

URL запроса содержит path-параметры `subscriberId` — уникальный идентификатор подписчика и `tableListId` — уникальный идентификатор табличного списка для указанного подписчика.

Параметры строки запроса описаны в таблице ниже.

Таблица 202. Параметры строки запроса `/api/event-tables/v2/subscribers/{subscriberId}/changes`

Параметр	Обязательный	Тип данных	Описание
token	Нет	String	Токен изменений табличного списка. Если параметр отсутствует — выборка из первого элемента
timeout	Нет	String (\$time-span)	Время максимального ожидания ответа сервера. Пример значения: 90.00:00:00

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 203. Поля ответа на запрос `/api/event-tables/v2/subscribers/{subscriberId}/tables/changes`

Поле	Тип данных	Описание
Параметры табличного списка		
<code>nextToken</code>	String	Токен для получения набора изменений табличного списка. Необязательный параметр
<code>changes</code>	Array	Набор изменений табличного списка. Необязательный параметр
<code>isSchemaValid</code>	Bool	Валидна ли схема табличного списка. Обязательный параметр
Параметры набора изменений табличного списка		
<code>added</code>	Array	Добавленные записи. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1, additionalProp2, additionalProp3 ... additionalPropN</code> . Обязательный параметр
<code>removed</code>	Array	Удаленные записи. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1, additionalProp2, additionalProp3 ... additionalPropN</code> . Обязательный параметр

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.10. Работа с учетными записями сканирования

С помощью запросов к API вы можете добавлять учетные записи сканирования с типами «логин — пароль», «пароль» и «сертификат», получать и обновлять информацию об учетных записях с такими типами, а также получать список всех учетных записей и их меток и удалять учетные записи.

В этом разделе

[Добавление учетной записи типа «логин — пароль» \(см. раздел 16.10.1\)](#)

[Получение учетной записи типа «логин — пароль» \(см. раздел 16.10.2\)](#)

[Обновление учетной записи типа «логин — пароль» \(см. раздел 16.10.3\)](#)

[Добавление учетной записи типа «пароль» \(см. раздел 16.10.4\)](#)

[Получение учетной записи типа «пароль» \(см. раздел 16.10.5\)](#)

[Обновление учетной записи типа «пароль» \(см. раздел 16.10.6\)](#)

[Добавление учетной записи типа «сертификат» \(см. раздел 16.10.7\)](#)

[Получение учетной записи типа «сертификат» \(см. раздел 16.10.8\)](#)

[Обновление учетной записи типа «сертификат» \(см. раздел 16.10.9\)](#)

[Получение всех учетных записей \(см. раздел 16.10.10\)](#)

[Удаление учетной записи \(см. раздел 16.10.11\)](#)

[Получение списка меток учетных записей \(см. раздел 16.10.12\)](#)

16.10.1. Добавление учетной записи типа «логин — пароль»

Запрос для добавления учетной записи типа «логин — пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v3/credentials/login_passwords

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 204. Параметры в теле запроса /api/v3/credentials/login_passwords

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Название учетной записи
description	Нет	String	Описание
credentialTags	Да	Array[String]	Метки учетной записи — если она создается для определенных методов сбора
domain	Нет	String	Имя домена — если для доступа к источнику используется доменная учетная запись
login	Да	String	Логин учетной записи
password	Нет	String	Пароль учетной записи

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 205. Поля ответа на запрос `/api/v3/credentials/login_passwords`

Поле	Тип данных	Описание
id	String	Идентификатор учетной записи

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

Пример

Запрос:

```
POST https://localhost/api/v3/credentials/login_passwords
{
  "name": "user",
  "login": "login",
  "credentialTags": []
}
```

16.10.2. Получение учетной записи типа «логин — пароль»

Запрос для получения учетной записи типа «логин — пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v3/credentials/login_passwords/{id}
```

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 206. Параметры ответа на запрос `/api/v3/credentials/login_passwords/{id}`

Параметр	Обязательный	Тип данных	Описание
type	Да	String	Тип учетной записи

Параметр	Обязательный	Тип данных	Описание
id	Да	String	Идентификатор учетной записи
name	Да	String	Название учетной записи
description	Да	String	Описание
credentialTags	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
domain	Нет	String	Имя домена — если для доступа к источнику используется доменная учетная запись
login	Да	String	Логин учетной записи

Возможные коды ошибок и их значения:

— 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.3. Обновление учетной записи типа «логин — пароль»

Запрос для обновления учетной записи типа «логин — пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/v3/credentials/login_passwords/{id}

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры строки запроса описаны в таблице ниже.

Таблица 207. Параметры строки запроса /api/v3/credentials/login_passwords/{id}

Поле	Обязательный	Тип данных	Описание
name	Да	String	Название учетной записи
description	Да	String	Описание

Поле	Обязательный	Тип данных	Описание
credentialTags	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
domain	Нет	String	Имя домена — если для доступа к источнику используется доменная учетная запись
login	Да	String	Логин учетной записи
shouldResetPassword	Да	Bool	Сбросить ли пароль от учетной записи
password	Да	String	Пароль учетной записи

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content). Ответ не содержит полей.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.4. Добавление учетной записи типа «пароль»

Запрос для добавления учетной записи типа «пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v3/credentials/passwords_only

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 208. Параметры в теле запроса `/api/v3/credentials/passwords_only`

Параметр	Обязательный	Тип данных	Описание
<code>name</code>	Да	String	Название учетной записи
<code>description</code>	Да	String	Описание
<code>credentialTags</code>	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
<code>password</code>	Да	String	Пароль учетной записи

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 209. Поля ответа на запрос `/api/v3/credentials/passwords_only`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор созданной учетной записи

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.10.5. Получение учетной записи типа «пароль»

Запрос для получения учетной записи типа «пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v3/credentials/passwords_only/{id}
```

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 210. Параметры ответа на запрос `/api/v3/credentials/passwords_only/{id}`

Поле	Тип данных	Описание
<code>type</code>	String	Тип учетной записи
<code>id</code>	String	Идентификатор учетной записи
<code>name</code>	String	Название учетной записи
<code>description</code>	String	Описание
<code>credentialTags</code>	String	Метки учетной записи — если она создается для определенных методов сбора

Возможные коды ошибок и их значения:

— 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.6. Обновление учетной записи типа «пароль»

Запрос для обновления учетной записи типа «пароль».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT `<Корневой URL API>/api/v3/credentials/passwords_only/{id}`

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры строки запроса описаны в таблице ниже.

Таблица 211. Параметры строки запроса `/api/v3/credentials/passwords_only/{id}`

Поле	Обязательный	Тип данных	Описание
<code>name</code>	Да	String	Название учетной записи
<code>description</code>	Да	String	Описание
<code>credentialTags</code>	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
<code>shouldResetPassword</code>	Да	Bool	Сбросить ли пароль от учетной записи

Поле	Обязательный	Тип данных	Описание
password	Да	String	Пароль учетной записи

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content). Ответ не содержит полей.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.7. Добавление учетной записи типа «сертификат»

Запрос для добавления учетной записи типа «сертификат».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/v3/credentials/certificates

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 212. Параметры в теле запроса /api/v3/credentials/certificates

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Название учетной записи
description	Да	String	Описание
credentialTags	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
login	Нет	String	Логин учетной записи
password	Нет	String	Пароль учетной записи
certificate	Да	String	Путь к файлу сертификата

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 213. Поля ответа на запрос `/api/v3/credentials/certificates`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор учетной записи

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.10.8. Получение учетной записи типа «сертификат»

Запрос для получения учетной записи типа «сертификат».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v3/credentials/certificates/{id}
```

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 214. Параметры ответа на запрос `/api/v3/credentials/certificates/{id}`

Поле	Тип данных	Описание
<code>type</code>	String	Тип учетной записи
<code>id</code>	String	Идентификатор учетной записи
<code>name</code>	String	Название учетной записи
<code>description</code>	String	Описание
<code>login</code>	String	Логин учетной записи
<code>credentialTags</code>	String	Метки учетной записи — если она создается для определенных методов сбора

Возможные коды ошибок и их значения:

— 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.9. Обновление учетной записи типа «сертификат»

Запрос для обновления учетной записи типа «сертификат».

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/v3/credentials/certificates/{id}

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры строки запроса описаны в таблице ниже.

Таблица 215. Параметры строки запроса /api/v3/credentials/certificates/{id}

Поле	Обязательный	Тип данных	Описание
name	Да	String	Название учетной записи
description	Да	String	Описание
credentialTags	Нет	String	Метки учетной записи — если она создается для определенных методов сбора
login	Нет	String	Логин учетной записи
shouldResetCertificatePassword	Да	Bool	Сбросить ли пароль от учетной записи
password	Да	String	Пароль учетной записи
certificate	Да	String	Путь к файлу сертификата

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content). Ответ не содержит полей.

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.10. Получение всех учетных записей

Запрос для получения всех учетных записей в краткой форме.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/v3/credentials

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 216. Параметры в теле запроса /api/v3/credentials

Поле	Обязательный	Тип данных	Описание
credentialTags	Нет	String	Метки учетных записей — если они созданы для определенных методов сбора

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 217. Параметры ответа на запрос /api/v3/credentials

Поле	Тип данных	Описание
type	String	Тип учетной записи
id	String	Идентификатор учетной записи
name	String	Название учетной записи
description	String	Описание
credentialTags	String	Метки учетной записи — если она создается для определенных методов сбора

16.10.11. Удаление учетной записи

Запрос для удаления учетной записи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/v3/credentials/{id}
```

URL запроса содержит path-параметр `id` — идентификатор учетной записи.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content). Ответ не содержит полей.

Возможные коды ошибок и их значения:

— 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.10.12. Получение списка меток учетных записей

Запрос для получения списка меток учетных записей.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v3/credential_tags
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ содержит список меток учетных записей.

16.11. Работа со сканами

С помощью запросов к API вы можете получать метаданные сырого и обработанного скана по идентификатору, получать содержание сырого и обработанного скана, получать коллекцию данных сырых и обработанных сканов, получать пакет информации о сырых и обработанных сканах, а также добавлять новые сканы.

В этом разделе

[Добавление нового скана \(см. раздел 16.11.1\)](#)

[Получение метаданных скана по идентификатору \(см. раздел 16.11.2\)](#)

[Получение содержания обработанного скана \(см. раздел 16.11.3\)](#)

[Получение коллекции данных обработанных сканов \(см. раздел 16.11.4\)](#)

[Получение пакета информации об обработанных сканах \(см. раздел 16.11.5\)](#)

[Получение метаданных сырого скана по идентификатору \(см. раздел 16.11.6\)](#)

[Получение содержания сырого скана в формате XML \(см. раздел 16.11.7\)](#)

[Получение коллекции данных сырых сканов \(см. раздел 16.11.8\)](#)

[Получение пакета информации о сырых сканах \(см. раздел 16.11.9\)](#)

16.11.1. Добавление нового скана

Запрос для добавления нового скана.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
PUT <Корневой URL API>/api/v1/scans/{scanId}
```

URL запроса содержит path-параметр `scanId` — идентификатор скана.

Параметры строки запроса описаны в таблице ниже.

Таблица 218. Параметры строки запроса `/api/v1/scans/{scanId}`

Параметр	Обязательный	Тип данных	Описание
<code>source</code>	Да	String	Источник сканирования
<code>scopeId</code>	Да	String	Идентификатор инфраструктуры
<code>time</code>	Да	String	Время сканирования
<code>jobId</code>	Нет	String	Идентификатор задачи сканирования
<code>noTtl</code>	Нет	Bool	Отключено ли устаревание актива
<code>replaceEntities</code>	Нет	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>
<code>createOnly</code>	Нет	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 219. Параметры в теле запроса /api/v1/scans/{scanId}

Параметр	Обязательный	Тип данных	Описание
content	Да	String	Тело скана

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (Everything is OK. Scan accepted).

Возможные коды ошибок и их значения:

- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.2. Получение метаданных скана по идентификатору

Запрос для получения метаданных скана по идентификатору.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans/{scanId}
```

URL запроса содержит path-параметр `scanId` — идентификатор скана.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 220. Поля ответа на запрос /api/v1/scans/{scanId}

Поле	Тип данных	Описание
id	String	Идентификатор скана
timeStamp	String	Время сканирования
source	String	Источник сканирования
scopeId	String	Идентификатор инфраструктуры
jobId	String	Идентификатор задачи сканирования
noTtl	Bool	Отключено ли устаревание актива
replaceEntities	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>

Поле	Тип данных	Описание
<code>createOnly</code>	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>
<code>type</code>	String	Тип скана
<code>isValid</code>	Bool	Валиден ли скан
<code>orderedId</code>	Integer	Порядковый номер скана

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.3. Получение содержания обработанного скана

Запрос для получения содержания валидного обработанного скана в формате XML.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans/{scanId}/content
```

URL запроса содержит path-параметр `scanId` — идентификатор скана.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.4. Получение коллекции данных обработанных сканов

Запрос для получения коллекции данных обработанных сканов начиная с указанного времени.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans
```

Параметры строки запроса описаны в таблице ниже.

Таблица 221. Параметры строки запроса /api/v1/scans

Параметр	Обязательный	Тип данных	Описание
fromDate	Да	String	Дата, начиная с которой нужно получить данные
offset	Да	Integer	Смещение от начала результатов выборки
limit	Да	Integer	Размер страницы выдачи
scopeId	Нет	String	Идентификатор инфраструктуры сканирования

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 222. Поля ответа на запрос /api/v1/scans

Поле	Тип данных	Описание
id	String	Идентификатор скана
timeStamp	String	Время сканирования
source	String	Источник сканирования
scopeId	String	Идентификатор инфраструктуры
jobId	String	Идентификатор задачи сканирования
noTtl	Bool	Отключено ли устаревание актива
replaceEntities	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>
createOnly	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>
type	String	Тип скана
isValid	Bool	Валиден ли скан
orderedId	Integer	Порядковый номер скана

Возможные коды ошибок и их значения:

— 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.5. Получение пакета информации об обработанных сканах

Запрос для получения пакета информации об обработанных сканах.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/v1/scans/packages

Параметры строки запроса описаны в таблице ниже.

Таблица 223. Параметры строки запроса /api/v1/scans/packages

Параметр	Обязательный	Тип данных	Описание
fromCheckpoint	Да	Integer	Токен предыдущего пакета

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 224. Поля ответа на запрос /api/v1/scans/packages

Поле	Тип данных	Описание
id	String	Идентификатор скана
timeStamp	String	Время сканирования
source	String	Источник сканирования
scopeId	String	Идентификатор инфраструктуры
jobId	String	Идентификатор задачи сканирования
noTtl	Bool	Отключено ли устаревание актива
replaceEntities	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>
createOnly	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>
type	String	Тип скана
isValid	Bool	Валиден ли скан
orderedId	Integer	Порядковый номер скана

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.6. Получение метаданных сырого скана по идентификатору

Запрос для получения метаданных сырого скана по идентификатору.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans/raw/{scanId}
```

URL запроса содержит path-параметр `scanId` — идентификатор скана.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 225. Поля ответа на запрос `/api/v1/scans/raw/{scanId}`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор скана
<code>timeStamp</code>	String	Время сканирования
<code>source</code>	String	Источник сканирования
<code>scopeId</code>	String	Идентификатор инфраструктуры
<code>jobId</code>	String	Идентификатор задачи сканирования
<code>noTtl</code>	Bool	Отключено ли устаревание актива
<code>replaceEntities</code>	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>
<code>createOnly</code>	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>
<code>type</code>	String	Тип скана

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.7. Получение содержания сырого скана в формате XML

Запрос для получения содержания сырого скана в формате XML.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans/raw/{scanId}/content
```

URL запроса содержит path-параметр `scanId` — идентификатор скана.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.8. Получение коллекции данных сырых сканов

Запрос для получения коллекции данных сырых сканов начиная с указанного времени.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/v1/scans/raw
```

Параметры строки запроса описаны в таблице ниже.

Таблица 226. Параметры строки запроса `/api/v1/scans/raw`

Параметр	Обязательный	Тип данных	Описание
<code>fromDate</code>	Да	String	Дата, начиная с которой нужно получить данные
<code>offset</code>	Да	Integer	Смещение от начала результатов выборки
<code>limit</code>	Да	Integer	Размер страницы выдачи
<code>scopeId</code>	Нет	String	Идентификатор инфраструктуры сканирования

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 227. Поля ответа на запрос `/api/v1/scans/raw`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор скана
<code>timeStamp</code>	String	Время сканирования
<code>source</code>	String	Источник сканирования
<code>scopeId</code>	String	Идентификатор инфраструктуры
<code>jobId</code>	String	Идентификатор задачи сканирования
<code>noTtl</code>	Bool	Отключено ли устаревание актива
<code>replaceEntities</code>	Bool	Замещаются ли все сущности при мердже. Значение по умолчанию — <code>false</code>
<code>createOnly</code>	Bool	Помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>
<code>type</code>	String	Тип скана

Возможные коды ошибок и их значения:

— 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.11.9. Получение пакета информации о сырых сканах

Запрос для получения пакета информации о сырых сканах.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/v1/scans/raw/packages

Параметры строки запроса описаны в таблице ниже.

Таблица 228. Параметры строки запроса `/api/v1/scans/raw/packages`

Параметр	Обязательный	Тип данных	Описание
<code>token</code>	Нет	String	Токен предыдущего пакета

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 229. Поля ответа на запрос `/api/v1/scans/raw/packages`

Поле	Тип данных	Описание
token	String	Токен пакета
scans	Array	Перечень описаний сканов: — <code>id</code> — идентификатор скана; — <code>timeStamp</code> — время сканирования; — <code>source</code> — источник сканирования; — <code>scopeId</code> — идентификатор инфраструктуры; — <code>jobId</code> — идентификатор задачи сканирования; — <code>noTtl</code> — отключено ли устаревание актива; — <code>replaceEntities</code> — замещаются ли все сущности при мердже; — <code>createOnly</code> — помечен ли скан как предназначенный только для создания актива. Значение по умолчанию — <code>false</code>; — <code>type</code> — тип скана

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Badly formatted request) — синтаксическая ошибка в запросе.

16.12. Работа с профилями сканирования

С помощью запросов к API вы можете создавать, обновлять и удалять профили сканирования, получать информацию о профиле сканирования, генерировать профили PenTest, проверять наличие уязвимости при сканировании в режиме пентеста, получать список схем профилей сканирования и локализацию параметров схемы модуля, а также удалять из базы ошибки, возникшие при миграции профиля.

В этом разделе

[Получение списка профилей сканирования \(см. раздел 16.12.1\)](#)

[Создание профиля сканирования \(см. раздел 16.12.2\)](#)

[Получение расширенной информации о профиле сканирования \(см. раздел 16.12.3\)](#)

[Обновление профиля сканирования \(см. раздел 16.12.4\)](#)

[Удаление профиля сканирования \(см. раздел 16.12.5\)](#)

[Генерация профиля PenTest \(см. раздел 16.12.6\)](#)

[Проверка наличия уязвимости при сканировании в режиме пентеста \(см. раздел 16.12.7\)](#)

[Получение списка схем профилей сканирования \(см. раздел 16.12.8\)](#)

[Получение локализации параметров схемы модуля \(см. раздел 16.12.9\)](#)

[Удаление из базы ошибок, возникших при миграции профиля \(см. раздел 16.12.10\)](#)

16.12.1. Получение списка профилей сканирования

Запрос для получения списка профилей сканирования с краткой информацией, с возможностью фильтрации по модулю.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v3/scanner_profiles
```

Параметры строки запроса описаны в таблице ниже.

Таблица 230. Параметры строки запроса /api/scanning/v3/scanner_profiles

Параметр	Обязательный	Тип данных	Описание
moduleId	Нет	String	Идентификатор модуля

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK) и список профилей сканирования с краткой информацией. Ответ может содержать поля, описанные в таблице ниже.

Таблица 231. Поля ответа на запрос /api/scanning/v3/scanner_profiles

Поле	Тип данных	Описание
id	String	Идентификатор профиля сканирования
name	String	Имя профиля сканирования
isSystem	Bool	Является ли системой
baseProfileName	String	Имя базового профиля

Поле	Тип данных	Описание
moduleName	String	Имя модуля
moduleId	String	Идентификатор модуля
output	String	Путь к профилям сканирования
agentRequired	Bool	Необходим ли коллектор
baseProfileId	String	Идентификатор базового профиля
validationErrors	Array	Ошибки валидации: — type — тип ошибки

16.12.2. Создание профиля сканирования

Запрос для создания профиля сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_profiles

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 232. Параметры в теле запроса /api/scanning/v3/scanner_profiles

Параметр	Обязательный	Тип данных	Описание
profile	Да	Array	Создаваемый профиль: — name — имя профиля, тип string; — description — описание профиля; тип string; — baseProfileId — идентификатор базового профиля; тип string; — overrides — переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: additionalProp1, additionalProp2, additionalProp3

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created). Ответ может содержать поля, описанные в таблице ниже.

Таблица 233. Поля ответа на запрос `api/scanning/v3/scanner_profiles`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор созданного профиля

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 234. Поля ответа на запрос `api/scanning/v3/scanner_profiles`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.12.3. Получение расширенной информации о профиле сканирования

Запрос для получения расширенной информации о профиле сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_profiles/{profileId}

URL запроса содержит path-параметр `profileId` — идентификатор профиля сканирования.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 235. Поля ответа на запрос `/api/scanning/v3/scanner_profiles/{profileId}`

Поле	Тип данных	Описание
<code>output</code>	String	Путь к профилю сканирования
<code>description</code>	String	Описание
<code>isSystem</code>	Bool	Является ли системой
<code>moduleName</code>	String	Имя модуля
<code>moduleId</code>	String	Идентификатор модуля
<code>baseProfileId</code>	String	Идентификатор базового профиля
<code>baseProfileName</code>	String	Имя базового профиля
<code>validationErrors</code>	Array	Ошибки валидации: — <code>type</code> — тип ошибки, тип string
<code>profile</code>	Array	Профиль. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code> , <code>additionalProp2</code> , <code>additionalProp3</code>
<code>overrides</code>	Array	Перекрытие параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code> , <code>additionalProp2</code> , <code>additionalProp3</code>

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 236. Поля ответа на запрос `api/scanning/v3/scanner_profiles/{profileId}`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.12.4. Обновление профиля сканирования

Запрос для обновления профиля сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/scanning/v3/scanner_profiles/{profileId}

URL запроса содержит path-параметр `profileId` — идентификатор профиля сканирования.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 237. Параметры в теле запроса `/api/scanning/v3/scanner_profiles/{profileId}`

Параметр	Обязательный	Тип данных	Описание
<code>profile</code>	Да	Array	Создаваемый профиль: — <code>name</code> — имя профиля, тип string; — <code>description</code> — описание профиля; тип string; — <code>overrides</code> — переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code>, <code>additionalProp2</code>, <code>additionalProp3</code>

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 204 (No Content) — скан не найден;
- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 238. Поля ответа на запрос `api/scanning/v3/scanner_profiles/{profileId}`

Поле	Тип данных	Описание
errors	Array	Ошибки: — error — ошибка: • type — тип ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.12.5. Удаление профиля сканирования

Запрос для удаления профиля сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

`DELETE <Корневой URL API>/api/scanning/v3/scanner_profiles/{profileId}`

URL запроса содержит path-параметр `profileId` — идентификатор профиля сканирования.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 204 (No Content) — профиль сканирования не найден;
- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 239. Поля ответа на запрос `api/scanning/v3/scanner_profiles/{profileId}`

Поле	Тип данных	Описание
errors	Array	Ошибки: — error — ошибка: • type — тип ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.12.6. Генерация профиля PenTest

Запрос для генерации профиля PenTest.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_profiles/generate_profile_for_pentest

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 240. Параметры в теле запроса `/api/scanning/v3/scanner_profiles/generate_profile_for_pentest`

Параметр	Обязательный	Тип данных	Описание
settings	Да	Object	Параметры: — useStandardPorts — признак использования стандартных портов, тип bool; — identifiers — идентификаторы; тип string

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 241. Поля ответа на запрос `/api/scanning/v3/scanner_profiles/generate_profile_for_pentest`

Поле	Тип данных	Описание
<code>baseProfileId</code>	String	Идентификатор базового профиля
<code>profile</code>	Array	Профиль сканирования. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code> , <code>additionalProp2</code> , <code>additionalProp3</code>

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 242. Поля ответа на запрос `api/scanning/v3/scanner_profiles/generate_profile_for_pentest`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.12.7. Проверка наличия уязвимости при сканировании в режиме пентеста

Запрос для проверки наличия уязвимости при сканировании в режиме пентеста.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/scanning/v3/scanner_profiles/pentest/validate_vulners_for_pentest
```

Параметры строки запроса описаны в таблице ниже.

Таблица 243. Параметры строки запроса /api/scanning/v3/scanner_profiles/pentest/validate_vulners_for_pentest

Параметр	Обязательный	Тип данных	Описание
identifier	Да	String	Идентификатор

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 244. Поля ответа на запрос /api/scanning/v3/scanner_profiles/pentest/validate_vulners_for_pentest

Поле	Тип данных	Описание
result	String	Результат поиска уязвимости

16.12.8. Получение списка схем профилей сканирования

Запрос для получения списка схем профилей сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v3/scanner_profile_schemas
```

Параметры строки запроса описаны в таблице ниже.

Таблица 245. Параметры строки запроса /api/scanning/v3/scanner_profile_schemas

Параметр	Обязательный	Тип данных	Описание
moduleId	Нет	String	Идентификатор модуля

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 246. Поля ответа на запрос `/api/scanning/v3/scanner_profile_schemas`

Поле	Тип данных	Описание
<code>moduleId</code>	String	Идентификатор модуля
<code>dataSchema</code>	Array	Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1, additionalProp2, additionalProp3</code>
<code>formSchema</code>	Array	Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1, additionalProp2, additionalProp3</code>

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 247. Поля ответа на запрос `api/scanning/v3/scanner_profile_schemas`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.12.9. Получение локализации параметров схемы модуля

Запрос для получения локализации параметров схемы модуля.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_profile_schemas/localization

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 248. Поля ответа на запрос /api/scanning/v3/scanner_profile_schemas/localization

Поле	Тип данных	Описание
additionalProp1	String	—
additionalProp2	String	—
additionalProp3	String	—

16.12.10. Удаление из базы ошибок, возникших при миграции профиля

Запрос для удаления из базы ошибок, возникших при миграции профиля.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

DELETE <Корневой URL API>/api/scanning/v3/scanner_profiles/{id}/migration_errors

URL запроса содержит path-параметр Id — идентификатор профиля сканирования.

Ответ на запрос

Если файл с ошибками не создан, сервис возвращает 204 (No Content).

16.13. Управление задачами сканирования

С помощью запросов к API вы можете создавать, обновлять, запускать, останавливать и удалять задачи сканирования, запрашивать отчеты по задачам (в том числе по идентификатору), точки сохранения задач, количество задач (в том числе по инфраструктуре), валидировать задачи, а также сбрасывать закладки для задач и удалять из базы ошибки, возникшие при миграции задач.

Контракт версии 3 используется для поддержания обратной совместимости. Контракт версии 4 актуальный, включает все последние обновления и улучшения производительности, рекомендуется для новых проектов.

В этом разделе

[Запрос отчета обо всех задачах \(см. раздел 16.13.1\)](#)

[Запрос отчета обо всех задачах \(API версии 4\) \(см. раздел 16.13.2\)](#)

[Создание задачи \(см. раздел 16.13.3\)](#)

[Создание задачи \(API версии 4\) \(см. раздел 16.13.4\)](#)

[Запрос количества задач \(см. раздел 16.13.5\)](#)

[Запрос отчета обо всех точках сохранения всех задач \(см. раздел 16.13.6\)](#)

[Запрос количества задач по инфраструктуре \(см. раздел 16.13.7\)](#)

[Запрос отчета по задаче по идентификатору \(см. раздел 16.13.8\)](#)

[Запрос отчета по задаче по идентификатору \(API версии 4\) \(см. раздел 16.13.9\)](#)

[Обновление задачи \(см. раздел 16.13.10\)](#)

[Обновление задачи \(API версии 4\) \(см. раздел 16.13.11\)](#)

[Удаление задачи \(см. раздел 16.13.12\)](#)

[Валидация задачи \(см. раздел 16.13.13\)](#)

[Сброс закладок для задачи сканирования \(см. раздел 16.13.14\)](#)

[Запуск задачи сканирования \(см. раздел 16.13.15\)](#)

[Остановка задачи \(см. раздел 16.13.16\)](#)

[Приостановка задачи \(см. раздел 16.13.17\)](#)

[Удаление из базы ошибок, возникших при миграции задачи \(см. раздел 16.13.18\)](#)

[Получение времени следующего запуска задачи \(API версии 4\) \(см. раздел 16.13.19\)](#)

16.13.1. Запрос отчета обо всех задачах

Запрос для получения отчета обо всех задачах с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v3/scanner_tasks
```

Параметры строки запроса описаны в таблице ниже.

Таблица 249. Параметры строки запроса /api/scanning/v3/scanner_tasks

Параметр	Обязательный	Тип данных	Описание
mainFilter	Нет	String	Основной фильтр
additionalFilter	Нет	String	Дополнительный фильтр

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 250. Поля ответа на запрос /api/scanning/v3/scanner_tasks

Поле	Тип данных	Описание
id	String	Идентификатор задачи
name	String	Название задачи
description	String	Описание задачи
component	Array	Компонент: — type — тип компонента
agent	Array	Коллектор: — id — идентификатор коллектора; — name — имя коллектора
scope	Array	Задача по инфраструктуре: — id — идентификатор задачи; — name — название задачи
profile	Array	Профиль: — id — идентификатор профиля; — name — название профиля
module	Array	Модуль: — id — идентификатор модуля; — name — название модуля

Поле	Тип данных	Описание
metatransports	Array	Метатранспорты и их параметры. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code> , <code>additionalProp2</code> , <code>additionalProp3</code>
status	Array	Статус задачи сканирования
created	String	Дата создания задачи в формате ISO 8601
lastRun	String	Дата последнего запуска в формате ISO 8601
nextRun	String	Дата следующего запуска в формате ISO 8601
lastRunErrorLevel	Array	Уровень ошибки последнего запуска
include	Array	Признак включения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
exclude	Array	Признак исключения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
isFqdnPriority	Bool	Является ли сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу
validationState	Array	Состояние валидности

Поле	Тип данных	Описание
<code>lastRunError</code>	Array	Ошибка последнего запуска: — <code>type</code> — тип ошибки; — <code>category</code> — категория ошибки
<code>hostDiscovery</code>	Array	Признак обнаружения узлов до начала сбора данных: — <code>enabled</code> — признак сканирования отвечающих узлов; — <code>profile</code> — профиль: • <code>id</code> — идентификатор профиля; • <code>name</code> — название профиля
<code>hasBookmarks</code>	Bool	Есть ли сохраненное состояние источника
<code>credentials</code>	Array	Учетные записи: — <code>id</code> — идентификатор учетной записи; — <code>name</code> — название учетной записи
<code>triggerParameters</code>	Array	Базовые параметры срабатывания по таймеру: — <code>type</code> — тип; — <code>fromDate</code> — дата начала срабатывания; — <code>toDate</code> — дата окончания срабатывания; — <code>isEnabled</code> — разрешено ли срабатывание по таймеру

16.13.2. Запрос отчета обо всех задачах (API версии 4)

Запрос для получения отчета обо всех задачах с помощью API версии 4.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v4/scanner_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 251. Параметры строки запроса /api/scanning/v4/scanner_tasks

Параметр	Обязательный	Тип данных	Описание
<code>mainFilter</code>	Нет	String	Основной фильтр

Параметр	Обязательный	Тип данных	Описание
<code>additionalFilter</code>	Нет	String	Дополнительный фильтр. Значение по умолчанию: <code>all</code>
<code>orderBy</code>	Нет	String	Сортировка в списке
<code>orderDirection</code>	Нет	String	Порядок сортировки
<code>validate</code>	Нет	Bool	Валидировать ли задачи. Значение по умолчанию: <code>true</code>
<code>token</code>	Нет	String	Токен выборки. Возвращает задачи из кэша. Если параметр не задан, задачи возвращаются из базы данных
<code>offset</code>	Нет	Integer	Смещение от начала выборки. Значение по умолчанию: <code>0</code>
<code>limit</code>	Нет	Integer	Количество запрашиваемых элементов выборки. Значение по умолчанию: <code>1000</code>

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 252. Параметры в теле запроса `/api/scanning/v4/scanner_tasks`

Параметр	Обязательный	Тип данных	Описание
<code>text</code>	Нет	String	Текст
<code>agents</code>	Нет	Array	Коллекторы
<code>agents → agentIds</code>	Да	Array[String]	Идентификаторы коллекторов
<code>agents → autoSelect</code>	Да	Bool	Включен ли автовыбор
<code>modules</code>	Нет	Array[String]	Модули
<code>profiles</code>	Нет	Array[String]	Профили
<code>statuses</code>	Нет	String	Статус задачи сканирования: — <code>new</code> — не запускалась; — <code>preparing</code> — подготавливается; — <code>waiting</code> — ожидает выполнения; — <code>running</code> — выполняется;

Параметр	Обязательный	Тип данных	Описание
			— <code>finishing</code> — завершается; — <code>finished</code> — завершена; — <code>suspending</code> — приостанавливается; — <code>suspended</code> — приостановлена
<code>credentials</code>	Нет	<code>Array[String]</code>	Учетные записи
<code>target</code>	Нет	<code>Object</code>	Информация об атакованных активах
<code>target → target</code>	Да	<code>Array</code>	Атакованные активы
<code>target → target → assets</code>	Да	<code>Array</code>	Активы: — <code>id</code> — идентификаторы активов, тип <code>String</code>; — <code>name</code> — названия активов, тип <code>String</code>
<code>target → target → targets</code>	Да	<code>Array[String]</code>	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
<code>target → target → assetsGroups</code>	Да	<code>Array</code>	Группы активов: — <code>id</code> — идентификаторы групп активов, тип <code>String</code>; — <code>name</code> — названия групп активов, тип <code>String</code>
<code>target → type</code>	Да	<code>String</code>	Тип атакованного актива
<code>scopes</code>	Нет	<code>Array[String]</code>	Задача по инфраструктуре

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 253. Поля ответа на запрос `/api/scanning/v4/scanner_tasks`

Поле	Тип данных	Описание
<code>id</code>	<code>String</code>	Идентификатор задачи
<code>name</code>	<code>String</code>	Название задачи
<code>description</code>	<code>String</code>	Описание задачи

Поле	Тип данных	Описание
agents	Object	Коллекторы и их компоненты, которые выбираются для выполнения задачи (расширенный вариант)
agents → agents	Array	Информация о коллекторах
agents → agents → id	String	Идентификаторы коллекторов
agents → agents → name	String	Названия коллекторов
agents → components	Object	Принадлежность заданных для выполнения задачи коллекторов к конкретному конвейеру или ядру
agents → components → siemIds	Array[String]	Конвейеры, коллекторы которых можно использовать для выполнения подзадачи
agents → components → useCoreAgents	Bool	Возможно ли использовать коллекторы ядра для выполнения подзадачи
scope	Object	Задача по инфраструктуре
scope → id	String	Идентификатор задачи
scope → name	String	Название задачи
profile	Object	Профиль сканирования
profile → id	String	Идентификатор профиля
profile → name	String	Название профиля
module	Object	Модуль
module → id	String	Идентификатор модуля
module → name	String	Название модуля
metatransports	Array	Метатранспорты и их параметры
status	String (Enum)	Статус задачи сканирования: — new — не запускалась; — preparing — подготавливается; — waiting — ожидает выполнения; — running — выполняется; — finishing — завершается; — finished — завершена;

Поле	Тип данных	Описание
		— <code>suspending</code> — приостанавливается; — <code>suspended</code> — приостановлена; — <code>imported</code> — импортирована
<code>created</code>	String (Date-time)	Дата создания задачи в формате ISO 8601
<code>lastRun</code>	String (Date-time)	Дата последнего запуска в формате ISO 8601
<code>nextRun</code>	String (Date-time)	Дата следующего запуска в формате ISO 8601
<code>lastRunErrorLevel</code>	String (Enum)	Уровень ошибки последнего запуска: — <code>green</code> — все подзадачи завершены без ошибок; — <code>yellow</code> — часть подзадач завершена с ошибками; — <code>red</code> — все подзадачи завершены с ошибками, задача не может быть выполнена
<code>include</code>	Object	Признак включения целей сбора данных
<code>include → assets</code>	Array	Активы
<code>include → assets → id</code>	String	Идентификатор актива
<code>include → assets → name</code>	String	Название актива
<code>include → targets</code>	Array[String]	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
<code>include → assetsGroups</code>	Array	Группы активов
<code>include → assetsGroups → id</code>	String	Идентификатор группы активов
<code>include → assetsGroups → name</code>	String	Название группы активов
<code>exclude</code>	Object	Признак исключения целей сбора данных
<code>exclude → assets</code>	Array	Активы
<code>exclude → assets → id</code>	String	Идентификатор актива

Поле	Тип данных	Описание
<code>exclude → assets → name</code>	String	Название актива
<code>exclude → targets</code>	Array[String]	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
<code>exclude → assetsGroups</code>	Array	Группы активов
<code>exclude → assetsGroups → id</code>	String	Идентификатор группы активов
<code>exclude → assetsGroups → name</code>	String	Название группы активов
<code>isFqdnPriority</code>	Bool	Является ли сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу
<code>validationState</code>	String	Статус корректности данных, указанных в задаче: — <code>valid</code> — все указанные данные корректны, задача может быть выполнена; — <code>invalid</code> — указанные данные некорректны, задача не может быть выполнена
<code>lastRunError</code>	Object	Последняя ошибка запуска
<code>lastRunError → type</code>	String	Тип ошибки
<code>lastRunError → category</code>	String	Категория ошибки: — <code>target</code> — цель сканирования; — <code>scope</code> — задача по инфраструктуре; — <code>profile</code> — профиль; — <code>task</code> — задача; — <code>agent</code> — коллектор; — <code>migration</code> — миграция
<code>hostDiscovery</code>	Array	Признак обнаружения узлов до начала сбора данных
<code>hostDiscovery → enabled</code>	Bool	Признак сканирования только отвечающих узлов
<code>hostDiscovery → profile</code>	Object	Профиль, используемый при сканировании

Поле	Тип данных	Описание
hostDiscovery → profile → id	String	Идентификатор профиля
hostDiscovery → profile → name	String	Название профиля
hasBookmarks	Bool	Сохранено ли состояние источника
credentials	Array	Учетные записи
credentials → id	String	Идентификатор учетной записи
credentials → name	String	Название учетной записи
triggerParameters	Array	Параметры срабатывания по таймеру
triggerParameters → type	String	Тип
triggerParameters → fromDate	String (Date-time)	Дата начала срабатывания
triggerParameters → toDate	String (Date-time)	Дата окончания срабатывания
triggerParameters → isEnabled	Bool	Признак того, что разрешено срабатывание по таймеру
token	String	Токен
totalCount	Integer	Общее количество

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 254. Поля ответа на запрос /api/scanning/v4/scanner_tasks

Поле	Тип данных	Описание
errors	Array	Ошибки
errors → error	Object	Ошибка
errors → error → type	String	Тип ошибки

Поле	Тип данных	Описание
errors → error → category	String	Категория ошибки: — target — цель сканирования; — scope — задача по инфраструктуре; — profile — профиль; — task — задача; — agent — коллектор; — migration — миграция
errors → source	Object	Источник ошибки
errors → source → displayName	String	Отображаемое имя
errors → source → hostName	String	Имя узла
errors → source → ipAddresses	Array[String]	IP-адреса
errors → sensitive	Bool	Признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.13.3. Создание задачи

Запрос для создания задачи с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_tasks

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 255. Параметры в теле запроса /api/scanning/v3/scanner_tasks

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Название задачи
description	Нет	String	Описание задачи

Параметр	Обязательный	Тип данных	Описание
component	Нет	Array of strings	Компонент, к которому подключены коллекторы
component → type	Да	String	Тип компонента: — <code>CoreComponent</code> — коллекторы подключены к MP 10 Core; — <code>SiemComponent</code> — коллекторы подключены к MP SIEM Server
component → siemId	Да, если указан тип <code>SiemComponent</code>	String	Идентификатор MP SIEM Server
agent	Нет	String	Идентификатор коллектора
scope	Да	String	Идентификатор задачи по инфраструктуре
profile	Да	String	Идентификатор основного профиля сканирования
include	Да	Array of objects	Признак включения целей сбора данных
include → assets	Да	Array of objects	Активы
include → assets → id	Да	String	Идентификатор актива
include → targets	Да	Array of strings	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
include → assetsGroups	Да	Array of objects	Группы активов
include → assetsGroups → id	Да	String	Идентификатор группы активов
exclude	Да	Array of objects	Признак исключения целей сбора данных
exclude → assets	Да	Array of objects	Активы
exclude → assets → id	Да	String	Идентификатор актива
exclude → targets	Да	Array of strings	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов

Параметр	Обязательный	Тип данных	Описание
<code>exclude → assetsGroups</code>	Да	Array of objects	Группы активов
<code>exclude → assetsGroups → id</code>	Да	String	Идентификатор группы активов
<code>hostDiscovery</code>	Да	Array of strings	Признак обнаружения узлов модулем HostDiscovery до начала сбора данных
<code>hostDiscovery → enabled</code>	Да	Boolean	Обнаружение выполняется (<code>true</code>) или не выполняется (<code>false</code>)
<code>hostDiscovery → profile</code>	Нет	String	Идентификатор профиля модуля HostDiscovery
<code>triggerParameters</code>	Да	Array of objects	Расписание сканирования
<code>triggerParameters → type</code>	Да	String	Параметры расписания: — <code>Daily</code> — ежедневно; — <code>Periodic</code> — раз в указанное число дней; — <code>RepeatableDaily</code> — несколько раз в день; — <code>Weekly</code> — в указанные дни недели; — <code>Fortnightly</code> — в указанные дни с интервалом в две недели; — <code>Monthly</code> — в указанные дни месяца; — <code>CronScheduler</code> — в формате cron
<code>triggerParameters → type → atTime</code>	Да, если указан тип <code>Daily</code> , <code>Periodic</code> , <code>Weekly</code> или <code>Fortnightly</code>	Time-span	Время срабатывания таймера
<code>triggerParameters → type → timeZone</code>	Да, если указан тип <code>Daily</code> , <code>Periodic</code>	String	Часовой пояс

Параметр	Обязательный	Тип данных	Описание
	с, Weekly, Fortnigh- tly, Monthly или CronSch- eduler		
triggerParameters → type → daysOfWeek	Да, если указан тип Daily, Weekly или Fortnigh- tly	String	Дни недели
triggerParameters → type → repeatInterval	Да, если указан тип Periodic или RepeatableDaily	Time-span	Интервал между срабатываниями
triggerParameters → type → onDays	Да, если указан тип Monthly	Integer	Дни месяца
triggerParameters → type → cronExpressions	Да, если указан тип CronScheduler	Array of strings	Время выполнения в формате cron
triggerParameters → fromDate	Нет	Date-time	Дата и время начала действия расписания
triggerParameters → toDate	Нет	Date-time	Дата и время окончания действия расписания

Параметр	Обязательный	Тип данных	Описание
triggerParameters → isEnabled	Да	Boolean	Задача запущена (true) или остановлена (false)
isFqdnPriority	Нет	Boolean	Подключаться к активам по FQDN (true) или по IP-адресу (false)
overrides	Нет	Array of objects	Переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: additionalProp1, additionalProp2, additionalProp3
savepoints	Нет	Array of objects	Точки сохранения
savepoints → target	Да	String	Цель задачи сканирования
savepoints → savepoint	Да	String	Точка сохранения в формате 64base
deniedScanSettings	Нет	Array of objects	Запрещенное время сканирования
deniedScanSettings → isEnabled	Да	Boolean	Запрещенные интервалы действуют (true) или отключены (false)
deniedScanSettings → periods	Нет	Array of objects	Параметры запрещенного интервала
deniedScanSettings → periods → daysOfWeek	Да	Array of strings	Дни недели
deniedScanSettings → periods → start	Да	String	Время начала запрещенного интервала в формате hh:mm:ss
deniedScanSettings → periods → end	Да	String	Время окончания запрещенного интервала в формате hh:mm:ss

Параметр	Обязательный	Тип данных	Описание
deniedScanSettings → periods → timeZone	Да	String	Часовой пояс
deniedScanSettings → periods → isWholeDayChosen	Да	Boolean	Запрещенный интервал действует весь день (<i>true</i>) или в указанное время (<i>false</i>)

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 256. Поля ответа на запрос `/api/scanning/v3/scanner_tasks`

Поле	Тип данных	Описание
id	String	Идентификатор задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 257. Поля ответа на запрос `/api/scanning/v3/scanner_tasks`

Поле	Тип данных	Описание
errors	Array of objects	Ошибки
errors → error	Array of objects	Ошибка
errors → error → category	Enum	Категория ошибки
errors → error → type	String	Тип ошибки
errors → source	Object	Источник ошибки
errors → source → displayName	String	Отображаемое имя

Поле	Тип данных	Описание
errors → source → hostName	String	Имя узла
errors → source → ipAddresses	Array of strings	IP-адреса
errors → sensitive	Boolean	Признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

Пример

Запрос:

```
{
  "name": "Service Discovery ежедневно",
  "description": "Service Discovery по группе Unmanaged hosts",
  "agent": "8f6c33a5-858e-4bcb-aaf2-311c70dacebb",
  "scope": "00000000-0000-0000-0000-000000000005",
  "profile": "bed53c75-f7eb-41b8-a784-3c343abd8d4c",
  "include": {
    "assets": [],
    "targets": [],
    "assetsGroups": [
      {
        "id": "00000000-0000-0000-0000-000000000003"
      }
    ]
  },
  "exclude": {
    "assets": [],
    "targets": [],
    "assetsGroups": []
  },
  "hostDiscovery": {
    "enabled": true,
    "profile": "e27ac04f-95fb-41e0-ad0e-185b6713f08c"
  },
  "triggerParameters": {
    "isEnabled": false,
    "fromDate": "2025-02-17",
    "timeZone": "+03:00",
    "type": "Daily",
  }
}
```

```

    "atTime": "09:00:00",
    "daysOfWeek": [
      "monday",
      "tuesday",
      "wednesday",
      "thursday",
      "friday",
      "saturday",
      "sunday"
    ]
  }
}

```

Ответ:

```

{
  "operationId": "14a5ca6b-f780-a001-0000-00000000003f"
}

```

16.13.4. Создание задачи (API версии 4)

Запрос для создания задачи с помощью API версии 4.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v4/scanner_tasks/create

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 258. Параметры в теле запроса /api/scanning/v4/scanner_tasks/create

Параметр	Обязательный	Тип данных	Описание
task	Нет	Object	Задача
Параметры задачи			
name	Да	String	Название задачи
description	Нет	String	Описание задачи

Параметр	Обязательный	Тип данных	Описание
agents	Нет	Array	Коллекторы и их компоненты, которые выбираются для выполнения задачи (расширенный вариант): — <code>agentIds</code> — идентификаторы коллекторов. Тип данных String; — <code>components</code> — принадлежность заданных для выполнения задачи коллекторов к конкретному конвейеру или ядру: • <code>siemIds</code> — конвейеры, коллекторы которых можно использовать для выполнения подзадачи. Тип данных String; • <code>useCoreAgents</code> — возможно ли использовать коллекторы ядра для выполнения подзадачи. Тип данных Bool
scope	Да	String	Задача по инфраструктуре
profile	Да	String	Основной профиль сканирования
include	Да	Array	Признак включения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива Тип данных String; • <code>name</code> — имя актива. Тип данных String; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов Тип данных String; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов. Тип данных String; • <code>name</code> — название группы активов. Тип данных String

Параметр	Обязательный	Тип данных	Описание
exclude	Да	Array	Признак исключения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива. Тип данных String; • <code>name</code> — имя актива. Тип данных String; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов. Тип данных String; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов. Тип данных String; • <code>name</code> — название группы активов. Тип данных String
hostDiscovery	Да	Array	Признак обнаружения узлов до начала сбора данных: — <code>enabled</code> — сканировать ли отвечающие узлы. Тип данных Bool; — <code>profile</code> — профиль: • <code>id</code> — идентификатор профиля. Тип данных String; • <code>name</code> — название профиля. Тип данных String
triggerParameters	Да	Array	Базовые параметры срабатывания по таймеру: — <code>type</code> — тип срабатывания. Тип данных String; — <code>fromDate</code> — дата начала срабатывания. Тип данных String; — <code>toDate</code> — дата окончания срабатывания. Тип данных String; — <code>isEnabled</code> — разрешено ли срабатывание по таймеру. Тип данных Bool
isFqdnPriority	Нет	Bool	Является ли сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу

Параметр	Обязательный	Тип данных	Описание
<code>overrides</code>	Нет	Array	Переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1, additionalProp2, additionalProp3</code>
<code>savepoints</code>	Нет	Array	Точки сохранения: — <code>target</code> — цель задачи сканирования. Тип данных String; — <code>savepoint</code> — точка сохранения. Тип данных String

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 259. Поля ответа на запрос `/api/scanning/v4/scanner_tasks/create`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 260. Поля ответа на запрос /api/scanning/v4/scanner_tasks/create

Поле	Тип данных	Описание
errors	Array	Ошибки: — error — ошибка: • type — тип ошибки; • category — категория ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.13.5. Запрос количества задач

Запрос для получения количества задач с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_tasks/count

Параметры строки запроса описаны в таблице ниже.

Таблица 261. Параметры строки запроса /api/scanning/v3/scanner_tasks/count

Параметр	Обязательный	Тип данных	Описание
mainFilter	Нет	String	Основной фильтр
additionalFilter	Нет	String	Дополнительный фильтр

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 262. Поля ответа на запрос /api/scanning/v3/scanner_tasks/count

Поле	Тип данных	Описание
total	Integer	Количество задач сканирования

16.13.6. Запрос отчета обо всех точках сохранения всех задач

Запрос для получения отчета обо всех точках сохранения всех задач с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_tasks/bookmarks

Параметры строки запроса описаны в таблице ниже.

Таблица 263. Параметры строки запроса /api/scanning/v3/scanner_tasks/bookmarks

Параметр	Обязательный	Тип данных	Описание
offset	Нет	Integer	Смещение страницы от начала выборки
limit	Нет	Integer	Количество запрашиваемых элементов выборки

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 264. Поля ответа на запрос /api/scanning/v3/scanner_tasks/bookmarks

Поле	Тип данных	Описание
items	Array	Представление конкретной точки сохранения: — taskId — идентификатор задачи; — scopeId — идентификатор скоупа; — target — цель; — savepoint — точка сохранения
totalItems	Integer	Количество точек сохранения

16.13.7. Запрос количества задач по инфраструктуре

Запрос для получения количества задач по инфраструктуре с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_tasks/scope/{scopeId}/count

URL запроса содержит path-параметр `scopeId` — идентификатор задачи по инфраструктуре.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 265. Поля ответа на запрос /api/scanning/v3/scanner_tasks/scope/{scopeId}/count

Поле	Тип данных	Описание
total	Integer	Количество задач по инфраструктуре

16.13.8. Запрос отчета по задаче по идентификатору

Запрос для получения отчета по задаче по идентификатору с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 266. Поля ответа на запрос /api/scanning/v3/scanner_tasks/{id}

Поле	Тип данных	Описание
id	String	Идентификатор задачи
name	String	Название задачи

Поле	Тип данных	Описание
description	String	Описание задачи
include	Array	Признак включения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
exclude	Array	Признак исключения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
isFqdnPriority	Bool	Является ли сканирование сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу
component	Array	Компонент: — <code>type</code> — тип компонента
agent	Array	Коллектор: — <code>id</code> — идентификатор коллектора; — <code>name</code> — имя коллектора
module	Array	Модуль: — <code>id</code> — идентификатор модуля; — <code>name</code> — название модуля

Поле	Тип данных	Описание
scope	Array	Задача по инфраструктуре: — id — идентификатор задачи; — name — название задачи
profile	Array	Профиль сканирования: — id — идентификатор профиля; — name — название профиля
completed	String	Дата завершения задачи
status	Array	Статус задачи сканирования
created	String	Дата создания задачи
lastRun	String	Дата последнего запуска
hasBookmarks	Bool	Сохранено ли состояние источника
lastRunErrorLevel	Array	Уровень ошибки последнего запуска
validationState	Array	Состояние валидности
hostDiscovery	Array	Признак обнаружения узлов до начала сбора данных: — enabled — признак сканирования отвечающих узлов; — profile — профиль: • id — идентификатор профиля; • name — название профиля
credentials	Array	Учетные записи: — id — идентификатор учетной записи — name — название учетной записи
triggerParameters	Array	Базовые параметры срабатывания по таймеру: — type — тип; — fromDate — дата начала срабатывания; — toDate — дата окончания срабатывания; — isEnabled — разрешено ли срабатывание по таймеру
overrides	Array	Переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: additionalProp1, additionalProp2, additionalProp3

Возможные коды ошибок и их значения:

404 (Task not found) — задача не найдена.

16.13.9. Запрос отчета по задаче по идентификатору (API версии 4)

Запрос для получения отчета по задаче по ее идентификатору с помощью API версии 4.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v4/scanner_tasks/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 267. Поля ответа на запрос /api/scanning/v4/scanner_tasks/{id}

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор задачи
<code>name</code>	String	Название задачи
<code>description</code>	String	Описание задачи
<code>include</code>	Array	Признак включения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов

Поле	Тип данных	Описание
<code>exclude</code>	Array	Признак исключения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
<code>isFqdnPriority</code>	Bool	Является ли сканирование сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу
<code>agents</code>	Array	Коллекторы и их компоненты, которые выбираются для выполнения задачи (расширенный вариант): — <code>agents</code> — коллекторы: • <code>id</code> — идентификаторы коллекторов; • <code>name</code> — названия коллекторов — <code>components</code> — принадлежность заданных для выполнения задачи коллекторов к конкретному конвейеру или ядру: • <code>siemIds</code> — конвейеры, коллекторы которых можно использовать для выполнения подзадачи; • <code>useCoreAgents</code> — возможно ли использовать коллекторы ядра для выполнения подзадачи
<code>module</code>	Array	Модуль: — <code>id</code> — идентификатор модуля; — <code>name</code> — название модуля
<code>scope</code>	Array	Задача по инфраструктуре: — <code>id</code> — идентификатор задачи; — <code>name</code> — название задачи

Поле	Тип данных	Описание
profile	Array	Профиль сканирования: — id — идентификатор профиля; — name — название профиля
completed	String	Дата завершения задачи
status	String	Статус задачи сканирования
created	String	Дата создания задачи
lastRun	String	Дата последнего запуска
hasBookmarks	Bool	Сохранено ли состояние источника
lastRunErrorLevel	String	Уровень ошибки последнего запуска
validationState	String (Enum)	Состояние валидности
hostDiscovery	Array	Признак обнаружения узлов до начала сбора данных: — enabled — признак сканирования отвечающих узлов; — profile — профиль: • id — идентификатор профиля; • name — название профиля
credentials	Array	Учетные записи: — id — идентификатор учетной записи — name — название учетной записи
triggerParameters	Array	Базовые параметры срабатывания по таймеру: — type — тип; — fromDate — дата начала срабатывания; — toDate — дата окончания срабатывания; — isEnabled — разрешено ли срабатывание по таймеру
overrides	Array	Переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: additionalProp1, additionalProp2, additionalProp3

Возможные коды ошибок и их значения:

- 404 (Not Found) — учетная запись с таким идентификатором не найдена.

16.13.10. Обновление задачи

Запрос для обновления задачи с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 268. Параметры в теле запроса /api/scanning/v3/scanner_tasks/{id}

Параметр	Обязательный	Тип данных	Описание
name	Да	String	Название задачи
description	Нет	String	Описание задачи
component	Нет	Object	Компонент
component → type	Да	String	Тип компонента
agent	Нет	String	Идентификатор коллектора
scope	Да	Object	Параметры инфраструктуры
scope → id	Да	String	Идентификатор инфраструктуры
scope → name	Да	String	Название инфраструктуры
profile	Да	Object	Параметры основного профиля сканирования
profile → id	Да	String	Идентификатор профиля сканирования
profile → name	Да	String	Название профиля сканирования
include	Да	Object	Цели сбора данных
include → assets	Да	Array	Активы
include → assets → id	Да	String	Идентификатор актива
include → targets	Да	Array[String]	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
include → assetsGroups	Да	Array	Группы активов

Параметр	Обязательный	Тип данных	Описание
include → assetsGroups → id	Да	String	Идентификатор группы активов
exclude	Да	Object	Исключения из целей сбора данных
exclude → assets	Да	Array	Активы
exclude → assets → id	Да	String	Идентификатор актива
exclude → targets	Да	Array[String]	IP-адреса, FQDN или маски подсетей конкретных сетевых адресов
exclude → assetsGroups	Да	Array	Группы активов
exclude → assetsGroups → id	Да	String	Идентификатор группы активов
hostDiscovery	Да	Object	Признак обнаружения узлов до начала сбора данных
hostDiscovery → enabled	Да	Bool	Признак сканирования отвечающих узлов
hostDiscovery → profile	Нет	String	Идентификатор профиля
triggerParameters	Да	Object	Базовые параметры срабатывания по таймеру
triggerParameters → type	Да	String	Тип
triggerParameters → fromDate	Нет	String	Дата начала срабатывания
triggerParameters → toDate	Нет	String	Дата окончания срабатывания
triggerParameters → isEnabled	Да	Bool	Разрешено ли срабатывание по таймеру
isFqdnPriority	Нет	Bool	Является ли сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу

Параметр	Обязательный	Тип данных	Описание
<code>overrides</code>	Нет	Object	Переопределение параметров профиля
<code>savepoints</code>	Нет	Array	Точки сохранения
<code>savepoints → target</code>	Да	String	Цель задачи сканирования
<code>savepoints → savepoint</code>	Да	String	Точка сохранения в формате base64
<code>deniedScanSettings</code>	Нет	Object	Параметры запрещенных интервалов для задачи
<code>deniedScanSettings → isEnabled</code>	Да	Bool	Признак включения приостановки задач во время действия запрещенного интервала сканирования
<code>deniedScanSettings → periods</code>	Нет	Array	Список запрещенных интервалов сканирования
<code>deniedScanSettings → periods → daysOfWeek</code>	Да	Array[String]	Дни недели, в которые действует запрещенный интервал сканирования: — <code>monday</code> — понедельник; — <code>tuesday</code> — вторник; — <code>wednesday</code> — среда; — <code>thursday</code> — четверг; — <code>friday</code> — пятница; — <code>saturday</code> — суббота; — <code>sunday</code> — воскресенье
<code>deniedScanSettings → periods → start</code>	Да	String	Время начала запрещенного интервала сканирования в формате hh:mm:ss
<code>deniedScanSettings → periods → end</code>	Да	String	Время окончания запрещенного интервала сканирования в формате hh:mm:ss
<code>deniedScanSettings → periods → timeZone</code>	Да	String	Часовой пояс в формате +hh:mm или -hh:mm

Параметр	Обязательный	Тип данных	Описание
deniedScanSettings → periods → isWholeDayChosen	Да	Bool	Признак того, что запрещенный период сканирования действует целый день
groups	Нет	Array	Группы, в которые входит задача
groups → id	Да	String	Идентификатор группы задач

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 269. Поля ответа на запрос /api/scanning/v3/scanner_tasks/{id}

Поле	Тип данных	Описание
id	String	Идентификатор задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 270. Поля ответа на запрос /api/scanning/v3/scanner_tasks/{id}

Поле	Тип данных	Описание
errors	Array	Ошибки
errors → error	Object	Ошибка
errors → error → type	String	Тип ошибки
errors → error → category	String	Категория ошибки: — target — цель сканирования; — scope — задача по инфраструктуре; — profile — профиль; — task — задача; — agent — коллектор;

Поле	Тип данных	Описание
		— <code>migration</code> — миграция; — <code>taskgroups</code> — группа задач
<code>errors → source</code>	Object	Источник ошибки
<code>errors → source → displayName</code>	String	Отображаемое имя
<code>errors → source → hostName</code>	String	Имя узла
<code>errors → source → ipAddresses</code>	Array[String]	IP-адреса
<code>errors → sensitive</code>	Bool	Признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.13.11. Обновление задачи (API версии 4)

Запрос для обновления задачи с помощью API версии 4.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
PUT <Корневой URL API>/api/scanning/v4/scanner_tasks/{id}
```

URL запроса содержит path-параметр `id` — идентификатор задачи.

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 271. Параметры в теле запроса `/api/scanning/v4/scanner_tasks/{id}`

Параметр	Обязательный	Тип данных	Описание
<code>ScannerTaskParam</code>	Нет	Object	Команда обновления
Параметры задачи			
<code>name</code>	Да	String	Название задачи
<code>description</code>	Нет	String	Описание задачи

Параметр	Обязательный	Тип данных	Описание
agents	Нет	Array	Коллекторы и их компоненты, которые выбираются для выполнения задачи (расширенный вариант): — <code>agentIds</code> — идентификаторы коллекторов; — <code>components</code> — принадлежность заданных коллекторов для выполнения задачи к конкретному конвейеру или ядру: • <code>siemIds</code> — конвейеры, коллекторы которых можно использовать для выполнения подзадачи; • <code>useCoreAgents</code> — возможно ли использовать коллекторы ядра для выполнения подзадачи
scope	Да	String	Задача по инфраструктуре
profile	Да	String	Основной профиль сканирования
include	Да	Array	Признак включения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов

Параметр	Обязательный	Тип данных	Описание
exclude	Да	Array	Признак исключения целей сбора данных: — <code>assets</code> — активы: • <code>id</code> — идентификатор актива; • <code>name</code> — имя актива; — <code>targets</code> — IP-адреса, FQDN или маски подсетей конкретных сетевых адресов; — <code>assetsGroups</code> — группы активов: • <code>id</code> — идентификатор группы активов; • <code>name</code> — название группы активов
hostDiscovery	Да	Array	Признак обнаружения узлов до начала сбора данных: — <code>enabled</code> — сканировать ли отвечающие узлы; — <code>profile</code> — профиль: • <code>id</code> — идентификатор профиля; • <code>name</code> — название профиля
triggerParameters	Да	Array	Базовые параметры срабатывания по таймеру: — <code>type</code> — тип срабатывания; — <code>fromDate</code> — дата начала срабатывания; — <code>toDate</code> — дата окончания срабатывания; — <code>isEnabled</code> — разрешено ли срабатывание по таймеру
isFqdnPriority	Нет	Bool	Является ли сканирование активов по FQDN более приоритетным, чем сканирование по IP-адресу
overrides	Нет	Array	Переопределение параметров профиля. Каждая строка представлена массивом значений столбцов строки в порядке, объявленном в схеме: <code>additionalProp1</code> , <code>additionalProp2</code> , <code>additionalProp3</code>
savepoints	Нет	Array	Точки сохранения: — <code>target</code> — цель задачи сканирования; — <code>savepoint</code> — точка сохранения

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 272. Поля ответа на запрос `/api/scanning/v4/scanner_tasks/{id}`

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 273. Поля ответа на запрос `/api/scanning/v4/scanner_tasks/create`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; • <code>category</code> — категория ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>message</code>	String	Сообщение
<code>code</code>	Integer	Код ошибки

16.13.12. Удаление задачи

Запрос для удаления задачи с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}
```


URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (Успешное удаление).

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе.

16.13.13. Валидация задачи

Запрос для валидации задачи с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/validation
```

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 274. Поля ответа на запрос `/api/scanning/v3/scanner_tasks/{id}/validation`

Поле	Тип данных	Описание
<code>errors</code>	Array	Ошибки: — <code>error</code> — ошибка: • <code>type</code> — тип ошибки; • <code>category</code> — категория ошибки; — <code>source</code> — источник ошибки: • <code>displayName</code> — отображаемое имя; • <code>hostName</code> — имя узла; • <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки

Поле	Тип данных	Описание
message	String	Сообщение
code	Integer	Код ошибки

16.13.14. Сброс закладок для задачи сканирования

Запрос для сброса закладок для задачи сканирования с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/cleanBookmarks

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (Успешный сброс).

16.13.15. Запуск задачи сканирования

Запрос для запуска задачи сканирования с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/start

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 275. Поля ответа на запрос /api/scanning/v3/scanner_tasks/{id}/start

Поле	Тип данных	Описание
id	String	Идентификатор задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 276. Поля ответа на запрос `/api/scanning/v3/scanner_tasks/{id}/start`

Поле	Тип данных	Описание
errors	Array	Ошибки: — error — ошибка: • type — тип ошибки; • category — категория ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.13.16. Остановка задачи

Запрос для остановки задачи сканирования с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/stop

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (Успешная остановка).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.13.17. Приостановка задачи

Запрос для приостановки задачи сканирования с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/suspend
```

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.13.18. Удаление из базы ошибок, возникших при миграции задачи

Запрос для удаления из базы ошибок, возникших при миграции задачи с помощью API версии 3.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/scanning/v3/scanner_tasks/{id}/migration_errors
```

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

16.13.19. Получение времени следующего запуска задачи (API версии 4)

Запрос для получения времени следующего запуска задачи с помощью API версии 4.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/scanning/v4/scanner_tasks/next_run
```

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 277. Параметры в теле запроса `/api/scanning/v4/scanner_tasks/next_run`

Параметр	Обязательный	Тип данных	Описание
<code>schedulerParameters</code>	Да	Object	Базовые параметры срабатывания по таймеру: — <code>type</code> — тип срабатывания; — <code>fromDate</code> — дата начала срабатывания; — <code>toDate</code> — дата завершения; — <code>isEnabled</code> — — включен ли запуск задачи по расписанию

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 278. Поля ответа на запрос `/api/scanning/v4/scanner_tasks/next_run`

Поле	Тип данных	Описание
<code>time</code>	String	Время запуска

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе. Ответ может содержать поля, описанные в таблице ниже.

Таблица 279. Поля ответа на запрос /api/scanning/v4/scanner_tasks/next_run

Поле	Тип данных	Описание
errors	Array	Ошибки: — error — ошибка: • type — тип ошибки; • category — категория ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
message	String	Сообщение
code	Integer	Код ошибки

16.14. Мониторинг задач сканирования

С помощью запросов к API вы можете получать историю запусков задачи, получать список подзадач и информацию об используемых в задаче коллекторах, получать информацию о подзадаче и ошибках при ее выполнении.

В этом разделе

[Получение истории запусков задачи \(см. раздел 16.14.1\)](#)

[Получение списка подзадач в составе задачи \(см. раздел 16.14.2\)](#)

[Получение информации о подзадаче \(см. раздел 16.14.3\)](#)

[Получение ошибок, произошедших при выполнении подзадачи \(см. раздел 16.14.4\)](#)

[Получение информации об используемых в задаче коллекторах \(см. раздел 16.14.5\)](#)

16.14.1. Получение истории запусков задачи

Запрос для получения истории запусков задачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v2/scanner_tasks/{taskId}/runs
```

URL запроса содержит path-параметр `taskId` — идентификатор задачи.

Параметры строки запроса описаны в таблице ниже.

Таблица 280. Параметры строки запроса `/api/scanning/v2/scanner_tasks/{taskId}/runs`

Параметр	Обязательный	Тип данных	Описание
<code>withErrors</code>	Нет	Bool	Применять ли значение «с ошибками» в фильтре
<code>timeFrom</code>	Нет	String (time-date)	Начало периода в фильтре в формате ISO 8601
<code>timeTo</code>	Нет	String (time-date)	Время завершения периода в фильтре в формате ISO 8601
<code>offset</code>	Нет	Integer	Смещение от начала результатов выборки
<code>limit</code>	Нет	Integer	Количество запрашиваемых в выборке элементов

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 281. Поля ответа на запрос `/api/scanning/v2/scanner_tasks/{taskId}/runs`

Поле	Тип данных	Описание
Поля с информацией о запусках задачи		
<code>items</code>	String	Список запусков задачи
<code>totalItems</code>	Integer	Общее количество запусков, попадающих в выборку
Параметры списка запусков задачи		
<code>id</code>	String	Идентификатор запуска
<code>status</code>	Array	Статусы запусков задачи
<code>startedAt</code>	String (time-date)	Время начала запуска в формате ISO 8601
<code>finishedAt</code>	String (time-date)	Время завершения запуска в формате ISO 8601
<code>jobCount</code>	Integer	Количество подзадач в запуске
<code>triggerType</code>	String	Способ запуска: вручную или по расписанию

Поле	Тип данных	Описание
startedBy	Array	Информация о пользователе, запустившем задачу: — <code>id</code> — идентификатор пользователя; — <code>login</code> — логин учетной записи; — <code>firstName</code> — имя; — <code>lastName</code> — фамилия
stoppedBy	Array	Информация о пользователе, остановившем задачу: — <code>id</code> — идентификатор пользователя; — <code>login</code> — логин учетной записи; — <code>firstName</code> — имя; — <code>lastName</code> — фамилия
errorStatus	Array	Статус ошибки запуска задачи

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе.

16.14.2. Получение списка подзадач в составе задачи

Запрос для получения списка подзадач в составе задачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/scanning/v2/runs/{runId}/jobs

URL запроса содержит path-параметр `runId` — идентификатор запуска задачи.

Параметры строки запроса описаны в таблице ниже.

Таблица 282. Параметры строки запроса /api/scanning/v2/runs/{runId}/jobs

Параметр	Обязательный	Тип данных	Описание
status	Нет	String	Фильтр по статусам
errorStatus	Нет	Array	Статус ошибки выполнения подзадачи
agentId	Нет	String	Фильтр по коллектору
targetPattern	Нет	String	Поиск по целям

Параметр	Обязательный	Тип данных	Описание
offset	Нет	Integer	Смещение от начала результатов выборки
limit	Нет	Integer	Количество запрашиваемых в выборке элементов
orderby	Нет	String	Сортировка по колонкам <code>startedAt</code> или <code>finishedAt</code> в направлении <code>asc</code> или <code>desc</code> . Пример значения: <code>orderby="startedAt desc"</code>

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 283. Поля ответа на запрос `/api/scanning/v2/runs/{runId}/jobs`

Поле	Тип данных	Описание
Поля списка подзадач		
items	String	Список подзадач
totalItems	Integer	Общее количество подзадач, попадающих в выборку
Параметры списка подзадач		
id	String	Идентификатор подзадачи
status	Array	Статусы подзадач
errorStatus	Array	Статус ошибки выполнения подзадачи
startedAt	String (time-date)	Время начала подзадачи в формате ISO 8601
finishedAt	String (time-date)	Время завершения подзадачи в формате ISO 8601
agent	Array	Информация о коллекторе: — <code>id</code> — идентификатор коллектора; — <code>name</code> — наименование; — <code>status</code> — статус
targets	String	Список целей сканирования
profile	Array	Профиль подзадачи: — <code>id</code> — идентификатор профиля; — <code>name</code> — наименование профиля

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе.

16.14.3. Получение информации о подзадаче

Запрос для получения информации о подзадаче.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Параметры строки запроса описаны в таблице ниже.

GET <Корневой URL API>/api/scanning/v2/jobs/{jobId}

URL запроса содержит path-параметр `jobId` — идентификатор подзадачи.

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 284. Поля ответа на запрос /api/scanning/v2/jobs/{jobId}

Поле	Тип данных	Описание
<code>id</code>	String	Идентификатор подзадачи
<code>status</code>	Array	Статусы подзадач
<code>errorStatus</code>	Array	Статус ошибки выполнения подзадачи
<code>startedAt</code>	String (time-date)	Время начала подзадачи в формате ISO 8601
<code>finishedAt</code>	String (time-date)	Время завершения подзадачи в формате ISO 8601
<code>agent</code>	Array	Информация о коллекторе: — <code>id</code> — идентификатор коллектора; — <code>name</code> — наименование; — <code>status</code> — статус
<code>targets</code>	String	Список целей сканирования
<code>profile</code>	Array	Профиль подзадачи: — <code>id</code> — идентификатор профиля; — <code>name</code> — наименование профиля

Возможные коды ошибок и их значения:

— 400 (Bad request) — синтаксическая ошибка в запросе.

16.14.4. Получение ошибок, произошедших при выполнении подзадачи

Запрос для получения ошибок, произошедших при выполнении подзадачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Параметры строки запроса описаны в таблице ниже.

GET <Корневой URL API>/api/scanning/v2/jobs/{jobId}/job_errors

URL запроса содержит path-параметр `jobId` — идентификатор подзадачи.

Параметры строки запроса описаны в таблице ниже.

Таблица 285. Параметры строки запроса /api/scanning/v2/jobs/{jobId}/job_errors

Параметр	Обязательный	Тип данных	Описание
offset	Нет	Integer	Смещение от начала результатов выборки
limit	Нет	Integer	Количество запрашиваемых в выборке элементов
orderby	Нет	String	Сортировка по колонке <code>occurredAt</code> в направлении <code>asc</code> или <code>desc</code> . Пример значения: <code>orderby="startedAt desc"</code>

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 286. Поля ответа на запрос /api/scanning/v2/jobs/{jobId}/job_errors

Поле	Тип данных	Описание
Поля списка ошибок подзадач		
items	String	Список ошибок
totalItems	Integer	Общее количество ошибок, попадающих в выборку
logsDownloadUnavailable	Bool	Возможно ли скачать журнал подзадачи с помощью коллектора
Параметры списка ошибок подзадач		
occurredAt	String	Время ошибки в формате ISO 8601
sourceName	String	Псевдоним источника ошибки
type	String	Тип ошибки, идентификатор шаблона для вывода
parameters	Array	Параметры для подстановки в шаблон

Поле	Тип данных	Описание
status	String	Статус ошибки

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе.

16.14.5. Получение информации об используемых в задаче коллекторах

Запрос для получения информации об используемых в задаче коллекторах.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Параметры строки запроса описаны в таблице ниже.

GET <Корневой URL API>/api/scanning/v2/runs/{runId}/agents

URL запроса содержит path-параметр `runId` — идентификатор запуска задачи.

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 287. Поля ответа на запрос /api/scanning/v2/runs/{runId}/agents

Поле	Тип данных	Описание
id	String	Идентификатор коллектора
name	String	Наименование коллектора
status	String	Статус коллектора

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе.

16.15. Журналирование действий пользователя

С помощью запросов к API вы можете получать отчеты о действиях пользователей (в том числе по фильтру), дерево категорий действий, возможные значения дополнительного поля, описания всех дополнительных полей для всех приложений, а также регистрировать события пользователя.

В этом разделе

[Получение дерева категорий действий \(см. раздел 16.15.1\)](#)

[Получение возможного значения дополнительного поля \(см. раздел 16.15.2\)](#)

[Получение описания всех дополнительных полей для всех приложений \(см. раздел 16.15.3\)](#)

[Получение отчетов о действиях пользователей \(см. раздел 16.15.4\)](#)

[Получение отчетов о действиях пользователей по фильтру \(см. раздел 16.15.5\)](#)

[Регистрация событий пользователя \(см. раздел 16.15.6\)](#)

16.15.1. Получение дерева категорий действий

Запрос для получения дерева категорий действий.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>:3334/ptms/api/ual/v2/action_categories
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (Success). Ответ может содержать поля, описанные в таблице ниже.

Таблица 288. Поля ответа на запрос /api/ual/v2/action_categories

Поле	Тип данных	Описание
domains	Array	Домены объектов
Параметры доменов объектов		
id	String	Идентификатор домена объекта
name	String	Название домена объекта
applicationId	UUID	Идентификатор приложения
applicationName	String	Название приложения
types	Array	Тип объекта
Параметры типа объекта		
id	String	Идентификатор типа объекта
name	String	Название типа объекта

16.15.2. Получение возможного значения дополнительного поля

Запрос для получения возможного значения дополнительного поля.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>:3334/ptms/api/ual/v2/user_actions/applications/{applicationId}/field_values/{key}
```

URL запроса содержит path-параметры: `applicationId` — идентификатор приложения и `key` — ключ поля.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (Success). Ответ может содержать поля, описанные в таблице ниже.

Таблица 289. Поля ответа на запрос `/api/ual/v2/user_actions/applications/{applicationId}/field_values/{key}`

Поле	Тип данных	Описание
<code>type</code>	Array	Тип поля
Параметры типа поля		
<code>value</code>	String	Значение
<code>relatedKeyValue</code>	String	Значение поля, к которому относится данное поле

16.15.3. Получение описания всех дополнительных полей для всех приложений

Запрос для получения описания всех дополнительных полей для всех приложений.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>:3334/ptms/api/ual/v2/user_actions/additional_fields
```

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (Success). Ответ может содержать поля, описанные в таблице ниже.

Таблица 290. Поля ответа на запрос `/api/ual/v2/user_actions/applications/{applicationId}/field_values/{key}`

Поле	Тип данных	Описание
type	Array	Тип поля
Параметры типа поля		
applicationId	String	Идентификатор приложения
fieldDefinitions	Array	Список описаний дополнительных полей
Параметры списка описаний дополнительных полей		
key	String	Ключ
localizedName	String	Локализованное название
isHidden	String	Признак сокрытия поля для отображения в списке дополнительных полей. Значение по умолчанию — <code>false</code>
useForFullTextSearch	Bool	Используются ли значения в полнотекстовом поиске
filter	Array	Описание доступного фильтра по данному полю
Параметры фильтра		
type	String	Дискриминатор типа фильтра
selectAllLocalizedName	String	Локализованное название строки «Выбрать все» при открытии фильтра

16.15.4. Получение отчетов о действиях пользователей

Запрос для получения отчетов о действиях пользователей.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>:3334/ptms/api/ual/v2/user_actions
```

Параметры строки запроса описаны в таблице ниже.

Таблица 291. Параметры строки запроса /api/ual/v2/user_actions

Параметр	Обязательный	Тип данных	Описание
userId	Да	String	Идентификатор пользователя для фильтрации. Примечание. Вы можете узнать идентификатор пользователя в приложении РТ МС. Для этого необходимо в главном меню приложения выбрать Пользователи → Пользователи и выбрать учетную запись пользователя. Идентификатор вида XX-XXXXXX-XXXX-XXXX-XXXXXXXXXXXX будет указан в адресной строке браузера в качестве значения параметра userId
category	Нет	String	Категория в формате applicationId.domainId.typeId
orderby	Нет	String	Сортировка, поддерживается только по времени. Пример — orderby=\"time desc\"
query	Нет	String	Строка для поиска, введенная пользователем
limit	Да	Integer (int32)	Количество запрашиваемых действий. Должен быть больше 0 и меньше 1000
offset	Нет	Integer (int32)	Смещение относительно начала списка действий
timeFrom	Нет	Long (int64)	Начало периода фильтрации (Unix time). Значение по умолчанию — 0
timeTo	Нет	Long (int64)	Конец периода фильтрации (Unix time). Значение по умолчанию — текущее время

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (Success). Ответ может содержать поля, описанные в таблице ниже.

Таблица 292. Поля ответа на запрос /api/ual/v2/user_actions

Поле	Тип данных	Описание
id	UUID	Идентификатор действия
beginDateTime	Date (date-time)	Время действия в формате ISO 8601
beginTime	Long (int64)	Время действия (Unix time)

Поле	Тип данных	Описание
beginUtcOffset	String	Сдвиг временной зоны действия относительно UTC
duration	String	Длительность
userId	String	Идентификатор пользователя
userLogin	String	Отображаемое имя пользователя
userDomain	String	Домен пользователя
objectDomainId	String	Идентификатор домена объекта
objectDomain	String	Домен объекта
objectTypeId	String	Идентификатор типа объекта
objectType	String	Тип объекта
objectId	String	Идентификатор объекта
objectDisplayName	String	Отображаемое имя объекта
applicationId	String	Идентификатор приложения
applicationName	String	Название приложения
code	String	Код действия
result	String	Результат действия
failReason	String	Причина ошибки
details	String	Подробное описание изменений
additionalFields	Object	Значение дополнительных полей событий

Пример

Запрос:

```
GET https://localhost:3334/ptms/api/ual/v2/user_actions?
userId=c3e157b1-329a-45ed-9b1b-f3b5da1b7dc4&limit=5&offset=50
```

16.15.5. Получение отчетов о действиях пользователей по фильтру

Запрос для получения отчетов о действиях пользователей по фильтру.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>:3334/ptms/api/uai/v2/user_actions

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 293. Параметры в теле запроса /api/uai/v2/user_actions

Параметр	Обязательный	Тип данных	Описание
filter	Да	UserActionFilter	Фильтр сбора событий
Параметры фильтра			
userIds	Нет	Array [String]	Массив идентификаторов пользователей
categories	Нет	Array [String]	Массив категорий в формате applicationId.domainId.typeId
orderby	Нет	String	Сортировка, поддерживается только по времени. Пример – orderby=\"time desc\"
query	Нет	String	Строка для поиска, введенная пользователем
timeFrom	Нет	Date (date-time)	Дата начала периода фильтрации
timeTo	Нет	Date (date-time)	Дата конца периода фильтрации
additionalFields	Нет	Array	Фильтры по дополнительным полям
Параметры фильтра по дополнительному полю			
key	Да	String	Ключ дополнительного поля
value	Да	String	Значение для поиска

Параметры строки запроса описаны в таблице ниже.

Таблица 294. Параметры URL запроса /api/uai/v2/user_actions

Параметр	Обязательный	Тип данных	Описание
limit	Да	Integer	Количество запрашиваемых действий
offset	Нет	Integer	Смещение относительно начала списка действий

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (Success). Ответ может содержать поля, описанные в таблице ниже.

Таблица 295. Поля ответа на запрос `/api/ual/v2/user_actions`

Поле	Тип данных	Описание
<code>totalItems</code>	Integer	Общее количество событий, удовлетворяющих условиям запроса
<code>items</code>	Array	Список действий пользователя
Параметры списка действий пользователя		
<code>id</code>	UUID	Идентификатор действия
<code>beginDateTime</code>	Date (date-time)	Время действия в формате ISO 8601
<code>beginTime</code>	Long (int64)	Время действия (Unix time)
<code>beginUtcOffset</code>	String	Сдвиг временной зоны действия от UTC
<code>duration</code>	String	Длительность
<code>userId</code>	String	Идентификатор пользователя
<code>userLogin</code>	String	Отображаемое имя пользователя
<code>userDomain</code>	String	Домен пользователя
<code>objectDomainId</code>	String	Идентификатор домена объекта
<code>objectDomain</code>	String	Домен объекта
<code>objectTypeId</code>	String	Идентификатор типа объекта
<code>objectType</code>	String	Тип объекта
<code>objectId</code>	String	Идентификатор объекта
<code>objectDisplayName</code>	String	Отображаемое имя объекта
<code>applicationId</code>	String	Идентификатор приложения
<code>applicationName</code>	String	Название приложения
<code>code</code>	String	Код действия
<code>result</code>	String	Результат действия
<code>failReason</code>	String	Причина ошибки
<code>details</code>	String	Подробное описание изменений

Поле	Тип данных	Описание
<code>additionalFields</code>	Object	Значение дополнительных полей событий

16.15.6. Регистрация событий пользователя

Запрос для регистрации событий пользователя.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>:3334/ptms/api/ual/v2/user_events

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 296. Параметры в теле запроса /api/ual/v2/user_events

Параметр	Обязательный	Тип данных	Описание
<code>actions</code>	Да	Array	Список событий пользователя
Параметры списка событий пользователя			
<code>id</code>	Да	UUID	Идентификатор действия
<code>time</code>	Да	Date (date-time)	Время действия
<code>type</code>	Да	String	Тип события
<code>utcOffset</code>	Да	String	Сдвиг временной зоны действия от UTC

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created).

16.16. Работа с активами

С помощью запросов к API вы можете сохранить статус завершенной задачи, отменить задачу обновления паспорта активов, запускать задачи на удаление активов, изменение расположения активов в группах, обновление паспортов активов, а также получать состояние этих задач.

В этом разделе

[Запуск задачи на удаление активов \(см. раздел 16.16.1\)](#)

[Получение состояния задачи по удалению активов \(см. раздел 16.16.2\)](#)

[Запуск задачи на изменение расположения активов в группах \(см. раздел 16.16.3\)](#)

[Получение состояния задачи по обновлению расположения активов в группах \(см. раздел 16.16.4\)](#)

[Запуск задачи на обновление паспортов активов \(см. раздел 16.16.5\)](#)

[Получение состояния задачи по обновлению паспорта активов \(см. раздел 16.16.6\)](#)

[Отмена задачи обновления паспорта активов \(см. раздел 16.16.7\)](#)

[Сохранение статуса завершенной задачи \(см. раздел 16.16.8\)](#)

16.16.1. Запуск задачи на удаление активов

Запрос для запуска задачи на удаление активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/assets_processing/v1/asset_operations/removeAssets

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 297. Параметры в теле запроса /api/assets_processing/v1/asset_operations/removeAssets

Параметр	Обязательный	Тип данных	Описание
assetsIds	Да	String	Идентификаторы удаляемых активов

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created. Возвращаем token для получения состояния задачи). Ответ может содержать поля, описанные в таблице ниже.

Таблица 298. Поля ответа на запрос /api/assets_processing/v1/asset_operations/removeAssets

Поле	Тип данных	Описание
operationId	String	Идентификатор операции

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 503 (ServiceUnavailable) — ошибка доступа.

16.16.2. Получение состояния задачи по удалению активов

Запрос для получения состояния задачи по удалению активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_processing/v1/asset_operations/removeAssets

Параметры строки запроса описаны в таблице ниже.

Таблица 299. Параметры строки запроса /api/assets_processing/v1/asset_operations/removeAssets

Параметр	Обязательный	Тип данных	Описание
operationId	Да	String	Идентификатор задачи

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 300. Поля ответа на запрос /api/assets_processing/v1/asset_operations/removeAssets

Поле	Тип данных	Описание
type	String	Тип
totalCount	Integer	Общее количество активов
succeedCount	Integer	Количество успешных удалений активов
failedCount	Integer	Количество неуспешных удалений активов

Поле	Тип данных	Описание
errorModel	Integer	Модель ошибок: — errors — ошибки: - error — ошибка: type — тип ошибки; - source — источник; - sensitive — признак значимости ошибки; — message — сообщение; — code — код ошибки
Параметры источника (source)		
displayName	String	Отображаемое имя
hostName	String	Имя узла
ipAddresses	String	IP-адрес

Возможные коды ошибок и их значения:

- 202 (Accepted) — задача еще выполняется;
- 400 (Bad request) — синтаксическая ошибка в запросе.

16.16.3. Запуск задачи на изменение расположения активов в группах

Запрос для запуска задачи на изменение расположения активов в группах.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/assets_processing/v1/asset_operations/updateGroupEntries

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 301. Параметры в теле запроса /api/assets_processing/v1/asset_operations/updateGroupEntries

Параметр	Обязательный	Тип данных	Описание
assetsIds	Да	String	Идентификаторы активов
includeInGroups	Да	String	Включенные в группы активы

Параметр	Обязательный	Тип данных	Описание
excludeFromGroups	Да	String	Исключенные из групп активы

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created. Возвращаем token для получения состояния задачи). Ответ может содержать поля, описанные в таблице ниже.

Таблица 302. Поля ответа на запрос `/api/assets_processing/v1/asset_operations/updateGroupEntries`

Поле	Тип данных	Описание
operationId	String	Идентификатор операции

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 503 (Service Unavailable) — ошибка доступа.

16.16.4. Получение состояния задачи по обновлению расположения активов в группах

Запрос для получения состояния задачи по обновлению расположения активов в группах.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_processing/v1/asset_operations/updateGroupEntries

Параметры строки запроса описаны в таблице ниже.

Таблица 303. Параметры строки запроса `/api/assets_processing/v1/asset_operations/updateGroupEntries`

Параметр	Обязательный	Тип данных	Описание
operationId	Да	String	Идентификатор задачи

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 304. Поля ответа на запрос /api/assets_processing/v1/asset_operations/updateGroupEntries

Поле	Тип данных	Описание
type	String	Тип
totalCount	String	Общее количество
succeedCount	String	Количество успешных задач
failedCount	String	Количество неуспешных задач
errorModel	Array	Данные ошибок
errorModel → errors	Array	Название ошибки: — error — ошибка: • type — тип ошибки; — source — источник ошибки: • displayName — отображаемое имя; • hostName — имя узла; • ipAddresses — IP-адрес; — sensitive — признак значимости ошибки
errorModel → message	String	Текст ошибки
errorModel → code	Integer	Код ошибки
updatedGroups	Array	Данные обновленной группы
updatedGroups → id	String	Идентификатор группы
updatedGroups → displayName	String	Название группы

Возможные коды ошибок и их значения:

- 202 (Accepted) — задача еще выполняется;
- 400 (Bad request) — синтаксическая ошибка в запросе.

16.16.5. Запуск задачи на обновление паспортов активов

Запрос для запуска задачи на обновление паспортов активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/assets_processing/v1/asset_operations/updatePassports

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 305. Параметры в теле запроса /api/assets_processing/v1/asset_operations/updatePassports

Параметр	Обязательный	Тип данных	Описание
assetsIds	Да	Array	Идентификаторы активов
changes	Да	Array	Изменение: - ChangeCommand: - changeType — тип изменения
operationId	Нет	UUID	Идентификатор операции

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 201 (Created. Возвращаем token для получения состояния задачи). Ответ может содержать поля, описанные в таблице ниже.

Таблица 306. Поля ответа на запрос /api/assets_processing/v1/asset_operations/updatePassports

Поле	Тип данных	Описание
operationId	String	Идентификатор операции

Возможные коды ошибок и их значения:

- 400 (Bad request) — синтаксическая ошибка в запросе;
- 401 (Unauthorized) — ошибка аутентификации;
- 503 (ServiceUnavailable) — ошибка доступа.

16.16.6. Получение состояния задачи по обновлению паспорта активов

Запрос для получения состояния задачи по обновлению паспорта активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_processing/v1/asset_operations/updatePassports

Параметры строки запроса описаны в таблице ниже.

Таблица 307. Параметры строки запроса `/api/assets_processing/v1/asset_operations/updatePassports`

Параметр	Обязательный	Тип данных	Описание
<code>operationId</code>	Да	String	Идентификатор задачи

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 308. Поля ответа на запрос `/api/assets_processing/v1/asset_operations/updatePassports`

Поле	Тип данных	Описание
<code>type</code>	String	Тип
<code>totalCount</code>	Integer	Общее количество задач
<code>succeedCount</code>	Integer	Количество успешных задач
<code>failedCount</code>	Integer	Количество неуспешных задач
<code>errorModel</code>	Array	Данные ошибок
<code>errorModel → errors</code>	Array	Название ошибки: — <code>error</code> — ошибка: <code>type</code> — тип ошибки; — <code>source</code> — источник ошибки: <code>displayName</code> — отображаемое имя; <code>hostName</code> — имя узла; <code>ipAddresses</code> — IP-адрес; — <code>sensitive</code> — признак значимости ошибки
<code>errorModel → message</code>	String	Текст ошибки
<code>errorModel → code</code>	Integer	Код ошибки

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция еще выполняется;
- 400 (Bad request) — синтаксическая ошибка в запросе.

16.16.7. Отмена задачи обновления паспорта активов

Запрос для отмены задачи по обновлению паспорта активов.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/assets_processing/v1/asset_operations/updatePassports/{operationId}
```

URL запроса содержит path-параметр `operationId` — идентификатор задачи обновления активов.

Параметры в теле запроса отсутствуют.

Ответ на запрос

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция еще выполняется;
- 400 (Bad request) — синтаксическая ошибка в запросе.

16.16.8. Сохранение статуса завершенной задачи

Запрос для сохранения статуса завершенной задачи.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
PUT <Корневой URL API>/api/assets_processing/v1/asset_operations/completed/{operationType}/{operationId}
```

URL запроса содержит path-параметры `operationType` — тип задачи и `operationId` — идентификатор задачи.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.17. Получение метаданных актива

Запрос для получения метаданных актива.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets/metadata/{type}

URL запроса содержит path-параметр `type` — тип модели метаданных.

Параметры строки запроса описаны в таблице ниже.

Таблица 309. Параметры строки запроса /api/assets/metadata/{type}

Параметр	Обязательный	Тип данных	Описание
<code>onlyUserProperties</code>	Нет	Bool	Используются ли только свойства пользователя

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 310. Поля ответа на запрос /api/assets/metadata/{type}

Поле	Тип данных	Описание
<code>name</code>	String	Имя типа модели метаданных
<code>root</code>	String	Корневая группа типа модели метаданных
<code>kind</code>	String	Род типа модели метаданных
<code>title</code>	String	Заголовок типа модели метаданных
<code>description</code>	String	Описание типа модели метаданных
<code>properties</code>	String	Свойства типа модели: — <code>isCollection</code> — является ли свойство коллекцией; — <code>name</code> — имя свойства; — <code>kind</code> — род свойства; — <code>type</code> — тип свойства; — <code>title</code> — заголовок свойства; — <code>description</code> — описание свойства

Возможные коды ошибок и их значения:

- 304 (Not modified) — запрашиваемая страница не обновлялась с момента последнего обращения;
- 400 (Invalid asset type) — неверный тип актива, синтаксическая ошибка в запросе.

16.18. Экспорт таблицы активов в CSV

Запрос для экспорта таблицы активов в CSV.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

GET <Корневой URL API>/api/assets_temporal_readmodel/v1/assets_grid/export

Параметры строки запроса описаны в таблице ниже.

Таблица 311. Параметры строки запроса /api/assets_temporal_readmodel/v1/assets_grid/export

Параметр	Обязательный	Тип данных	Описание
pdqlToken	Да	String	Токен (см. раздел 16.3) PDQL-запроса
gridSide	Нет	Enum	Сторона таблицы при наличии группировки по полю

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе;
- 403 (Forbidden) — доступ к запрашиваемой странице запрещен;
- 404 (Not Found) — учетная запись с таким идентификатором не найдена;
- 500 (Internal Server Error) — внутренняя ошибка сервера.

Внимание! При интеграции со другими системами, импортирующими данные из MaxPatrol SIEM, следует перед импортом предусмотреть приведение строковых данных, полученных в результате выполнения запроса, к единому регистру. Импорт данных, не приведенных к единому регистру, может вызвать ошибки и некорректную работу интеграций.

16.19. Массовые операции над задачами

С помощью запросов к API вы можете запускать, останавливать, приостанавливать и удалять выбранные задачи сканирования, устанавливать и сравнивать расписание запуска задач, сбрасывать закладки для задач сканирования, получать информацию о массовой операции над задачами и удалять созданную массовую операцию.

В этом разделе

[Запуск задач сканирования \(см. раздел 16.19.1\)](#)

[Остановка задач сканирования \(см. раздел 16.19.2\)](#)

[Приостановка задач сканирования \(см. раздел 16.19.3\)](#)

[Удаление задач сканирования \(см. раздел 16.19.4\)](#)

[Установка расписания задач сканирования \(см. раздел 16.19.5\)](#)

[Сравнение расписаний запуска задач \(см. раздел 16.19.6\)](#)

[Сброс закладок для задач сканирования \(см. раздел 16.19.7\)](#)

[Получение информации о массовой операции \(см. раздел 16.19.8\)](#)

[Удаление массовой операции \(см. раздел 16.19.9\)](#)

16.19.1. Запуск задач сканирования

Запрос для запуска выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/start_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 312. Параметры строки запроса /api/scanning/v3/task_bulk_operations/start_tasks

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.2. Остановка задач сканирования

Запрос для остановки выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/stop_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 313. Параметры строки запроса /api/scanning/v3/task_bulk_operations/stop tasks

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.3. Приостановка задач сканирования

Запрос для приостановки выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/suspend_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 314. Параметры строки запроса /api/scanning/v3/task_bulk_operations/suspend_tasks

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.4. Удаление задач сканирования

Запрос для удаления выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/delete_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 315. Параметры строки запроса /api/scanning/v3/task_bulk_operations/delete_tasks

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.5. Установка расписания задач сканирования

Запрос для установки расписания выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/update_tasks_schedule

Параметры строки запроса описаны в таблице ниже.

Таблица 316. Параметры строки запроса /api/scanning/v3/task_bulk_operations/update_tasks_schedule

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 317. Параметры в теле запроса /api/scanning/v3/task_bulk_operations/update_tasks_schedule

Параметр	Обязательный	Тип данных	Описание
schedulerParameters → trigger	Нет	Object	Базовые параметры запуска задачи по расписанию: — type — тип запуска; — fromDate — дата запуска задачи по расписанию; — toDate — дата завершения задачи по расписанию; — isEnabled — включен ли запуск задачи по расписанию

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.6. Сравнение расписаний запуска задач

Запрос для сравнения расписания запуска выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST <Корневой URL API>/api/scanning/v3/task_bulk_operations/compare_scheduler_tasks

Параметры строки запроса описаны в таблице ниже.

Таблица 318. Параметры строки запроса `/api/scanning/v3/task_bulk_operations/compare_scheduler_tasks`

Параметр	Обязательный	Тип данных	Описание
<code>selectionId</code>	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 319. Поля ответа на запрос `/api/scanning/v3/task_bulk_operations/compare_scheduler_tasks`

Поле	Тип данных	Описание
<code>triggersAreSame</code>	Bool	Одинаковы ли расписания запуска для задач, выбранных для использования в массовых операциях
<code>schedulersState</code>	String	Состояние расписания всех задач
<code>schedulerParameters → trigger</code>	Object	Базовые параметры запуска задачи по расписанию: — <code>type</code> — тип запуска; — <code>fromDate</code> — дата запуска задачи по расписанию; — <code>toDate</code> — дата завершения задачи по расписанию; — <code>isEnabled</code> — включен ли запуск задачи по расписанию

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.7. Сброс закладок для задач сканирования

Запрос для сброса закладок для выбранных задач сканирования.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

POST `<Корневой URL API>/api/scanning/v3/task_bulk_operations/clean_bookmarks`

Параметры строки запроса описаны в таблице ниже.

Таблица 320. Параметры строки запроса /api/scanning/v3/task_bulk_operations/clean_bookmarks

Параметр	Обязательный	Тип данных	Описание
selectionId	Да	String	Идентификатор выборки задач для использования в массовых операциях

Параметры в теле запроса отсутствуют.

Возможные коды ошибок и их значения:

- 202 (Accepted) — операция выполняется. Ответ содержит идентификатор массовой операции;
- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.8. Получение информации о массовой операции

Запрос для получения информации о массовой операции.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
GET <Корневой URL API>/api/scanning/v3/task_bulk_operations/{id}
```

URL запроса содержит path-параметр `id` — идентификатор массовой операции.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 321. Поля ответа на запрос /api/scanning/v3/task_bulk_operations/{id}

Поле	Тип данных	Описание
status	String	Статус массовой операции

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.19.9. Удаление массовой операции

Запрос для удаления массовой операции.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
DELETE <Корневой URL API>/api/scanning/v3/task_bulk_operations/{id}
```

URL запроса содержит path-параметр `id` — идентификатор массовой операции.

Параметры в теле запроса отсутствуют.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.20. Работа с выбором задач

С помощью запросов к API вы можете выбирать задачи для использования в массовых операциях, изменять параметры выбора, отменить выбор задач, а также получать информацию о количестве выбранных задач и доступных операциях с ними по их идентификатору.

В этом разделе

[Выбор задач для использования в массовых операциях \(см. раздел 16.20.1\)](#)

[Изменение параметров выбора задач \(см. раздел 16.20.2\)](#)

[Получение информации о выбранных задачах по их идентификатору \(см. раздел 16.20.3\)](#)

[Отмена выбора задач \(см. раздел 16.20.4\)](#)

16.20.1. Выбор задач для использования в массовых операциях

Запрос для выбора задач для использования в массовых операциях.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

```
POST <Корневой URL API>/api/scanning/v3/task_selections
```

Параметры строки запроса описаны в таблице ниже.

Таблица 322. Параметры строки запроса /api/scanning/v3/task_selections

Параметр	Обязательный	Тип данных	Описание
mainFilter	Нет	String	Основной фильтр
additionalFilter	Нет	String	Дополнительный фильтр. Значение по умолчанию: <code>all</code>
token	Нет	String	Токен выборки. Возвращает задачи из кэша. Если параметр не задан, задачи возвращаются из базы данных

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 323. Параметры в теле запроса /api/scanning/v3/task_selections

Параметр	Обязательный	Тип данных	Описание
filter	Нет	Object	Параметры фильтра
tasksIds	Да	Array	Список идентификаторов выбранных и невыбранных задач: — <code>allSelected</code> — выбраны ли все задачи; — <code>unselected</code> — список идентификаторов невыбранных задач. Если параметр задан, параметр <code>selected</code> не задается; — <code>selected</code> — список идентификаторов выбранных задач. Если параметр задан, параметр <code>unselected</code> не задается

Параметры фильтра

text	Нет	String	Текст
agents	Нет	Array	Коллектор: — <code>agentIds</code> — идентификаторы коллекторов; — <code>autoSelect</code> — включен ли автовыбор
modules	Нет	String	Модули
profiles	Нет	String	Профили
statuses	Нет	String (Enum)	Статус задачи сканирования
credentials	Нет	String	Учетные записи

Параметр	Обязательный	Тип данных	Описание
target → target	Да	Array	Атакованные активы: — assets — активы: - id — идентификаторы активов; - name — названия активов — targets — атакованные активы; — assetsGroups — группы активов: - id — идентификаторы групп активов; - name — названия групп активов
target → type	Да	String	Тип атакованного актива
scopes	Нет	String	Задача по инфраструктуре

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 324. Поля ответа на запрос /api/scanning/v3/task_selections

Поле	Тип данных	Описание
selectionId	String	Идентификатор выборки
selectedTasksCount	Integer	Количество выбранных задач
availableOperations	Array	Список доступных операций над задачами: — operation — операция над задачей
token	String	Токен

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.20.2. Изменение параметров выбора задач

Запрос для изменения параметров выбора задач.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/scanning/v3/task_selections/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Параметры строки запроса описаны в таблице ниже.

Таблица 325. Параметры строки запроса /api/scanning/v3/task_selections/{id}

Параметр	Обязательный	Тип данных	Описание
<code>mainFilter</code>	Нет	String	Основной фильтр
<code>additionalFilter</code>	Нет	String	Дополнительный фильтр
<code>token</code>	Нет	String	Токен выборки. Возвращает задачи из кэша. Если параметр не задан, задачи возвращаются из базы данных

Тело запроса может содержать параметры, описанные в таблице ниже.

Таблица 326. Параметры в теле запроса /api/scanning/v3/task_selections/{id}

Параметр	Обязательный	Тип данных	Описание
<code>filter</code>	Нет	Object	Параметры фильтра
<code>tasksIds</code>	Да	Array	Список идентификаторов выбранных и невыбранных задач: — <code>allSelected</code> — выбраны ли все задачи; — <code>unselected</code> — список идентификаторов невыбранных задач. Если параметр задан, параметр <code>selected</code> не задается; — <code>selected</code> — список идентификаторов выбранных задач. Если параметр задан, параметр <code>unselected</code> не задается

Параметры фильтра

<code>text</code>	Нет	String	Текст
<code>agents</code>	Нет	Array	Коллектор: — <code>agentIds</code> — идентификаторы коллекторов; — <code>autoSelect</code> — включен ли автовыбор
<code>modules</code>	Нет	String	Модули
<code>profiles</code>	Нет	String	Профили

Параметр	Обязательный	Тип данных	Описание
statuses	Нет	String (Enum)	Статус задачи сканирования
credentials	Нет	String	Учетные записи
target → target	Да	Array	Атакованные активы: — assets — активы: • id — идентификаторы активов; • name — названия активов — targets — атакованные активы; — assetsGroups — группы активов: • id — идентификаторы групп активов; • name — названия групп активов
target → type	Да	String	Тип атакованного актива
scopes	Нет	String	Задача по инфраструктуре

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 327. Поля ответа на запрос /api/scanning/v3/task_selections/{id}

Поле	Тип данных	Описание
selectionId	String	Идентификатор выборки
selectedTasksCount	Integer	Количество выбранных задач
availableOperations	Array	Список доступных операций над задачей: — Operation — операция над задачей
token	String	Токен

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

16.20.3. Получение информации о выбранных задачах по их идентификатору

Запрос для получения информации о количестве выбранных задач и доступных операциях с ними по их идентификатору.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

PUT <Корневой URL API>/api/scanning/v3/task_selections/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 200 (OK). Ответ может содержать поля, описанные в таблице ниже.

Таблица 328. Поля ответа на запрос /api/scanning/v3/task_selections/{id}

Поле	Тип данных	Описание
<code>selectionId</code>	String	Идентификатор выборки
<code>selectedTasksCount</code>	Integer	Количество выбранных задач
<code>availableOperations</code>	Array	Список доступных операций над задачами: — <code>Operation</code> — операция над задачей
<code>token</code>	String	Токен

Возможные коды ошибок и их значения:

— 400 (Bad Request) — синтаксическая ошибка в запросе.

16.20.4. Отмена выбора задач

Запрос для отмены выбора задач.

Для выполнения запроса требуется аутентификация по протоколу OAuth с токеном доступа типа Bearer.

Метод и URL запроса:

DELETE <Корневой URL API>/api/scanning/v3/task_selections/{id}

URL запроса содержит path-параметр `id` — идентификатор задачи.

Ответ на запрос

В ответ на успешный запрос сервис возвращает код 204 (No Content).

Возможные коды ошибок и их значения:

- 400 (Bad Request) — синтаксическая ошибка в запросе.

17. Отправка уведомлений через POST-запрос

В MaxPatrol SIEM предусмотрена возможность отправки уведомлений на внешний сервер через POST-запросы. Вы можете настроить отправку таких уведомлений для следующих случаев:

- изменение состава активов в системе или в выбранной группе;
- получение события, удовлетворяющего выбранному фильтру;
- запуск или остановка задачи сбора данных;
- выход параметров потока событий от источника из выбранной группы за пределы допустимых значений;
- появление уведомления о состоянии системы.

Для приема запросов на сервере должно быть установлено специализированное ПО.

Для отправки уведомлений через POST-запросы в веб-интерфейсе MaxPatrol SIEM в разделе **Система** на странице **Уведомления** нужно создать задачу на отправку уведомлений. В параметрах задачи нужно включить отправку уведомления через POST-запрос и указать URL внешнего сервера. Вы можете настроить отправку мгновенных уведомлений или уведомлений за период времени (см. Руководство администратора).

После создания задачи в случае выполнения условия отправки уведомления MaxPatrol SIEM автоматически собирает данные для уведомления и отправляет POST-запрос на внешний сервер. Специализированное ПО на внешнем сервере принимает POST-запрос и по ссылке в запросе получает из MaxPatrol SIEM данные уведомления.

Для приема POST-запросов и получения данных уведомления к ПО выдвигаются следующие требования:

- поддержка обмена данными по протоколам HTTP или HTTPS;
- прием POST-запросов без аутентификации;
- отправка подтверждения с кодом ответа 2xx при приеме POST-запроса;

Примечание. При отсутствии подтверждения получения запроса от внешнего сервера (с кодом ответа 2xx) POST-запрос будет отправляться повторно до 10 раз с интервалом в минуту.

- получение данных уведомления GET-запросом по ссылке в POST-запросе;
- при получении данных по протоколу HTTPS поддержка проверки подлинности сервера с помощью сертификата SSL.

В этом разделе

[Поля POST-запроса для уведомления \(см. раздел 17.1\)](#)

[Тестирование приема POST-запросов \(см. раздел 17.2\)](#)

17.1. Поля POST-запроса для уведомления

MaxPatrol SIEM отправляет POST-запрос в формате JSON. В зависимости от типа запрос может содержать следующие поля:

- `message_id` — идентификатор POST-запроса.
- `notification_type` — тип POST-запроса: `test` — тестовый запрос, `event` — для мгновенного уведомления, `schedule` — для уведомления за период времени.
- `notification_source` — условие создания уведомления: `AssetStateMetaTrigger` — изменение состава активов в системе; `EventsMetaTrigger` — получение события, удовлетворяющего выбранному фильтру; `EventsMonitoringControlsMetaTrigger` — выход параметров потока событий от источника из выбранной группы за пределы допустимых значений; `GroupContentMetaTrigger` — изменение состава активов в выбранной группе; `HealthMonitoringIssuesMetaTrigger` — появление уведомления о состоянии системы; `ScannerTaskNotificationsMetaTrigger` — запуск или остановка задачи на сбор данных.
- `notification_uid` — идентификатор уведомления.
- `notification_name` — название задачи MaxPatrol SIEM на отправку уведомления.
- `uri` — ссылка на данные уведомления.

Внимание! Данные уведомления доступны по ссылке в течение 24 часов с момента создания уведомления.

- `schema_uri` — ссылка на схему данных уведомления в формате JSON.
- `event_time_stamp` — дата и время создания мгновенного уведомления (UTC+0).
- `time_interval` — для уведомления за период содержит поля `event_time_start` и `event_time_end` с датами и временем начала и конца периода (UTC+0).
- `time_stamp` — дата и время создания POST-запроса (UTC+0).

Пример тестового POST-запроса:

```
{
  "notification_type": "test",
  "notification_source": "AssetStateMetaTrigger",
  "schema_uri": "https://siem-server.ru:8733/api/assets_triggers/v1/triggers_data/meta_triggers/AssetStateMetaTrigger/reactions/webhook_notification/schema",
  "time_stamp": "2019-04-15T11:20:49.7031727Z"
}
```

Пример POST-запроса для уведомления за период времени:

```
{
  "message_id": 1194,
  "notification_type": "schedule",
  "notification_source": "AssetStateMetaTrigger",
  "notification_uid": "110a1602-d980-0001-0000-000000000006",
}
```

```

    "notification_name": "1",
    "uri": "https://siem-server.ru:8733/api/assets_triggers/v1/triggers_data/
meta_triggers/AssetStateMetaTrigger/reactions/webhook_notification/
110a718e7300000100000000000000261",
    "schema_uri": "https://siem-server.ru:8733/api/assets_triggers/v1/triggers_data/
meta_triggers/AssetStateMetaTrigger/reactions/webhook_notification/schema",
    "event_time_stamp": "2019-04-12T09:50:21.4309566Z",
    "time_interval": {
        "event_time_start": "2019-04-12T09:50:21.4309566Z",
        "event_time_end": "2019-04-12T09:55:21.4309566Z"
    },
    "time_stamp": "2019-04-12T09:55:33.0096243Z"
}

```

17.2. Тестирование приема POST-запросов

В качестве примера ПО для приема POST-запросов и получения данных уведомления приводится сценарий на языке Python. В сценарии реализованы следующие возможности:

- При получении POST-запроса в интерфейс командной строки выводится его текст.
- При получении POST-запроса в MaxPatrol SIEM отправляется подтверждение с кодом ответа 2xx.
- По ссылке в POST-запросе из MaxPatrol SIEM скачиваются не более 10 страниц данных уведомления и выводятся в интерфейс командной строки.
- При необходимости по ссылке в POST-запросе из MaxPatrol SIEM скачивается схема данных уведомления и выводится в интерфейс командной строки.
- При получении данных из MaxPatrol SIEM по протоколу HTTPS для проверки подлинности сервера может быть предоставлен сертификат SSL.

Для выполнения сценария на сервере должны быть установлены операционные системы Windows 2012 R2 или Debian 9 и интерпретатор языка Python версии 3.6 или 3.7 с библиотеками Flask и Requests.

Примечание. Для отправки уведомлений на сервер в веб-интерфейсе MaxPatrol SIEM нужно создать задачу на отправку уведомлений через POST-запросы. В параметрах задачи нужно указать URL сервера в виде `http://<IP-адрес> или FQDN>/handle`.

- ▶ Чтобы настроить прием POST-запросов и получение данных уведомления:
 1. Создайте файл с расширением .py и скопируйте в него код сценария.
 2. Если требуется, с помощью параметра `HOST` укажите сетевой интерфейс сервера.
 3. Если требуется выводить в интерфейс командной строки схему данных уведомления, для параметра `SHOW_SCHEMA` укажите значение `True`.
 4. Если требуется, с помощью параметров `HTTP_PORT` и `HTTPS_PORT` измените порты для приема запросов по протоколам HTTP и HTTPS.

5. Если требуется использовать протокол HTTPS, для параметра USE_HTTPS укажите значение True.

Внимание! Для проверки подлинности при получении сценарием данных из MaxPatrol SIEM по протоколу HTTPS нужно выпустить сертификат SSL. Файлы сертификата и ключа к нему нужно поместить в одну папку с файлом сценария. С помощью параметров сценария `certfile` и `keyfile` нужно указать имена файлов сертификата и ключа.

6. Сохраните файл.
7. Откройте интерфейс командной строки и запустите сценарий:
`python <Имя файла сценария>.py`

Прием POST-запросов настроен. Данные уведомлений будут выводиться в интерфейс командной строки.

Код сценария для приема POST-запросов

```
"""pip install flask requests"""
import requests
from pathlib import Path
from flask import Flask, request
HOST = '0.0.0.0'
HTTP_PORT = 10080
HTTPS_PORT = 10443
SHOW_SCHEMA = False
USE_HTTPS = False
cert_dir = Path(__file__).parent
certfile = cert_dir / '<Имя файла сертификата>.crt'
keyfile = cert_dir / '<Имя файла ключа>.pem'
app = Flask('ExampleTriggerService')
@app.route('/handle', methods=['POST'])
def handle():
    data = request.get_json()
    print('INCOMING DATA', data)
    if data['notification_type'] == 'test':
        return print('TEST') or 'test'
    if SHOW_SCHEMA:
        print('GET SCHEMA', data['schema_uri'])
        schema_data = requests.get(data['schema_uri'], verify=False).text
        print('SCHEMA DATA', schema_data)
    page_uri = data['uri']
    for page_num in range(10):
        if not page_uri:
            break
        print('GET PAGE', page_num, page_uri)
        page_data = requests.get(page_uri, verify=False).json()
```

```
        print('PAGE DATA', page_data)
        page_uri = page_data['dataSetInfo']['nextDataSetUri']
    return print('OK') or 'ok'
if __name__ == '__main__':
    if USE_HTTPS:
        import ssl
        context = ssl.SSLContext(ssl.PROTOCOL_TLS)
        # context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2) # для Python версии 3.6
        context.load_cert_chain(str(certfile), str(keyfile))
        port = HTTPS_PORT
    else:
        context = None
        port = HTTP_PORT
    app.run(ssl_context=context, host=HOST, port=port, debug=True)
```


18. О технической поддержке

Гарантийная поддержка доступна для всех владельцев действующих лицензий на MaxPatrol SIEM в течение периода предоставления обновлений и включает в себя следующий набор услуг.

Предоставление доступа к обновленным версиям продукта

Обновления продукта выпускаются в порядке и в сроки, определяемые Positive Technologies. Positive Technologies предоставляет обновленные версии продукта в течение оплаченного периода получения обновлений, указанного в бланке лицензии приобретенного продукта Positive Technologies.

Positive Technologies не производит установку обновлений продукта в рамках гарантийной поддержки и не несет ответственности за инциденты, возникшие в связи с некорректной или несвоевременной установкой обновлений продукта.

Восстановление работоспособности продукта

Специалист Positive Technologies проводит диагностику заявленного сбоя и предоставляет рекомендации для восстановления работоспособности продукта. Восстановление работоспособности может заключаться в выдаче рекомендаций по установке продукта заново с потенциальной потерей накопленных данных либо в восстановлении версии продукта из доступной резервной копии (резервное копирование должно быть настроено заблаговременно). Positive Technologies не несет ответственности за потерю данных в случае неверно настроенного резервного копирования.

Примечание. Помощь оказывается при условии, что конечным пользователем продукта соблюдены все аппаратные, программные и иные требования и ограничения, описанные в документации к продукту.

Устранение ошибок и дефектов в работе продукта в рамках выпуска обновленных версий

Если в результате диагностики обнаружен дефект или ошибка в работе продукта, Positive Technologies обязуется включить исправление в ближайшие возможные обновления продукта (с учетом релизного цикла продукта, приоритета дефекта или ошибки, сложности требуемых изменений, а также экономической целесообразности исправления). Сроки выпуска обновлений продукта остаются на усмотрение Positive Technologies.

Рассмотрение предложений по доработке продукта

При обращении с предложением по доработке продукта необходимо подробно описать цель такой доработки. Вы можете поделиться рекомендациями по улучшению продукта и оптимизации его функциональности. Positive Technologies обязуется рассмотреть все предложения, однако не принимает на себя обязательств по реализации каких-либо

доработок. Если Positive Technologies принимает решение о доработке продукта, то способы и сроки ее реализации остаются на усмотрение Positive Technologies. Заявки принимаются [на портале технической поддержки](#).

Портал технической поддержки

[На портале технической поддержки](#) вы можете круглосуточно создавать и обновлять заявки и читать новости продуктов.

Для получения доступа к portalу технической поддержки нужно создать учетную запись, используя адрес электронной почты в официальном домене вашей организации. Вы можете указать другие адреса электронной почты в качестве дополнительных. Добавьте в профиль название вашей организации и контактный телефон — так нам будет проще с вами связаться.

Поддержка на портале предоставляется на русском и английском языках.

Условия предоставления гарантийной поддержки

Для получения гарантийной поддержки необходимо оставить заявку [на портале технической поддержки](#) и предоставить следующие данные:

- номер лицензии на использование продукта;
- файлы журналов и наборы диагностических данных, которые требуются для анализа;
- снимки экрана.

Positive Technologies не берет на себя обязательств по оказанию услуг гарантийной поддержки в случае вашего отказа предоставить запрашиваемую информацию или отказа от внедрения предоставленных рекомендаций.

Если заказчик не предоставляет необходимую информацию по прошествии 20 календарных дней с момента ее запроса, специалист технической поддержки имеет право закрыть заявку. Оказание услуг может быть возобновлено по вашей инициативе при условии предоставления запрошенной ранее информации.

Услуги по гарантийной поддержке продукта не включают в себя услуги по переустановке продукта, решение проблем с операционной системой, инфраструктурой заказчика или сторонним программным обеспечением.

Заявки могут направлять уполномоченные сотрудники заказчика, владеющего продуктом на законном основании (включая наличие действующей лицензии). Специалисты заказчика должны иметь достаточно знаний и навыков для сбора и предоставления диагностической информации, необходимой для решения заявленной проблемы.

Услуги по гарантийной поддержке оказываются в отношении поддерживаемых версий продукта. Информация о поддерживаемых версиях содержится в «Политике поддержки версий программного обеспечения» и (или) иных информационных материалах, размещенных на официальном веб-сайте Positive Technologies.

Время реакции и приоритизация заявок

При получении заявки ей присваивается регистрационный номер, специалист службы технической поддержки классифицирует ее (присваивает тип и уровень значимости) и выполняет дальнейшие шаги по ее обработке.

Время реакции рассчитывается с момента получения заявки до первичного ответа специалиста технической поддержки с уведомлением о взятии заявки в работу. Время реакции зависит от уровня значимости заявки. Специалист службы технической поддержки оставляет за собой право переопределить уровень значимости в соответствии с приведенными ниже критериями. Указанные сроки являются целевыми, но возможны отклонения от них по объективным причинам.

Таблица 329. Время реакции на заявку

Уровень значимости заявки	Критерии значимости заявки	Время реакции на заявку
Критический	Аварийные сбои, приводящие к невозможности штатной работы продукта (исключая первоначальную установку) либо оказывающие критически значимое влияние на бизнес заказчика	До 4 часов
Высокий	Сбои, проявляющиеся в любых условиях эксплуатации продукта и оказывающие значительное влияние на бизнес заказчика	До 8 часов
Средний	Сбои, проявляющиеся в специфических условиях эксплуатации продукта либо не оказывающие значительного влияния на бизнес заказчика	До 8 часов
Низкий	Вопросы информационного характера либо сбои, не влияющие на эксплуатацию продукта	До 8 часов

Указанные часы относятся к рабочему времени (рабочие дни с 9:00 до 18:00 UTC+3) специалистов технической поддержки. Под рабочим днем понимается любой день за исключением субботы, воскресенья и дней, являющихся официальными нерабочими днями в Российской Федерации.

Обработка и закрытие заявок

По мере рассмотрения заявки и выполнения необходимых действий специалист службы поддержки сообщает вам:

- о ходе диагностики по заявленной проблеме и ее результатах;
- планах выпуска обновленной версии продукта (если требуется для устранения проблемы).

Если по итогам обработки заявки необходимо внести изменения в продукт, Positive Technologies включает работы по исправлению в ближайшее возможное плановое обновление продукта (с учетом релизного цикла продукта, приоритета дефекта или ошибки, сложности требуемых изменений, а также экономической целесообразности исправления). Сроки выпуска обновлений продукта остаются на усмотрение Positive Technologies.

Специалист закрывает заявку, если:

- предоставлены решение или возможность обойти проблему, не влияющие на критически важную функциональность продукта (по усмотрению Positive Technologies);
- диагностирован дефект продукта, собрана техническая информация о дефекте и условиях его воспроизведения, исправление дефекта запланировано к выходу в рамках планового обновления продукта;
- заявленная проблема вызвана программным обеспечением или оборудованием сторонних производителей, на которые не распространяются обязательства Positive Technologies;
- заявленная проблема классифицирована специалистами Positive Technologies как неподдерживаемая.

Примечание. Если продукт приобретался вместе с аппаратной платформой в виде программно-аппаратного комплекса (ПАК), решение заявок, связанных с ограничением или отсутствием работоспособности аппаратных компонентов, происходит согласно условиям и срокам, указанным в гарантийных обязательствах (гарантийных талонах) на такие аппаратные компоненты.

Приложение А. Типы данных

При использовании функций и операторов языка XP выполняется приведение типов данных. Кроме того, нужно учитывать тип данных при заполнении [полей событий \(см. раздел 10.3\)](#).

Таблица 330. Используемые типы данных

Тип	Описание	Пример
Bool	Логическое значение	True, False
Buffer	Массив байтов	[0x68, 0x65, 0x6c, 0x6c, 0x6f, 0x20, 0x77, 0x6f, 0x72, 0x6c, 0x64]
DateTime	Для фильтрации событий и инцидентов время в формате ISO 8601: YYYY-MM-DD'T'HH:MM:SS'Z' Для фильтрации активов и табличных списков время в формате YYYY-MM-DD'T'HH:MM:SS	2016-04-20T23:04:38Z, 2020-07-22T18:08:38
Enum	Один из элементов предопределенного списка значений	—
IPAddress	IP-адрес стандарта IPv4 или IPv6	192.0.2.235, 1080:0:0:0:8:800:200C:4 17A
KeyValue	Ассоциативный массив пар «ключ — значение»	{"Красный": "Каждый", "Оранжевый": "Охотник", "Желтый": "Желает"}
List	Список. Один список может содержать элементы разных типов	["Порт ", 22, " открыт"]
MACAddress	MAC-адрес	00:53:00:B8:DF:B8
Network	Адрес подсети с маской в формате CIDR	192.0.2.0/24
Null	Отсутствие данных	null
Number	Целое число от -9223372036854775808 до 9223372036854775807	-3, 0, 12
String	Строка	"Порт 22 открыт"
StringList	Список, элементы которого имеют тип дан- ных String	["Красный", "Оранже- вый", "Желтый", "Зеле- ный"]
UUID	Идентификатор в формате RFC 4122 — 16- байтный (128-битный) номер	123e4567-e89b-12d3- a456-426655440000

Тип	Описание	Пример
UUIDList	Список идентификаторов UUID	["00000005-9d7c-011d-f000-0001e5c6e294", "00000005-9d7c-011d-f000-0001e5c6e295"]

Приложение Б. Схема полей событий

Схема полей событий представляет собой набор полей и их допустимых значений для сохранения информации о событиях на всех этапах их обработки. Столбец «Обязательное» указывает на необходимость обязательно заполнить поле в правиле нормализации или корреляции.

Таблица 331. Схема полей событий

Поле	Обязательное	Тип данных	Описание
Поля корреляционного события			
alert.context	Нет	String	Дополнительная информация о событии ИБ
alert.key	Нет	String	Дополнительная техническая информация о событии ИБ
alert.ioc_provider	Нет	String	Поставщик индикатора компрометации из события
alert.ioc_value	Нет	String	Значение индикатора компрометации из события
alert.regex_match	Нет	String	Фрагмент строки, попавшей под регулярное выражение, вызвавшей срабатывание правила
assets	Автоматически	StringList	IP-адреса и имена узлов, вовлеченных в инцидент. Эти узлы указаны в событиях, на основании которых зарегистрировано корреляционное событие
attacking_assets	Автоматически	StringList	IP-адреса и имена узлов, являющихся источниками атаки. Эти узлы указаны в событиях, на основании которых зарегистрировано корреляционное событие
correlation_name	Нет	String	Название правила корреляции (уникальное в рамках всего набора правил)

Поле	Обязательное	Тип данных	Описание
<code>correlation_type</code>	При корреляции	Enum	Результат выполнения правила корреляции: <code>draft</code> — событие, сгенерированное правилом, находящимся на этапе тестирования или отладки; <code>event</code> — корреляционное событие; <code>incident</code> — корреляционное событие и инцидент; <code>subrule</code> — событие, сгенерированное вспомогательным правилом для работы другого верхнеуровневого правила
<code>count.subevents</code>	Нет	Number	Количество событий, на основании которых зарегистрировано агрегированное или корреляционное событие
<code>detect</code>	Нет	String	Описание атаки
<code>labels</code>	Нет	String	Метки события ИБ
<code>subevents</code>	Автоматически	UUIDList	Список идентификаторов событий, на основании которых зарегистрировано агрегированное или корреляционное событие
<code>subevents.time</code>	Автоматически	StringList	Время регистрации событий, на основании которых зарегистрировано агрегированное или корреляционное событие
<code>subevents.missed</code>	Автоматически	UUIDList	Список идентификаторов событий, на основании которых было создано корреляционное событие, но которые были недоступны при его обработке
Поля категоризации событий			
<code>category.generic</code>	Нет	String	Уровень категоризации generic (см. приложение Д)
<code>category.high</code>	Нет	String	Уровень категоризации high
<code>category.low</code>	Нет	String	Уровень категоризации low

Поле	Обязательное	Тип данных	Описание
Адресные поля источника события			
<code>assigned_src_host</code>	Нет	String	Полное доменное имя, соответствующее IP-адресу, назначенному источнику
<code>assigned_src_ip</code>	Нет	IPAddress	IP-адрес, назначенный источнику. Адрес может быть назначен, например, DHCP-сервером, при трансляции адресов NAT или при подключении к VPN
<code>assigned_src_port</code>	Нет	Number	Порт, назначенный источнику при трансляции NAT
<code>external_src.fqdn</code>	Нет	String	FQDN узла — источника, внешнего по отношению к взаимодействию сторон
<code>external_src.geo.asn</code>	Нет	Number	Номер автономной системы узла-источника, внешнего по отношению к взаимодействию сторон
<code>external_src.geo.city</code>	Нет	String	Город узла-источника, внешнего по отношению к взаимодействию сторон
<code>external_src.geo.country</code>	Нет	String	Страна узла-источника, внешнего по отношению к взаимодействию сторон
<code>external_src.geo.org</code>	Нет	String	Организация, которой принадлежит узел-источник, внешний по отношению к взаимодействию сторон
<code>external_src.host</code>	Нет	String	Заполняется автоматически на основании значений других атрибутов группы полей <code>external_src</code>
<code>external_src.hostname</code>	Нет	String	Имя узла-источника (без имени домена), внешнего по отношению к взаимодействию сторон

Поле	Обязательное	Тип данных	Описание
external_src.ip	Нет	IPAddress	IP-адрес (IPv4 или IPv6) источника, внешнего по отношению к взаимодействию сторон
src.asset	Нет	UUID	Идентификатор актива, являющегося субъектом взаимодействия
src.fqdn	Нет	String	Полное доменное имя узла-источника при взаимодействии сторон (в формате FQDN)
src.geo.asn	Нет	Number	Номер автономной системы узла-источника при взаимодействии сторон
src.geo.city	Нет	String	Город узла-источника при взаимодействии сторон
src.geo.country	Нет	String	Страна узла-источника при взаимодействии сторон
src.geo.org	Нет	String	Организация, которой принадлежит узел-источник, при взаимодействии сторон
src.host	Нет	String	Заполняется автоматически на основании одного из заполненных полей — <code>src.ip</code> или <code>src.hostname</code>
src.hostname	Нет	String	Имя узла источника при взаимодействии сторон (без имени домена)
src.ip	Нет	IPAddress	IP-адрес источника при взаимодействии сторон (IPv4 или IPv6)
src.mac	Нет	MACAddress	MAC-адрес источника при взаимодействии сторон
src.port	Нет	Number	Порт источника при взаимодействии сторон
Поля, описывающие источник события			
event_src.asset	Нет	UUID	Идентификатор актива, являющегося источником события

Поле	Обязательное	Тип данных	Описание
event_src.category	При нормализации	Enum	Категория источника события (см. приложение Г), например IDS/IPS, Firewall, Mail server
event_src.fqdn	Нет	String	Полное доменное имя узла-источника события (в формате FQDN)
event_src.host	Автоматически	String	Узел источника. Заполняется автоматически на основании полей event_src.hostname, event_src.ip, recv_ipv4, recv_ipv6
event_src.hostname	Нет	String	Имя узла источника события (без имени домена)
event_src.id	Нет	String	Уникальный идентификатор источника события
event_src.ip	Нет	IPAddress	IP-адрес источника события (IPv4 или IPv6)
event_src.rule	Нет	String	Название правила, службы или приложения, благодаря которым на источнике зарегистрировано событие
event_src.subsys	Нет	String	Подсистема источника события, в которой генерируются события этого типа
event_src.title	При нормализации	String	Название приложения (источника), которое зарегистрировало событие. Указывается в нижнем регистре без пробелов, например windows, asa, ios
event_src.vendor	При нормализации	String	Производитель приложения (источника), которое зарегистрировало событие. Указывается в нижнем регистре без пробелов, например microsoft, cisco
Адресные поля получателя			
assigned_dst_host	Нет	String	Полное доменное имя, соответствующее IP-адресу, который назначен узлу получателя

Поле	Обязательное	Тип данных	Описание
<code>assigned_dst_ip</code>	Нет	IPAddress	IP-адрес, назначенный узлу получателя при трансляции NAT
<code>assigned_dst_port</code>	Нет	Number	Порт, назначенный узлу получателя при трансляции NAT
<code>dst.asset</code>	Нет	UUID	Идентификатор актива, являющегося объектом взаимодействия
<code>dst.fqdn</code>	Нет	String	Полное доменное имя узла назначения при взаимодействии сторон (в формате FQDN)
<code>dst.geo.asn</code>	Нет	Number	Номер автономной системы узла назначения при взаимодействии сторон
<code>dst.geo.city</code>	Нет	String	Город узла назначения при взаимодействии сторон
<code>dst.geo.country</code>	Нет	String	Страна узла назначения при взаимодействии сторон
<code>dst.geo.org</code>	Нет	String	Организация, которой принадлежит узел назначения, при взаимодействии сторон
<code>dst.host</code>	Нет	String	Заполняется автоматически на основании одного из заполненных полей — <code>dst.ip</code> или <code>dst.hostname</code>
<code>dst.hostname</code>	Нет	String	Имя узла назначения при взаимодействии сторон (без имени домена)
<code>dst.ip</code>	Нет	IPAddress	IP-адрес (IPv4 или IPv6) назначения при взаимодействии сторон
<code>dst.mac</code>	Нет	MACAddress	MAC-адрес назначения при взаимодействии сторон
<code>dst.port</code>	Нет	Number	Порт назначения
<code>external_dst.fqdn</code>	Нет	String	Полное доменное имя узла назначения, внешнего по отношению к взаимодействию сторон (в формате FQDN)

Поле	Обязательное	Тип данных	Описание
external_dst.geo.asn	Нет	Number	Номер автономной системы узла назначения, внешнего по отношению к взаимодействию сторон
external_dst.geo.city	Нет	String	Город узла назначения, внешнего по отношению к взаимодействию сторон
external_dst.geo.country	Нет	String	Страна узла назначения, внешнего по отношению к взаимодействию сторон
external_dst.geo.org	Нет	String	Организация, которой принадлежит узел назначения, внешний по отношению к взаимодействию сторон
external_dst.host	Нет	String	Заполняется автоматически на основании значений других атрибутов группы полей external_dst
external_dst.hostname	Нет	String	Имя узла назначения (без имени домена), внешнего по отношению к взаимодействию сторон
external_dst.ip	Нет	IPAddress	IP-адрес (IPv4 или IPv6) назначения, внешнего по отношению к взаимодействию сторон
Параметры взаимодействия			
action	Обязательно	Enum	Характер воздействия на объект (см. приложение В)
chain_id	Нет	String	Идентификатор последовательности событий
direction	Нет	String	Направление взаимодействия сторон (например, направление сетевого соединения, направление доверительных отношений между доменами)
duration	Нет	Number	Продолжительность процесса в секундах

Поле	Обязательное	Тип данных	Описание
importance	Обязательно	Enum	Уровень важности события для информационной безопасности. Возможные значения: — <code>info</code> — информационное событие, — <code>low</code> — низкая, — <code>medium</code> — средняя, — <code>high</code> — высокая, — <code>critical</code> — критически важное событие
logon_auth_method	Нет	Enum	Способ подключения при аутентификации на источнике: <code>local</code> — локальный; <code>remote</code> — удаленный
logon_service	Нет	String	Имя службы (модуля), через которую выполнена аутентификация на источнике, или название ПО (терминала), используемого при взаимодействии с источником
logon_type	Нет	Number	Тип аутентификации на источнике
protocol	Нет	String	Протокол передачи данных (ниже уровня приложения по модели OSI)
protocol.layer7	Нет	String	Протокол уровня приложения (по модели OSI)
reason	Нет	String	Описание причины, по которой произошло событие (например, обрыв соединения, неудачная попытка начать сессию, неудачная аутентификация)
start_time	Нет	DateTime	Время регистрации первого события в последовательности
status	Обязательно	Enum	Конечный результат воздействия

Поле	Обязательное	Тип данных	Описание
Поля, описывающие инцидент			
<code>incident.aggregation.closed_behavior</code>	Нет	Enum	Действие, если зарегистрированный ранее инцидент с тем же ключом имеет статус «Закрит»
<code>incident.aggregation.key</code>	Нет	String	Ключ для агрегации инцидентов (регистронезависимый)
<code>incident.aggregation.time_window</code>	Нет	Enum	Способ отсчета времени агрегации инцидентов
<code>incident.aggregation.timeout</code>	Нет	Number	Время агрегации инцидентов в секундах
<code>incident.assigned_to_user_id</code>	Нет	String	Идентификатор пользователя, который назначается ответственным за расследование инцидента
<code>incident.attacking_addresses</code>	Нет	StringList	Сетевые адреса — источники атаки
<code>incident.attacking_assets</code>	Нет	StringList	Источники атаки, которые не могут быть отнесены к активам, группам активов, сетям и сетевым адресам
<code>incident.category</code>	Нет	Enum	Тип инцидента
<code>incident.compromised_assets</code>	Нет	StringList	Скомпрометированные узлы, которые не могут быть отнесены к активам, группам активов, сетям и сетевым адресам
<code>incident.description</code>	Нет	String	Описание инцидента
<code>incident.name</code>	Нет	String	Название инцидента
<code>incident.related_addresses</code>	Нет	StringList	Названия активов, на которые направлена атака
<code>incident.related_assets</code>	Нет	StringList	Цели атаки, которые не могут быть отнесены к активам, группам активов, сетям и сетевым адресам

Поле	Обязательное	Тип данных	Описание
<code>incident.severity</code>	Нет	Enum	Опасность инцидента. Возможные значения: — <code>low</code> — низкая; — <code>medium</code> — средняя; — <code>high</code> — высокая; — <code>critical</code> — критически высокая
<code>incident.severity_behavior</code>	Нет	Enum	Уровень опасности инцидента при агрегации
Поля, описывающие субъект при взаимодействии			
<code>subject</code>	Нет	Enum	Субъект воздействия
<code>subject.account.contact</code>	Нет	String	Контакты пользователя учетной записи, например телефон или адрес электронной почты
<code>subject.account.dn</code>	Нет	String	Отличительное имя учетной записи (Distinguished Name)
<code>subject.account.domain</code>	Нет	String	Домен учетной записи
<code>subject.account.fullname</code>	Нет	String	Данные, указанные пользователем учетной записи, например его имя и фамилия
<code>subject.account.group</code>	Нет	String	Имена групп, в которые входит учетная запись, или их идентификаторы, перечисленные через « »
<code>subject.account.id</code>	Нет	String	Идентификатор учетной записи
<code>subject.account.name</code>	Нет	String	Логин учетной записи. Значение может быть взято из поля <code>subject.account.id</code> , если в событии нет информации о логине

Поле	Обязательное	Тип данных	Описание
subject.account.privileges	Нет	String	Права и привилегии учетной записи
subject.account.provider	Нет	Enum	Хранилище учетной записи в инфраструктуре
subject.application.account.domain	Нет	String	Домен учетной записи в приложении
subject.application.account.id	Нет	String	Идентификатор учетной записи в приложении
subject.application.account.name	Нет	String	Логин учетной записи в приложении
subject.application.account.privileges	Нет	String	Права и привилегии доступа учетной записи в приложении
subject.application.account.session_id	Нет	String	Идентификатор сессии учетной записи в приложении
subject.application.name	Нет	String	Название приложения
subject.account.session_id	Нет	String	Идентификатор сессии, например Logon ID
subject.domain	Нет	String	Название домена субъекта
subject.group	Нет	String	Имя группы, в которую входит субъект
subject.id	Нет	String	Идентификатор субъекта
subject.name	Нет	String	Имя субъекта
subject.privileges	Нет	String	Привилегии субъекта

Поле	Обязательное	Тип данных	Описание
subject.process.chain	Нет	String	Цепочка порождения процессов для процесса — субъекта
subject.process.cmdline	Нет	String	Команда запуска процесса (включая аргументы)
subject.process.cwd	Нет	String	Рабочая папка процесса
subject.process.fullpath	Нет	String	Полный путь к исполняемому файлу процесса (объединение строк из полей subject.process.path и subject.process.name)
subject.process.guid	Нет	String	GUID процесса
subject.process.hash	Нет	String	Названия алгоритмов и хеш-суммы исполняемого файла процесса в виде: <Функция 1>:<Сумма 1> ... <Функция N>:<Сумма N>. Блоки должны быть разделены пробелами
subject.process.hash.imphash	Нет	String	Хеш-сумма исполняемого файла (по алгоритму ImpHash)
subject.process.hash.md5	Нет	String	Хеш-сумма исполняемого файла (по алгоритму MD5)
subject.process.hash.sha1	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-1)
subject.process.hash.sha256	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-256)
subject.process.id	Нет	String	PID процесса
subject.process.meta	Нет	String	Метаданные исполняемого файла процесса в виде: Description:<Описание> Product:<Название> Company:<Производитель>
subject.process.name	Нет	String	Имя исполняемого файла процесса
subject.process.original_name	Нет	String	Имя исполняемого файла процесса, указанное разработчиком в метаданных

Поле	Обязательное	Тип данных	Описание
<code>subject.process.parent.cmdline</code>	Нет	String	Команда запуска родительского процесса (включая аргументы)
<code>subject.process.parent.fullpath</code>	Нет	String	Полный путь к исполняемому файлу родительского процесса (объединение строк из полей <code>subject.process.parent.path</code> и <code>subject.process.parent.name</code>)
<code>subject.process.parent.guid</code>	Нет	String	GUID родительского процесса
<code>subject.process.parent.hash</code>	Нет	String	Названия алгоритмов и хеш-суммы исполняемого файла родительского процесса в виде: <Функция 1>:<Сумма 1> ... <Функция N>:<Сумма N>. Блоки должны быть разделены пробелами
<code>subject.process.parent.hash.imphash</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму ImpHash)
<code>subject.process.parent.hash.md5</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму MD5)
<code>subject.process.parent.hash.sha1</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-1)
<code>subject.process.parent.hash.sha256</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-256)
<code>subject.process.parent.id</code>	Нет	String	PID родительского процесса
<code>subject.process.parent.name</code>	Нет	String	Имя исполняемого файла родительского процесса
<code>subject.process.parent.path</code>	Нет	String	Путь к исполняемому файлу родительского процесса (без имени)
<code>subject.process.path</code>	Нет	String	Путь к исполняемому файлу процесса (без имени)

Поле	Обязательное	Тип данных	Описание
subject.process.version	Нет	String	Версия исполняемого файла процесса
subject.state	Нет	String	Состояние субъекта
subject.type	Нет	String	Тип субъекта
subject.version	Нет	String	Версия субъекта
Поля, описывающие объект при взаимодействии			
object	Обязательно	Enum	Объект воздействия
object.account.contact	Нет	String	Контактная информация пользователя учетной записи (номер телефона или адрес электронной почты)
object.account.dn	Нет	String	Уникальное в рамках Active Directory имя учетной записи (Distinguished Name)
object.account.domain	Нет	String	Домен учетной записи
object.account.fullname	Нет	String	Данные, указанные пользователем учетной записи, например его имя и фамилия
object.account.group	Нет	String	Перечень групп, в которых состоит учетная запись. Идентификаторы или названия групп должны быть разделены вертикальной чертой с пробелами « »
object.account.id	Нет	String	Идентификатор учетной записи
object.account.name	Нет	String	Логин учетной записи. Значение может быть взято из поля object.account.id, если в событии нет информации о логине
object.account.privileges	Нет	String	Права и привилегии учетной записи

Поле	Обязательное	Тип данных	Описание
<code>object.account.provider</code>	Нет	Enum	Хранилище учетной записи в инфраструктуре
<code>object.account.session_id</code>	Нет	String	Идентификатор сессии учетной записи, например Logon ID
<code>object.application.account.domain</code>	Нет	String	Домен учетной записи в приложении
<code>object.application.account.id</code>	Нет	String	Идентификатор учетной записи в приложении
<code>object.application.account.name</code>	Нет	String	Логин учетной записи в приложении
<code>object.application.account.privileges</code>	Нет	String	Права и привилегии доступа учетной записи в приложении
<code>object.application.account.session_id</code>	Нет	String	Идентификатор сессии учетной записи в приложении
<code>object.application.name</code>	Нет	String	Название приложения
<code>object.domain</code>	Нет	String	Имя домена объекта
<code>object.fullpath</code>	Нет	String	Полный путь к объекту (объединение строк из полей <code>object.path</code> и <code>object.name</code>)
<code>object.group</code>	Нет	String	Имя группы, в которую входит объект
<code>object.hash</code>	Нет	String	Хеш-сумма объекта
<code>object.hash.imphash</code>	Нет	String	Хеш-сумма объекта (по алгоритму ImpHash)
<code>object.hash.md5</code>	Нет	String	Хеш-сумма объекта (по алгоритму MD5)

Поле	Обязательное	Тип данных	Описание
<code>object.hash.sha1</code>	Нет	String	Хеш-сумма объекта (по алгоритму SHA-1)
<code>object.hash.sha256</code>	Нет	String	Хеш-сумма объекта (по алгоритму SHA-256)
<code>object.id</code>	Нет	String	Идентификатор объекта
<code>object.name</code>	Нет	String	Имя объекта
<code>object.new_value</code>	Нет	String	Конечное значение измененного свойства объекта
<code>object.num_value</code>	Нет	Number	Исходное значение измененного свойства объекта (численное значение)
<code>object.path</code>	Нет	String	Путь к объекту
<code>object.process.chain</code>	Нет	String	Цепочка порождения процессов для процесса-объекта
<code>object.process.cmdline</code>	Нет	String	Командная строка запуска процесса
<code>object.process.cwd</code>	Нет	String	Рабочая папка процесса
<code>object.process.fullpath</code>	Нет	String	Полный путь к исполняемому файлу процесса (объединение строк из полей <code>object.process.path</code> и <code>object.process.name</code>)
<code>object.process.guid</code>	Нет	String	GUID процесса
<code>object.process.hash</code>	Нет	String	Названия алгоритмов и хеш-суммы исполняемого файла процесса в виде: <Функция 1>:<Сумма 1> ... <Функция N>:<Сумма N>. Блоки должны быть разделены пробелами
<code>object.process.hash.imphash</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму ImpHash)
<code>object.process.hash.md5</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму MD5)
<code>object.process.hash.sha1</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-1)

Поле	Обязательное	Тип данных	Описание
<code>object.process.hash.sha256</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-256)
<code>object.process.id</code>	Нет	String	Идентификатор процесса (PID)
<code>object.process.meta</code>	Нет	String	Метаданные исполняемого файла процесса в виде: <code>Description:<Описание> Product:<Название> Company:<Производитель></code> . Блоки должны быть разделены вертикальной чертой с пробелами « »
<code>object.process.name</code>	Нет	String	Имя исполняемого файла процесса
<code>object.process.original_name</code>	Нет	String	Имя исполняемого файла процесса, указанное разработчиком и полученное из метаданных
<code>object.process.parent.cmdline</code>	Нет	String	Команда строка запуска родительского процесса
<code>object.process.parent.fullpath</code>	Нет	String	Полный путь к исполняемому файлу родительского процесса (объединение строк из полей <code>object.process.parent.path</code> и <code>object.process.parent.name</code>)
<code>object.process.parent.guid</code>	Нет	String	GUID родительского процесса
<code>object.process.parent.hash</code>	Нет	String	Названия алгоритмов и хеш-суммы исполняемого файла родительского процесса в виде: <code><Функция 1>:<Сумма 1> ... <Функция N>:<Сумма N></code> . Блоки должны быть разделены пробелами
<code>object.process.parent.hash.imphash</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму ImpHash)
<code>object.process.parent.hash.md5</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму MD5)

Поле	Обязательное	Тип данных	Описание
<code>object.process.parent.hash.sha1</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-1)
<code>object.process.parent.hash.sha256</code>	Нет	String	Хеш-сумма исполняемого файла (по алгоритму SHA-256)
<code>object.process.parent.id</code>	Нет	String	Идентификатор родительского процесса (PID)
<code>object.process.parent.name</code>	Нет	String	Имя исполняемого файла родительского процесса
<code>object.process.parent.path</code>	Нет	String	Путь к папке, в которой находится исполняемый файл родительского процесса
<code>object.process.path</code>	Нет	String	Путь к папке, в которой находится исполняемый файл процесса
<code>object.process.version</code>	Нет	String	Версия исполняемого файла процесса
<code>object.property</code>	Нет	String	Изменяемое свойства объекта
<code>object.query</code>	Нет	String	Выполненный запрос
<code>object.state</code>	Нет	String	Состояние объекта
<code>object.storage.fullpath</code>	Нет	String	Полный путь к объекту в хранилище (объединение строк из полей <code>object.storage.name</code> и <code>object.storage.path</code>)
<code>object.storage.id</code>	Нет	String	Идентификатор или имя хранилища, в котором расположен объект (например, название общей папки или общего диска)
<code>object.storage.name</code>	Нет	String	Имя объекта в хранилище (если отличается от публичного)
<code>object.storage.path</code>	Нет	String	Путь к объекту в хранилище (если отличается от публичного)

Поле	Обязательное	Тип данных	Описание
object.type	Нет	String	Тип объекта
object.value	Нет	String	Значение изменяемого свойства объекта
object.vendor	Нет	String	Производитель объекта
object.version	Нет	String	Версия объекта
Точка сбора			
recv_asset	Нет	UUID	Идентификатор актива, являющегося передатчиком события
recv_host	Автоматически	String	Имя узла, от которого получено событие
recv_ipv4	Автоматически	IPAddress	IPv4-адрес узла, от которого получено событие
recv_ipv6	Автоматически	IPAddress	IPv6-адрес узла, от которого получено событие
recv_time	Автоматически	DateTime	Время сбора события MP 10 Collector (в часовом поясе UTC+0)
Информационные поля			
asset_ids	Автоматически	UUIDList	Перечень идентификаторов в хранилище активов
count	Автоматически	Number	Количество агрегированных событий
count.bytes	Нет	Number	Суммарное количество байтов в ответе сервера
count.bytes_in	Нет	Number	Количество входящих байтов
count.bytes_out	Нет	Number	Количество исходящих байтов
count.packets	Нет	Number	Суммарное количество пакетов
count.packets_in	Нет	Number	Количество входящих пакетов

Поле	Обязательное	Тип данных	Описание
count.packets_out	Нет	Number	Количество исходящих пакетов
datafield1 ... datafield20	Нет	String	Данные, не укладываемые в схему полей событий. Примечание. В полях datafield1 ... datafield20 строковые значения типа данных Date поддерживают только тип данных DateTime2
external_link	Нет	String	Ссылка на внешний ресурс с подробными сведениями о событии
incorrect_time	Автоматически	Bool	Корректность времени события. Если в поле original_time указано время регистрации события более суток назад и поле historical = false, то возможно, что время события указано неверно (incorrect_time = true). В иных случаях время считается корректным (incorrect_time = false)
interface	Нет	String	Название сетевого интерфейса, через который происходит взаимодействие
msgid	Нет	String	Идентификатор типа события, уникальный в рамках событий одного источника
nas_fqdn	Нет	String	Полное доменное имя (FQDN) NAS-сервера
numfield1 ... numfield5	Нет	Number	Числовые данные, не укладываемые в схему полей событий
nas_ip	Нет	IPAddress	IP-адрес NAS-сервера
original_time	Автоматически	DateTime	Время регистрации события на источнике

Поле	Обязательное	Тип данных	Описание
tcp_flag	Нет	String	Флаг в заголовке сегмента TCP (ACK, URG, FIN). Значение поля может состоять из нескольких флагов
time	Обязательно при нормализации. Автоматически при корреляции	DateTime	Время события в MaxPatrol SIEM (в часовом поясе UTC+0). Обычно <code>time = original_time</code>, для ненормализованных событий <code>time = recv_time</code>. Кроме того, если в поле <code>original_time</code> указано время регистрации события более суток назад и поле <code>historical = false</code>, то <code>incorrect_time = true</code> и <code>time = recv_time</code>. Примечание. Суточный интервал устаревания события может быть изменен в конфигурационном файле <code>siem.conf</code> в объекте <code>normalizer</code> в параметре <code>expected_event_time_deviation</code>

Сервисные поля

agent_id	Автоматически	UUID	Идентификатор MP 10 Collector, получившего событие
aggregation_name	Автоматически	String	Название правила агрегации
bad.risk_score	Автоматически	Number	Оценка риска события, присвоенная компонентом BAD
bad.triggered_models	Автоматически	String	Перечень срабатываний моделей машинного обучения BAD
body	Автоматически	String	Текст необработанного события в кодировке UTF-8
event_type	Автоматически	String	Не используется
generator	Автоматически	String	Не используется

Поле	Обязательное	Тип данных	Описание
<code>generator.type</code>	Автоматически	Enum	Название компонента, зарегистрировавшего событие: — <code>logcollector</code> — служба нормализации или агрегации, — <code>correlationengine</code> — служба корреляции, — <code>srcmonitoring</code> — мониторинг источников, — <code>iocengine</code> — модуль <code>batcheventprocessing</code>, — <code>bad</code> — компонент BAD
<code>generator.version</code>	Автоматически	String	Версия службы, зарегистрировавшей событие
<code>historical</code>	Автоматически	Bool	Совпадает со значением параметра <code>historical</code> , указанным в профиле при сборе события: <code>true</code> — устаревшее событие, <code>false</code> — обычное событие
<code>id</code>	Обязательно	String	Идентификатор правила нормализации в виде <Пространство имен>_<Производитель>_<Название источника>_<Транспорт>_<Тип события>
<code>input_id</code>	Автоматически	UUID	Идентификатор источника события
<code>job_id</code>	Автоматически	UUID	Идентификатор подзадачи MP 10 Collector на сбор событий
<code>mime</code>	Автоматически	String	MIME-тип данных необработанного события: — <code>text/plain</code> — простой текст, — <code>application/json</code> — формат JSON, — <code>application/x-pt-eventlog</code> — событие журнала Windows,

Поле	Обязательное	Тип данных	Описание
			— <code>text/csv</code> — ассоциативный массив в формате JSON, — <code>text/xml</code> — XML
<code>normalized</code>	Автоматически	Bool	Статус нормализации события: <code>true</code> — нормализованное или корреляционное событие, <code>false</code> — необработанное событие
<code>origin_app_id</code>	Автоматически	UUID	Идентификатор приложения, которое получило или зарегистрировало событие
<code>primary_siem_app_id</code>	Автоматически	UUID	Идентификатор приложения, которое обработало событие
<code>remote</code>	Автоматически	Bool	Статус репликации события: <code>true</code> — событие было зарегистрировано на одной площадке и передано на другую по правилу репликации, <code>false</code> — репликация события не проводилась
<code>scope_id</code>	Автоматически	UUID	Идентификатор пользовательской инфраструктуры в MaxPatrol SIEM
<code>siem_alias</code>	Автоматически	String	Псевдоним конвейера
<code>siem_id</code>	Автоматически	UUID	Идентификатор MP SIEM Server
<code>site_id</code>	Автоматически	UUID	Идентификатор площадки
<code>subevents.missed</code>	Автоматически	UUIDList	Список идентификаторов исходных событий, на основании которых было создано корреляционное событие, но которые были недоступны при его обработке
<code>tag</code>	Автоматически	String	Название модуля MP 10 Collector, получившего необработанное событие
<code>task_id</code>	Автоматически	UUID	Идентификатор задачи на сбор событий
<code>taxonomy_version</code>	Автоматически	String	Версия схемы полей событий

Поле	Обязательное	Тип данных	Описание
tenant_id	Автоматически	UUID	Идентификатор дочерней организации в рамках площадки
type	Автоматически	String	Не используется
uuid	Автоматически	UUID	Идентификатор события. Событие в необработанном и нормализованном виде имеет один и тот же UUID

Приложение В. Значения полей с типом данных Enum

Поля событий `action`, `object`, `status` и `subject` могут принимать значения, указанные в таблицах ниже.

Таблица 332. Значения поля `action`

Значение	Действие
<code>access</code>	Получение доступа
<code>alert</code>	Получение предупреждения
<code>allow</code>	Разрешение
<code>apply</code>	Применение
<code>assign</code>	Сопоставление одного объекта с другим
<code>backup</code>	Резервное копирование
<code>bind</code>	Присвоение IP-адреса, добавление политики в группу
<code>call</code>	Звонок
<code>check</code>	Проверка
<code>clean</code>	Очистка
<code>close</code>	Закрытие
<code>configure</code>	Настройка
<code>connect</code>	Соединение
<code>copy</code>	Копирование
<code>create</code>	Создание
<code>decrypt</code>	Расшифровка
<code>deelevate</code>	Завершение сессии с измененными привилегиями и возврат привилегий в исходное состояние
<code>deescalate</code>	Завершение сессии с повышенными привилегиями
<code>deliver</code>	Доставка
<code>deny</code>	Запрет
<code>detect</code>	Обнаружение
<code>disable</code>	Выключение
<code>disconnect</code>	Разъединение
<code>down</code>	Изменение состояния интерфейса на <code>down</code> (выключение)

Значение	Действие
download	Скачивание
elevate	Начало сессии с привилегиями, измененными другой учетной записью или процессом
enable	Включение
encrypt	Зашифровка
escalate	Начало сессии с повышенными привилегиями
exclude	Исключение
execute	Выполнение
export	Экспорт
extract	Извлечение
grant	Предоставление права, привилегии
import	Импорт
include	Включение (добавление)
info	События, носящие информационный характер, без возможности выделить действия
initiate	Инициирование
inject	Внедрение внешнего содержимого
install	Установка
leak	Утечка данных
lock	Блокирование
login	Вход в систему
logout	Выход из системы
modify	Изменение
move	Перемещение
open	Открытие
print	Распечатывание
protect	Защита
quarantine	Изолирование
read	Чтение или загрузка содержимого
receive	Принятие
reject	Отмена
remove	Удаление

Значение	Действие
rename	Переименование
request	Выполнение запроса на предоставление объекта или информации об объекте
reset	Сброс к состоянию по умолчанию
restart	Перезапуск
restore	Восстановление
revoke	Отзыв права, привилегии
rollback	Откат к предыдущему состоянию
scan	Сканирование
search	Поиск
send	Отправка
start	Запуск
state	Изменение состояния (на другое из заранее заданного перечня)
stop	Остановка
sync	Синхронизация
terminate	Аварийное завершение
truncate	Удаление содержимого объекта (с сохранением объекта)
unassign	Отмена сопоставления одного объекта с другим
unbind	Удаление политики из группы, освобождение IP-адреса
uninstall	Удаление продукта
unlock	Разблокирование
up	Изменение состояния интерфейса на up (включение)
update	Обновление (минорное)
upgrade	Обновление (мажорное)
upload	Загрузка
validate	Валидация
view	Просмотр

Таблица 333. Значения поля object

Значение	Описание
account	Учетная запись
alert	Оповещение

Значение	Описание
application	Приложение
arp_table	Таблица ARP
attack	Атака
certificate	Сертификат
check	Проверка
client	Клиент
cmdlet	Командлет
command	Команда
compliance	Соответствие требованиям безопасности
computer	Компьютер
configuration	Конфигурация
connection	Соединение
database	База данных
db_object	Объект базы данных
device	Устройство
ds_object	Объект службы каталогов
file	Файл
file_object	Объект файловой системы
flow	Поток данных
handle	Ссылка на ресурс в памяти процесса (файловый дескриптор)
host	Узел, для которого известен только IP-адрес или MAC-адрес
infected_object	Инфицированный объект
interface	Интерфейс
ip_address	IP-адрес
link	Ссылка
log	Файл журнала записи событий
logging	Журналирование
mailbox	Почтовый ящик
malware	Вредоносный объект
message	Сообщение
mode	Режим

Значение	Описание
module	Модуль
network	Сеть
node	Сетевой узел
package	Дистрибутив приложения
packet	Пакет
policy	Политика
port	Порт
process	Процесс
profile	Профиль
reg_object	Объект системного реестра
report	Отчет
request	Запрос
resource	Ресурс
rule	Правило
scan	Результат сканирования
service	Сервис
session	Сессия
socket	Программный интерфейс для передачи данных
system	Система
table	Таблица
task	Задача
thread	Поток команд
transaction	Транзакция
translation	Трансляция сетевого адреса (запись в таблице NAT)
trust	Доверительное отношение (например, между доменами)
tunnel	Туннель
update	Обновление
user_group	Группа пользователей
virtual_key	Цифровой сертификат, токен или ключ аутентификации (включая файлы с идентификационной информацией и лицензией)
virtual_machine	Виртуальная машина

Значение	Описание
volume	Том
vulnerability	Уязвимость

Таблица 334. Значения поля status

Значение	Описание
failure	Неуспешная попытка
success	Успешная попытка
ongoing	Действие происходит в данный момент. Результат выполнения еще неизвестен

Таблица 335. Значения поля subject

Значение	Описание
account	Пользователь (действия выполняются от имени учетной записи)
application	Приложение
host	Узел
process	Процесс
rule	Правило
service	Служба
system	Система

Приложение Г. Правила заполнения поля event_src.category

Правила заполнения поля нормализованных событий `event_src.category` для категоризации источников событий представлены в таблице ниже.

Таблица 336. Значения поля `event_src.category`

Значение	Описание
AAA	Аутентификация, авторизация, учетные записи
Anti-virus	Антивирусное программное обеспечение
Application security	Защита приложений
Application server	Сервер приложений (например, SAP)
Backup server	Система резервного копирования и восстановления
Certification authority	Служба сертификации (например, Microsoft Windows CA, OpenSSL)
CI/CA	Контроль целостности, контроль доступности
Database server	Сервер баз данных
DHCP server	Сервер DHCP
Directory service	Служба каталогов (например, Microsoft AD, Novell eDirectory, OpenLDAP)
DLP	DLP
DNS server	Сервер DNS
DPI	Система DPI
File service	Файловая служба (например, Samba)
Firewall	Межсетевой экран
Honeypot	Honeypot
Host security	Защита узлов
IDS/IPS	IDS/IPS
Load balancer	Балансировщик нагрузки сети
Mail server	Почтовый сервер
Mobile	Мобильное устройство
Network device	Сетевое оборудование

Значение	Описание
Network monitoring system	Система мониторинга сети
Network security	Защита сети
Network traffic analysis	Система анализа сетевого трафика
Operating system	Операционная система
Other	Другое
PAM	Контроль действий привилегированных пользователей
Physical security	Система физической защиты (например, СКУД, видеонаблюдение)
PKI	Управление носителями ключей и цифровыми сертификатами
Proxy server	Прокси-сервер
SCADA	Система SCADA
Server management	Управление сервером
SIEM	Система SIEM
Software supply chain	Цепочка поставок программного обеспечения
Storage device	Система хранения данных
Telecom	Система связи
Terminal services	Службы удаленных рабочих столов (ранее – службы терминалов)
Virtualization	Система виртуализации
Voice over IP	IP-телефония
VPN	VPN
Vulnerability scanner	Сканер уязвимостей
Web application	Веб-приложение
Web security	Сетевая защита
Web server	Веб-сервер
Wireless	Беспроводная сеть

Приложение Д. Правила заполнения полей category.generic, category.high, category.low для нормализованных событий

Правила заполнения полей нормализованных событий представлены в таблице ниже.

Таблица 337. Правила заполнения полей category.generic, category.high, category.low

Generic	High	Low
Access	Authentication	Default Credentials
		Host
		Local
		Remote
		Service
		Unknown Type
	Authorization	Host
		Network
		Object
		User
Accounting	Network Accounting	Address Translation
		Connections & Sessions
		Firewall Rules Usage
		Traffic
	User Accounting	Administrative Privilege Use
		Application & Component Launching
		Command Execution
		Data Usage
		Execution
		System Accounts Use
		System Application Launching
Anomaly	Network Anomaly	Detection
	System Anomaly	

Generic	High	Low
	User Behavior Anomaly	
Asset & Asset Group Management	Asset Group Management	Activation
		Creation
		Deactivation
		Deletion
		Modification
	Asset Management	Domain Activation
		Domain Creation
		Domain Data Collection
		Domain Deactivation
		Domain Deletion
		Host Activation
		Host Creation
		Host Data Collection
		Host Deactivation
		Host Deletion
		Hypervisor Activation
		Hypervisor Creation
		Hypervisor Data Collection
		Hypervisor Deactivation
		Hypervisor Deletion
		Link Activation
		Link Creation
		Link Data Collection
		Link Deactivation
		Link Deletion
		Network Activation
		Network Creation
		Network Data Collection
		Network Deactivation

Generic	High	Low
		Network Deletion
		Virtual Host Activation
		Virtual Host Creation
		Virtual Host Data Collection
		Virtual Host Deactivation
		Virtual Host Deletion
Attacks & Recon	Attack	Bruteforce
		Complex Attack
		DDoS
		DoS
		HIPS Alert
		Identity Theft
		IDS/IPS Alert
		Miscellaneous
		Network Attack
		Post Compromise
		Potential Attack
		Privilege Escalation
		Spam Attack
		Vulnerability Exploitation
	Recon	Crawling/Dictionary Bruteforce
		Enumeration
		Fingerprinting
		Network Scan
		OS Discovery
		Port Discovery
Business Continuity	Backup/Restore	Backup Jobs
		Backup Schedule
		Restore Jobs
	Cluster Nodes	Connection

Generic	High	Low
		Creation
		Deletion
		Replication
	Replication	Data
		Heartbeat
		Settings
Compliance	Policy Violation	Application Policy Violation
		AV Policy Violation
		Confidentiality Policy Violation
		Database Policy Violation
		IM Policy Violation
		IP Access Policy Violation
		Mail Policy Violation
		P2P Policy Violation
		Remote Access Policy Violation
		System Update Policy Violation
Configuration Management	Network Configuration	Web Policy Violation
		Access Rule Modification
		Interface Modification
		Link Modification
		Port Detection
		Port State Change
	Rights Management	Routing Changes
		Critical Privileges
		Group Assignment
		Group Creation
		Group Deletion
		Group Modification
		Object Rights Modification
		Rights Assignment
		Rights Revocation

Generic	High	Low
		Role Assignment
		Role Creation
		Role Deletion
		Role Modification
		Role Revocation
		User Membership Modification
	System Configuration Management	Application Object Modification
		Application Settings Modification
		Network Service Detection
		Network Service Modification
		Network Service Resumption
		Network Service Stoppage
		OS Service Detection
		OS Service Modification
		OS Service Resumption
		OS Service Stoppage
		OS Settings Modification
		Security Settings Modification
	User Management	Creation
		Deletion
		Detection
		External User Connection
		Locking
		Modification
		Unlocking
Hacktool	Bruteforcer	Deletion
	DoS	Detection
	Encryption	Exploitation
	Enumeration	Installation
	Exploit Kit	
	Fingerprint	

Generic	High	Low
	Miscellaneous	
	MITM	
	Remote Shell	
	Scanner	
	Shellcode	
	Sniffer	
	Spoofing	
	Tunnel	
	Web Scanner	
Hardware Management	CPU Management	CPU Detection
		CPU Resources Change
	External Device Management	Bluetooth pairing
		Mobile device
		Printer
		Scanner
	External Storage Management	USB Device
		External Disk Detection
		External Disk Mounting
		External Disk Unmounting
		SAN/NAS Detection
		SAN/NAS Mounting
		SAN/NAS Unmounting
	Hardware inventory	Hardware Configuration Errors
		Hardware Detection
		Hardware Installation
		Hardware Removal
	Memory Management	Memory Resizing
	Storage Management	Local Storage
		Network Storage
Incident Management	Incident	Creation
		Deletion

Generic	High	Low
Information Management		Detection
		Modification
		Remediation
	Data Loss	Accidental
		Deliberate
	Data Modification	By System
		By User
	Information Access Control	Assignment
		Object Access
		Settings
	Information Labeling	Label Assignment
		Label Management
	Information Leak	Confidential Information
		Critical Information
	Leakage Channel	External Drive
		Mail
		Mobile Device
		Printer
		Web
Malware	Backdoor	Curing
	Bootkit	Detection
	Botnet	Epidemic
	Miscellaneous	Mitigation
	Rootkit	
	Trojan	
	Virus	
	Worm	
Monitoring	Errors	Application
		Hardware
		Network
		System

Generic	High	Low
	Network Monitoring	Interface State
		Performance
	System Monitoring	Notification
		Performance
		Processes
		System State
Network Interaction	Network Interaction	Application
		Database
		File transfer
		Mail transfer
		Remote Management
		Suspicious
		Unknown
		VPN
		Web
Software Management	Operating System Management	Deletion
		Detection
		Installation
		Update
	Patch Management	Deletion
		Installation
	Security Software Management	Deletion
		Detection
		Installation
		Update
	Software Inventory	Deletion
		Detection
		Installation
		Update
Vulnerability Management	Vulnerability	Knowledge Base Updates
		Potential Exploit Detection

Generic	High	Low
		Vulnerability Detection
		Vulnerability Exception
		Vulnerability Remediation

Приложение Е. Правила заполнения полей инцидента category и type

Для инцидента могут быть указаны категория в поле `category` и тип в поле `type`. Для инцидентов категории «Атака» также могут быть указаны параметры.

Таблица 338. Правила заполнения полей `category`, `type` и полей параметров инцидента

category	Категория	type	Тип	Поля для параметров	
Attack	Атака	BotNetCC	Командный центр бот-сети	BotNetInfo	Общие сведения
				BotNetServers	Серверы C&C
				BotNetType	Тип бот-сети
		BotNetNode	Узел бот-сети	BotNetInfo	Общие сведения
				BotNetServers	Серверы C&C
				BotNetType	Тип бот-сети
		BruteForce	Подбор данных	BytesPerSecond	Мощность атаки — байт в секунду
				PacketsPerSecond	Мощность атаки — пакетов в секунду
				ProtocolType	Тип протокола
		DoSDDoS	Отказ в обслуживании	AttackType	Тип атаки
				BytesPerSecond	Мощность атаки — байт в секунду

category	Категория	type	Тип	Поля для параметров	
		MaliciousResources	Вредоносные ресурсы	PacketsPerSecond	Мощность атаки — пакетов в секунду
				MaliciousHashSum	Хеш-суммы файлов
				MaliciousResource	Вредоносные ресурсы
				MaliciousResourceType	Тип вредоносного ресурса
				MaliciousSoftwareIds	Идентификаторы вредоносного ПО
				MaliciousSoftwareType	Тип вредоносного ПО
		NetworkScan	Сканирование сети	ScanMethods	Методы сканирования
				ScanPorts	Список сканируемых портов
		Phishing	Фишинг	LegitimateResource	Легитимные ресурсы
				MaliciousResources	Вредоносные ресурсы
		Spam	Спам	SpamMailCount	Количество почтовых сообщений

category	Категория	type	Тип	Поля для параметров	
		UnauthorizedAccess	НСД (несанкционированный доступ)	UnauthorizedAccessEffect	Последствия НСД
				UnauthorizedAccessProtocol	Протокол
				UnauthorizedAccessType	Способ получения НСД
		VulnerabilityExploitation	Эксплуатация уязвимости	VulnerabilityExploitationEffect	Последствия эксплуатации уязвимости
				VulnerabilityExploitationType	Класс уязвимости
DataSecurity	Безопасность данных	DataLeakage	Утечка данных	—	
		UnauthorizedDataChange	Несанкционированное изменение данных	—	
		UnauthorizedDataDeletion	Несанкционированное удаление данных	—	
Malfunction	Нарушение работоспособности	BackupMalfunction	Нарушение работоспособности при резервном копировании данных	—	
		DOSAttack	Атака типа «Отказ в обслуживании»	—	
		ProtectionMalfunction	Неработоспособность средств защиты	—	
		ServiceMalfunction	Нарушение работоспособности сервиса	—	

category	Категория	type	Тип	Поля для параметров
		SoftwareMalfunction	Нарушение работоспособности ПО	—
MalwareDetection	Обнаружение вредоносного ПО	HackToolsDetection	Обнаружение хакерского ПО	—
		TrojanHorseDetection	Обнаружение троянской программы	—
		VirusDetection	Обнаружение вируса	—
		WormDetection	Обнаружение сетевого червя	—
PolicyViolation	Нарушение политик ИБ	ForbiddenServiceUsage	Использование запрещенных сервисов	—
		PatchPolicyViolation	Нарушение регламента по установке обновлений	—
		PermissionViolation	Превышение служебных полномочий	—
		SoftwareInstallationPolicyViolation	Установка запрещенного ПО	—
UnauthorizedAccess	Неавторизованный доступ	HostOrSoftCompromising	Компрометация узла/ПО	—
		ProtectionBypassing	Обход средств защиты	—

category	Категория	type	Тип	Поля для параметров
		SecurityPerimeterBreach	Нарушение границ внешнего периметра	—
		UserCompromising	Компрометация пользователя	—
Undefined	Не определена	SoftwareSuspiciousActivity	Подозрительные операции с ПО	—
		Undefined	Не определен	—
VulnerabilityManagement	Управление уязвимостями	CriticalVulnerabilityDetection	Обнаружение критически опасной уязвимости	—
		SecurityBackdoorDetection	Обнаружение бэкдора	—

Приложение Ж. Экранирование специальных символов в регулярных выражениях

В таблице ниже представлены примеры экранирования специальных символов, используемых в регулярных выражениях для правил корреляции.

Таблица 339. Экранирование специальных символов в регулярных выражениях

Специальный символ	Пример экранирования
+	\+
-	-
/	/
\	\\
	\
'	'
"	\"
{ }	{ }
[]	\[\]
()	\(\)
*	*
#	#
!	!
?	\?
\$	\\$
,	,
.	\.
,	,
\a	\\a
\v	\\v
\r	\\r
\n	\\n
\t	\\t
\f	\\f
\b	\\b

Специальный символ	Пример экранирования
<code>\S</code>	<code>\S</code>
<code>\s</code>	<code>\s</code>
<code>\w</code>	<code>\w</code>

Предметный указатель

Символы

!=, оператор	26
#, символ	31
&&, оператор	28
*, оператор	24
*, оператор последовательности	187
*, оператор токена	92
~, оператор	23
'...', оператор	31
?, оператор последовательности	186
?, оператор токена	92
[...], оператор	32
[N,M], оператор последовательности	187
, оператор токена	93
, оператор	29
+, оператор	23
+, оператор последовательности	186
+, оператор токена	91
<, оператор	27
<=, оператор	28
=, оператор	22
=, оператор токена	91
==, оператор	26
>, оператор	27
->, оператор последовательности	183
>=, оператор	28

A

aggregate, директива	129
aggregator-cli, утилита	330
and, оператор	28
and, оператор последовательности	184
append, функция	63
as, оператор последовательности	190
avg, агрегатная функция	170, 240

B

BASE64, токен	93
bool, функция	41
buffer_from_base64, функция	42

C

clear_table, инструкция	222
close, команда	192
coalesce, функция	70
COND, ключевое слово	88
correlator-cli, утилита	342
count, агрегатная функция	170, 240
CSV, токен	94
csv, функция	49

D

DATETIME, токен	95
datetime, функция	35
datetime_to_epoch, функция	36

datetime_to_epoch_ms, функция	36
datetime_to_win_ticks, функция	37
day, функция	40
decode, функция	43
div, оператор	25
drop, команда	115, 160
DURATION, токен	97
duration, функция	37

E

emit, директива	226
endsubformula, ключевое слово	116
enrich, директива	135
enrich_fields, инструкция	136
enricher-cli, утилита	337
enrichment, директива	135
epoch_ms_to_datetime, функция	38
epoch_to_datetime, функция	38
event, директива	128, 134, 174
EVENTLOG, ключевое слово	108
exec_query, функция	77, 78, 160, 165, 228, 234
exec_query_first, функция	160, 228
export_data, утилита	320

F

filter, директива	120
filter, инструкция	134, 178
filter::, префикс	120
find_substr, функция	50
flip_endianness16, функция	70
flip_endianness32, функция	70

flip_endianness64, функция	70
fpta_filler, утилита	332

H

hex_to_string, функция	43
HOSTNAME, токен	98
hour, функция	39

I

if, оператор	33
in_list, функция	64
in_subnet, функция	71, 160, 228
init, инструкция	190
insert_dec, функция insert_into	150, 205
insert_inc, функция insert_into	150, 205
insert_into, инструкция	138, 143, 149, 193, 198, 204, 211
insert_max, функция insert_into	150, 205
insert_min, функция insert_into	150, 205
IPV4, токен	98
ipv4, функция	44
IPV6, токен	99
ipv6, функция	44
is_list, функция	65

J

join, функция	45
JSON, ключевое слово	107

K

kbtools, утилита	376
------------------	-----

key, инструкция	128, 177
KEYVALUE, токен	100
keyvalue, функция	51

L

length, функция	52
limit, инструкция	165, 234
LITERAL, токен	101
lower, функция	52

M

MACADDR, токен	101
macaddr, функция	46
match, функция	53
max, агрегатная функция	170, 240
median, агрегатная функция	170, 240
min, агрегатная функция	170, 240
minute, функция	39
mod, оператор	25
month, функция	40

N

normalizer-cli, утилита	326
not, оператор	30
not, оператор последовательности	184
NTUSER, токен	102
NUMBER, токен	102
number, функция	47
number16, функция	48
number8, функция	47

O

on, инструкция	191
or, оператор	29
or, оператор последовательности	184

Q

qhandler, инструкция	78, 165, 234
query, директива	160, 228

R

rcs, утилита	324, 329, 335, 339
regex, функция	54
regex_match, функция	83, 160, 228
remove, функция	66
remove_from, инструкция	155, 217
replace, функция	57
REST, токен	103
rot13, функция	59
router-cli, утилита	340
rule, директива	179

S

second, функция	39
select_query_first, функция	81
skip, инструкция	165, 234
STRING, токен	103
string, функция	48
strip, функция	60
subformula, ключевое слово	116
submessage, функция	116

substr, функция	61
sum, агрегатная функция	170, 240
switch, оператор	33

T

table::, префикс	75
TABULAR, ключевое слово	105
TEXT, ключевое слово	89
timeout_timer, оператор последовательности	189
timer, оператор последовательности	189
timezone, функция	39

U

UNTIL, токен	104
upper, функция	62

W

win_ticks_to_datetime, функция	40
with different, оператор последовательности	185
within, оператор последовательности	188
WORD, токен	104
WORDDASH, токен	105

X

XML, ключевое слово	112
---------------------	-----

Y

year, функция	40
---------------	----

A

агрегатная функция	78, 170, 240
агрегация событий	17

И

инцидент	17
агрегация	311, 317
поля	312
регистрация	174, 311

К

корреляция событий	17
многоуровневая	178
на активах	179

M

макрос	120
--------	-----

Н

нормализация события	16
----------------------	----

О

обогащение событий	17
--------------------	----

П

правило	
агрегации	17
корреляции	17

нормализации	16
обогащения	17
профиль	16

С

событие	
агрегированное	17
корреляционное	17
нормализованное	16

Т

табличный список	17
арифметические операции над результатами запроса	78, 165, 234
арифметические операции при записи	149, 204
запись	138, 193, 211
инлайн-запрос, префикс	75
очистка	222
удаление строк	155, 217
условная запись	143, 198
токен	90

Ф

формула нормализации, см. правило нормализации	16
функция правила корреляции	
append	63
bool	41
coalesce	70
datetime_to_epoch	36

datetime_to_epoch_ms	36
datetime_to_win_ticks	37
day	40
duration	37
epoch_ms_to_datetime	38
epoch_to_datetime	38
exec_query	77, 78
find_substr	50
hex_to_string	43
hour	39
in_list	64
in_subnet	71
ipv4	44
ipv6	44
join	45
length	52
lower	52
match	53
minute	39
month	40
number	47
number16	48
number8	47
regex	54
regex_match	83
remove	66
replace	57
second	39
select_query_first	81
string	48
strip	60

substr	61	join	45
timezone	39	keyvalue	51
upper	62	length	52
win_ticks_to_datetime	40	lower	52
year	40	macaddr	46
функция правила нормализации		minute	39
append	63	month	40
bool	41	number	47
buffer_from_base64	42	number16	48
coalesce	70	number8	47
csv	49	remove	66
datetime	35	replace	57
datetime_to_epoch	36	rot13	59
datetime_to_epoch_ms	36	second	39
datetime_to_win_ticks	37	string	48
day	40	strip	60
decode	43	submessage	116
duration	37	substr	61
epoch_ms_to_datetime	38	timezone	39
epoch_to_datetime	38	upper	62
find_substr	50	win_ticks_to_datetime	40
flip_endianness16	70	year	40
flip_endianness32	70	функция правила обогащения	
flip_endianness64	70	append	63
hex_to_string	43	bool	41
hour	39	coalesce	70
in_list	64	datetime_to_epoch	36
in_subnet	71	datetime_to_epoch_ms	36
ipv4	44	datetime_to_win_ticks	37
ipv6	44	day	40
is_list	65	duration	37

epoch_ms_to_datetime	38
epoch_to_datetime	38
exec_query	77, 78
find_substr	50
hour	39
in_list	64
in_subnet	71
ipv4	44
ipv6	44
join	45
length	52
lower	52
match	53
minute	39
month	40
number	47
number16	48
number8	47
regex	54
regex_match	83
remove	66
replace	57
second	39
select_query_first	81
string	48
strip	60
substr	61
timezone	39
upper	62
win_ticks_to_datetime	40
year	40



Positive Technologies — один из лидеров в области результативной кибербезопасности. Компания является ведущим разработчиком продуктов, решений и сервисов, позволяющих выявлять и предотвращать кибератаки до того, как они причинят неприемлемый ущерб бизнесу и целым отраслям экономики. Positive Technologies — первая и единственная компания из сферы кибербезопасности на Московской бирже (MOEX: POSI). Количество акционеров превышает 220 тысяч.